



Document History

[illegible]

본 문서의 목적

본 교안은 SW Expert Academy에 있는 'Design Patterns(기본 이론)' 동영상 강의에서 사용된 교재입니다. 해당 동영상 강의는 1) Associate Architect 양성과정 선발 평가 준비, 2) SW개발자의 SW구조설계 역량에 도움을 제공하려는 목적으로 활용하고 있습니다.

'Associate Architect 양성과정 선발 평가'는 2과목으로 구성되어 있습니다.

- 과목 1: OOP/OOAD/UML(기본 이론)
- 과목 2: Design Patterns(기본 이론) (본 파일)

두 과목 중 어느 것을 먼저 학습하셔도 관계없으나 일반적으로는 OOP/OOAD/UML(기본 이론)을 먼저, 이후 Design Pattern(기본 이론)을 학습하시는 것이 무난하겠습니다.



보안 및 저작권 관련 주의 사항

1. 보안 관련 사항

본 자료에 있는 강의 내용은 특정 기업의 영업에 관한 사항이 아니고 일반적인 SW설계 이론입니다. 따라서 보안 민감도가 낮은 자료라 할 수 있습니다. 그러나 본 자료를 출력물 또는 이메일 등 전자 자료의 형태로 '반출'시 어떤 절차가 필요할지는 각 부서 및 사업장의 보안 담당 부서에서 해석하는 것이 맞다고 할 수 있겠습니다. 따라서 교육주관처(삼성전자 첨단기술연수소)에서는 위에서 말씀드린 바와 같이 본 자료의 내용에 대해서만 설명드릴 수 있으며, 반출 관한 절차에 대해서는 반출자가 판단하시고 직접 진행하시기 바랍니다.

2. 저작권 관련 사항

- ① 본 교안은 타/개인 단체에 저작권이 있는 자료를 포함하고 있으며, 그러한 경우 출처를 명확하게 표기하여 저작권 소유자의 권리를 존중하고 있습니다.
- ② 본 교안은 절대 영리 목적으로 사용되어서는 안되며, 개인의 S/W 구조설계 역량 강화를 위한 학습 목적으로만 사용되어야 합니다.
- ③ 본 교안을 개인 학습 이외의 목적으로 사용하거나, 개인 학습 목적으로 사용하더라도 출처를 명확하게 표기하지 않아 발생하는 모든 분쟁은 사용자 개인에게 있음을 알립니다.

1. INTRODUCTION TO DESIGN PATTERNS

2020. 04. 16

중앙대학교 소프트웨어학부

이찬근 교수



Contents

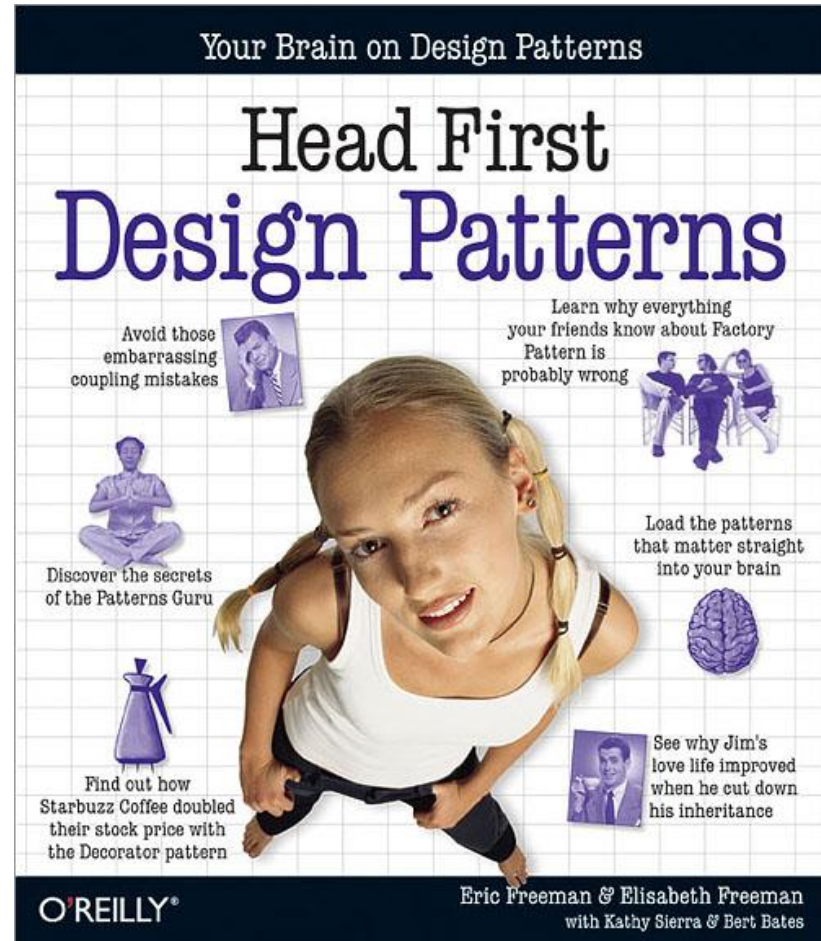
- 1. Course outlines**
- 2. What is a pattern?**
- 3. Software pattern history**
- 4. Design pattern categories**
- 5. Benefits of patterns**
- 6. Levels of patterns**

Course Outlines

- Prerequisite
 - Java or C++ experience
 - OOAD
- Intro. to Design Patterns
- OOP Review
- Design Principles
 - SOLID Principles
 - GRASP Principles
- GoF Design Patterns
 - Behavioral Patterns
 - Creational Patterns
 - Structural Patterns
- Compound Pattern (MVC)

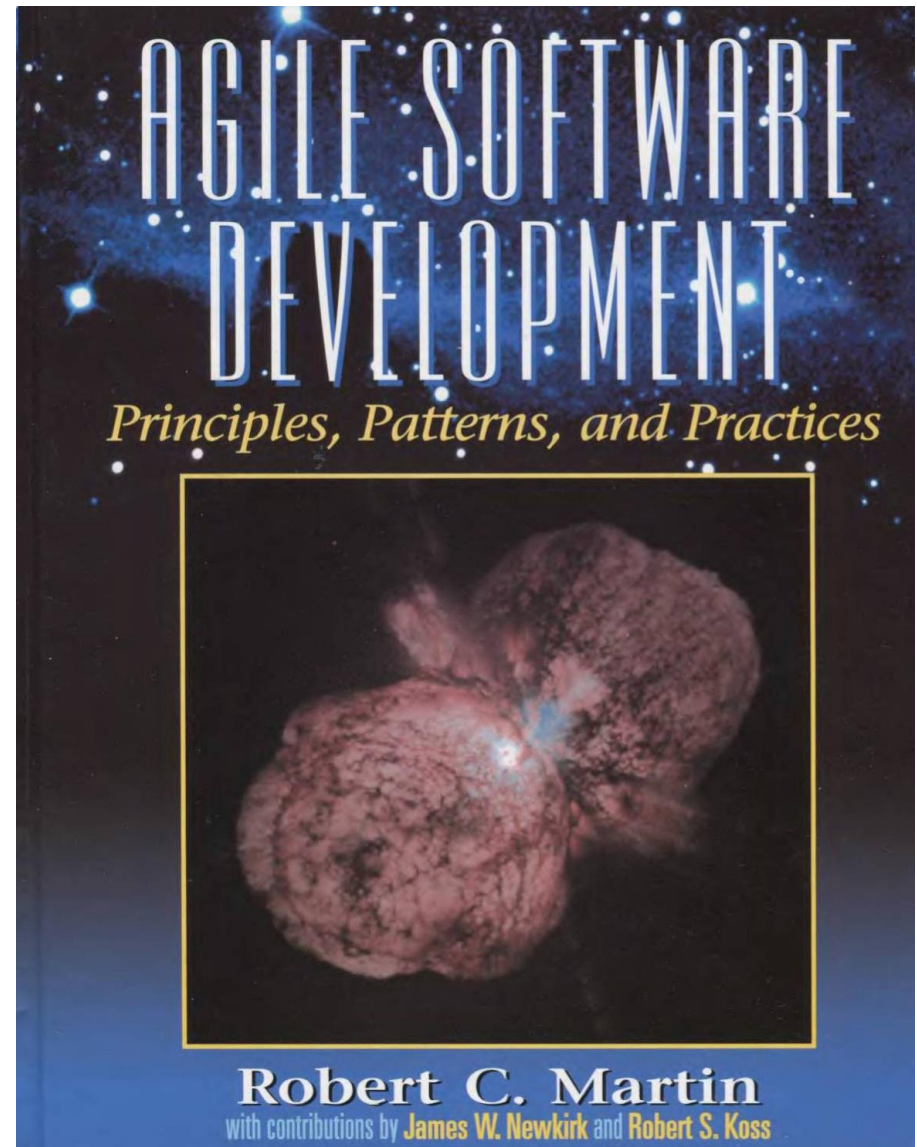
Main Textbook

- Required
 - **Head First Design Patterns** by Eric Freeman, Elisabeth Freeman, Kathy Sierra, and Bert Bates. O'Reilly



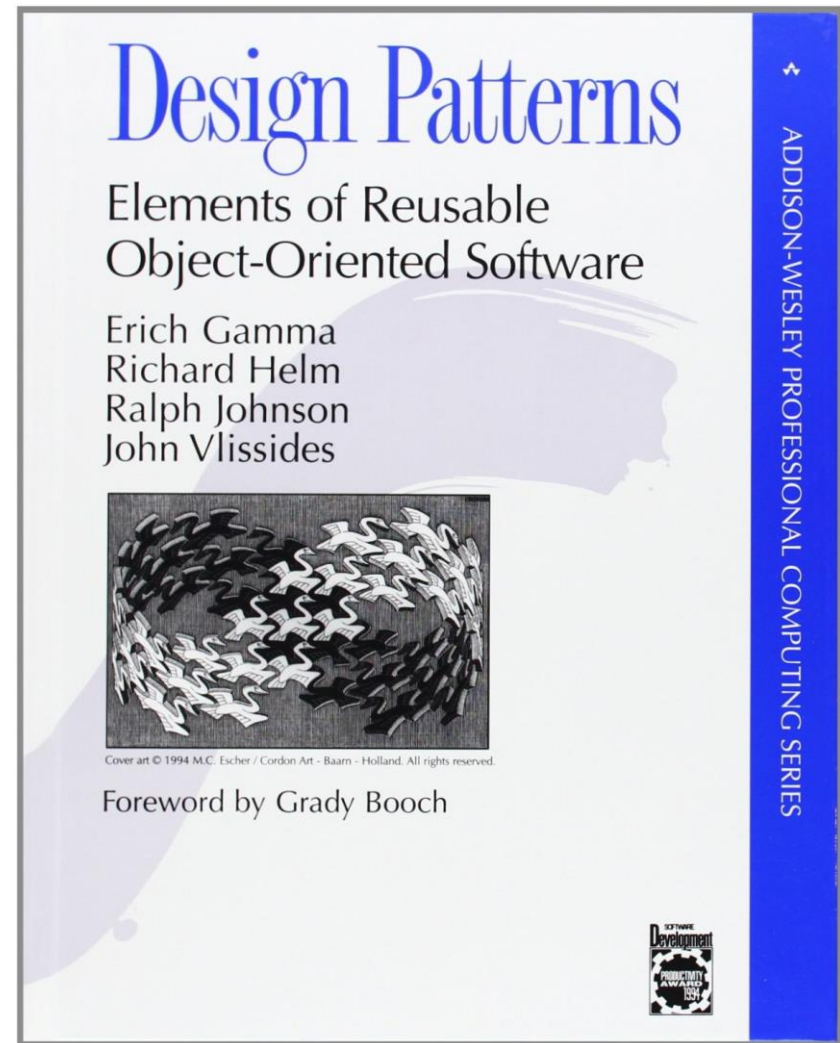
Reference

- Recommended
 - Agile Software Development: Principles, Patterns, and Practices
by Robert. C. Martin.
Prentice Hall



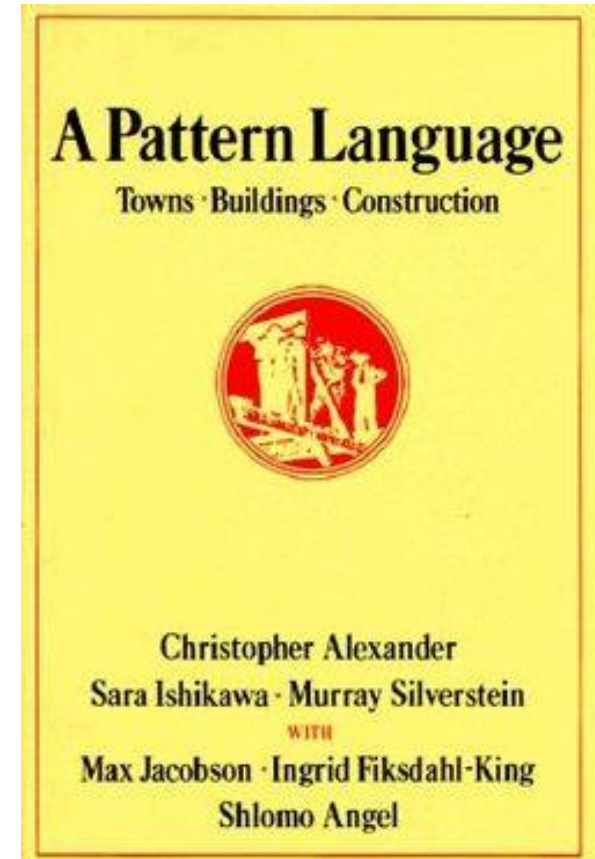
Reference

- Recommended
 - **Design Patterns: Elements of Reusable Object-Oriented Software** by Gamma, Helm, Johnson, and Vlissides (GoF). Addison-Wesley. 1995



What is a pattern?

- **Christopher Alexander, an architect**, studied ways to improve the process of designing **buildings** and **urban areas**
- Experts often recall a similar problem they have already solved and reuse the essence of its solution to solve a new problem.
- Common Definition:
 - “A solution to a problem in a context”
- Later, some people realized that the patterns are **applicable to software development**, too!



Software Patterns History

- 1977: The architect Christopher Alexander, A Pattern Language: Towns, Buildings, Construction
- 1987: Kent Beck and Ward Cunningham, OOPSLA Paper
 - Adopted Alexander's pattern idea for Smalltalk GUI design
- 1991: Erich Gamma, Ph. D. thesis
- 1995: Gamma, Helm, Johnson, Vlissides (Gang of Four) Design Patterns: Elements of Reusable Object-Oriented Software
- 1994-present: Pattern Languages of Programs (PLoP) Conferences and books

Why do we use patterns?

- Designing object-oriented software is hard, **Designing reusable object-oriented software is even harder**” – Erich Gamma
- **Experienced designers reuse solutions** which were proved to work in the past
- **Well-structured object-oriented systems** have recurring **patterns** of classes and objects
- **Knowledge of patterns** allow a designer to be **more productive** and the resulting designs to be **more flexible and reusable**
- Facilitate **communication** among developers by providing a common language
- **Someone has already solved your problems !**

Three Part Rule of Design Pattern

- “A solution to a problem in a context”
 - The **context** is the situation in which the pattern applies. This should be a **recurring** situation
 - The **problem** refers to the **goal** you are trying to achieve in this context, but it also refers to any **constraints** that occur in the context
 - The **solution** is **what you're after**: a general design that anyone can apply which resolves the goal and the set of constraints
 - Structure with components and relationships
 - Run-time mechanism

Example of Design Pattern: Iterator

■ Iterator Pattern

■ Context

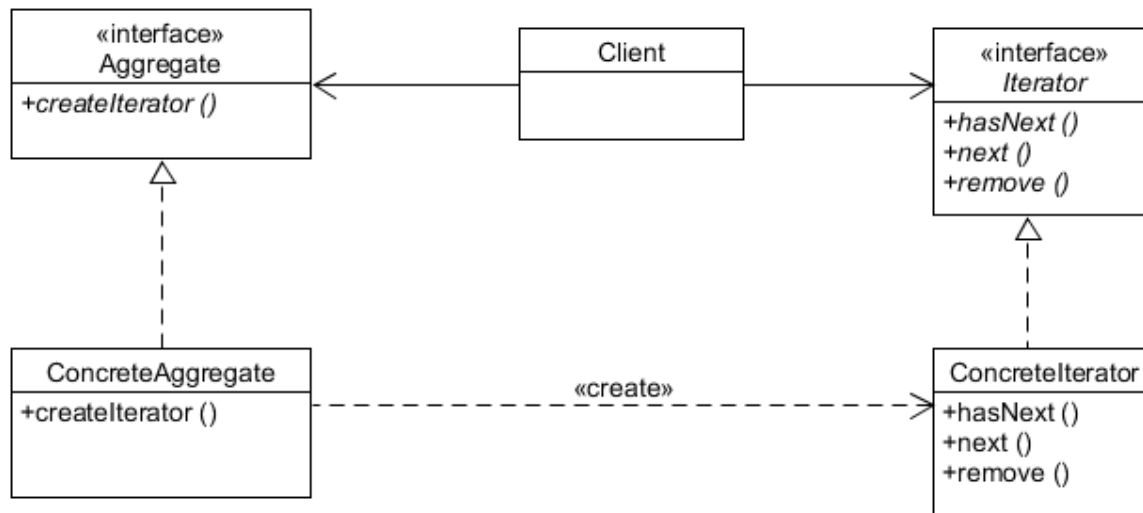
- You have a **collection of objects**

■ Problem

- You need to **step through** the objects **without exposing the collection's implementation**

■ Solution

- **Encapsulate the iteration** into a separate class



Category of GoF Patterns

		Purpose		
		Defer object creation to another class or object	Describe ways to assemble objects	Address problems of assigning responsibilities
		Creational	Structural	Behavioral
Scope	Class	Factory Method	Adapter	Interpreter Template
	Object	Abstract Factory Builder Prototype Singleton	Adapter Bridge Composite Decorator Façade Flyweight Proxy	CoR Command Iterator Mediator Memento Observer State Strategy Visitor

Category of GoF Patterns

- **Creational**

- Address problems of creating an object in a flexible way
- **Separate creation** from operation/use

- **Structural**

- Address problems of using OO constructs like inheritance to **organize classes and objects**

- **Behavioral**

- Address problems of **assigning responsibilities** to classes
- Suggest both **static relationships** and **patterns of communication**

Key Features of Design Patterns

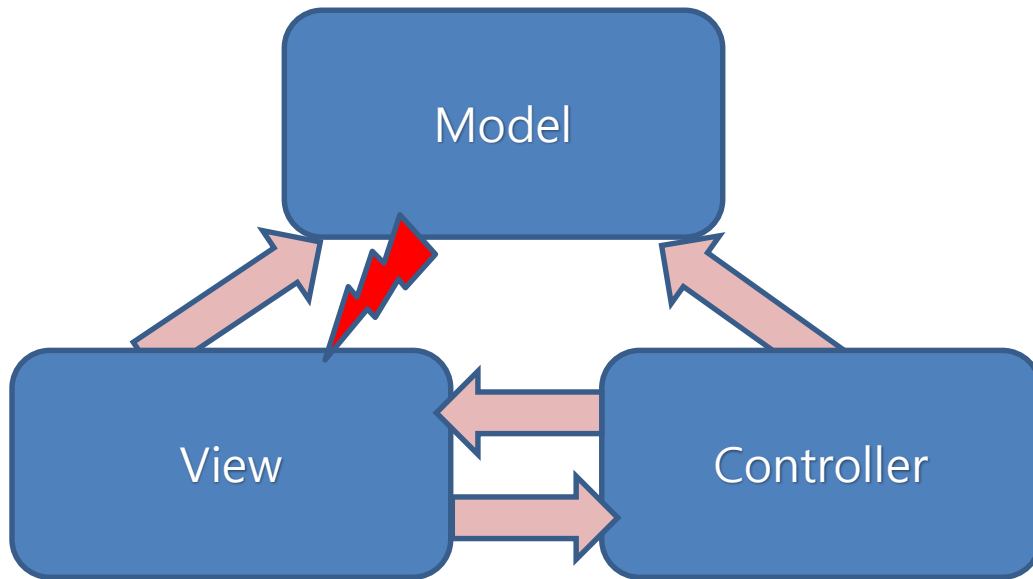
Pattern name	A concise, meaningful name for a pattern improves communication among developers
Intent	The purpose of the pattern
Problem	The problem that the pattern is trying to solve
Solution	How the pattern provides a solution to the problem in the context where it shows up. Emphasizes their relationships, responsibilities and collaborations; rather an abstract description
Participants and collaborators	The entities involved in the pattern
Consequences	The pros and cons of using the pattern (includes impacts on reusability, portability, extensibility)
Implementation	How the pattern can be implemented. Note: Implementations are just concrete manifestations of the pattern the pattern and should not be considered as the pattern itself
Generic structure	A standard diagram showing a typical structure for the pattern

Levels of Patterns [POSA]

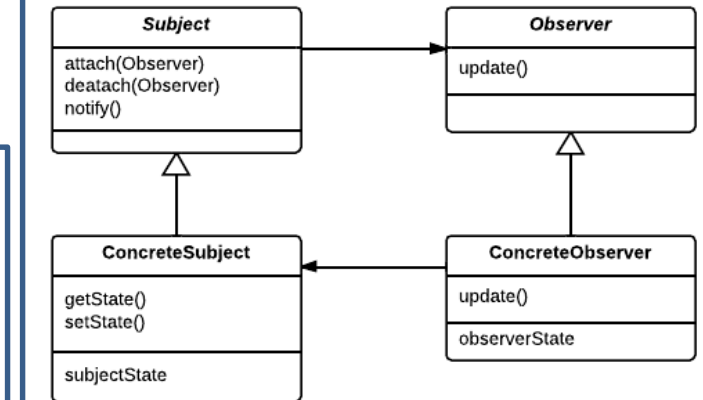
- Architectural pattern
 - **fundamental structural** organization or schema
 - provides **predefined subsystems**, specifies their **responsibilities**, and includes **rules and guidelines for organizing relationships** between them
 - **affects the overall skeletal structure and organization** of a software
- Design pattern
 - **refine subsystems or components**, or **relationships** between them.
 - describes commonly recurring structure of components that solves a general design problem within a particular context
 - **does not influence overall system structure**, but instead define *micro-architectures* of subsystems and components
- Coding pattern (or programming idiom)
 - **low-level pattern** specific to a **programming language**
 - describes **how to implement particular aspects** of components or the relationships between them **using the features of the given language**.

Levels of Patterns

MVC: Architecture Pattern



Observer: Design Pattern



Counter Pointer: Coding Idiom

It makes memory management of dynamically-allocated shared objects in C++ easier. It introduces a reference counter to a body class that is updated by handle objects ...

Summary

- Software pattern and pattern history
- Design pattern categories
 - creational, structural, behavioral
- Benefits of patterns
 - Reusable design
 - Communication language
- Levels of patterns
 - Architectural pattern, design pattern, coding pattern

DP Intro Quiz : Problem

다음 중 잘못된 설명은?

1. 설계 작업에서 설계패턴의 이름을 통해 보다 명확하게 의사전달을 할 수 있다.
2. 설계패턴은 설계 시 자주 반복되는 문제에 대한 해결책을 담고 있다.
3. 아키텍처 패턴은 주로 컴포넌트 내부의 설계에 사용되며, 설계패턴은 시스템의 전체 구조를 결정하는데 사용된다.
4. 코딩 패턴은 특정한 프로그래밍 언어의 특징에 종속적일 수 있다.

다음 중 생성(creational) 패턴이 아닌 것은?

1. Factory Method Pattern
2. Abstract Factory Pattern
3. Singleton Pattern
4. State Pattern

DP Intro Quiz : Problem

다음 중 잘못된 설명은?

1. 설계 작업에서 설계패턴의 이름을 통해 보다 명확하게 의사전달을 할 수 있다.
2. 설계패턴은 설계 시 자주 반복되는 문제에 대한 해결책을 담고 있다.
3. 아키텍처 패턴은 주로 컴포넌트 내부의 설계에 사용되며, 설계패턴은 시스템의 전체 구조를 결정하는데 사용된다.
4. 코딩 패턴은 특정한 프로그래밍 언어의 특징에 종속적일 수 있다.

다음 중 생성(creational) 패턴이 아닌 것은?

1. Factory Method Pattern
2. Abstract Factory Pattern
3. Singleton Pattern
4. State Pattern → 행위(Behavioral) Pattern

2. OBJECT-ORIENTED PARADIGM REVIEW

2020. 04. 16

중앙대학교 소프트웨어학부

이찬근 교수

Contents

- Abstract Data Type
- Object-Oriented Paradigm
- Inheritance
- Polymorphism
- Method Overloading
- Method Overriding
- What is an Interface?
- Polymorphism Examples
- Class Relationships and Change Propagations

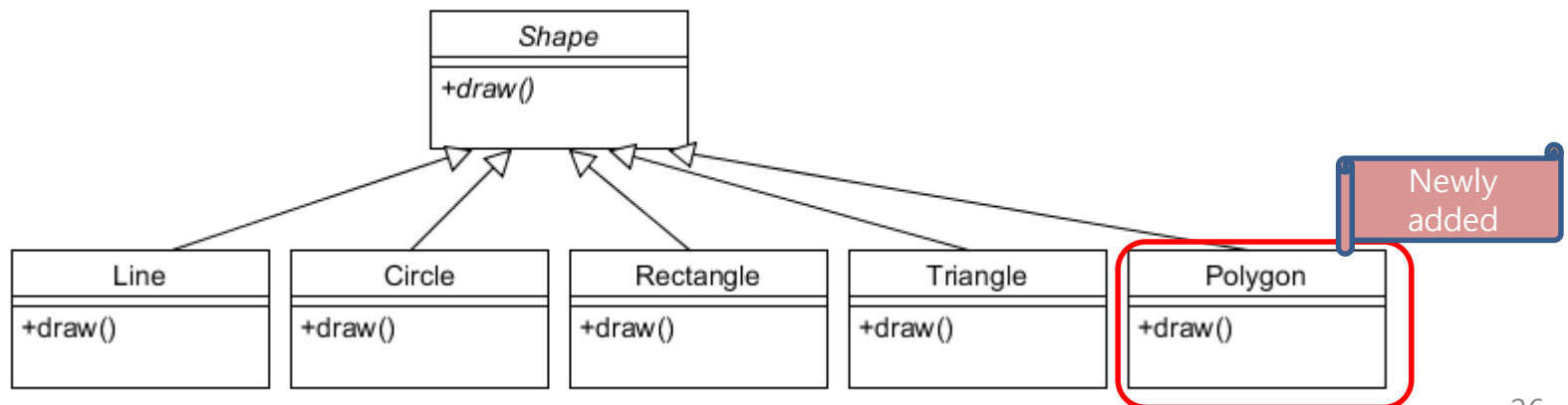
Data Abstraction

- **Abstract Data Type (ADT)**
 - **Encapsulation** of data and related operations into a **single syntactic unit**
 - Single syntactic unit
 - increases program **organization, modifiability** (everything associated with a data structure is together), and **separate compilation**
 - Encapsulation
 - increases **reliability**; by hiding the data representations, user code cannot directly access objects of the type or depend on the representation, **allowing the representation to be changed without affecting user code**

Object-Oriented Paradigm

Class = ADT + Inheritance + Polymorphism

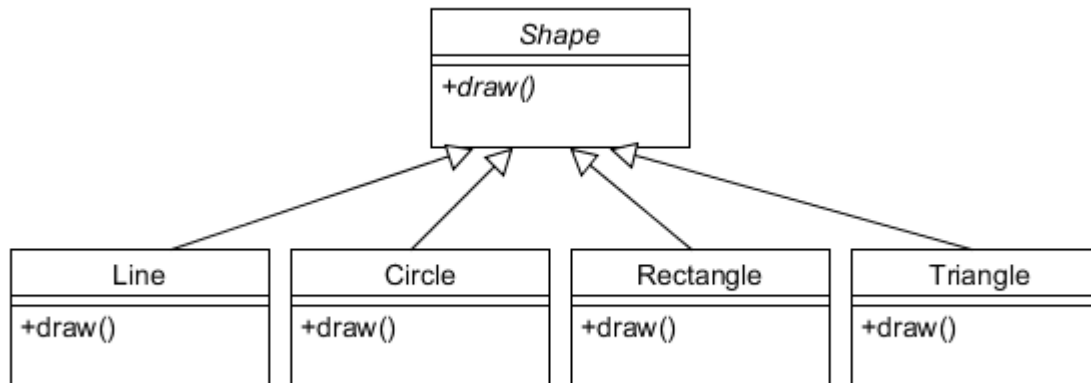
Promotes reusability and flexibility



Object-Oriented Paradigm

Class = ADT + Inheritance + Polymorphism

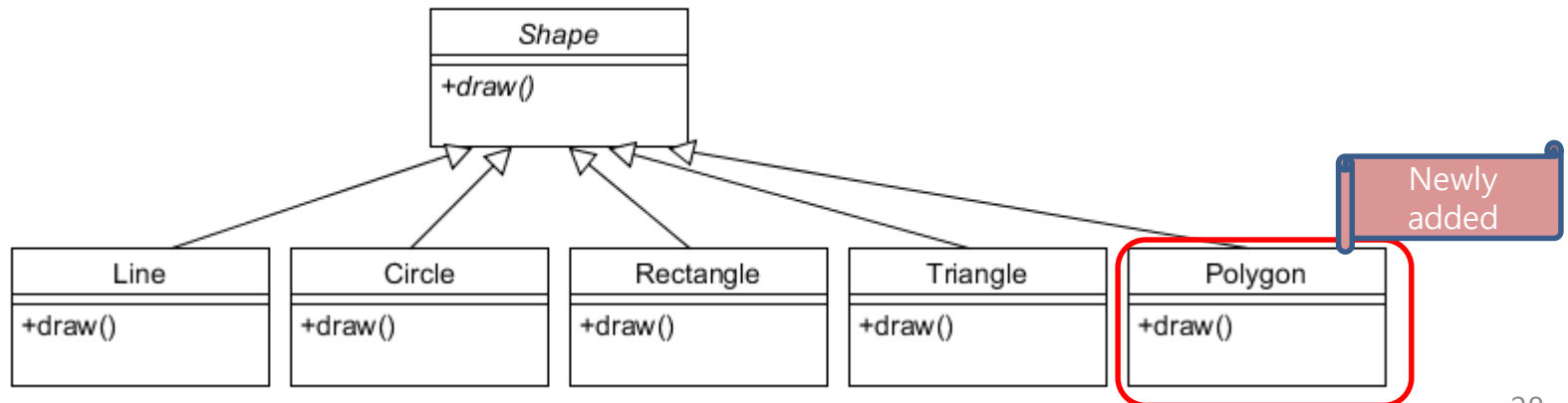
```
class ClientOfShape
{
    void renderShape(Shape s)
    {
        if (s.type == LINE) DrawLine(s.data);
        else if ((s.type == CIRCLE) DrawCircle(s.data);
        else if ((s.type == RECT) DrawRect(s.data);
        else if ((s.type == TRIANGLE) DrawTriangle(s.data);
    }
}
```



Object-Oriented Paradigm

Class = ADT + Inheritance + Polymorphism

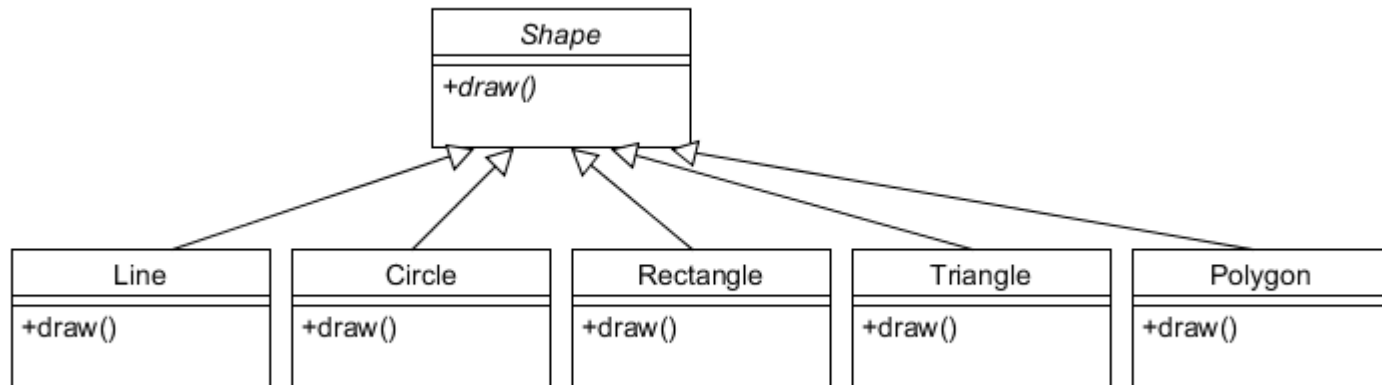
```
class ClientOfShape
{
    void rendershape(Shape s)
    {
        if (s.type == LINE) DrawLine(s.data);
        else if ((s.type == CIRCLE) DrawCirlce(s.data);
        else if ((s.type == RECT) DrawRect(s.data);
        else if ((s.type == TRIANGLE) DrawTriangle(s.data);
        else if ((s.type == POLYGON) DrawPolygon(s.data);
    }
}
```



Object-Oriented Paradigm

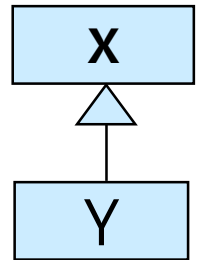
Class = ADT + Inheritance + Polymorphism

```
class ClientOfShape
{
    void renderShape(Shape s)
    {
        s.draw(); // No Change!
    }
}
```



Inheritance

- When class Y inherits from class X
 - Y inherits all the methods of X
 - Instances of Y can be sent all the messages that X responds to
 - Y inherits all the data from X
 - Instances Y have instances of all the data of X
 - As a consequence we can say
 - Y is a X
 - Every instance of Y is also an instance of X
 - **We can use the instance of Y wherever the instance of type X is expected**



Polymorphism

- A property of object oriented software by which **an operation** (typically virtual) **may be performed in different ways** in different classes.
 - Requires that there be multiple methods of the same name
 - The choice of which one to execute **depends on the object** that is in a variable
 - **Reduces** the need for programmers to code **many if-else or switch statements**
- Types of Polymorphism
 - Runtime polymorphism (Dynamic polymorphism)
 - Method Overriding
 - Compile time polymorphism (Static polymorphism)
 - Method Overloading

Method Overloading

```
class X {  
    void methodA(int num) {  
        System.out.println ("methodA:" + num);  
    }  
    void methodA(int num1, int num2) {  
        System.out.println ("methodA:" + num1 + "," + num2);  
    }  
    double methodA(double num) {  
        System.out.println("methodA:" + num);  
        return num;  
    }  
}  
  
public class test {  
    public static void main (String args []) {  
        X Obj = new X();  
        double result;  
        Obj.methodA(20);  
        Obj.methodA(20, 30);  
        result = Obj.methodA(5.5);  
        System.out.println("Answer is:" + result);  
    }  
}
```

methodA:20

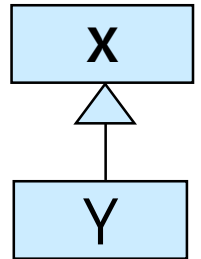
methodA:20,30

methodA:5.5

Answer is:5.5

Method Overriding

```
public class X {  
    public void methodA() { //Base class method  
        System.out.println ("hello, I'm methodA of class X");  
    }  
}  
  
public class Y extends X {  
    public void methodA() { //Derived Class method  
        System.out.println ("hello, I'm methodA of class Y");  
    }  
}  
  
public class Z {  
    public static void main (String args []) {  
        X obj1 = new X(); // Reference and object X  
        X obj2 = new Y(); // X reference but Y object  
        obj1.methodA();  
        obj2.methodA();  
    }  
}
```



```
hello, I'm methodA of class X  
hello, I'm methodA of class Y
```

(Run-time) Polymorphism

- A *polymorphic variable* can be defined in a class that is able **to reference** (or point to) **objects of the class** and **objects of any of its descendants**
- When a class hierarchy includes classes that override methods and such methods are called through a polymorphic variable, the binding to the **correct method** will be dynamic
- Allows software systems to be **more easily extended during both development and maintenance**

Abstract Class and Abstract Method (in Java)

- An *abstract method* is one that does not include a definition (it only defines a protocol)
- A class defined with “abstract” keyword is an abstract class
- A class with at least one abstract method should become an abstract class (but not vice versa)
- An abstract class cannot be instantiated
- But, a variable of abstract class type can refer to any objects of its decedents!

Polymorphism + Abstract class

```
abstract class Animal {  
    protected String name;  
    abstract public void say();  
}
```

```
class Cat extends Animal {  
    private void meow() { ... }  
    public void say() { meow(); }  
}
```

```
abstract class Canine extends Animal {  
    protected bool likeBones;  
}
```

```
class Dog extends Canine {  
    private void bark() { ... }  
    public void say() { bark(); }  
}
```

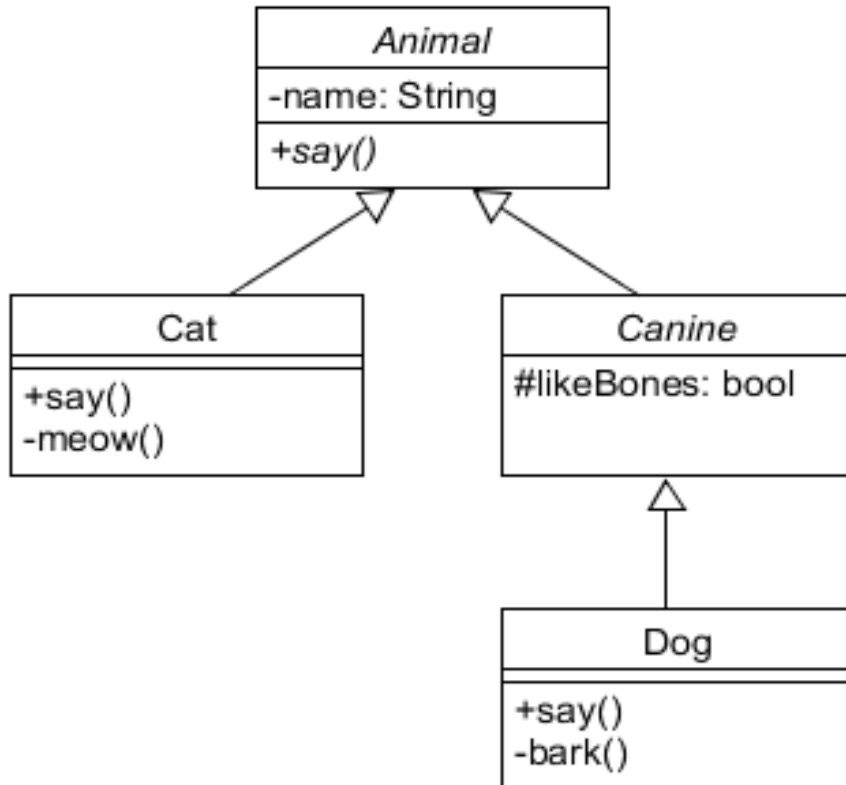
```
Animal a = null;  
Dog baduki = new Dog();  
Cat nabi = new Cat();
```

```
a = baduki; a.say();  
a = nabi; a.say();
```

```
Animal c1 = new Animal();  
Animal c2 = new Cat();  
Cat c3 = new Animal();  
Cat c4 = new Cat();
```

```
Animal d1 = new Canine();  
Animal d2 = new Dog();  
Canine d3 = new Dog();  
Canine d4 = new Cat();  
Dog d5 = new Canine();  
Dog d6 = new Dog();
```


Polymorphism + Abstract class



```
Animal a = null;  
Dog baduki = new Dog();  
Cat nabi = new Cat();
```

```
a = baduki; a.say();  
a = nabi; a.say();
```

```
Animal c1 = new Animal();  
Animal c2 = new Cat();  
Cat c3 = new Animal();  
Cat c4 = new Cat();
```

```
Animal d1 = new Canine();  
Animal d2 = new Dog();  
Canine d3 = new Dog();  
Canine d4 = new Cat();  
Dog d5 = new Canine();  
Dog d6 = new Dog();
```

Polymorphism + Abstract class

<pre>abstract class Animal { protected String name; abstract public void say(); } class Cat extends Animal { private void meow() { ... } public void say() { meow(); } } abstract class Canine extends Animal { protected bool likeBones; } class Dog extends Canine { private void bark() { ... } public void say() { bark(); } }</pre>	<pre>Animal a = null; Dog baduki = new Dog(); Cat nabi = new Cat(); a = baduki; a.say(); a = nabi; a.say(); Animal c1 = new Animal(); //Compile Error! Animal c2 = new Cat(); Cat c3 = new Animal(); //Compile Error! Cat c4 = new Cat(); Animal d1 = new Canine(); //Compile Error! Animal d2 = new Dog(); Canine d3 = new Dog(); Canine d4 = new Cat(); //Compile Error! Dog d5 = new Canine(); //Compile Error! Dog d6 = new Dog();</pre>
---	---

How a decision is made about which method to run

Step 1: If there is a concrete method for the operation in the current class, run that method.

Step 2: Otherwise, check in the immediate superclass to see if there is a method there; if so, run it.

Step 3: Repeat step 2, looking in successively higher superclasses until a concrete method is found and run.

Step 4: If no method is found, then there must be an error

- In Java and C++ the program would not have compiled

What is an Interface?

- An interface is similar to an abstract class with the following exceptions:
 - All methods defined in an interface are abstract; Interfaces cannot contain any implementation
 - Interfaces cannot contain instance variables.
 - However, they can contain public static final variables (i.e. constant class variables)
- Interfaces are declared using the "interface" keyword
 - If an interface is public, it must be contained in a file which has the same name.
- Interfaces are more abstract than abstract classes
- Interfaces are implemented by classes using the "implements" keyword.

Declaring an Interface

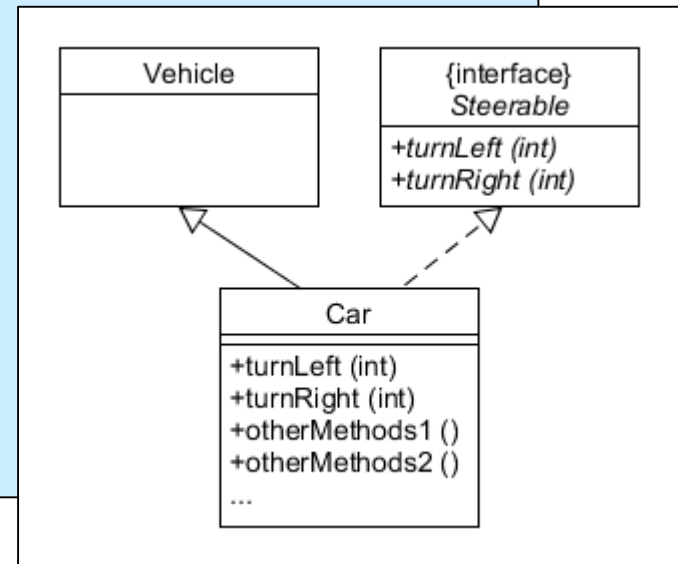
In Steerable.java:

```
public interface Steerable
{
    public void turnLeft(int degrees);
    public void turnRight(int degrees);
}
```

In Car.java:

```
public class Car extends Vehicle implements Steerable
{
    public void turnLeft(int degrees)
    { ... }
    public void turnRight(int degrees)
    { ... }
    public otherMethods1() { ... }
    public otherMethods2() { ... }
    ...
}
```

When a class "implements" an interface, the compiler ensures that it provides an implementation for all methods defined within the interface.



Interfaces as Types (Important !)

- When a class is defined, the compiler views the class as a new type.
- The same thing is true of interfaces. The compiler regards an interface as a type.
 - It can be used to **declare variables or method parameters**

Abstract Classes V.S. Interfaces

- When should one use abstract class instead of interface?
 - If the subclass-superclass relationship is genuinely an "is a" relationship.
 - If the abstract class can provide an implementation at the appropriate level of abstraction
- When should one use interface in place of abstract class?
 - When the methods defined represent a small portion of a class
 - When the subclass needs to inherit from another class
 - When you cannot reasonably implement any of the methods

Polymorphism with Interface

```
abstract class Animal {
    protected String name;
    abstract public void say();    }

class Cat extends Animal implements Sayable {
    private void meow() { ... }
    public void say() { meow(); }    }

abstract class Canine extends Animal {
    protected boolean likeBones;    }

class Dog extends Canine implements Sayable {
    private void bark() { ... }
    public void say() { bark(); }    }

class Robot implements Sayable {
    private void printOut() { ... }
    public void say() { printOut(); }    }

interface Sayable {
    public void say();
}
```

```
Dog  baduki = new Dog();
Cat  nabi = new Cat();
Robot robo = new Robot();

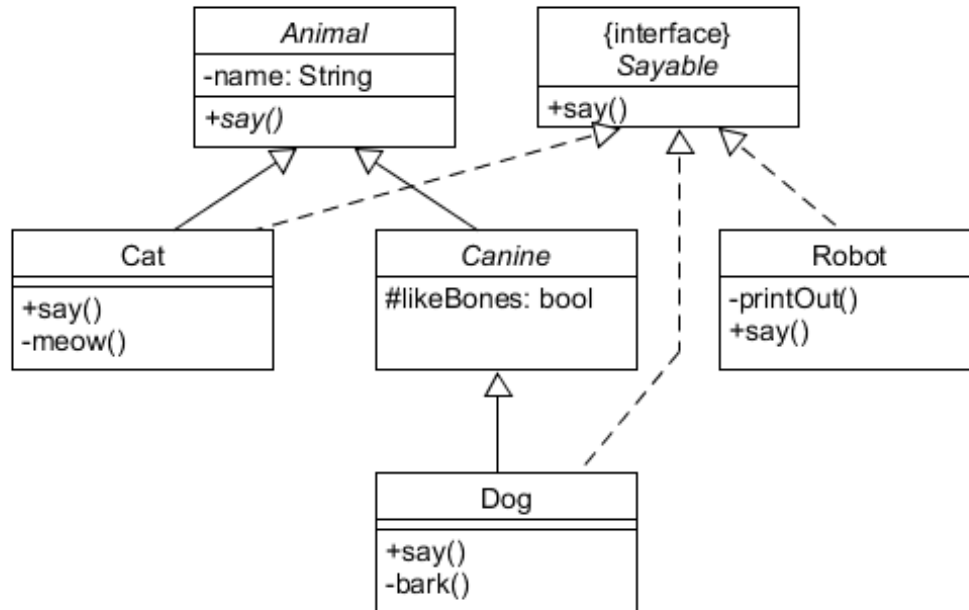
Animal aref = null;
Sayable sref = null;
Canine cref = null;

aref = baduki; aref.say();
aref = nabi; aref.say();
aref = robo; aref.say();

sref = baduki; sref.say();
sref = nabi; sref.say();
sref = robo; sref.say();

cref = baduki; cref.say();
cref = nabi; cref.say();
cref = robo; cref.say();
```


Polymorphism with Interface



```
Dog baduki = new Dog();
Cat nabi = new Cat();
Robot robo = new Robot();
```

```
Animal aref = null;
Sayable sref = null;
Canine cref = null;
```

```
aref = baduki; aref.say();
aref = nabi; aref.say();
aref = robo; aref.say();
```

```
sref = baduki; sref.say();
sref = nabi; sref.say();
sref = robo; sref.say();
```

```
cref = baduki; cref.say();
cref = nabi; cref.say();
cref = robo; cref.say();
```

Polymorphism with Interface

```
abstract class Animal {
    protected String name;
    abstract public void say(); }

class Cat extends Animal implements Sayable {
    private void meow() { ... }
    public void say() { meow(); } }

abstract class Canine extends Animal {
    protected boolean likeBones; }

class Dog extends Canine implements Sayable {
    private void bark() { ... }
    public void say() { bark(); } }

class Robot implements Sayable {
    private void printOut() { ... }
    public void say() { printOut(); } }

interface Sayable {
    public void say(); }
```

```
Dog baduki = new Dog();
Cat nabi = new Cat();
Robot robo = new Robot();

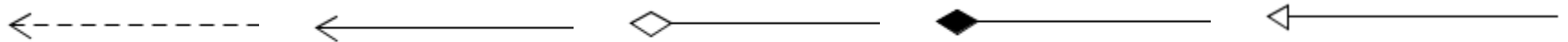
Animal aref = null;
Sayable sref = null;
Canine cref = null;

aref = baduki; aref.say();
aref = nabi; aref.say();
aref = robo; aref.say(); // Error!

sref = baduki; sref.say();
sref = nabi; sref.say();
sref = robo; sref.say();

cref = baduki; cref.say();
cref = nabi; cref.say(); // Error!
cref = robo; cref.say(); // Error!
```

Class Relationships



Dependency	Association	Aggregation	Composition	Inheritance
dashed arrow	simple connecting line	empty diamond arrow	filled diamond arrow	empty arrow
when objects of one class work briefly with objects of another class	when objects of one class work with objects of another class for some prolonged amount of time	when one class owns but shares a reference to objects of another class	when one class contains objects of another class	when one class is a type of another class

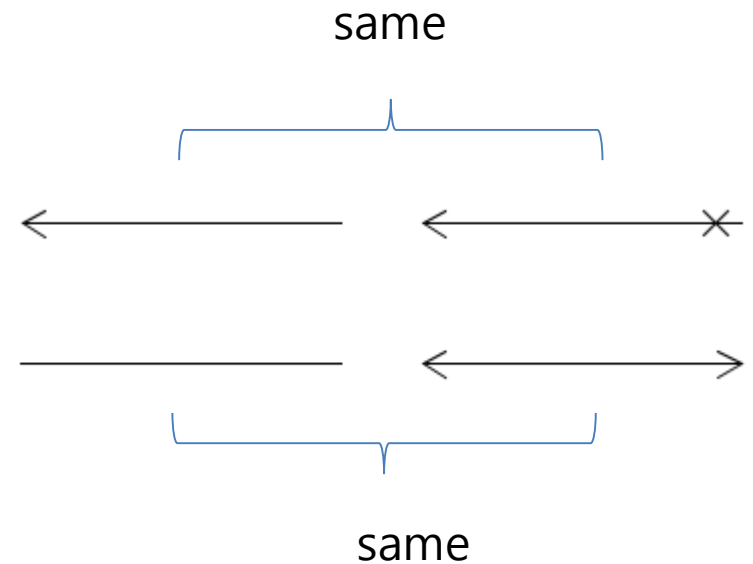
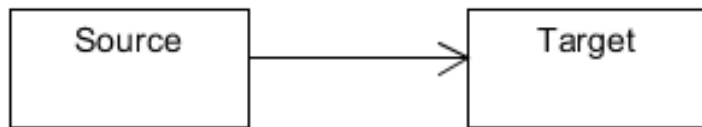
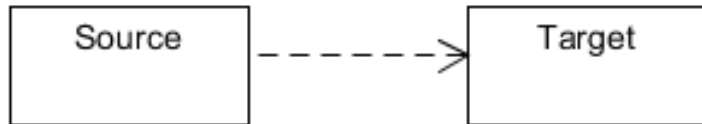
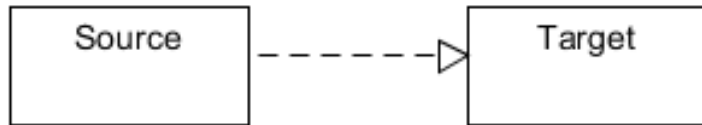
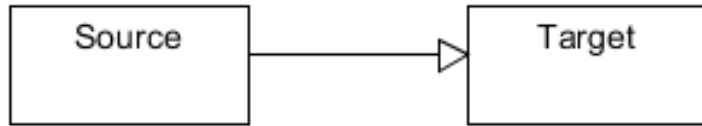


weaker relationship



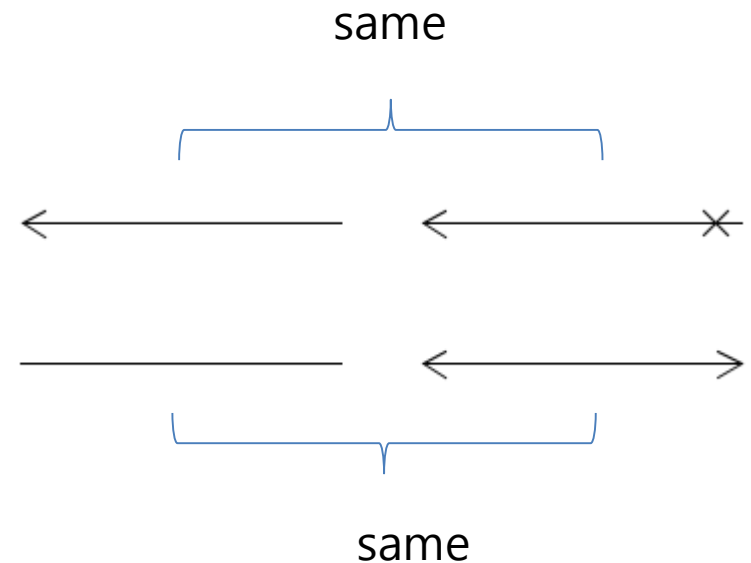
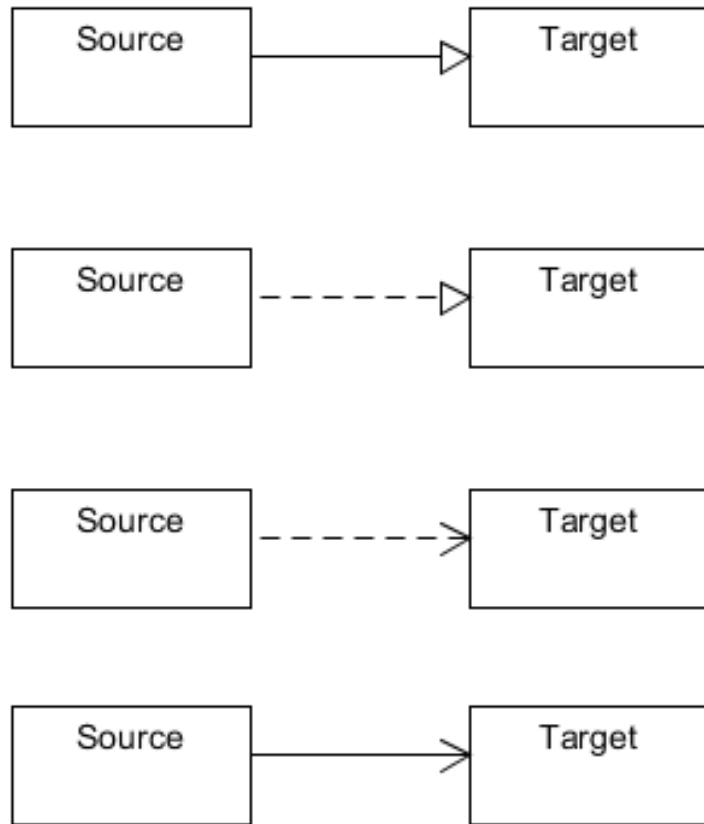
stronger relationship

Class Relationships



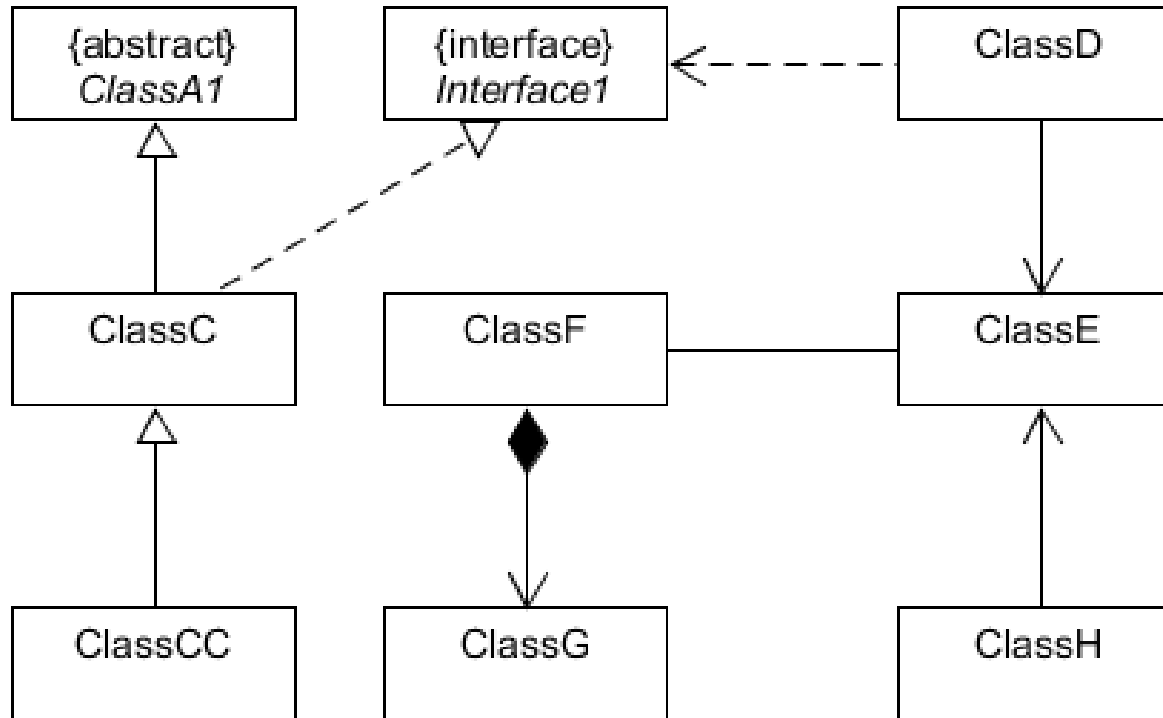
If Target changes, then Source may be affected.
Target is ignorant of Source.

Class Relationships and Change Propagations



If Target changes, then Source may be affected.
Target is ignorant of Source, hence the change of Source is not propagated.

Class Change Propagations



(start) ClassA1 -> ClassC -> ClassCC (end)

(start) ClassD (end)

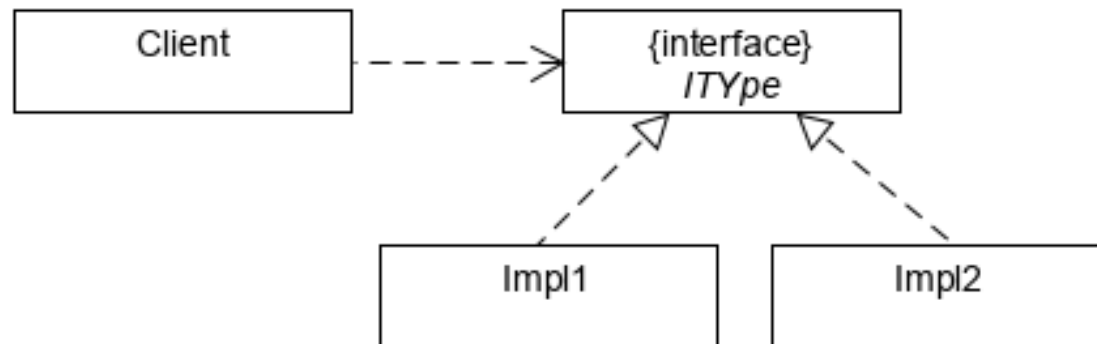
(start) Interface1 -> ClassC, ClassD -> ClassCC

(start) ClassG -> ClassF -> ClassE -> ClassD, ClassH

(start) ClassE -> ClassD, ClassH, ClassF (end)

Information Hiding

- Design principles for **hiding internal design decisions**
 - The internal design decisions are most likely to change
 - Thus, **reduce the side effects** of any future maintenance or modification of the design and hence minimizing the effect on the other modules in the design



Summary

- Object-Oriented Paradigm
 - class, inheritance, polymorphism
- Polymorphism
 - method overloading
 - method overriding
- Class Relationships and Change propagations

OOP Review Quiz : Problem

```
interface IType {}  
abstract class Type1 implements IType {}  
class Type2 implements IType {}  
class Type3 extends Type1 {}  
class Type4 extends Type2 {}
```

다음 중 에러 없이 컴파일 되는 코드는?

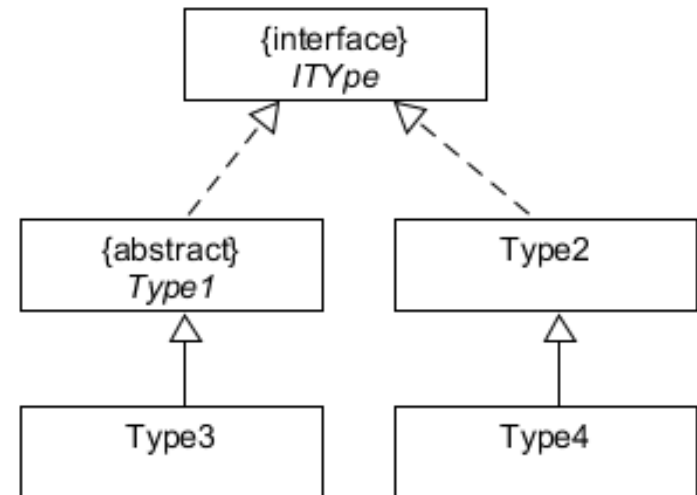
- (a) Type1 a1 = new Type1();
- (b) IType a2 = new Type2();
- (c) Type1 a3 = new Type2();
- (d) IType a4 = new Type4();
- (e) Type2 a5 = new IType();
- (f) Type1 a6 = new Type3();
- (g) Type3 a7 = new Type4();

OOP Review Quiz : Answer

```
interface IType {}  
abstract class Type1 implements IType {}  
class Type2 implements IType {}  
class Type3 extends Type1 {}  
class Type4 extends Type2 {}
```

다음 중 에러 없이 컴파일 되는 코드는?

- (a) Type1 a1 = new Type1();
- (b) IType a2 = new Type2();
- (c) Type1 a3 = new Type2();
- (d) IType a4 = new Type4();
- (e) Type2 a5 = new IType();
- (f) Type1 a6 = new Type3();
- (g) Type3 a7 = new Type4();



3. SOLID PRINCIPLES

2020. 04. 16

중앙대학교 소프트웨어학부

이찬근 교수

Contents

- Hierarchy of Pattern Knowledge
- OO Principles
- Dependency management
- RC Martin's Software Design Principles (SOLID)

Hierarchy of Pattern Knowledge

Design Pattern

Strategy pattern
defines a family of algorithms,
encapsulates each one, and makes
them interchangeable. Strategy lets
the algorithm vary independently
from clients using it

OO Principles

Encapsulate what varies
Favor composition over inheritance
Program to interface, not implementations

OO Basics

Abstraction
Encapsulation
Polymorphism
Inheritance

Design Smells

Design smells = various signs and symptoms of bad design

Name	Symptoms
Rigidity(경직성)	The system is hard to change, because every time you change one thing, you have to change something else in a never ending succession of changes.
Fragility(취약성)	A change to one part of the system causes it to break in many other, completely unrelated parts.
Immobility(부동성)	It is hard to disentangle the system into components that can be reused in other systems.
Viscosity(점착성)	If it is easier to add a hack than it is to add code that fits into the design, then the system has high viscosity.
Needless Complexity (불필요한 복잡성)	There are lots of very clever code structures that aren't actually necessary right now, but could be very useful one day.
Needless Repetition (불필요한 반복)	The code looks like it was written by two programmers named Cut and Paste.
Opacity(불투명성)	Elucidation of the originator's intent presents certain difficulties related to convolution of expression

Dependency Management

- Design smells are resulted from **mismanaged dependencies**
- Mismanaged dependencies
→ **Tangled mass of couplings (spaghetti code)**
- OO languages provide tools helping managing dependencies
 - Interfaces: break or invert the direction of certain dependencies
 - Polymorphism: allows modules to invoke methods dynamically
 - Lots of power to shape the dependencies, indeed
- **So, how do we want them shaped?**

Object-Oriented Design Principles

- SOLID principles by R.C. Martin
- GRASP principles by C. Larman
- Program to interfaces, not implementations
- Favor object composition over class inheritance
- Encapsulate what varies
- ...

R.C. Martin's Software design principles (SOLID)

- The **S**ingle-Responsibility Principle (SR.P)
- The **O**pen-Closed Principle (OCP)
- The **L**iskov Substitution Principle (LSP)
- The **I**nterface Segregation Principle (ISP)
- The **D**ependency Inversion Principle (DIP)



SINGLE RESPONSIBILITY PRINCIPLE

Just Because You Can, Doesn't Mean You Should

Single Responsibility Principle (SR.P)

A class should have one, and only one, reason to change

The Single Responsibility Principle

- Responsibility
 - “a contract or obligation of a class”
 - reason to change
 - More responsibilities == More likelihood of change
 - The more a class changes, the more likely we will introduce bugs
 - Changes can impact the others
- SR.P: Separate coupled responsibilities into separate classes
 - Related measure – Cohesion: how strongly-related and focused are the various responsibilities of a module

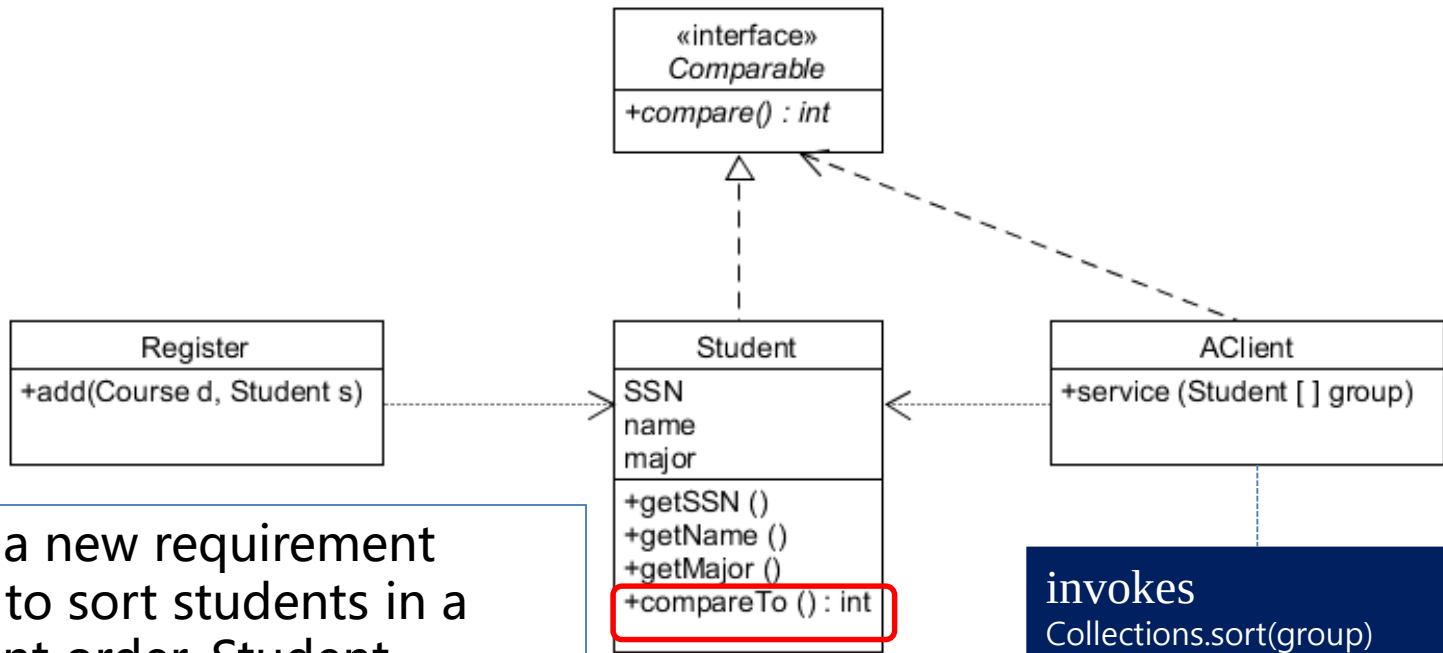
Example of SR.P violation

- Often we need to sort students by their name, or SSN. So one may make Class Student implement the Java Comparable interface.

```
class Student implements Comparable {  
    ...  
    int compareTo(Object o) { ... }  
    ...  
}
```

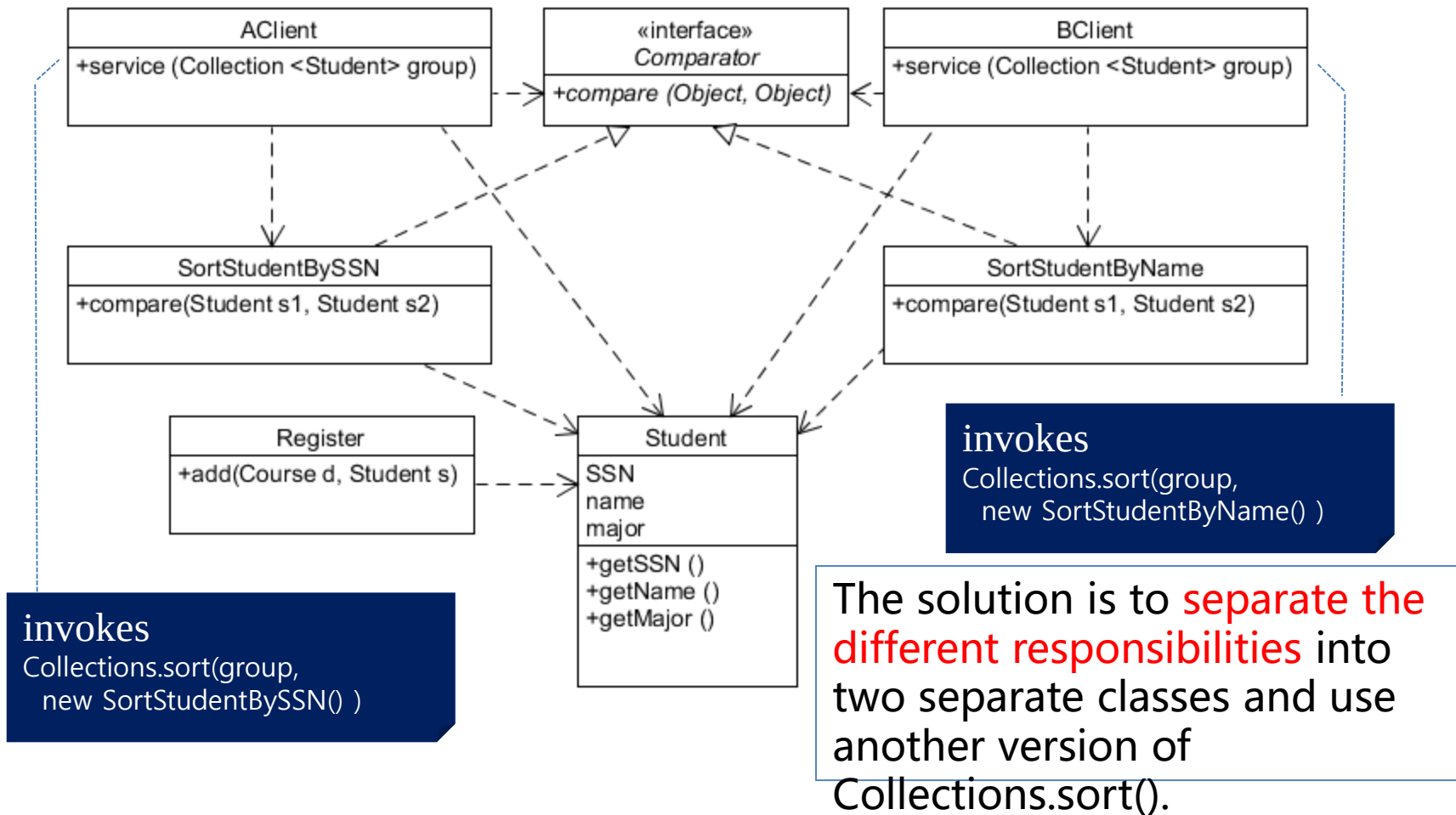
- Student is a business entity, it does not know in what order it should be sorted since the order of sorting is imposed by the client of Student.
- Worse: every time students need to be ordered differently, we have to recompile Student and all its client.
- Cause of the problems: we bundled two separate responsibilities (i.e., student as a business entity with ordering) into one class – **a violation of SR.P**

Example of SR.P violation: Case 1

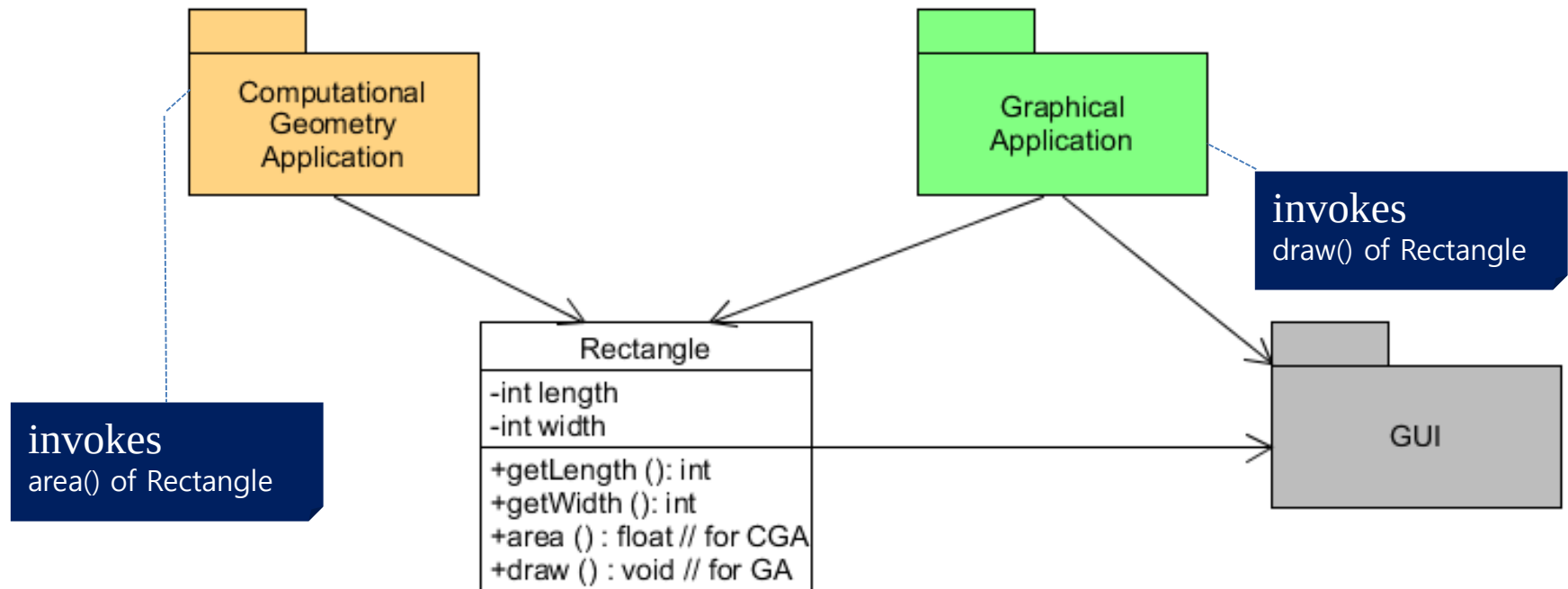


When a new requirement needs to sort students in a different order, Student, Register, and AClient all need to be recompiled, even though Register has nothing to do with any ordering of Students.

Example of design following SR.P

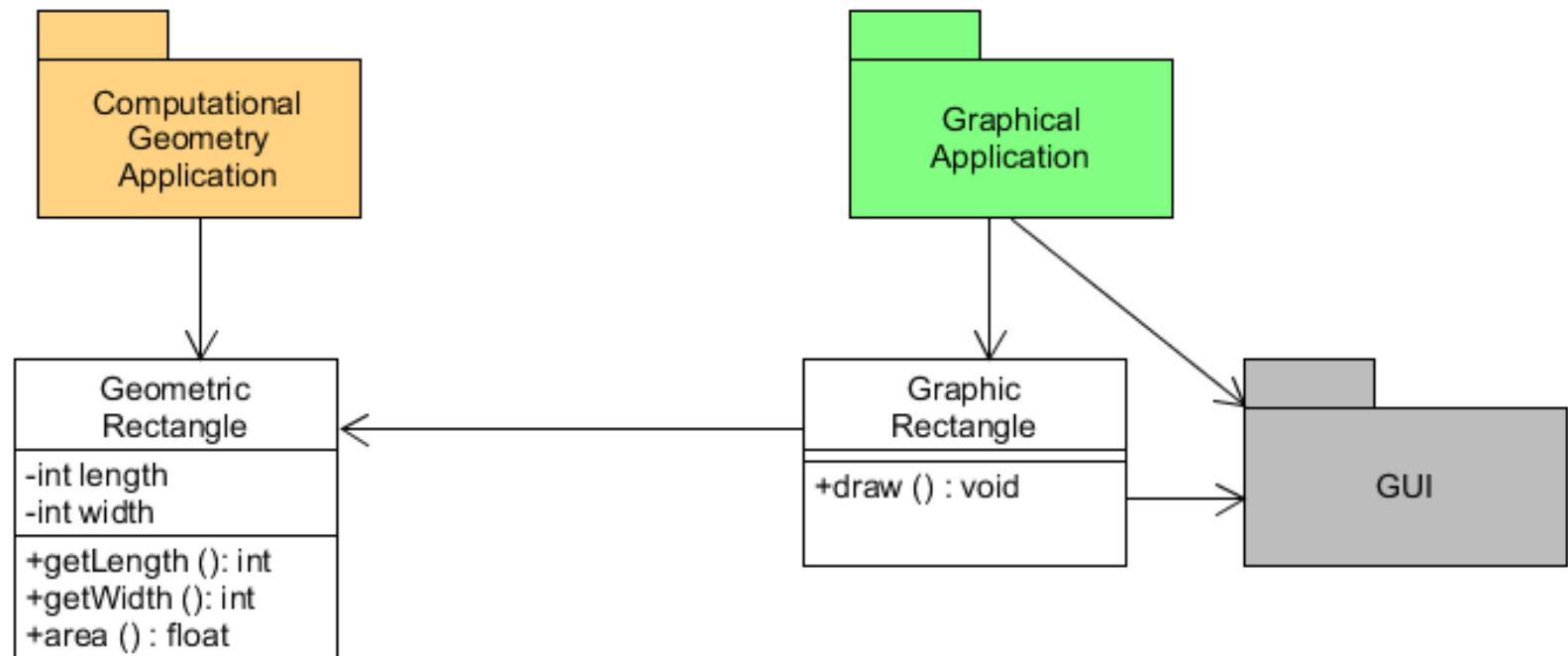


Example of SR.P violation: Case 2



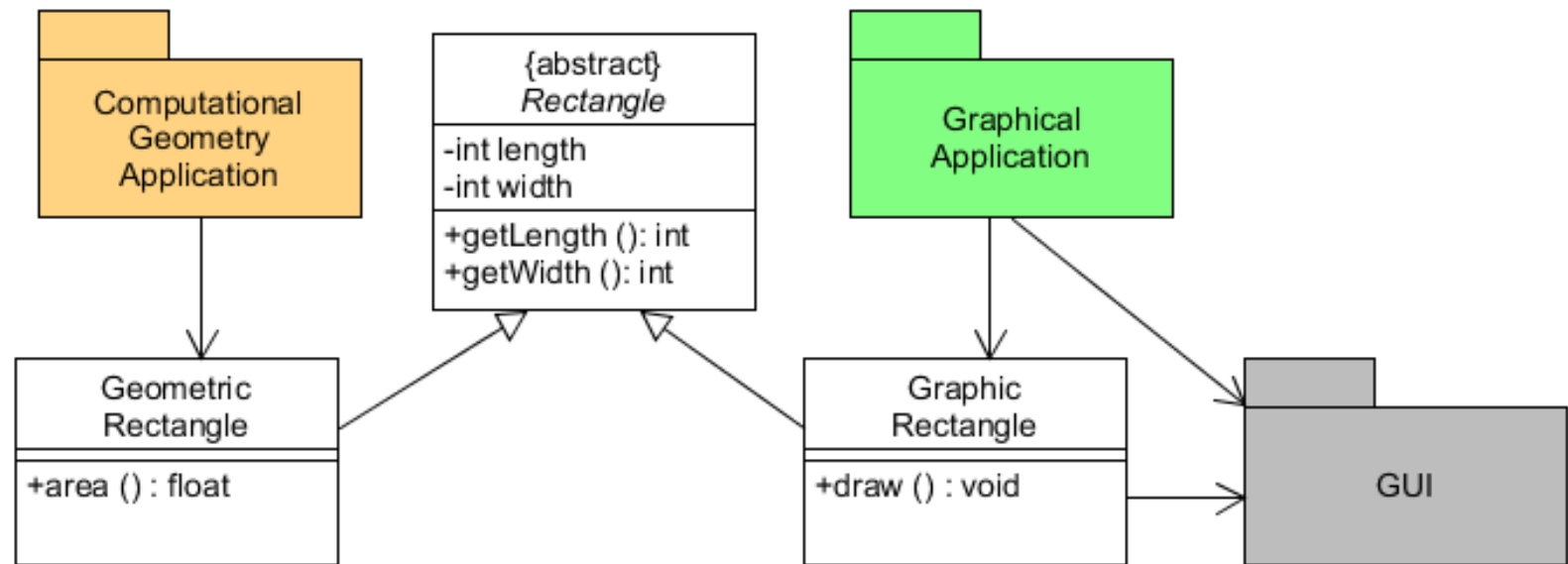
Class **Rectangle** may be forced to make changes from two different unrelated sources. One is from the Computational Geometry Application (CGA). E.g., modifying `area()`. The other is from Graphical Application (GA). E.g., modifying `draw()` for different platforms. A change from either of the two source would still cause the other application to recompile.

Example of design following SR.P (not enough yet)



- Package **CGA** is no longer dependent on graphical side of **Rectangle** and thus it becomes independent of package **GUI**. Any change caused by graphical application no longer requires CGA to be recompiled.
- However, any changes from the **CGA** side may cause **GA** to be recompiled.

Example of design following SR.P



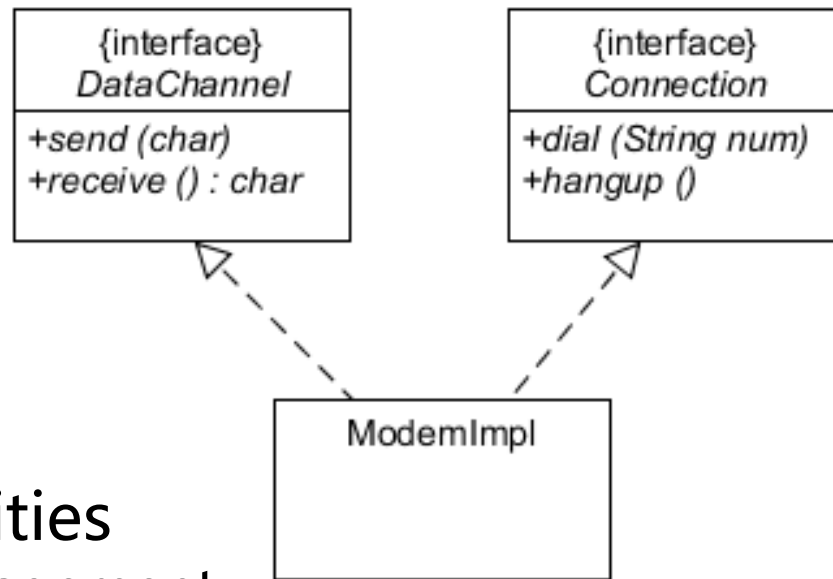
Class **Rectangle** contains the most primitive attributes and operations of rectangles. Classes **GeometricRectangle** and **GraphicRectangle** are independent of each other. A change from either side of **CGA** or **GA**, it would not cause the other side to be recompiled.

Identifying Responsibilities Can be Tricky

- Responsibility (in SR.P)
 - A reason for change
 - Note: sometimes hard to see multiple responsibilities

```
interface Modem
{
    public void dial (String num);
    public void hangup ();
    public void send (char c);
    public char receive ();
}
```

Identifying Responsibilities Can be Tricky



- Two responsibilities
 - Connection management
 - Data communication
- Lesson: It depends on how the application is changing!
- Beware: Avoid Needless Complexity
 - If there is no symptom, it is not wise to apply the SR.P or any other principle!



OPEN CLOSED PRINCIPLE

Open Chest Surgery Is Not Needed When Putting On A Coat

Open Closed principle (OCP)

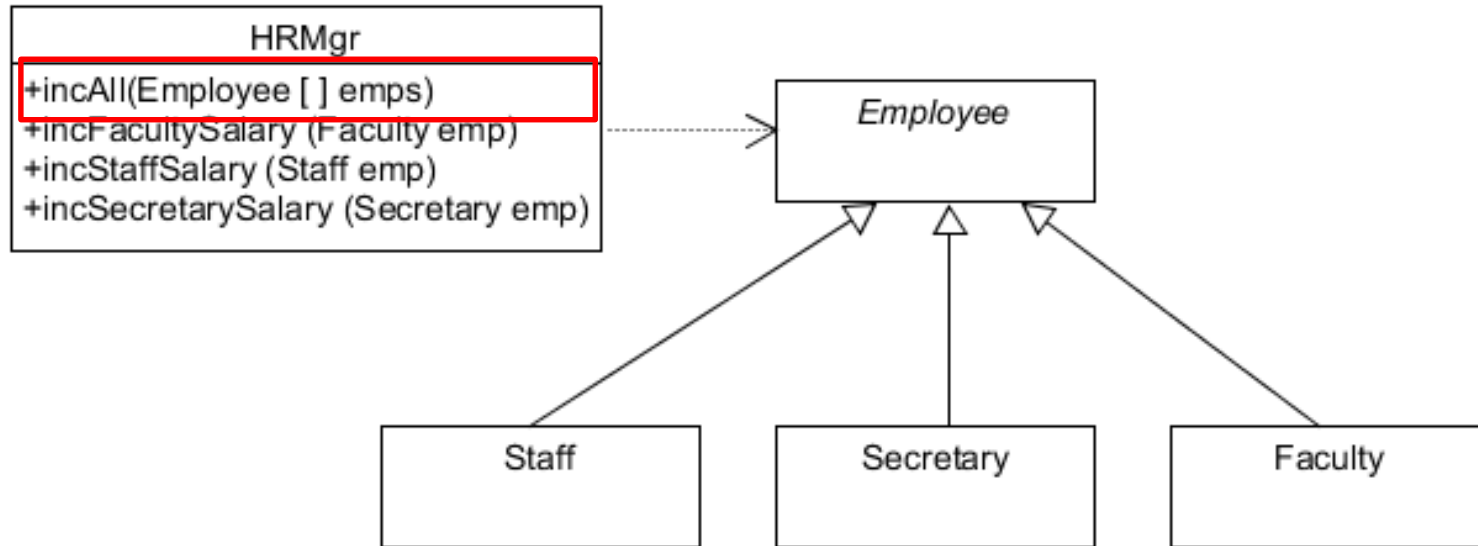
Software entities (classes, modules, functions, etc.) should be open for extension but closed for modification.

You should be able to extend a class's behavior, without modifying it.

Conforming to OCP

- Open for extension
 - Behavior of the module can be extended
 - We are able to change what the module does
- Closed for modification
 - Extending behavior does not result in excessive modification such as architectural changes of the module
- Violation Indicator: Design Smell of Rigidity
 - A single change to a program results in a **cascade of changes to dependent modules**

Bad Design Example



```
void incAll(Employee[] emps) {
    for (int i = 0; i < emps.size(); i++) {
        if (emps[i].empType == FACULTY)
            incFacultySalary((Faculty)emps[i]);
        else if (emps[i].empType == STAFF)
            incStaffSalary((Staff)emps[i]);
        else if (emps[i].empType == SECRETARY)
            incSecretarySalary((Secretary)emps[i]);
    }
}
```


Problems of Bad Design

- Rigid
 - Adding new employee type requires significant changes
- Fragile
 - Many switch/case or if/else statements
 - Hard to find and understand

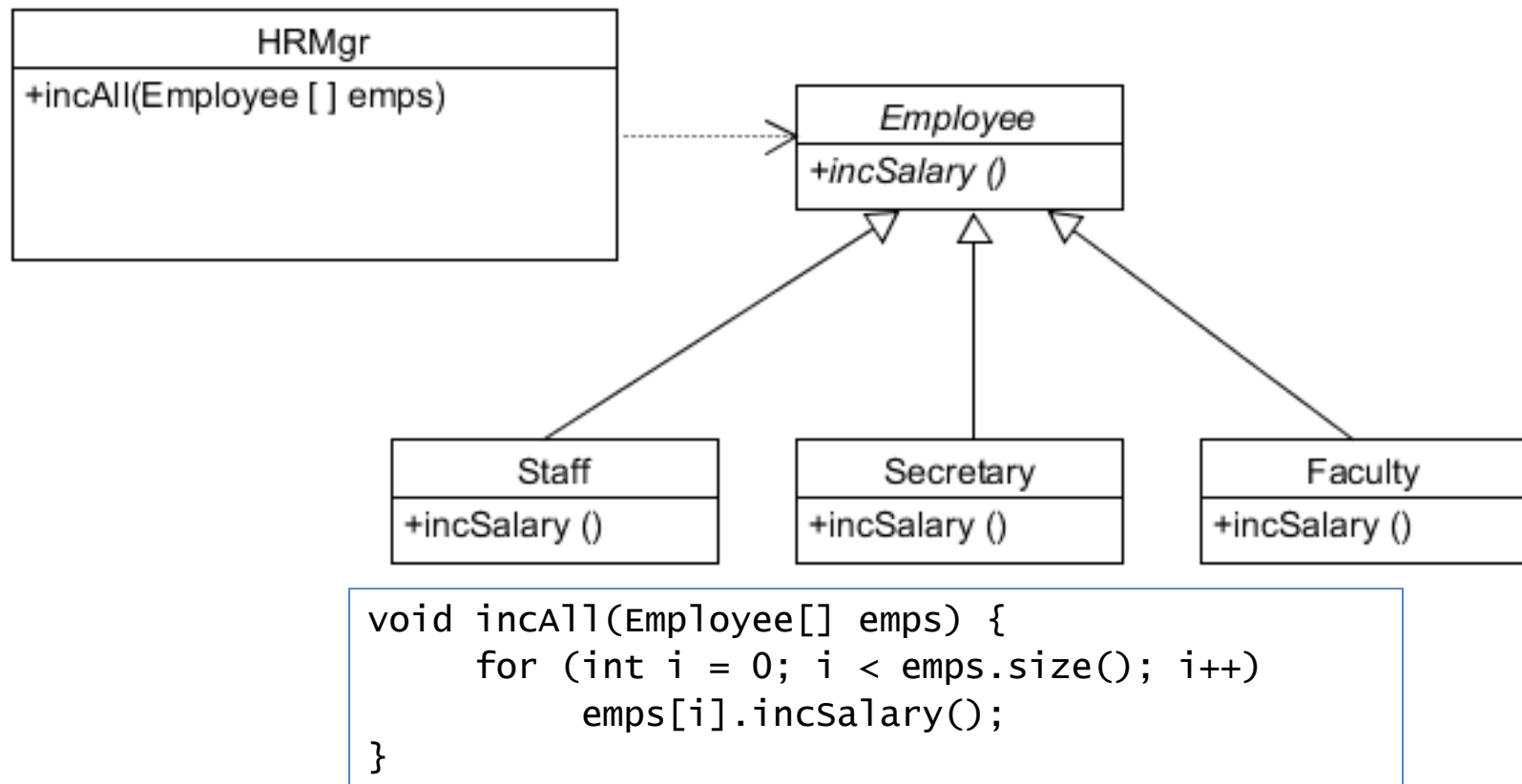
```
void incAll(Employee[] emps) {  
    for (int i = 0; i < emps.size(); i++) {  
        if (emps[i].empType == FACULTY)  
            incFacultySalary((Faculty)emps[i]);  
        else if (emps[i].empType == STAFF)  
            incStaffSalary((Staff)emps[i]);  
        else if (emps[i].empType == SECRETARY)  
            incSecretarySalary((Secretary)emps[i]);  
        else if (emps[i].empType == ENGINEER)  
            incEngineersSalary((Engineer)emps[i]);  
    }  
}  
void incEngineersSalary (Engineer e) { ... }
```

Problems of Bad Design

- Rigid
 - Adding new employee type requires significant changes
- Fragile
 - Many switch/case or if/else statements
 - Hard to find and understand
- Immobile
 - To reuse incAll() → we need Faculty, Staff, Secretary, too!
 - What if we need just Faculty and Staff only?

```
void incAll(Employee[] emps) {  
    for (int i = 0; i < emps.size(); i++) {  
        if (emps[i].empType == FACULTY)  
            incFacultySalary((Faculty)emps[i]);  
        else if (emps[i].empType == STAFF)  
            incStaffSalary((Staff)emps[i]);  
        else if (emps[i].empType == SECRETARY)  
            incSecretarySalary((Secretary)emps[i]);  
    }  
}
```

Better Design

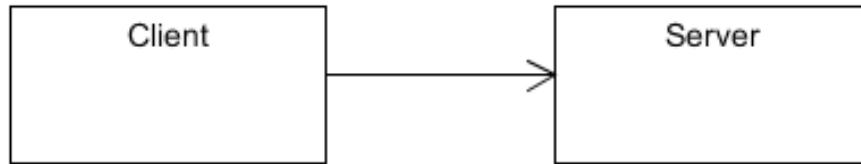


- When Engineer is added, `incAll()` does not even need to recompile.
- This design is open to extension, closed for modification.

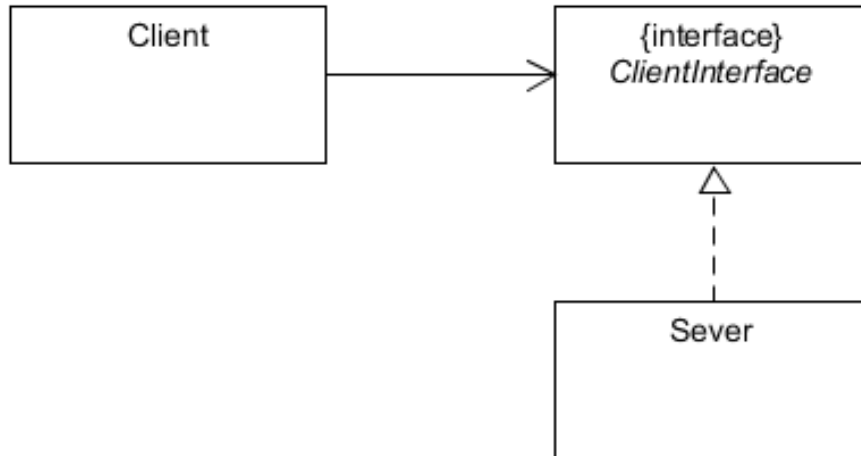
Abstraction is the Key!

- Abstractions
 - Fixed and yet represent an unbounded group of possible behaviors
 - Abstract base class : fixed
 - All the possible derived classes : unbounded group of possible behaviors
- Program the class
 - to interfaces (or abstract classes)
 - not to implementation (concrete classes)

Abstraction is the Key!



What if we need to use other servers ?



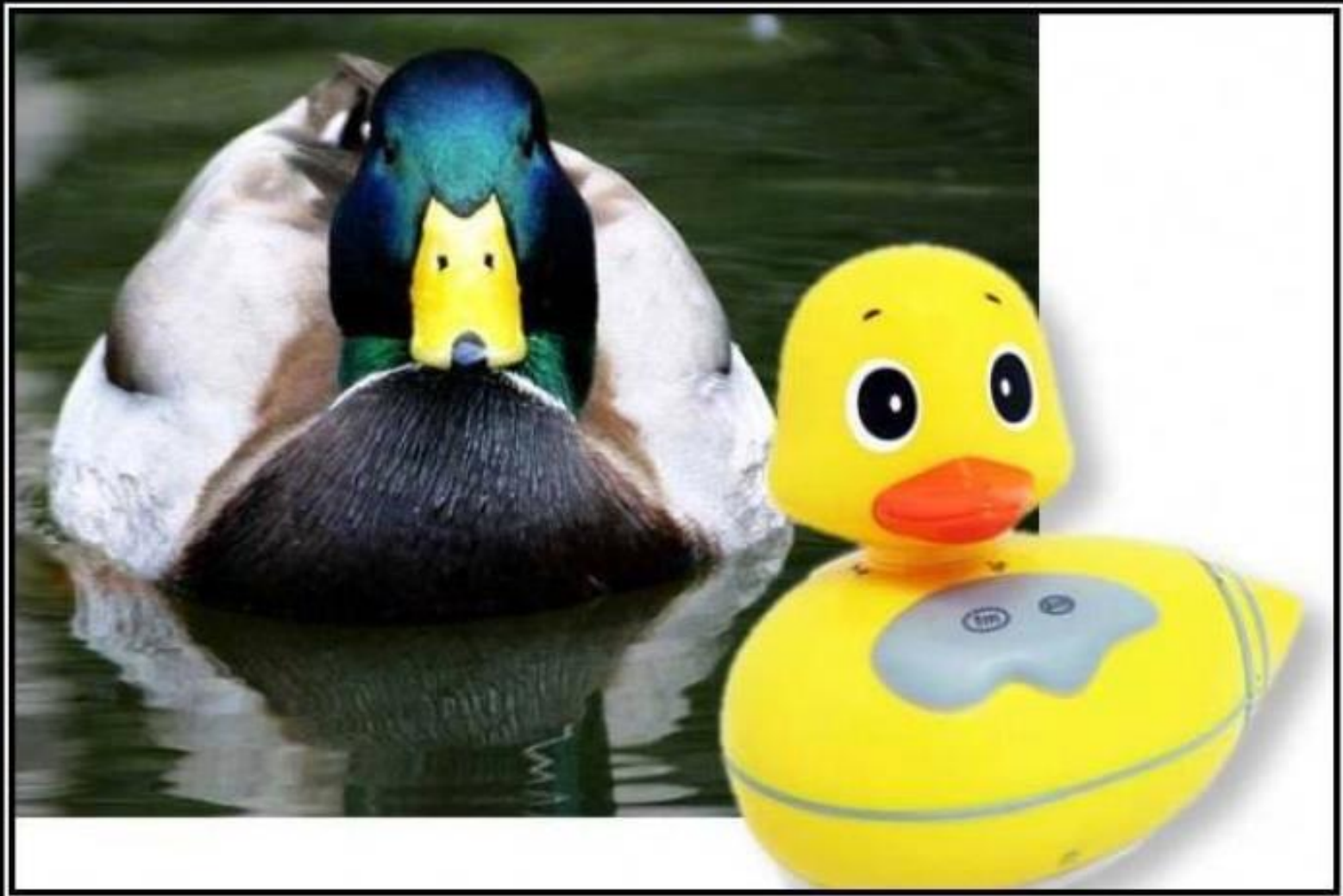
Abstract classes (or interfaces) are more closely associated to their clients than to the classes that implement them

Anticipating Future Changes

- Strategy is needed
 - Choose the kinds of changes against which to close design
 - Guess the most likely kinds of changes, and then construct abstractions to protect him from those changes.
- Beware: Consider the cost!
 - Conforming to OCP is expensive
 - Time and effort to create appropriate abstractions
 - Abstractions also increase complexity

Anticipating Future Changes

- Do not put hooks in for changes that might happen
 - "Fool me once, shame on you. Fool me twice, shame on me."
 - Initially write the code expecting it to not change.
 - When a change occurs, implement the abstractions that protect from future changes of that kind.
 - It's better to take the first hit as early as possible.
 - We want to know what kind of changes are likely before going too far in the development.
 - Use TDD and listen to the tests.



LISKOV SUBSTITUTION PRINCIPLE

If It Looks Like A Duck, Quacks Like A Duck, But Needs Batteries - You Probably Have The Wrong Abstraction

Liskov Substitution Principle (LSP)

***Subtypes must be substitutable
for their base types***

Derived classes must be substitutable for their
base classes

Liskov Substitution Principle

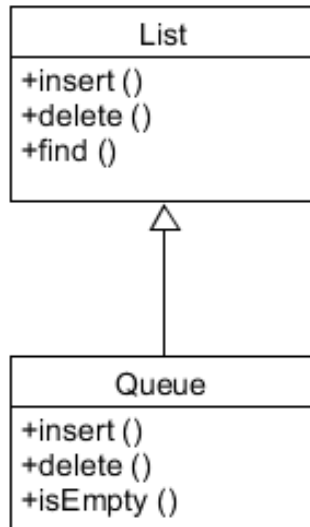
- A rule that you want to **check when you decide to use inheritance or not**
- If **C is a subtype of P** , then **objects of type P may be replaced with objects of type C without altering** any of the desirable properties of the program

Subtyping VS Implementation Inheritance

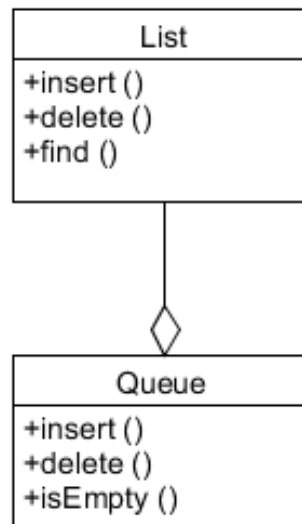
- Subtyping
 - establishes an IS_A relationship
 - also known as *interface inheritance*
- Implementation Inheritance
 - only reuses implementation and establishes a syntactic relationship not necessarily a semantic relationship
 - Also known as *code inheritance*
- Most OOP languages like Java, C++, and C#, inheritance keyword such as “extends” does the both Subtyping and Implementation Inheritance
 - But some languages distinguish them

The Liskov Substitution Principle and Reuse

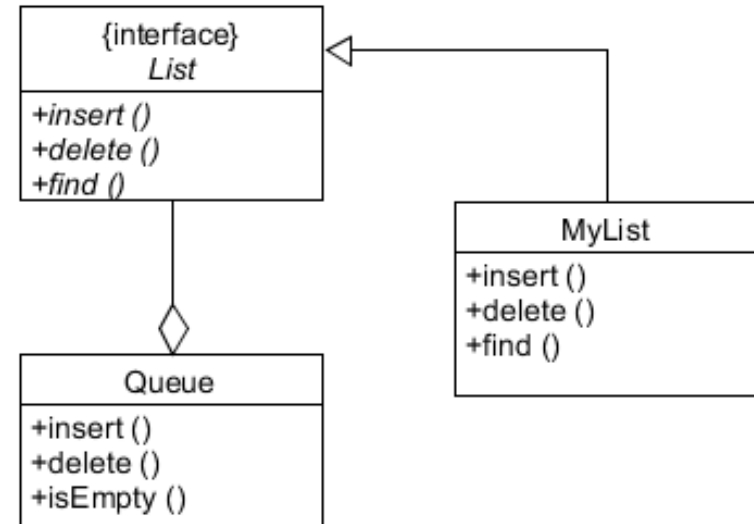
- Think twice when you decide to use Inheritance!
 - If you want to reuse implementation of List, you had better exploit **object composition**, **not inheritance**.
 - If you inherit Queue from List, then you violate LSP since Queue object cannot be substitutable for List



Violation of LSP!

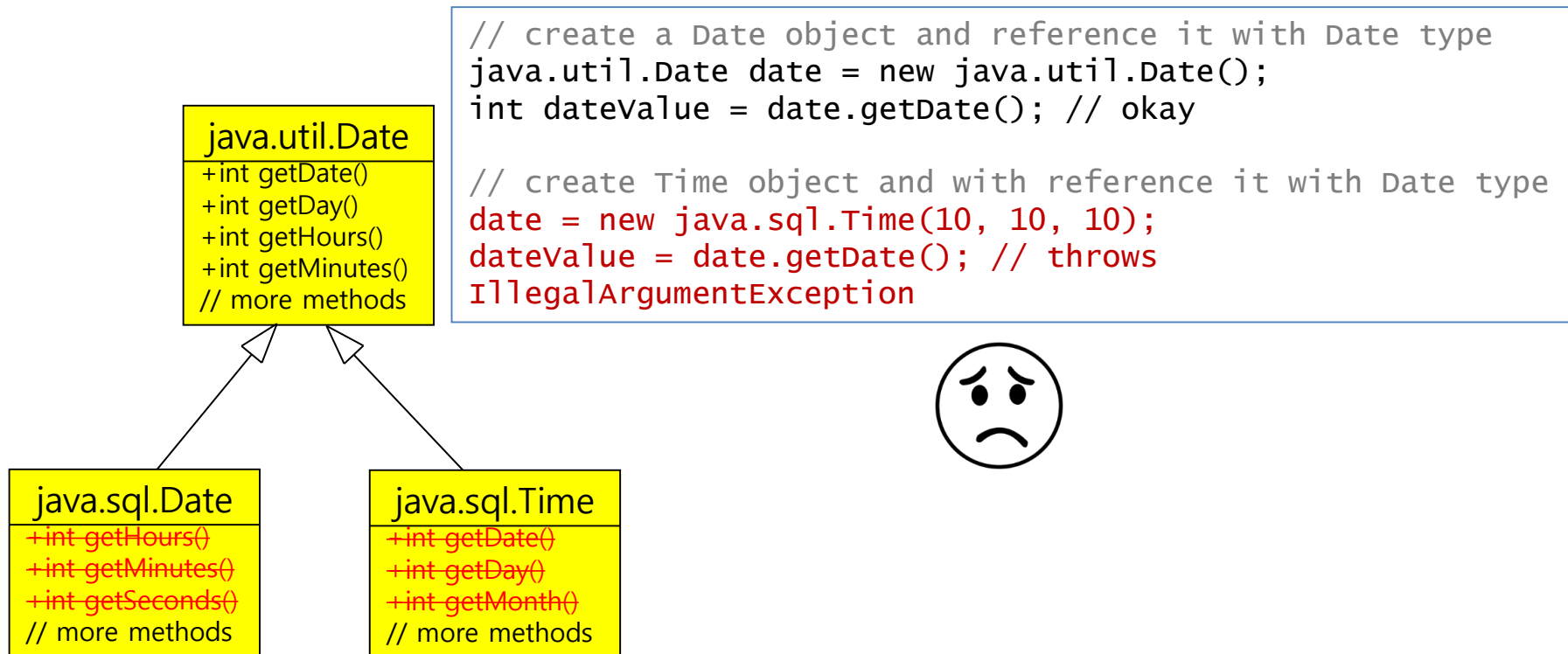


Good



Better!

Violation Example of LSP

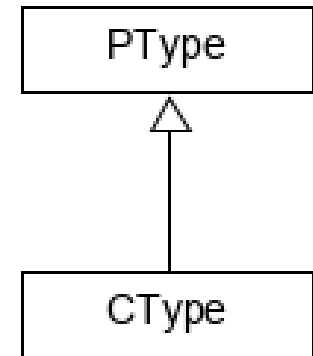


Violation of LSP may lead to another violation

- Assume that when *CType* is passed to *f()* instead of *PType*, it causes *f* to misbehave; it means that *CType* violates the LSP

→ *CType* is fragile in the presence of *f*

```
void f(PType x) {  
    // some code misbeavaves when x is CType  
    ...  
}
```



The owner of *f* might want to put test code for *Ctype*

```
void f(PType x) {  
    if (x instanceof( CType)) throw new RuntimeException();  
    // some code misbeavaves when x is CType  
    ...  
}
```

The above reaction is worse. Now, it is violating OCP, too!

→ *f* is **not closed** to all various **subtypes of *PType***



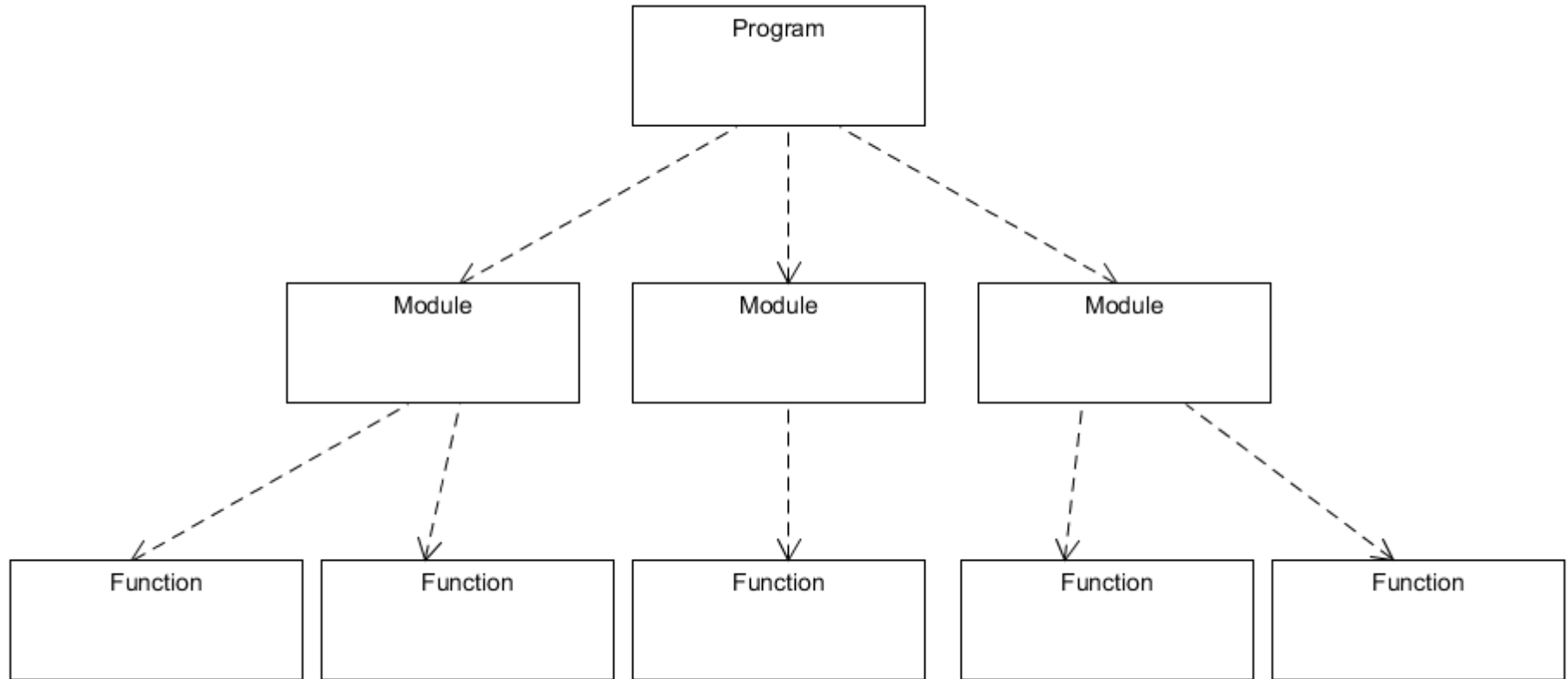
DEPENDENCY INVERSION PRINCIPLE

Would You Solder A Lamp Directly To The Electrical Wiring In A Wall?

Dependency Inversion Principle (DIP)

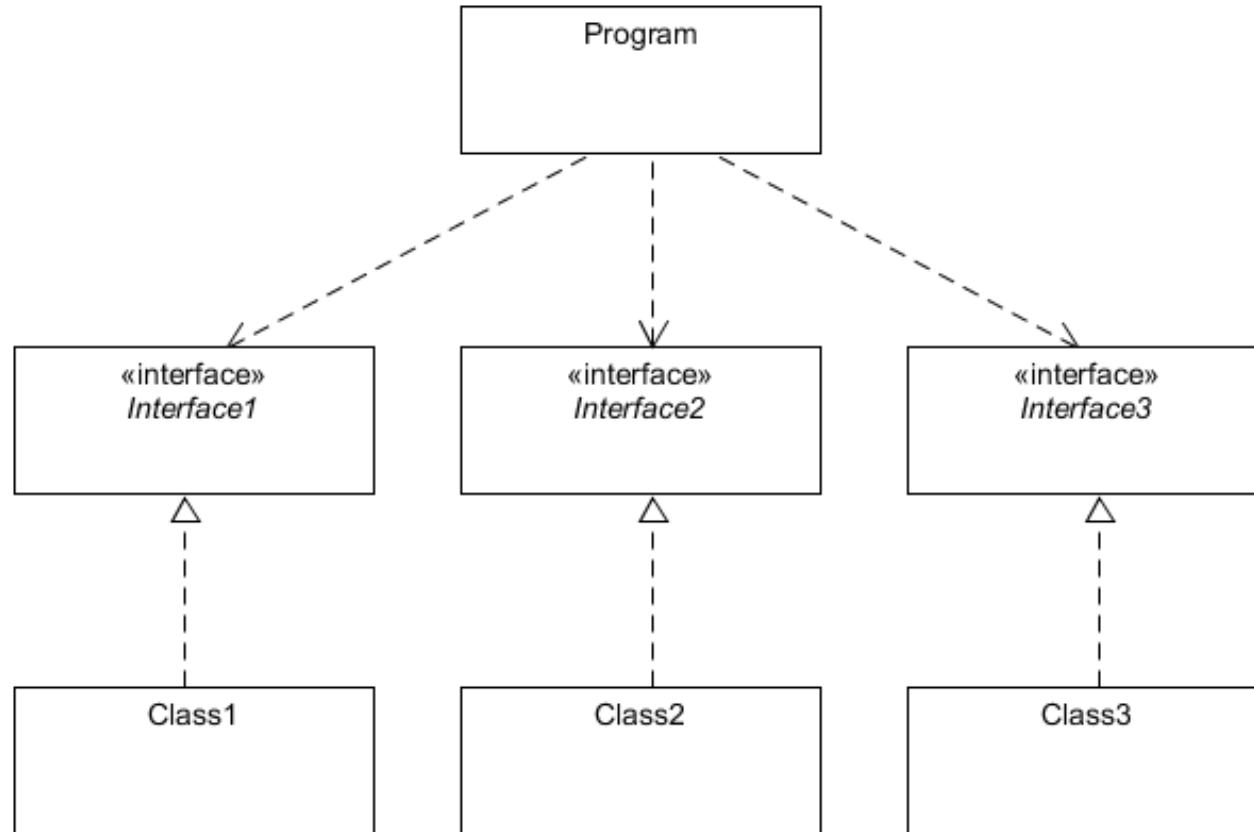
- *High-level modules should not depend on low-level modules. Both should depend on abstractions*
- *Abstractions should not depend on details. Details should depend on abstractions.*
- Why Inversion?
 - DIP attempts to “invert” the dependencies that result from a structured analysis and design approach

Typical in Structured Analysis & Design



Dependency Inversion Principle

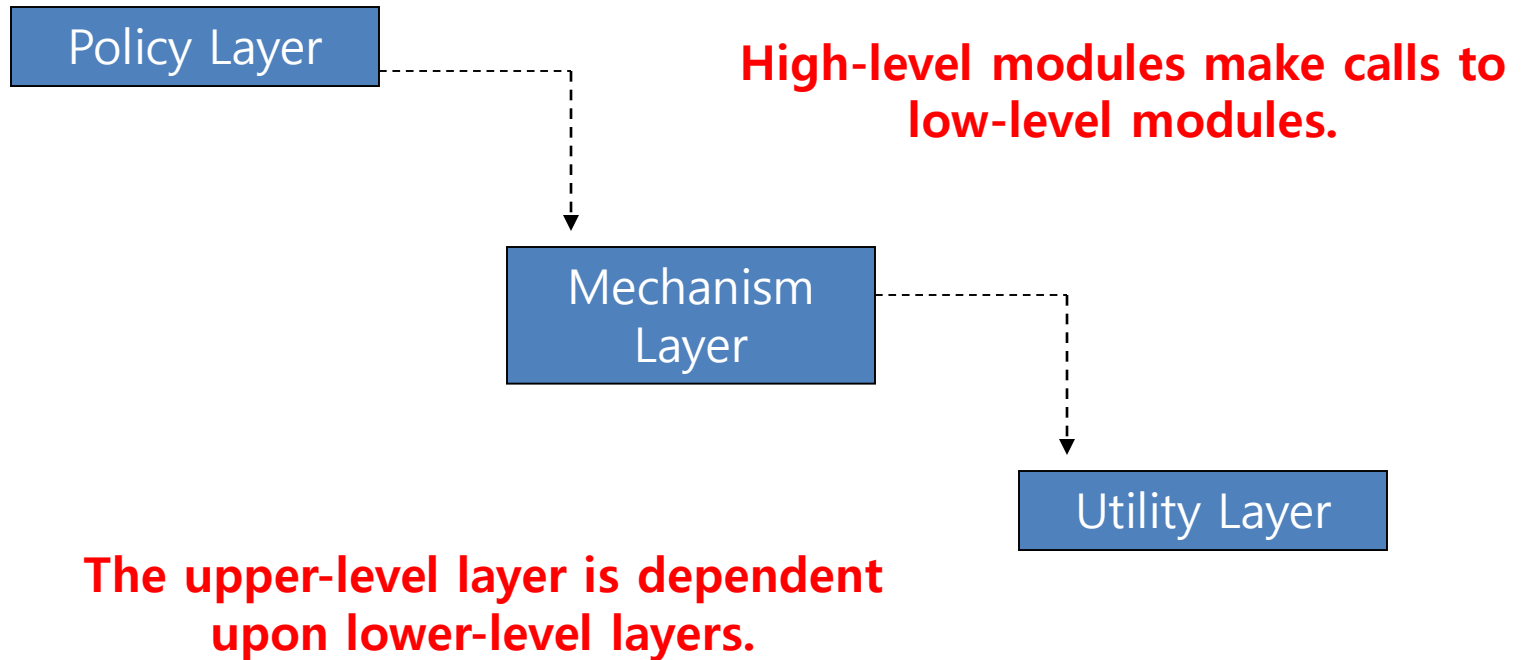
Interfaces and abstract classes are high-level resources
Concrete classes are low-level resources



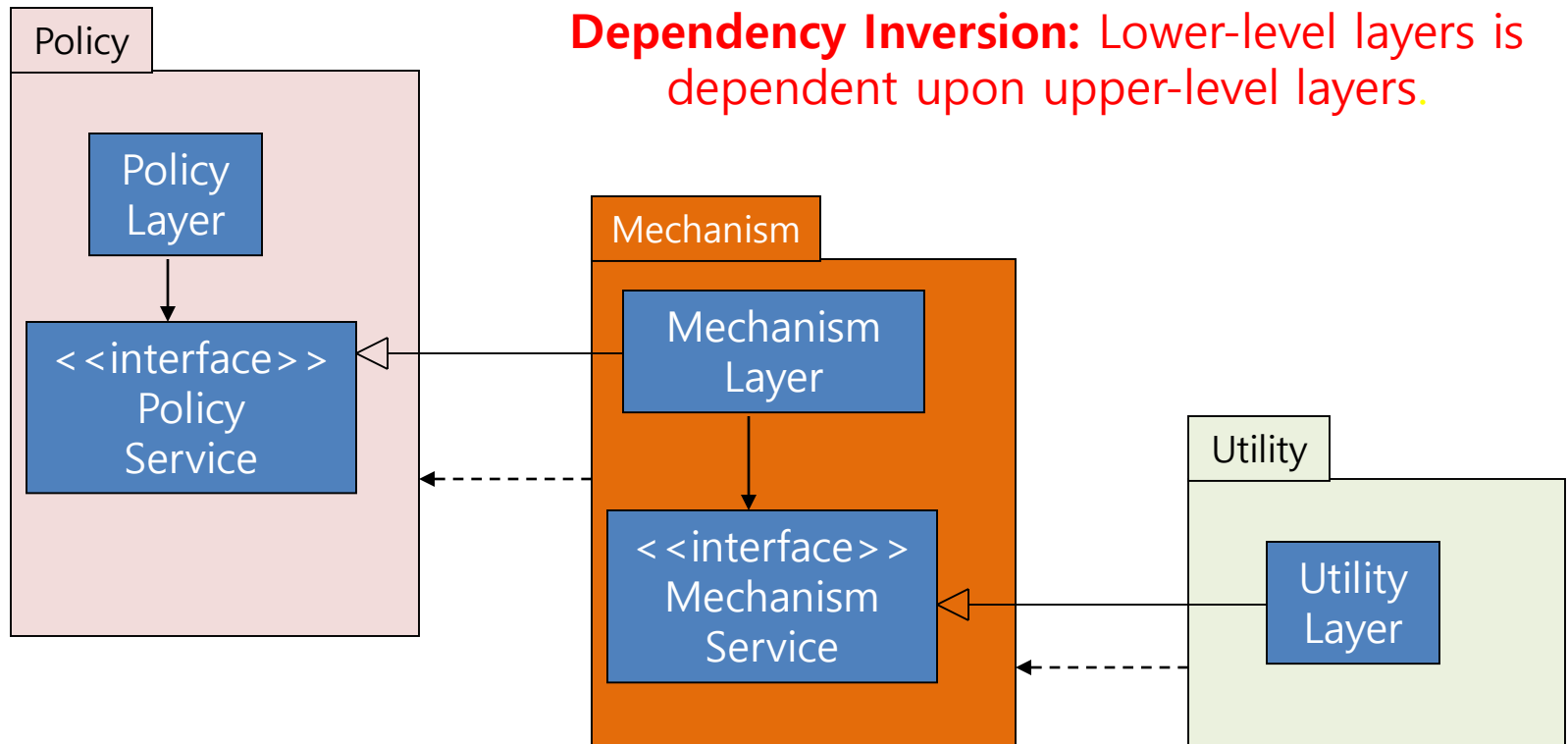
Inversion of Ownership

- Its not just an inversion of **dependency**, DIP also inverts **ownership**
 - Typically a service interface is “owned” or declared by the server, here the client is specifying what they want from the server
 - DIP → Clients should own the interface!

The dependency inversion principle



The dependency inversion principle



The client (upper-level layer) owns the interface, not the lower-level layers



INTERFACE SEGREGATION PRINCIPLE

You Want Me To Plug This In, Where?

Interface Segregation Principle (ISP)

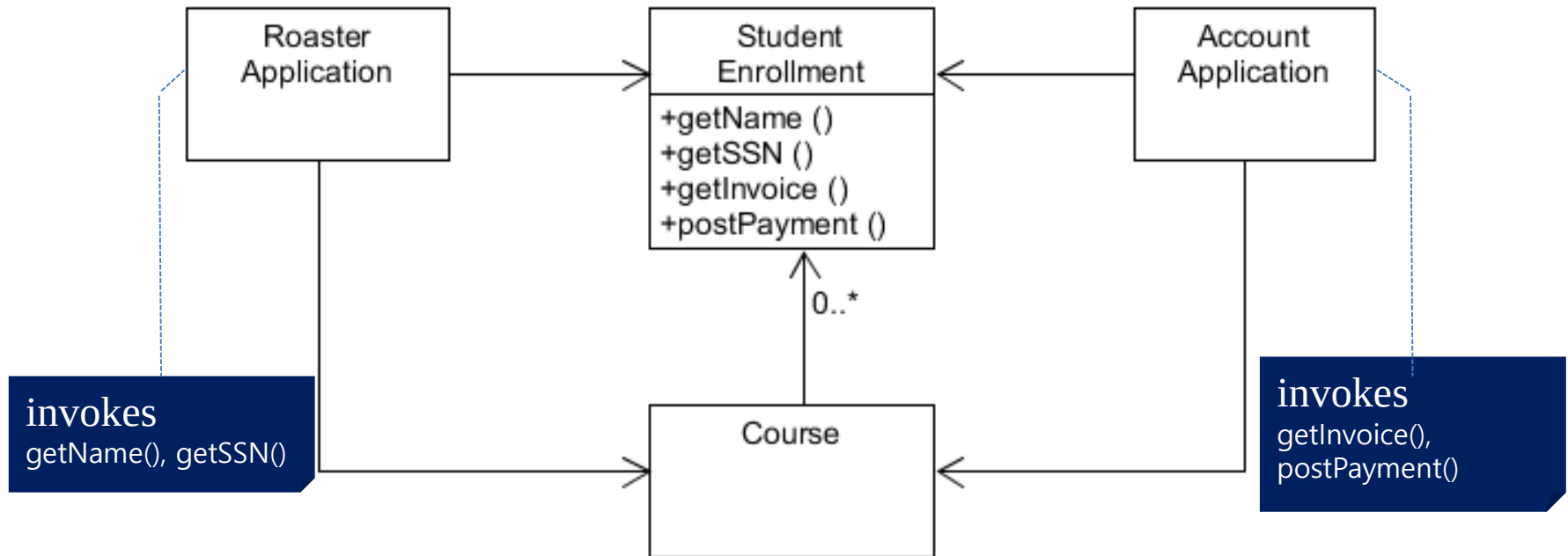
Clients should not be forced to depend on methods they do not use

Make fine grained interfaces that are client specific

Fat interface

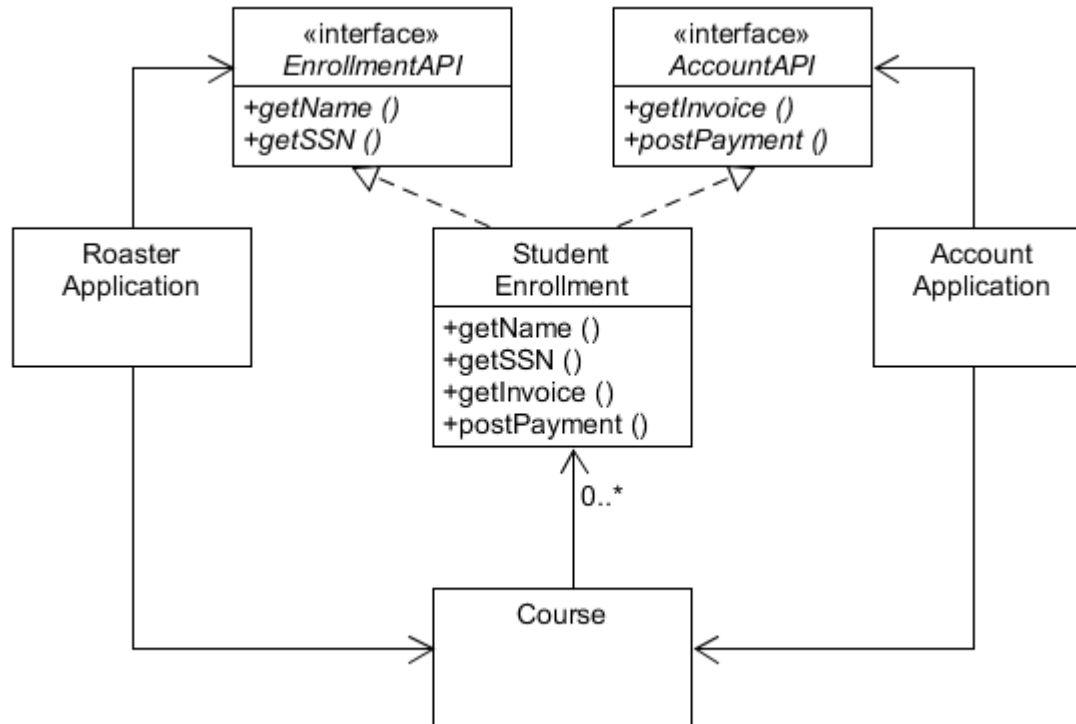
- Bundling functions for different clients into one interface create unnecessary coupling among the clients.
 - When one client causes the interface to change, all other clients are forced to recompile.
 - **Solution: Break the interface into cohesive groups**
- ISP solves non-cohesive interfaces
 - Clients should know only abstract base classes that have cohesive interfaces

ISP Example: Original Design



- Suppose that RoasterApplication does not invoke methods getInvoice() or postPayment()
- Also suppose AccountApplication does not invoke the methods getName() or getSSN()
- **Requirements change: add a new argument to the postPayment()**
- This change force us to **recompile and redeploy RoasterApplication**, which does not care at all about the postPayment()

ISP Example: Better Design



- Now, each user of a **Student Enrollment** object is given an interface that provides just the methods that it is interested in
- This protects the user from changes in methods that don't concern it
- It also protects the user from knowing too much about the implementation of the object it is using.

Summary

- SOLID
 - Single-Responsibility Principle
 - Open-Closed Principle
 - Liskov Substitution Principle
 - Dependency Inversion Principle
 - Interface Segregation Principle
- Design Principles
 - Help manage dependency
 - Better maintainability, flexibility, robustness, and reusability
 - Abstraction is important

SOLID Quiz : Problem

- LSP에 대한 설명으로 옳지 않은 것은?
 1. 클래스 간 상속관계가 올바른지 체크해 볼 수 있는 원칙이다..
 2. 문법적 규칙이기 때문에 위반하면 컴파일러가 에러를 검출한다.
 3. LSP를 위반하는 경우 상속 대신 객체 합성(object composition)을 고려해 볼 수 있다.
 4. 부모 클래스의 인스턴스 대신에 자식 클래스의 인스턴스가 쓰일 수 있어야 한다는 의미이다.

다음 각 문장이 옳은지 O/X 로 답하시오

- 모듈 설계에 OCP를 적용하면 일반적으로 수행 속도가 향상된다 (O / X)
- OCP를 위반하면 다른 설계원칙이 또한 위반되는 경우가 있다 (O / X)

SOLID Quiz : Answer

- LSP에 대한 설명으로 옳지 않은 것은?
 1. 클래스 간 상속관계가 올바른지 체크해 볼 수 있는 원칙이다..
 2. 문법적 규칙이기 때문에 위반하면 컴파일러가 에러를 검출한다.
 3. LSP를 위반하는 경우 상속 대신 객체 합성(object composition)을 고려해 볼 수 있다.
 4. 부모 클래스의 인스턴스 대신에 자식 클래스의 인스턴스가 쓰일 수 있어야 한다는 의미이다.

다음 각 문장이 옳은지 O/X 로 답하시오

- 모듈 설계에 OCP를 적용하면 일반적으로 수행 속도가 향상된다 (O / X)
- OCP를 위반하면 다른 설계원칙이 또한 위반되는 경우가 있다 (O / X)

4. GENERAL RESPONSIBILITY ASSIGNMENT SOFTWARE PATTERNS (GRASP)

2020. 04. 16

중앙대학교 소프트웨어학부

이찬근 교수

Contents

- Intro. to GRASP Principles
- Modularity: Coupling and Cohesion
- Case Study: POS System and its Domain Model
- Applying GRASP Principles

GRASP

- General Responsibility Assignment Software Patterns

- Craig Larman's 9 principles

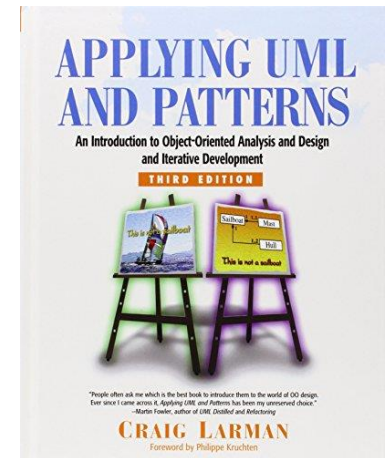
- Typical tasks in constructing a software with OOAD

- Identify the requirements
- Create a domain model
- Create design model
 - Add attributes and methods to the software classes
 - Define the interactions between the objects

- Okay, I got it. But, how ???

- GRASP

- will guide you how to assign responsibilities to collaborating objects

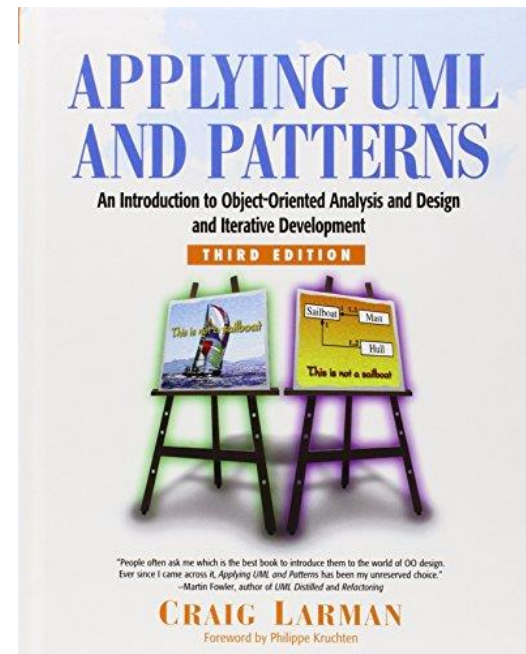


Responsibilities

- Responsibility = “a contract or obligation of a class”
- **Knowing** responsibilities:
 - knowing about private encapsulated data
 - knowing about related objects
 - knowing about things it can derive or calculate
 - Ex) "a *Sale* is responsible for *knowing its total*"
- **Doing** responsibilities:
 - doing something itself
 - initiating action in other objects
 - controlling and coordinating activities in other objects
 - Ex) "a *Sale* is responsible for *creating SalesLineItems*"
- A responsibility is not the same as a method, but **methods are implemented to fulfil responsibilities**.

GRASP: General Principles in Assigning Responsibilities

- From Craig Larman's 9 principles:
 - Information Expert
 - Creator
 - Controller
 - Low Coupling
 - High Cohesion
 - Polymorphism
 - Pure Fabrication
 - Indirection
 - Protected variations



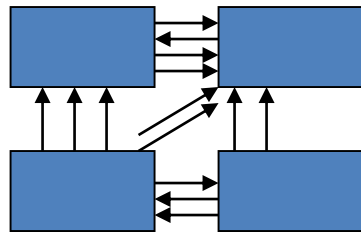
Modularity

- Goal of design
 - partition the system into modules and assign responsibility among the components
- Modularity reduces the total complexity a programmer has to deal with at any one time:
 - functions are assigned to modules in a way that groups similar functions together (**Separation of Concerns**), and
 - there are small, simple, well-defined interfaces between modules (**Information Hiding**)
- Measuring Modularity
 - Coupling: measure of the **inter-dependencies** between modules
 - Cohesion: measure of how strongly the elements in a module are **related**

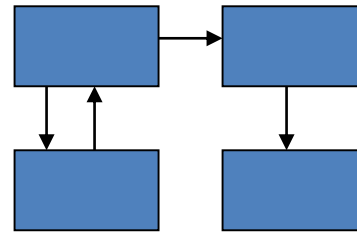
Concept of Coupling

■ Coupling

- measure of the inter-dependencies between classes (or modules)
- a class with high (or strong) coupling relies on many other classes.
- when the independent element changes, it affects the dependent one.



Highly coupled



Loosely coupled

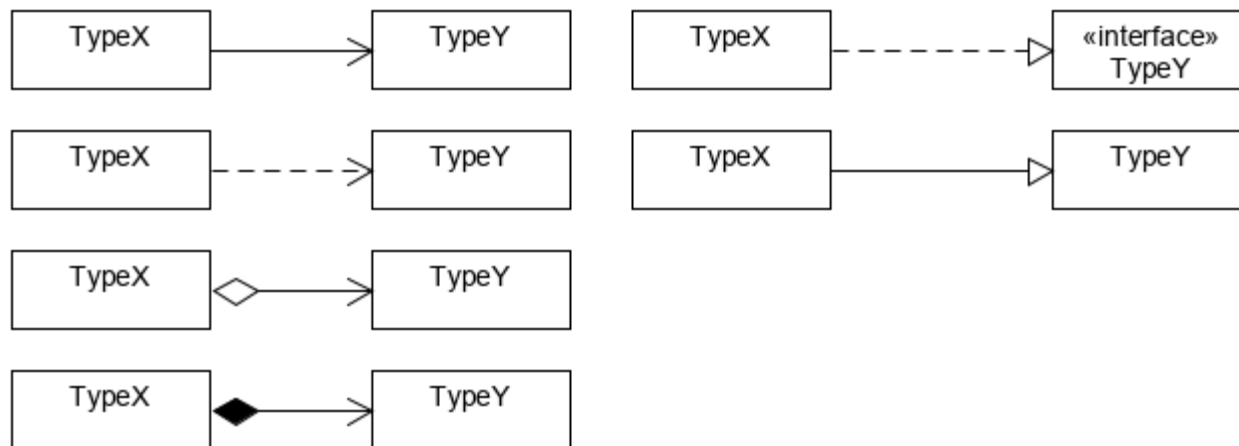
Pursue Low (or loose) Coupling!

- Disadvantages of High Coupling:
 - Forced changes due to changes in depending classes
 - Harder to understand in isolation
 - Harder to reuse

Common forms of coupling in OO languages

TypeX depends on TypeY if

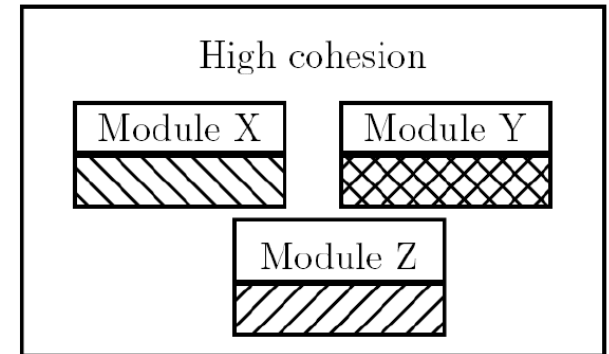
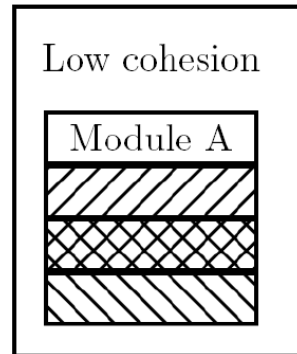
- TypeX has an **attribute referencing** a TypeY instance, or TypeY itself.
- TypeX has a **method referencing** a TypeY instance, or TypeY itself, by any means. (e.g. a parameter or local variable of type TypeY, or the returned object is an instance of TypeY, etc)
- TypeX is a direct or indirect **subclass** of TypeY.
- TypeX **implements** interface TypeY.



Concept of Cohesion

■ Cohesion

- measure of how strongly the elements in a class (module) are related
- the **related responsibilities** are placed into one manageable unit.
- related to Single Responsibility Principle (SR.P)



Pursue High Cohesion!

■ Advantages of High Cohesion

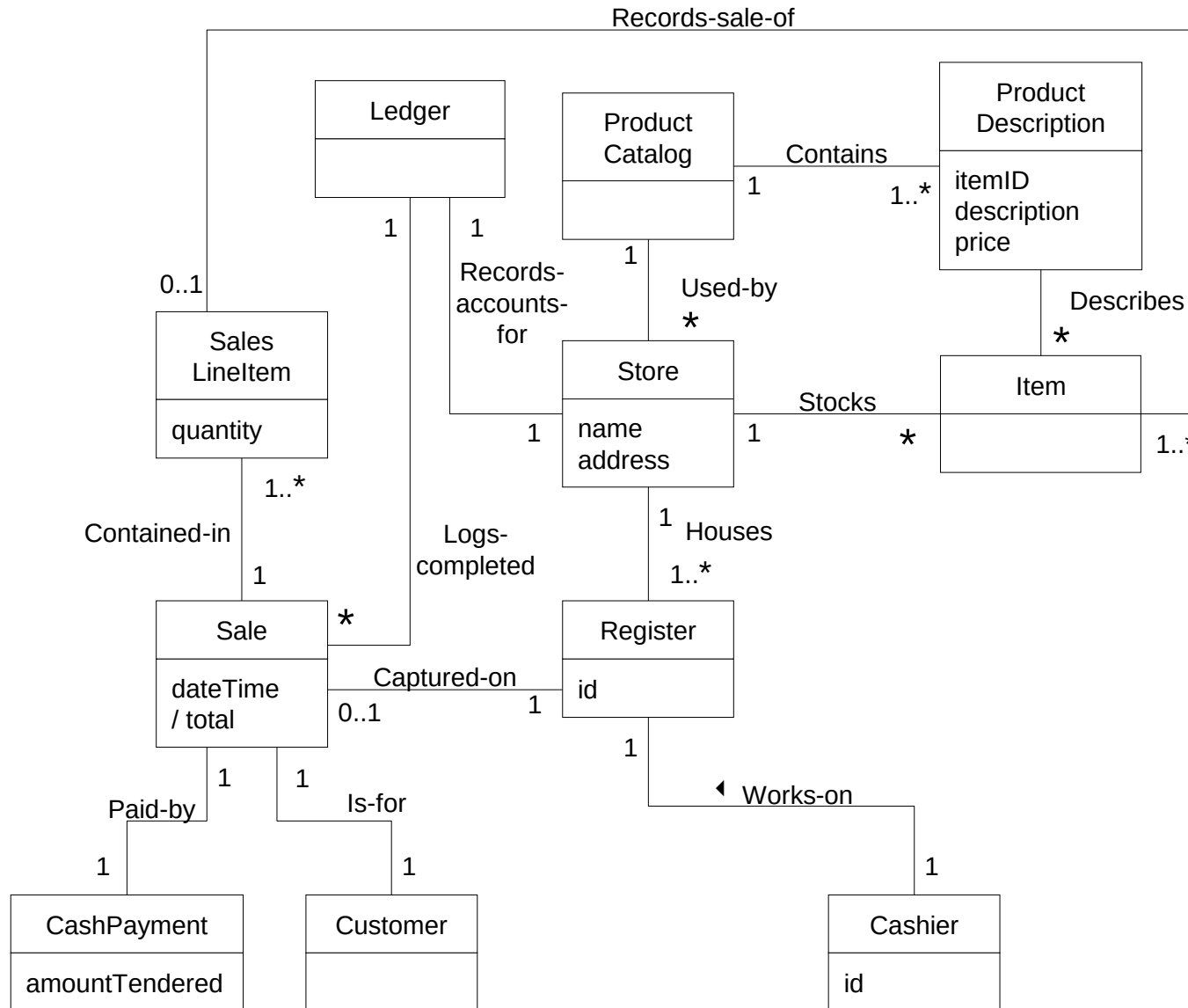
- Easily understandable and maintainable
- Code reuse
- Low coupling

Case Study – Point-Of-Sale (POS) System

- is used to record sales and handle payments; it is typically used in a retail store.
- includes hardware components such as a computer and bar code scanner, and software to run the system.
- interfaces to various service applications, such as a third-party tax calculator and inventory control.



Domain Model



Creator Pattern

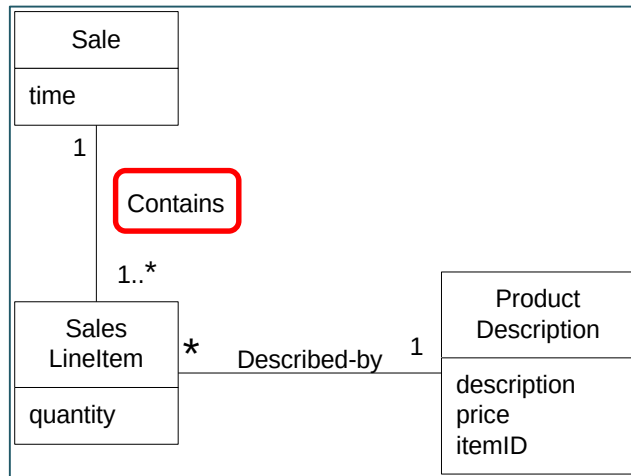
Name	Creator
Problem	Who creates an A?
Solution	Assign class B the responsibility to create an instance of class A, if one of these is true (the more the better): • B contains or aggregates A. • B records A. • B closely uses A. • B has the initializing data for A. if there is a choice, prefer “B contains or aggregates A”

Creator Example

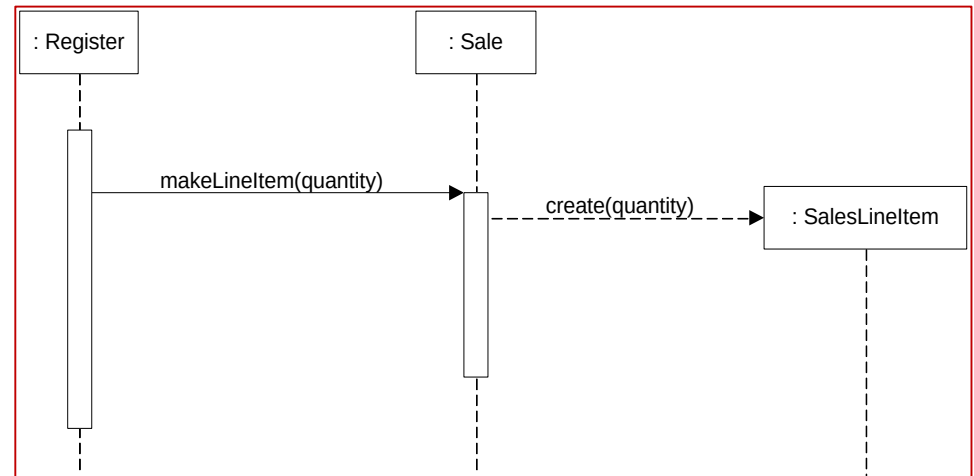
- Who should be responsible for creating a [SalesLineItem](#) instance?

Assign class B the responsibility to create an instance of class A, if :

- B contains or aggregates A.
- B records A.
- B closely uses A.
- B has the initializing data for A.



Domain Model



Design Model

Creation Pattern

- **Discussion:**

- Guides assigning creation responsibilities
- Contradictions: creation may require significant complexity
 - using recycled instances for performance reasons
 - conditionally creating an instance from one of a family of similar classes
 - sometimes desired to outsource object wiring (“dependency injection”)

- **Benefits:**

- Low Coupling is supported

- **Related patterns:**

- Abstract Factory, Singleton

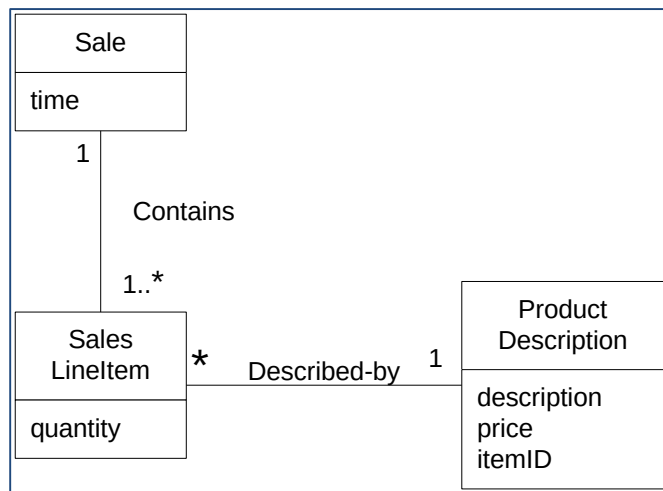
Information Expert Pattern

Name	Information Expert
Problem	What is a basic principle by which to assign responsibilities to objects?
Solution	Assign a responsibility to the class that has the information needed to fulfill it.

Information Expert Example

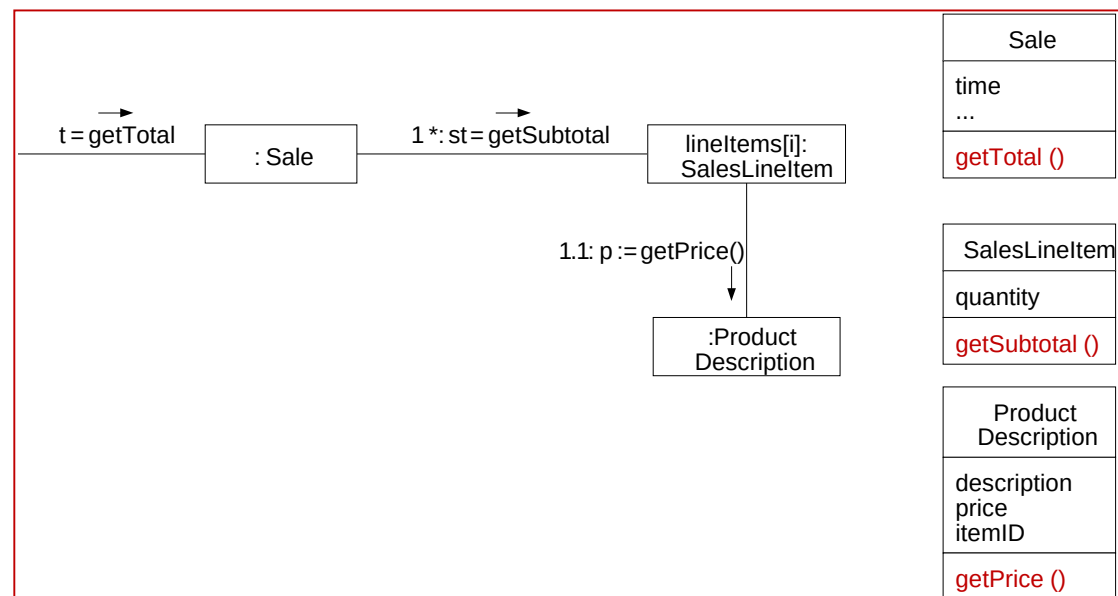
- Who should be responsible for knowing the grand total of a sale?

Assign a responsibility to the class that has the information needed to fulfill it.



Domain Model

Design Class	Responsibility
Sale	knows sale total
SalesLineItem	knows line item subtotal
ProductDescription	knows product price



Design Model

Information Expert Pattern

▪ Discussion

- **Most used principle** in the assignment of responsibilities
- Information is spread across different objects -> they need to interact
- Contraindication: Conflict with **Separation of Concerns**
 - Example: Responsibility for saving a sale in the database
 - Adding this responsibility to Sale
 - distribute database logic over many classes : low cohesion

▪ Benefits

- Information encapsulation → supports low coupling

Controller Pattern

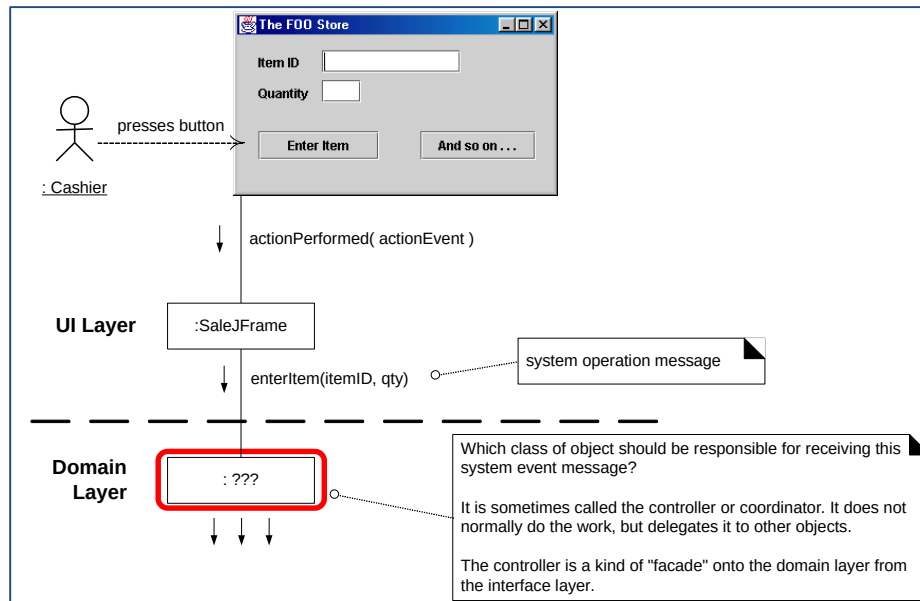
Name	Controller
Problem	What first object beyond the UI layer receives and coordinates ("controls") a system operation?
Solution	Assign the responsibility to an object representing one of these choices: • 1st option: Overall "system," a "root object," a device that the software is running within, or a major subsystem (<i>facade controller</i>) • 2nd option: Use case scenario within which the system operation occurs (<i>use case</i> or <i>session controller</i>)

Controller Example

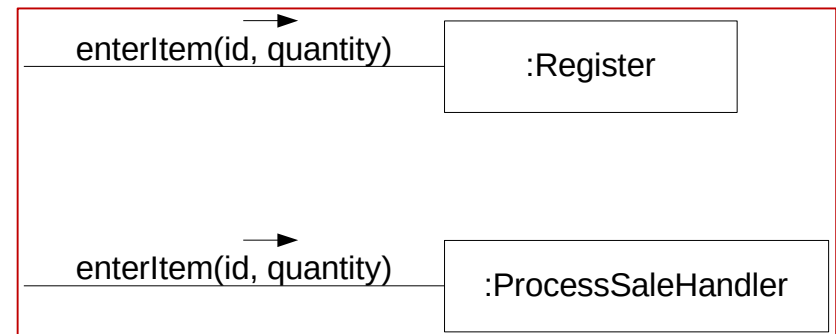
- Who should be the controller for system events such as `enterItem` and `endSale`?

Assign the responsibility to an object representing one of these choices:

- 1st option: Overall "system," a "root object," a device that the software is running within, or a major subsystem (these are all variations of a *facade controller*).
- 2nd option: Use case scenario within which the system operation occurs (a use case or *session controller*)



By the Controller pattern, two choices:



Design Model

Controller Example

▪ Choices

- Register (or POSSystem): represents the overall "system," device, or subsystem
→ Façade controller
- ProcessSaleHandler (or ProcessSaleSession): represents a receiver or handler of all system events of a use case scenario or session
→ Use-case controller (or session controller)

Controller Summary

▪ Discussion

- Controller class is called bloated if overloaded with too many responsibilities
 - Add more controllers (façade -> use-case controllers)
 - Delegates to other objects

▪ Benefits

- No application logic in the GUI
- Increased potential for reusable components
- Reason about the state of the use case
 - Useful when operations must be performed in a specific order
Example) MakePayment cannot occur until EndSale has occurred

Low Coupling Pattern

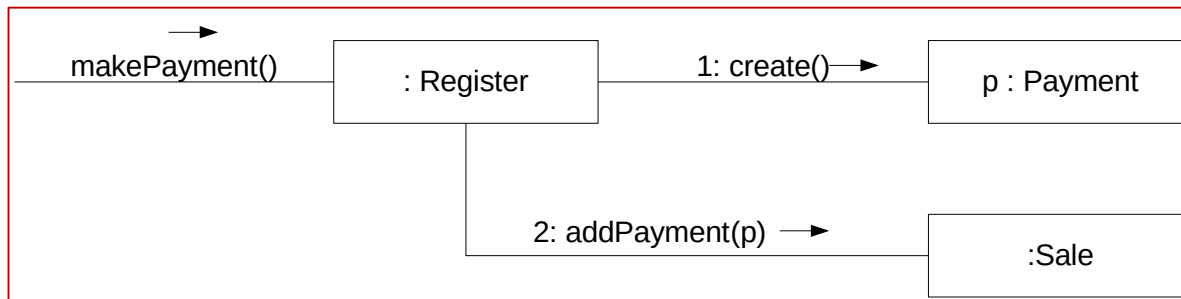
Name	Low Coupling
Problem	How to reduce the impact of change?
Solution	Assign responsibilities so that (unnecessary) coupling remains low. Use this principle to evaluate alternatives .

Low Coupling Example

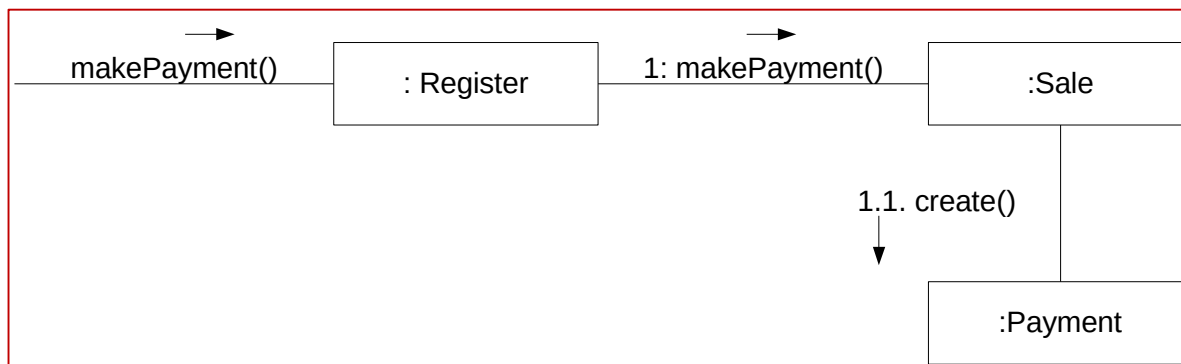
- We need to **create a Payment instance** and **associate it with the Sale**. What class should be responsible for this?

Assign responsibilities so that (unnecessary) coupling remains low
Use this principle to **evaluate alternatives**.

- Option 1: Register "records" a Payment in the real-world domain, the **Creator pattern suggests Register** as a candidate for creating the Payment



- Option 2: Sale creates the *Payment* and associate it with the *Sale*



Low Coupling Pattern

▪ Discussion

- Low coupling encourages designs to be more independent which reduces the impact of change.
- Needs to be considered with other patterns such as Information Expert and High Cohesion.
- High coupling to stable “global” objects is not problematic
Example) coupling to Java libraries such as `java.util`

High Cohesion Pattern

Name	High Cohesion
Problem	How to keep objects focused, understandable, and manageable, and as a side effect, support Low Coupling?
Solution	Assign responsibilities so that cohesion remains high. Use this to evaluate alternatives.

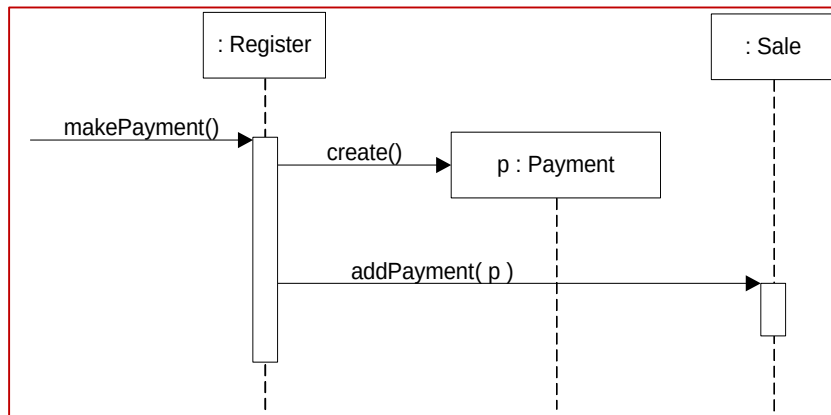
High Cohesion Example

- Where to place the responsibility for making a payment?

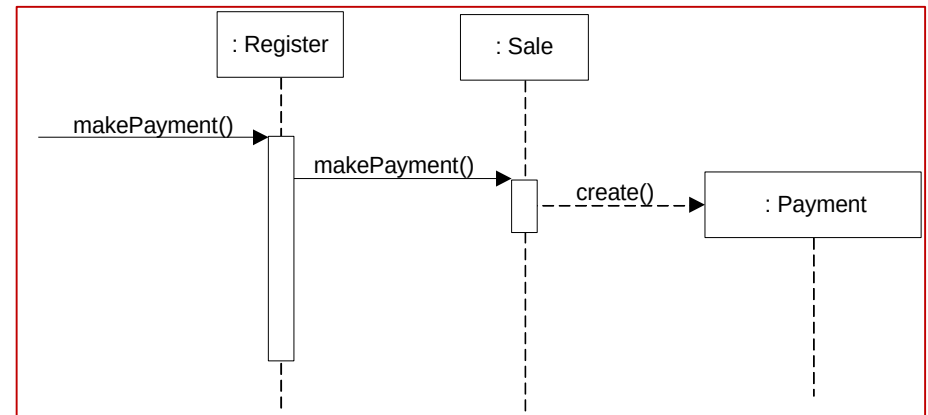
Assign responsibilities so that cohesion remains high.
Use this principle to **evaluate alternatives**.

Option 1: Assign to the Register. In this isolated example, this is acceptable; but the Register will get bigger

Option 2: Delegates the payment creation responsibility to the Sale



Option 1



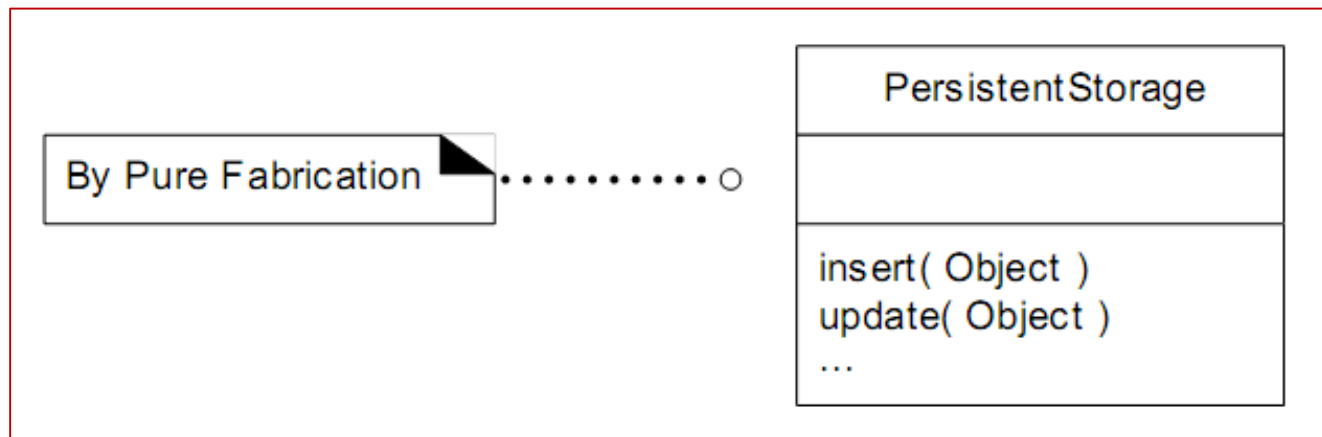
Option 2 😊

Pure Fabrication Pattern

Name	Pure Fabrication
Problem	Who is responsible when you are desperate, and do not want to violate high cohesion and low coupling?
Solution	Assign a highly cohesive set of responsibilities to an artificial or convenience "behavior" class that does not represent a problem domain concept something made up , in order to support high cohesion, low coupling, and reuse .

Pure Fabrication Example

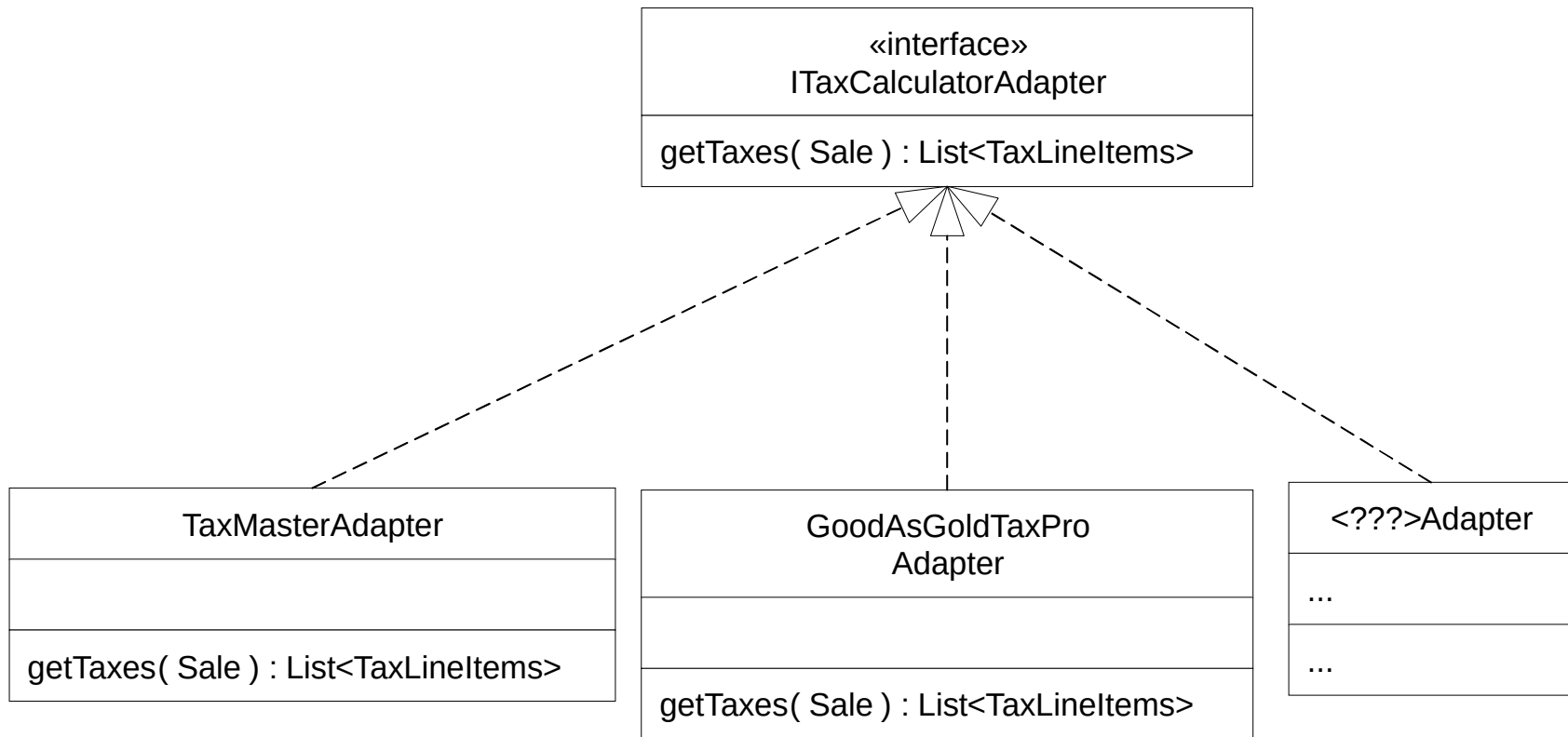
- We want to save Sale instances in a DB.
 - By Information Expert, let's assign this responsibility to Sale class.
 - the task requires many DB related operations and the Sale class becomes **low cohesive**.
 - **poor reuse** or **lots of duplication** in other classes that do the same thing.
 - By Pure Fabrication
 - **create a new class** that is **solely responsible for saving** objects in a persistent storage medium



Polymorphism Pattern

Name	Polymorphism
Problem	Who is responsible when behavior varies by type?
Solution	When related alternatives or behaviors vary by type (class), assign responsibility for the behavior using polymorphic operations to the types for which the behavior varies.

Polymorphism Example



By Polymorphism, multiple tax calculator adapters have their own similar, but varying behavior for adapting to different external tax calculators

Polymorphism Pattern

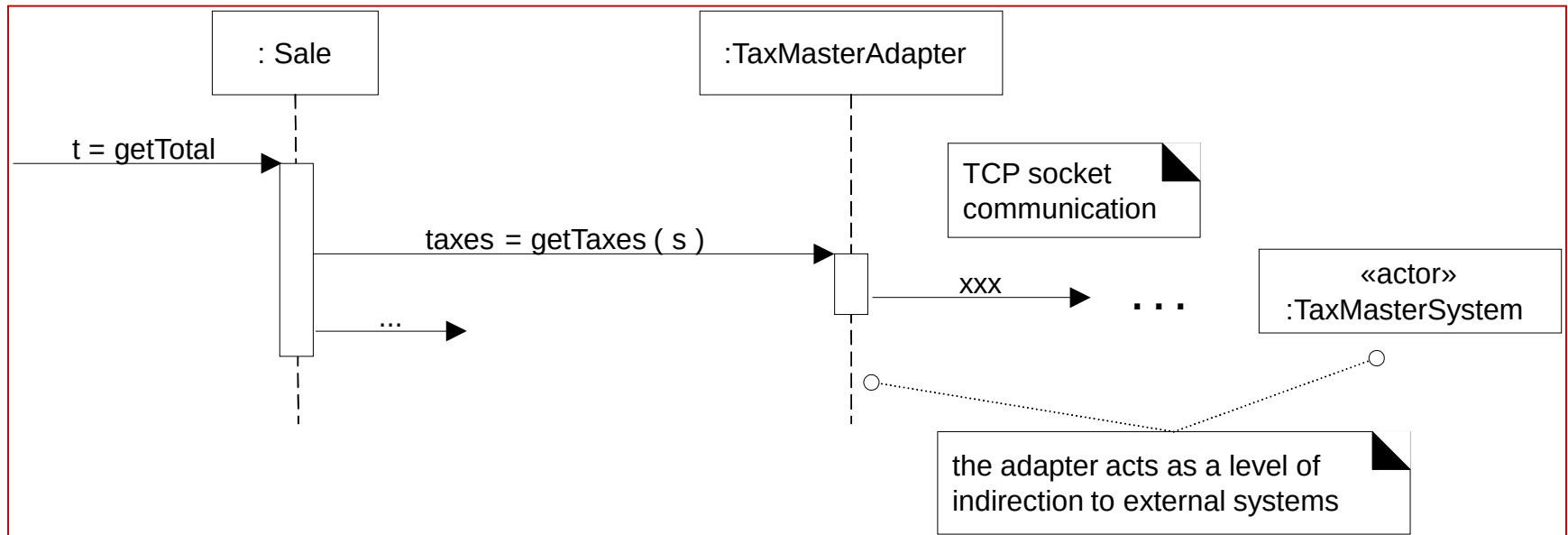
- Benefits
 - Easier and more reliable than using explicit selection logic
 - Easier to add additional behaviors later on
- Costs
 - Increases the number classes in a design
 - May make the code less easy to follow
- Discussion
 - fundamental principle in design: how to handle similar variations.
 - Avoid excessive use for “future-proofing” against yet unknown potential future variations
 - Agile approach: no significant “upfront design” and add the variation point only when the need arises

Indirection Pattern

Name	Indirection
Problem	How to assign responsibilities to avoid direct coupling?
Solution	Assign the responsibility to an intermediate object to mediate between other components or services, so that they are not directly coupled.

Indirection Example

- By adding a level of **indirection** and adding **polymorphism**, the adapter objects protect the inner design against variations in the external interfaces



Indirection Pattern

- How can we avoid a direct coupling between two or more elements?
- Indirection **introduces an intermediate units** (classes) to communicate between the other units, so that the other units are not directly coupled.
- Benefits
 - Low Coupling
- Example: Adapter, Facade, Proxy, Mediator, ...

Protected Variations Pattern

Name	Protected Variations
Problem	How to assign responsibilities to objects, subsystems, and systems so that the variations or instability in these elements do not have an undesirable impact on other elements?
Solution	Identify points of predicted variation or instability Assign responsibilities to create a stable "interface" around them.

Protected Variations Pattern

- OCP and PV are two expressions of the same principle
 - protection against change to the existing code and design at variation and evolution points with minor differences in emphasis
- Benefits
 - Flexibility and Protection from Variations
 - Provides more structured design
- Mechanisms motivated by PV
 - Uniform access in C#, Ada, and Eiffel
 - Express both method and field access in the same way
 - Service lookup
 - JNDI
 - Reflective or meta-model designs

Summary

- GRASP
 - Information Expert
 - Creator
 - Controller
 - Low Coupling
 - High Cohesion
 - Polymorphism
 - Pure Fabrication
 - Indirection
 - Protected variations

GRASP Quiz : Problem



- Which class should determine the final mark the student receives in a course? Which GRASP pattern does support your answer?

GRASP Quiz : Answer



- WorkItem class can determine the value of an individual items, but it **cannot determine the final mark.**
- **Student class** should be assigned this responsibility
 - **Because it knows about all of the work items.**
 - Student class will rely on WorkItem class to determine individual marks.
 - It's based on **Information Expert pattern.**

5. STRATEGY PATTERN

2020. 04. 16

중앙대학교 소프트웨어학부

이찬근 교수

Contents

- Intro. to the problem
- How the strategy pattern is discovered
- Structure of strategy pattern
- Examples using the strategy pattern

Strategy Pattern

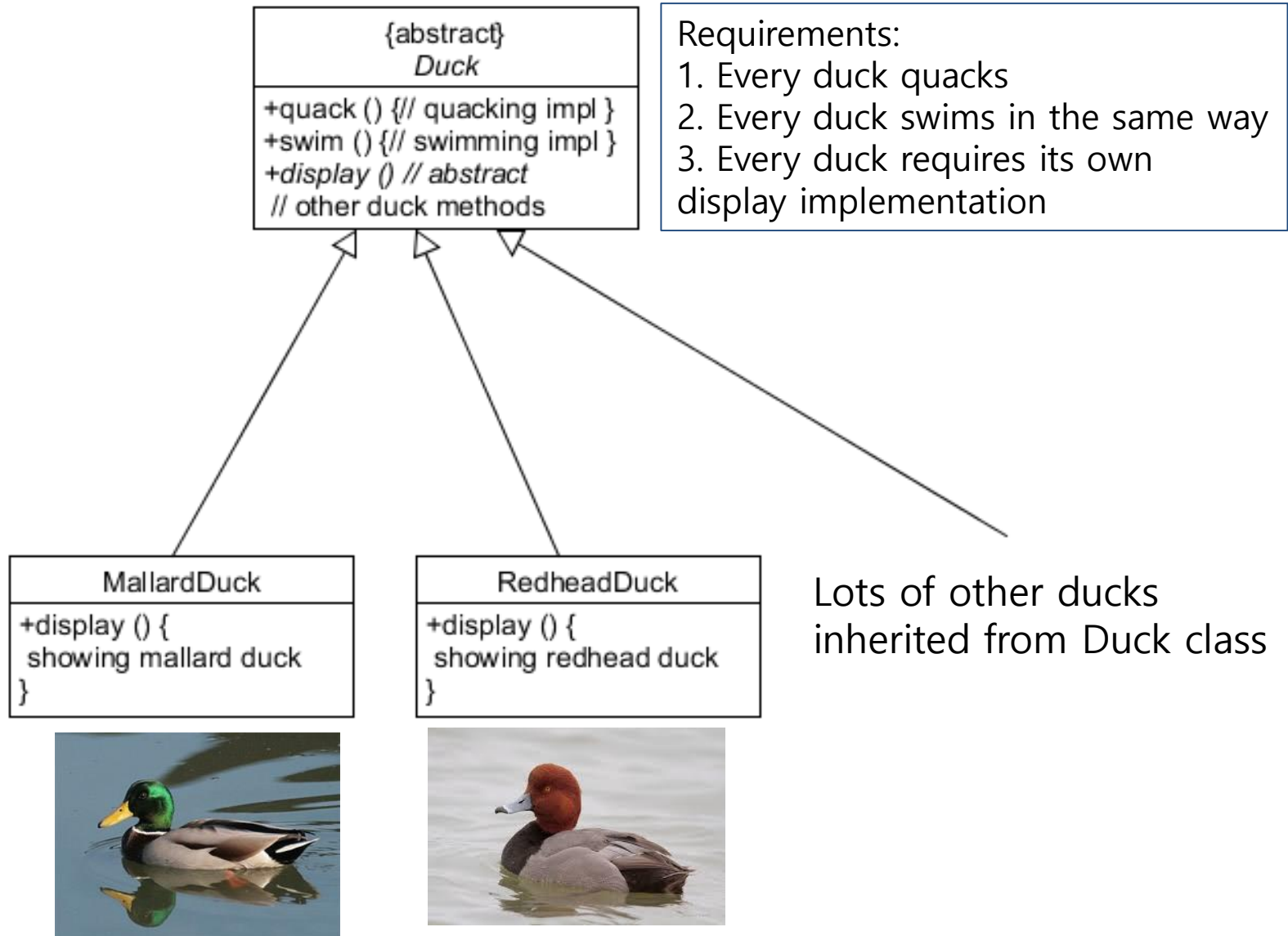
- **Purpose**

- Defines a set of encapsulated algorithms that can be swapped to carry out a specific behavior.

- **Use When**

- The only difference between many related classes is their behavior.
 - Multiple versions or variations of an algorithm are required.
 - The behavior of a class should be defined at runtime.
 - Conditional statements are complex and hard to maintain.

Our Original Design



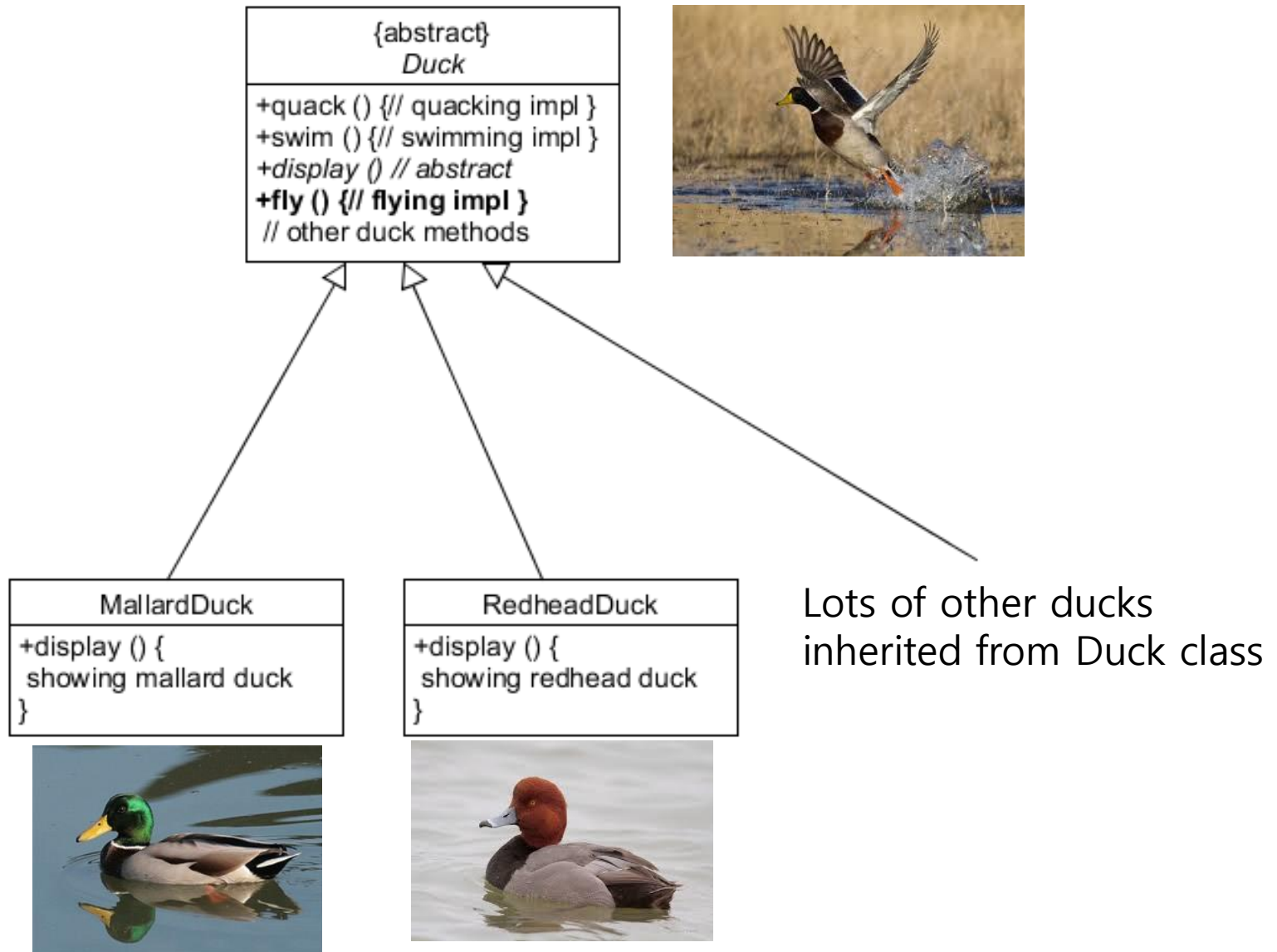
Requirement Changes ...

- The original design has been okay
- We just got additional requirement
 - We need to add fly behavior to ducks

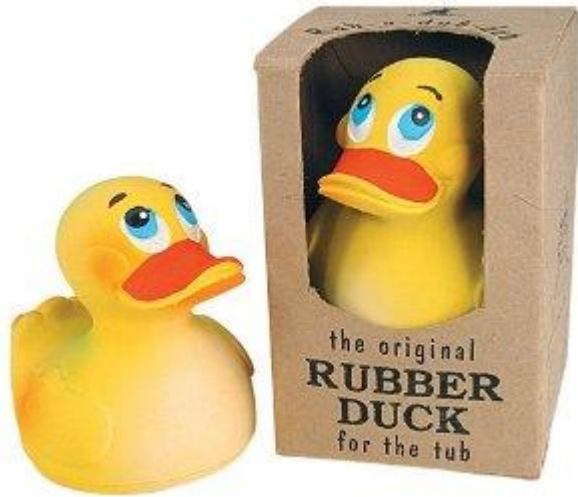


- Okay, let's add it to the base class!

Our First Attempt

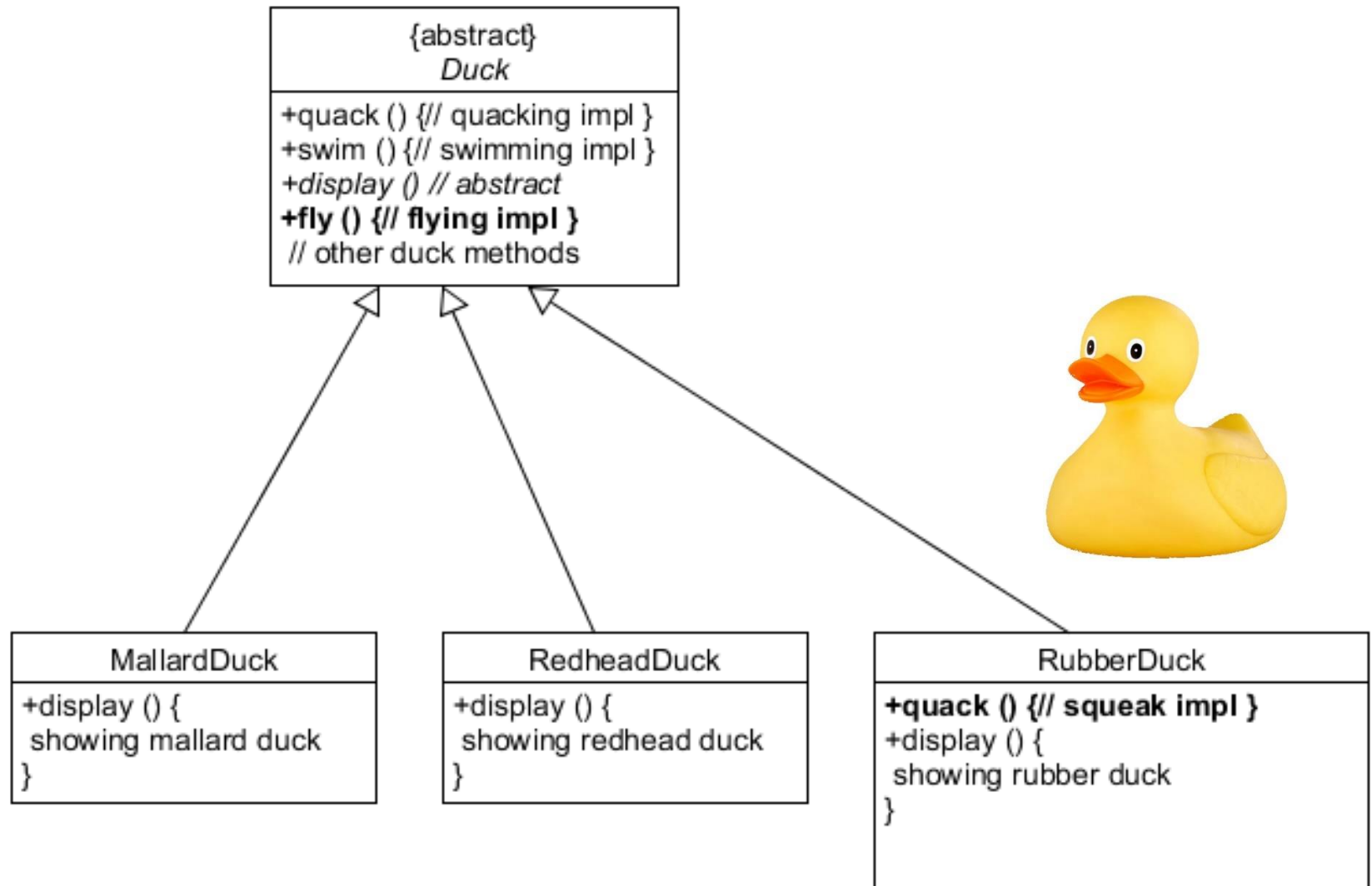


What???

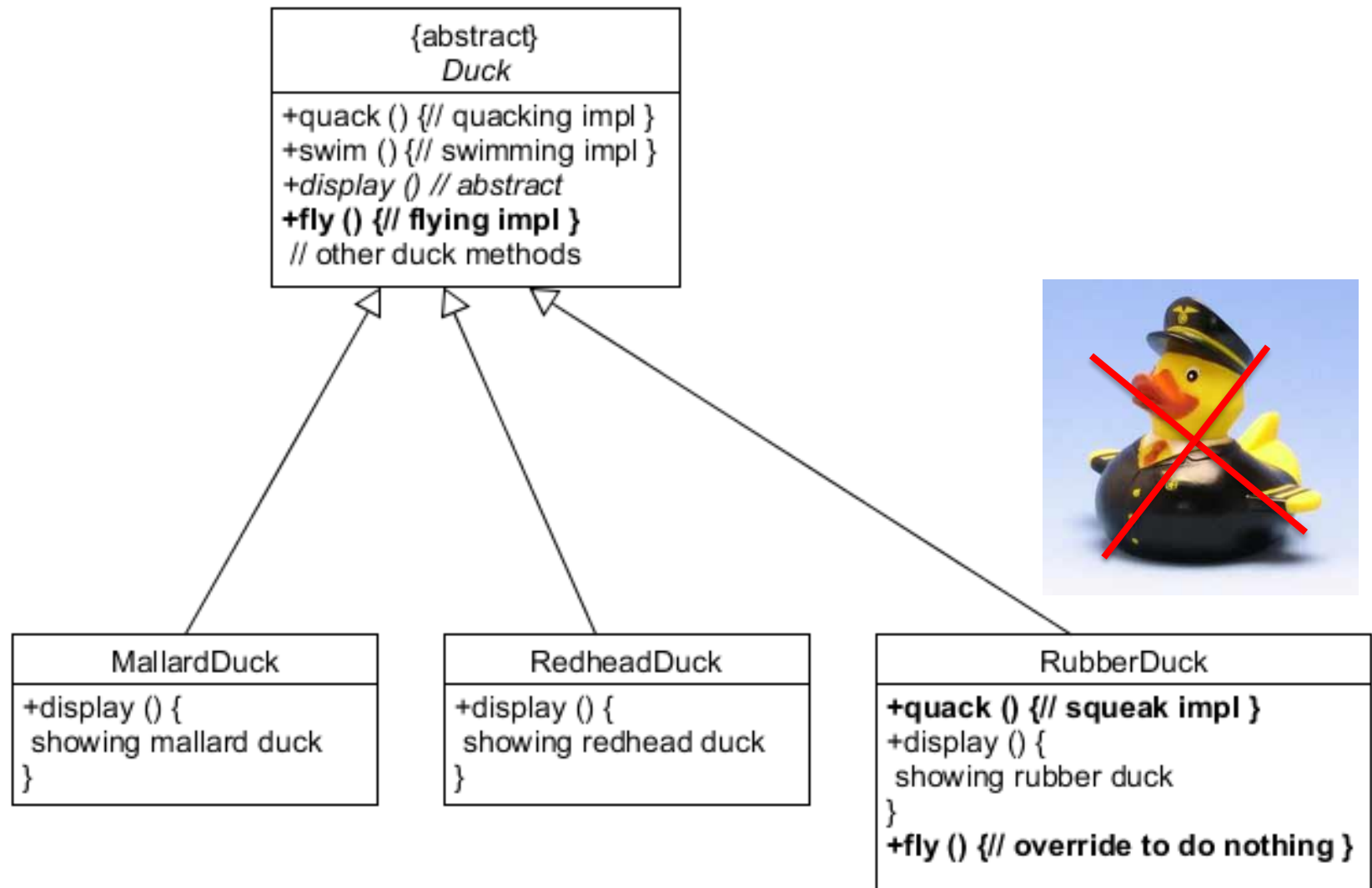


A rubber duck can't fly, at least in our application!

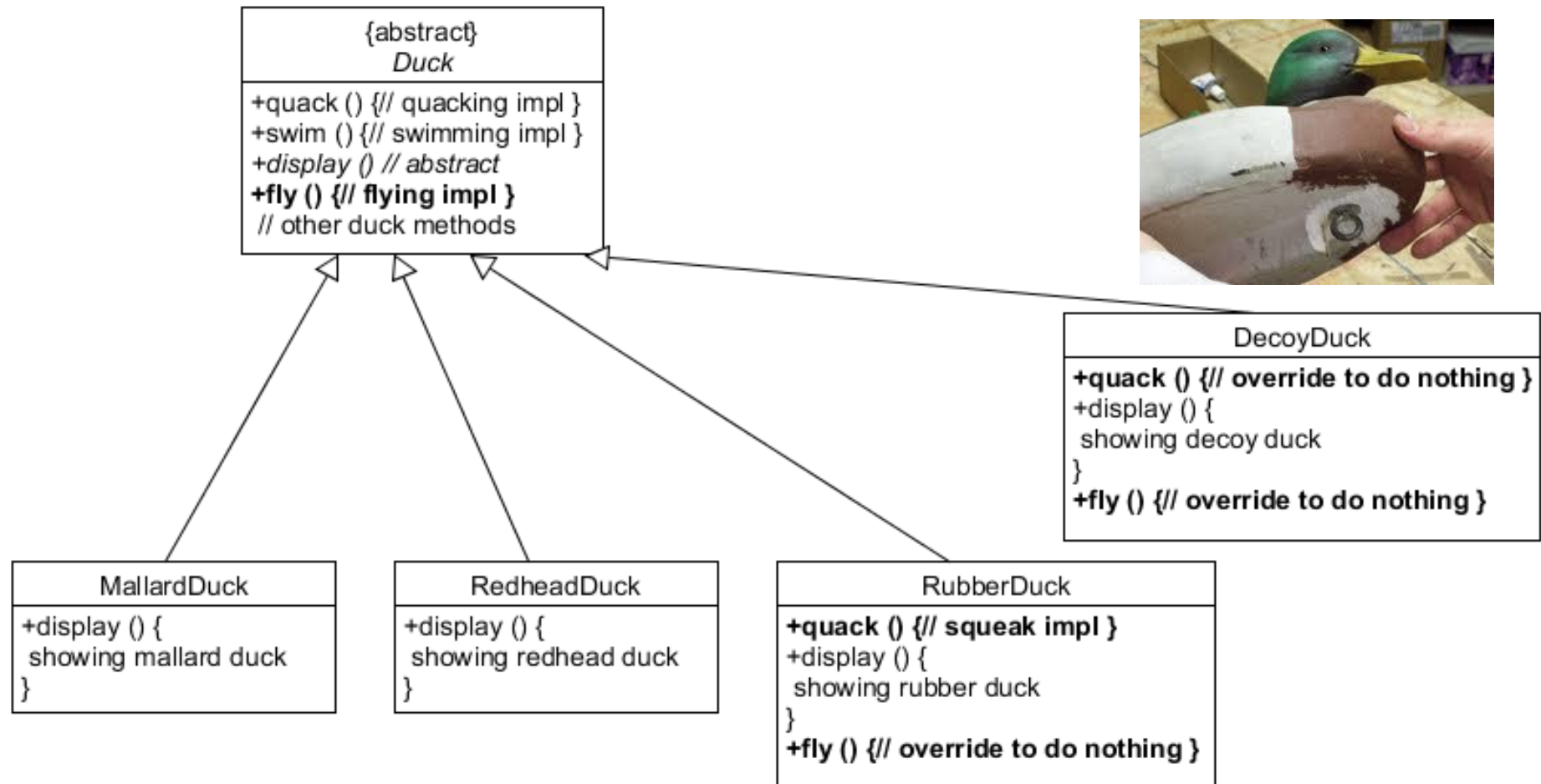
Looking into Our Design



Overriding RubberDuck Would Solve the Problem



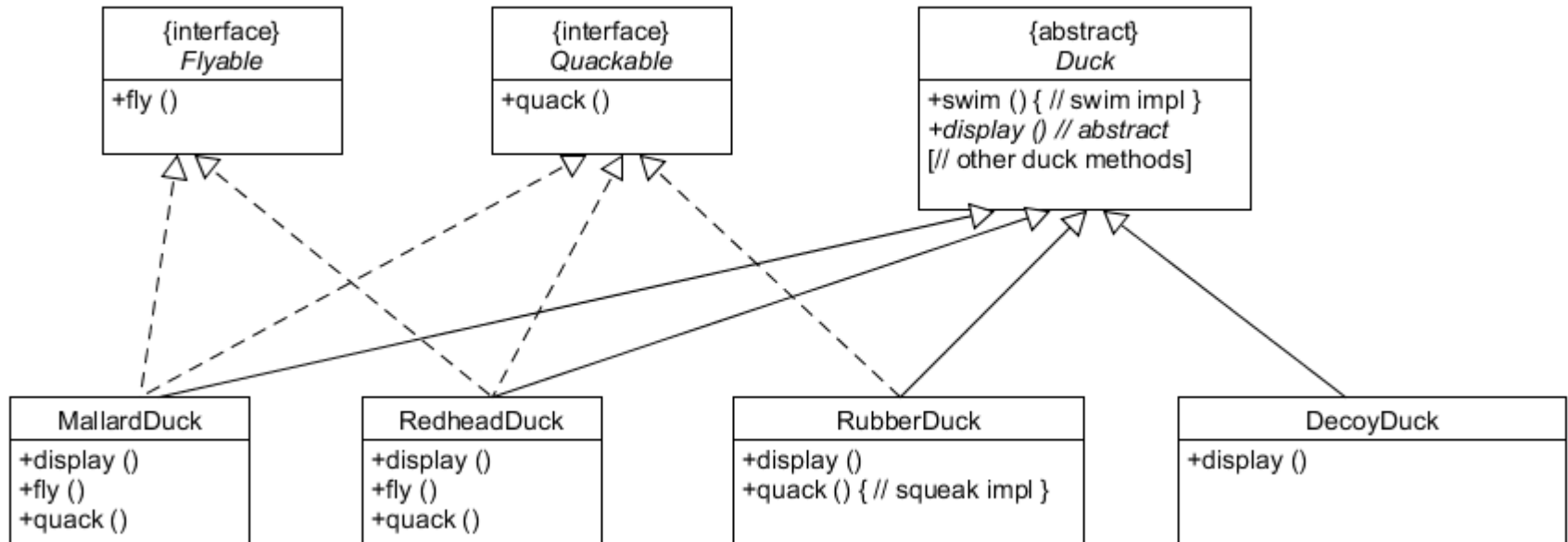
More and More Ducks to Override



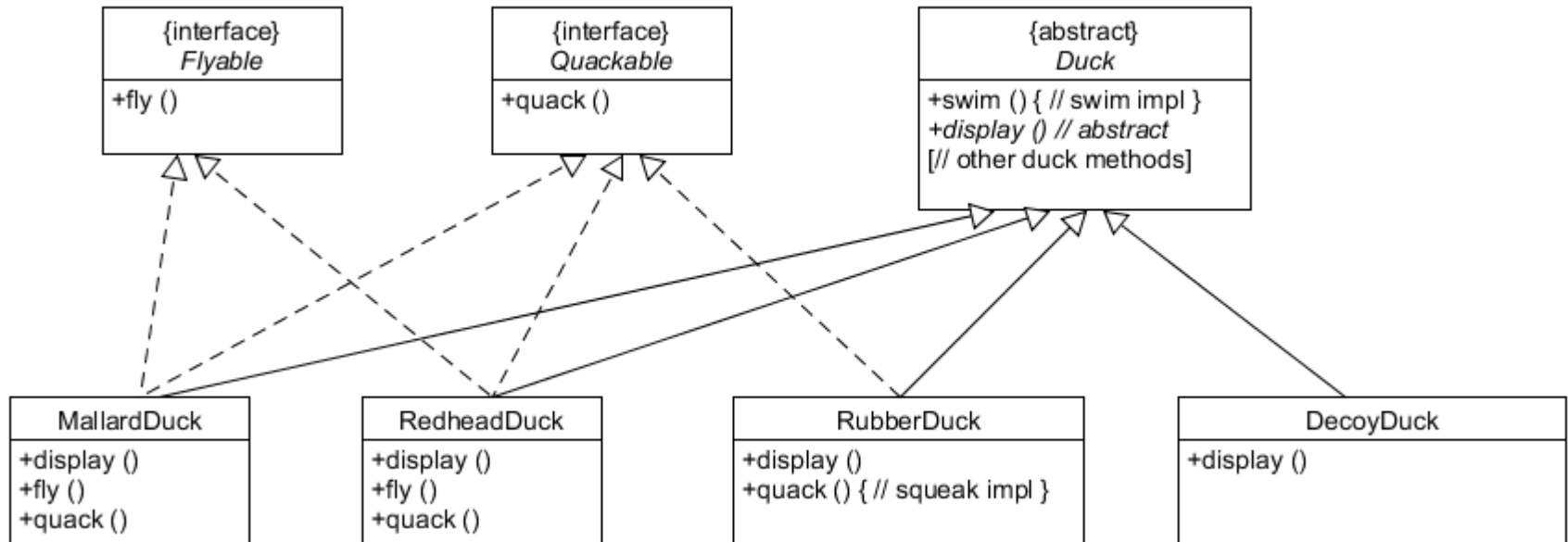
What we have here

- A localized update to the code caused a non-local side effect!
- A great use of inheritance for the purpose of reuse hasn't turned out so well when time comes to maintenance

Our Second Attempt: How about an Interface?



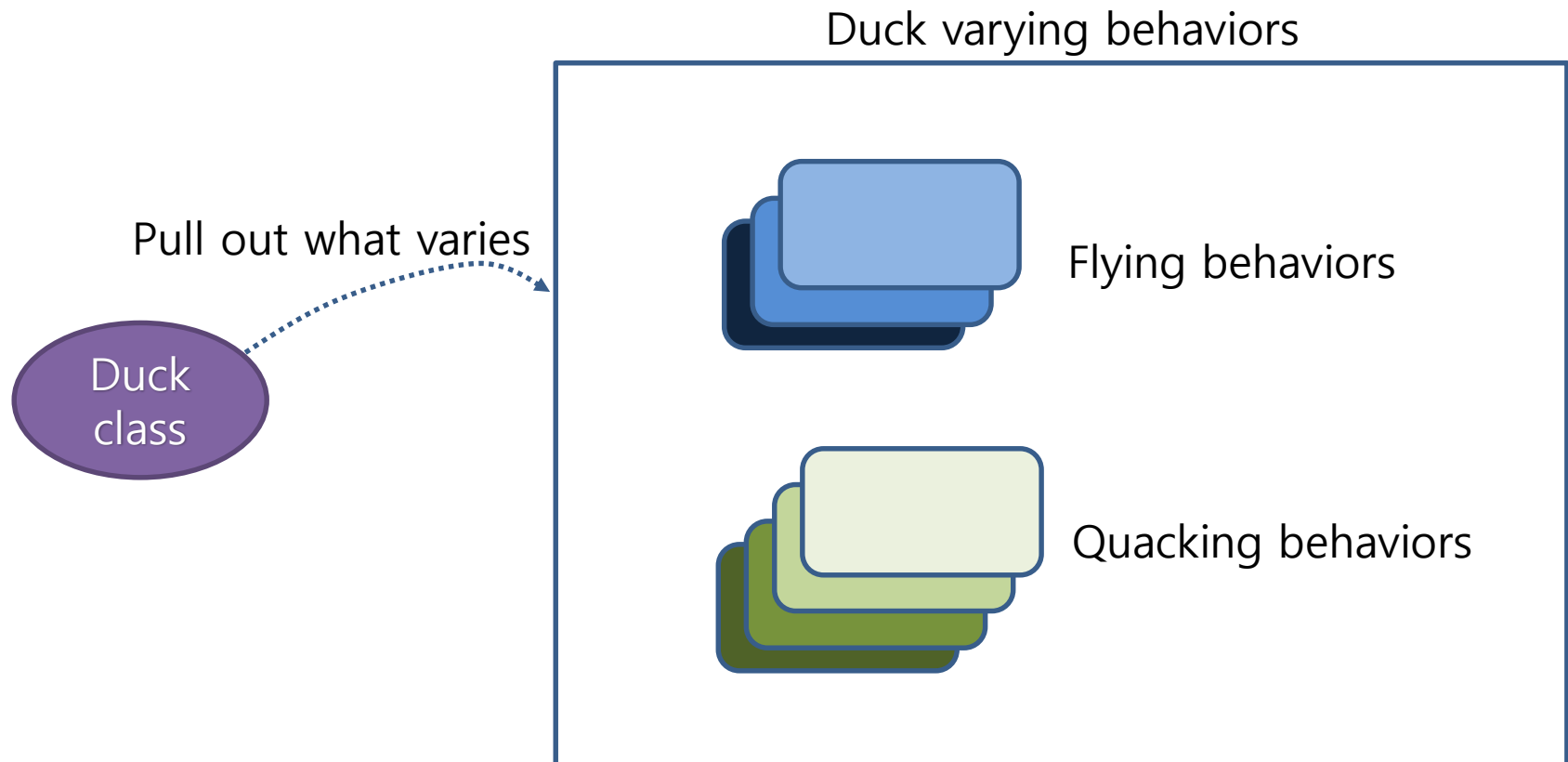
Our Second Attempt: How about an Interface?



- Completely **destroys code reuse**
- Creates a *different* **maintenance nightmare**
- There might be more than one kind of flying behavior even among the ducks that do fly
 - Turkey, Chicken, ...

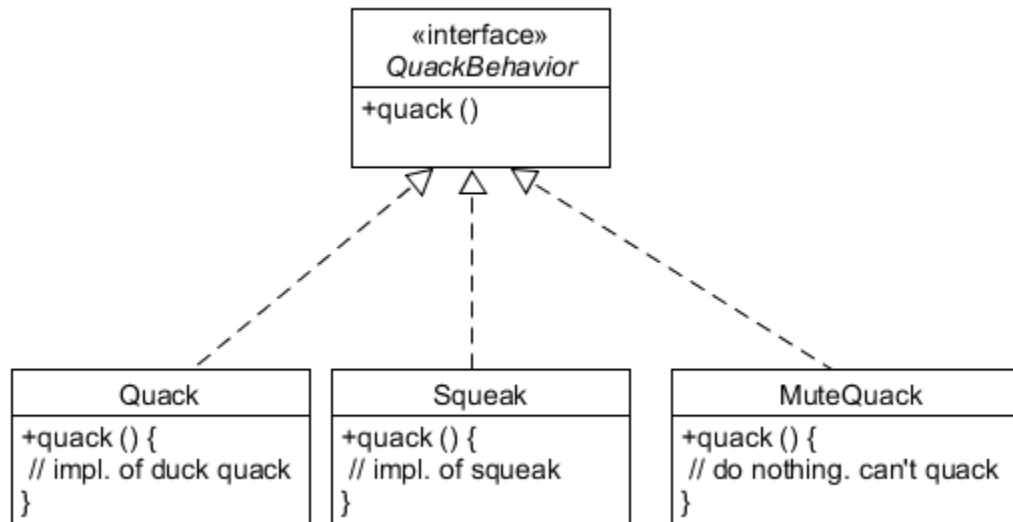
Design Principle: Encapsulate what varies

- Identify the aspects of your application that vary and separate them from what stays the same!



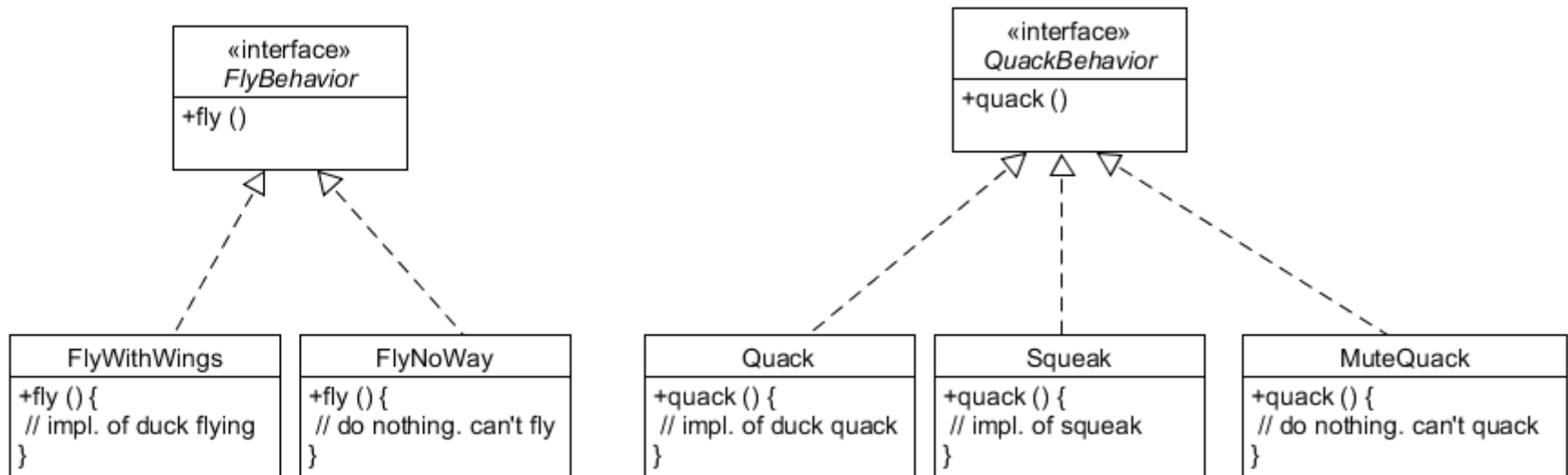
Separating what changes from what stays the same

- What is **changing** in SimDuck?
 - **Flying behavior** and **quack behavior**
 - Each type of behavior has a **family**
- To separate these behaviors from the Duck class, we'll
 - pull both methods out of the Duck class
 - **Create a new set of classes** to represent each **behavior**



Design Principle

- Program to an interface, not an implementation
 - Programming to a super type
 - Exploit Polymorphism
 - The declared type of the variables should be a super type
 - The class declaring them doesn't have to know the actual object types



Program to interface, not an implementation

- Programming to an implementation would be:

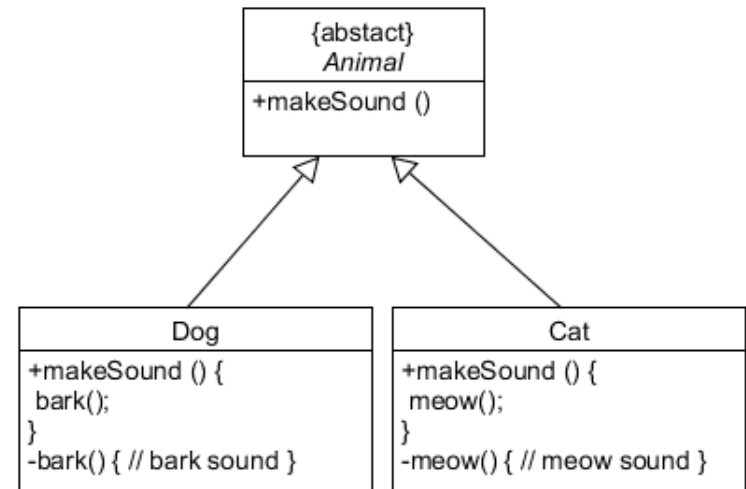
```
Dog d = new Dog();  
d.bark();
```

- Programming to an interface/supertype would be:

```
Animal a = new Dog();  
a.makeSound();
```

- Even better, rather than hard-coding the instantiation of the subtype into the code, **assign the concrete implementation object at runtime:**

```
Animal a = getAnimal("Dog");  
a.makeSound();
```



Improved Design

New Duck

Duck
FlyBehavior flyBehavior QuackBehavior quackBehavior
<pre>performQuack(){ quackBehavior.quack(); } swim() { //swimming impl} display() //abstract performFly(){ flyBehavior.fly(); } //Other duck-like methods</pre>

V.S.

Old Duck

Duck
<pre>quack() { //quacking impl} swim() { //swimming impl} display() //abstract fly() { //fly impl} //Other duck-like methods</pre>

- We can reuse fly and quack behaviors.
- We can add new behaviors without modifying any of existing behavior classes or touching any of Duck classes.

Class Duck

```
public abstract class Duck {  
    FlyBehavior flyBehavior;  
    QuackBehavior quackBehavior;  
    public Duck() {  
    }  
  
    public abstract void display();  
  
    public void performFly() {  
        flyBehavior.fly();  
    }  
  
    public void performQuack() {  
        quackBehavior.quack();  
    }  
  
    public void swim() {  
        System.out.println("All ducks float, even decoys!");  
    }  
}
```

Class Duck

```
public abstract class Duck {  
    FlyBehavior flyBehavior;  
    QuackBehavior quackBehavior;  
    public Duck() {  
    }  
  
    public abstract void display();  
  
    public void performFly() {  
        flyBehavior.fly();  
    }  
  
    public void performQuack() {  
        quackBehavior.quack();  
    }  
  
    public void swim() {  
        System.out.println("All ducks float, even decoys!");  
    }  
}
```

object composition
object composition
delegation
delegation

Interface FlyBehavior

```
public interface FlyBehavior {  
    public void fly();  
}
```

```
public class Flywithwings implements FlyBehavior {  
    public void fly() {  
        System.out.println("I'm Flying!!");  
    }  
}
```

```
public class FlyNoway implements FlyBehavior {  
    public void fly() {  
        System.out.println("I can't fly");  
    }  
}
```

Interface QuackBehavior

```
public interface QuackBehavior {  
    public void quack();  
}  
  
public class Quack implements QuackBehavior {  
    public void quack() {  
        System.out.println("Quack");  
    }  
}  
  
public class MuteQuack implements QuackBehavior {  
    public void quack() {  
        System.out.println("<< silence >>");  
    }  
}  
  
public class Squeak implements QuackBehavior {  
    public void quack() {  
        System.out.println("Squeak");  
    }  
}
```

Concrete Ducks

```
public class MallardDuck extends Duck {  
    public MallardDuck() {  
        quackBehavior = new Quack();  
        flyBehavior = new Flywithwings();  
    }  
  
    public void display() {  
        System.out.println("I'm a real Mallard Duck");  
    }  
}
```

Test Drive!

```
public class MiniDucksSimulator {  
    public static void main(String[] args)  
    {  
        Duck mallard = new MallardDuck();  
        mallard.performQuack();  
        mallard.performFly();  
    }  
}
```

```
public abstract class Duck {  
    ...  
    QuackBehavior quackBehavior;  
    FlyBehavior flyBehavior;  
    public void performQuack() {  
        quackBehavior.quack();  
    }  
    public void performFly() {  
        flyBehavior.fly();  
    }  
    ...  
}
```

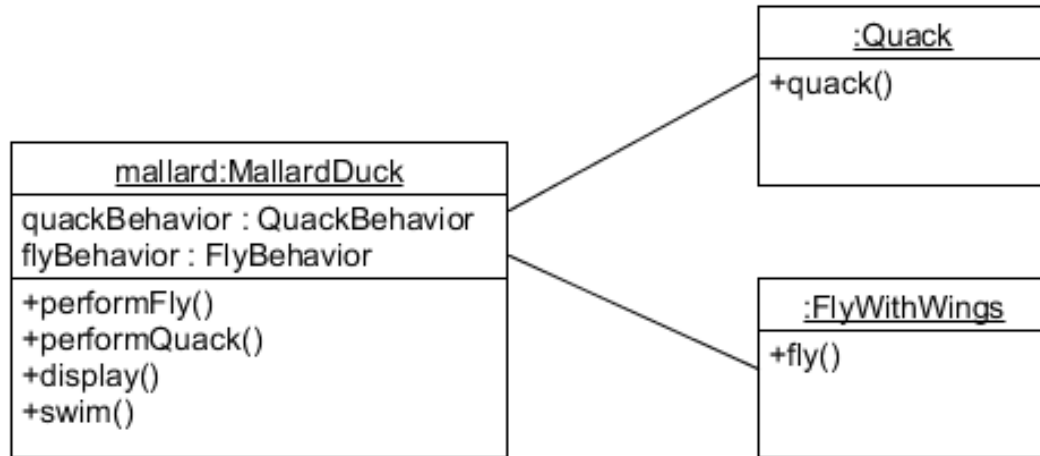
Output

```
Quack  
I'm flying!!
```

```
public MallardDuck() {  
    quackBehavior = new Quack();  
    flyBehavior = new Flywithwings();  
}
```

```
public class Quack implements QuackBehavior  
{  
    public void quack() {  
        System.out.println("Quack");  
    }  
}
```

Object Diagram for a Mallard Duck



```
public abstract class Duck {  
    ...  
    QuackBehavior quackBehavior;  
    FlyBehavior flyBehavior;  
    public void performQuack() {  
        quackBehavior.quack();  
    }  
    public void performFly() {  
        flyBehavior.fly();  
    }  
    ...  
}
```

```
public MallardDuck() {  
    quackBehavior = new Quack();  
    flyBehavior = new Flywithwings();  
}
```

```
public class Quack implements QuackBehavior  
{  
    public void quack() {  
        System.out.println("Quack");  
    }  
}
```

Setting Behavior Dynamically

Duck
FlyBehavior flyBehavior QuackBehavior quackBehavior
performQuack(){} swim() { //swimming impl} display() //abstract performFly(){} setFlyBehavior() setQuackBehavior() //Other duck-like methods

Setting behavior dynamically

(1) Add two new methods to the Duck class

```
public void setFlyBehavior(FlyBehavior fb) {  
    flyBehavior = fb;  
}  
  
public void setQuackBehavior(QuackBehavior qb) {  
    quackBehavior = qb;  
}
```

We can call these methods anytime we want to change the behavior of a duck on the fly.

Duck
FlyBehavior flyBehavior QuackBehavior quackBehavior
performQuack(){} swim() { //swimming impl} display() //abstract performFly(){} setFlyBehavior() setQuackBehavior() //Other duck-like methods

Setting behavior dynamically (cont'd)

(2) Make a new Duck type (ModelDuck.java)

```
public class ModelDuck extends Duck {  
    public ModelDuck() {  
        flyBehavior = new FlyNoway();  
        quackBehavior = new Quack();  
    }  
  
    public void display() {  
        System.out.println("I'm a model duck");  
    }  
}
```

(3) Make a new FlyBehavior type

(FlyRocketPowered.java)

```
public class FlyRocketPowered implements FlyBehavior {  
    public void fly() {  
        System.out.println("I'm flying with a rocket!");  
    }  
}
```


Setting behavior dynamically (cont'd)

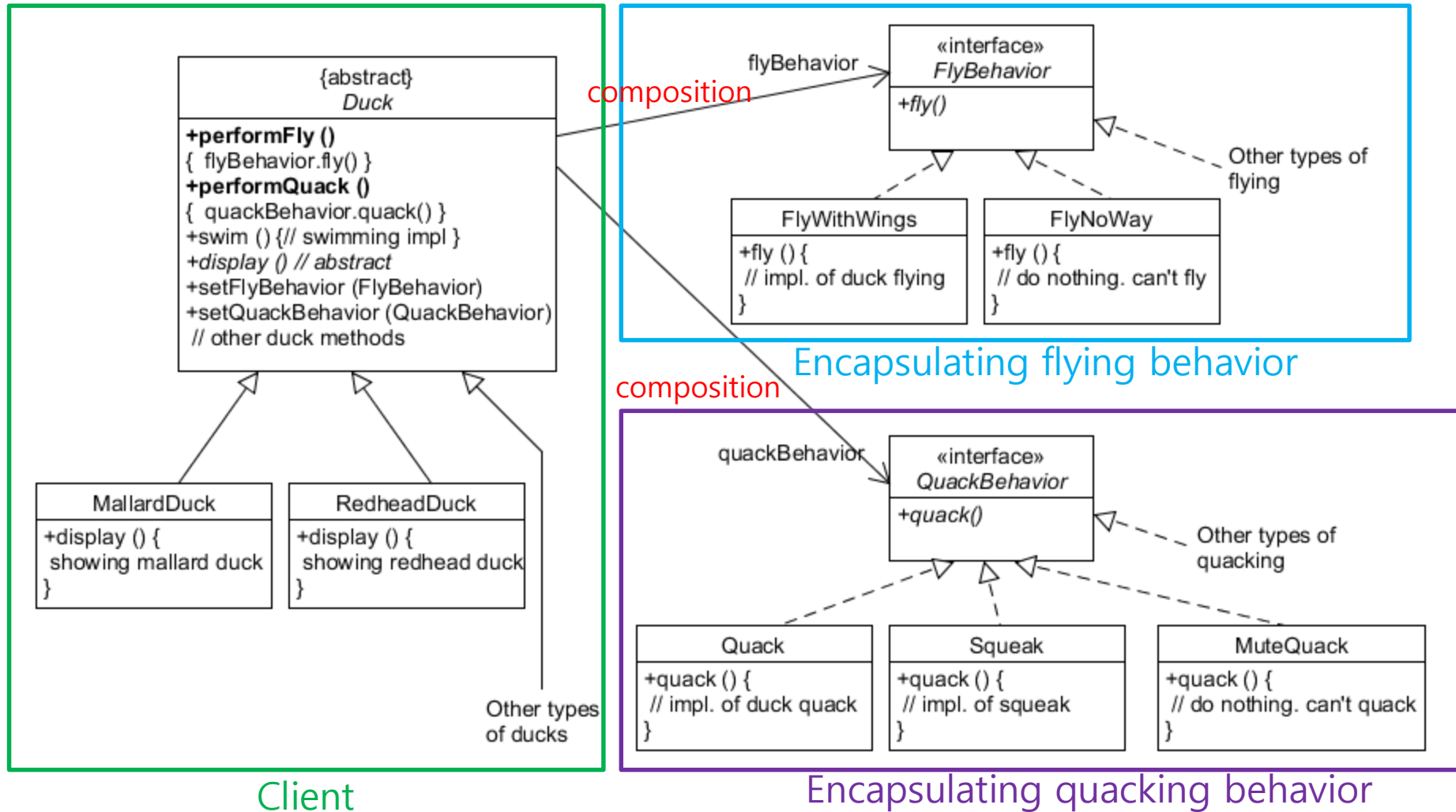
(4) Change the test driver, and make ModelDuck rocket-enabled!

```
public class MiniDuckSimulator {  
    public static void main(String [] args) {  
        Duck mallard = new MallardDuck();  
        mallard.performQuack();  
        mallard.performFly();  
  
        Duck model = new ModelDuck();  
        model.performFly();  
        model.setFlyBehavior(new FlyRocketPowered());  
        model.performFly();  
    }  
}
```

Output

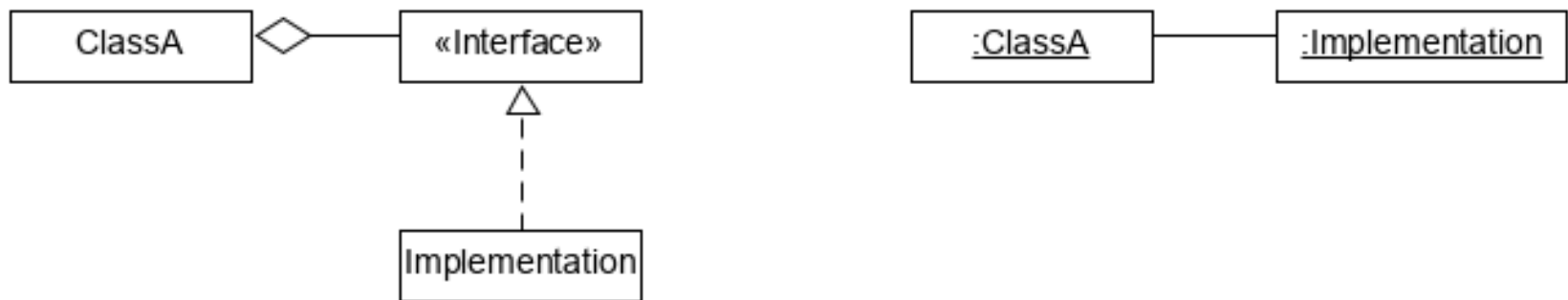
```
Quack  
I'm flying!!  
I can't fly  
I'm flying with a rocket
```

The Big Picture



Design Principle: Favor composition over inheritance (or Composing Objects Principle)

- Classes should achieve polymorphic behavior and code reuse
 - **by their composition rather than inheritance** (= by containing instances of other classes implementing the desired functionality)
- Delegation
 - passing method calls to a composed object
- **Favor (Object) Composition over (Class) Inheritance!**
 - Most design patterns emphasize object composition over inheritance whenever it is possible



Reuse in Object-Oriented System

1. Class Inheritance

- subclass' implementation is defined in terms of the parent class' implementation.
- the parent class implementation is often visible to the subclasses
- While-box reuse

(+) done at compile-time and is easy to use

(-) the subclass becomes dependent on the parent class implementation

(-) the inherited implementation cannot be changed at run-time

2. Object Composition

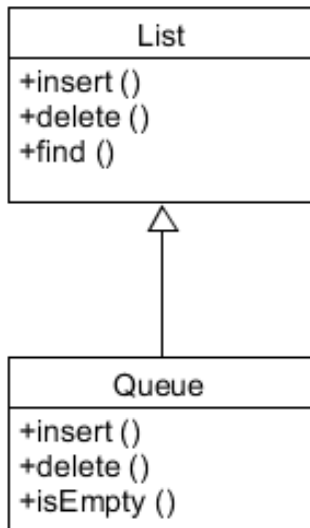
- objects are composed to achieve more complex functionality
- needs well-defined interfaces since the internals of the objects are unknown
- functionality is acquired dynamically at run-time by utilizing references to other objects
- Black-box reuse

(+) implementations can be replaced at run-time

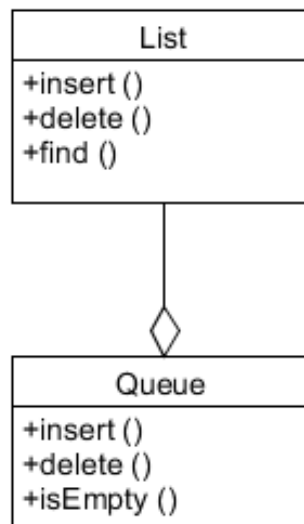
(+) less implementation dependencies

(-) harder to understand

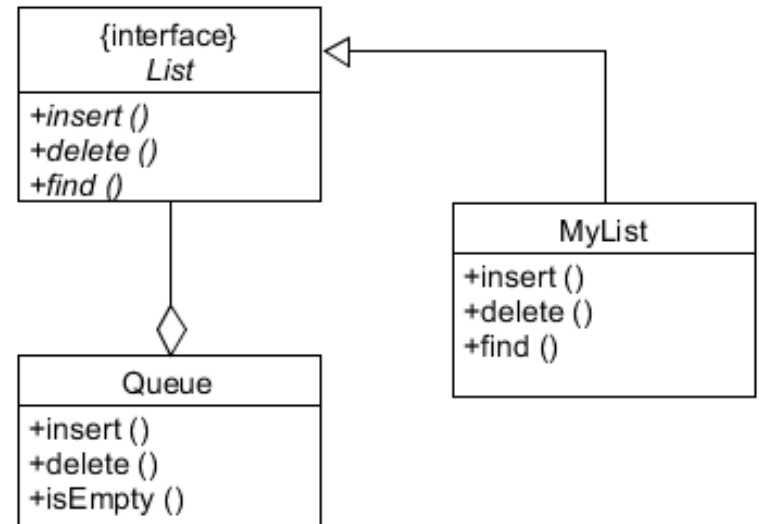
Favor Composition over Inheritance



Violation of LSP!



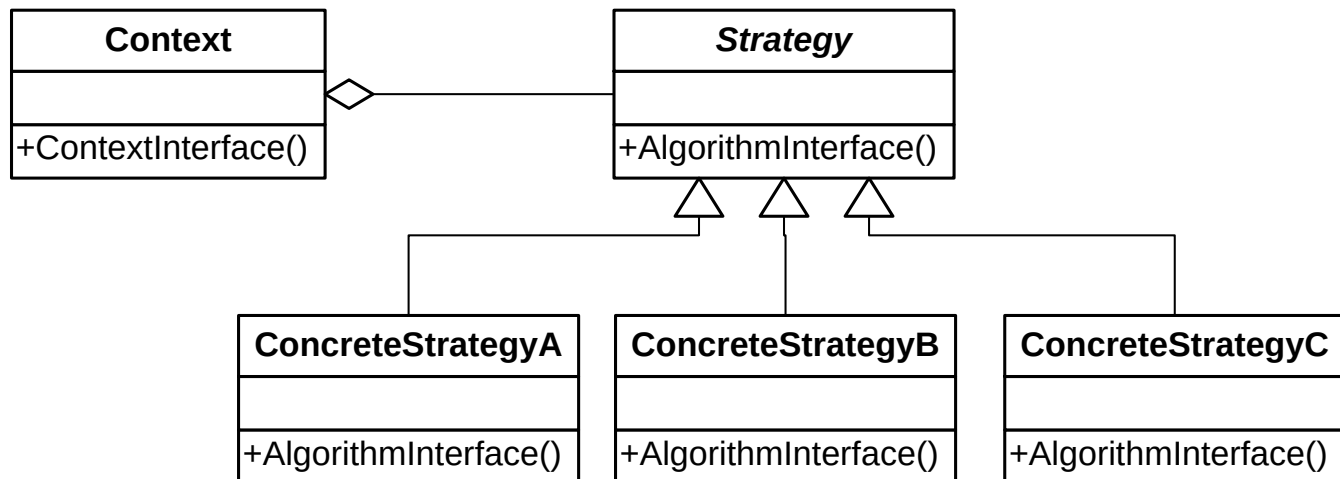
Good



Better!

The Strategy Pattern

- Strategy Pattern
 - Defines a family of algorithms,
 - Encapsulates each one,
 - And makes them interchangeable
- Strategy Pattern lets the algorithm vary independently from clients that use it



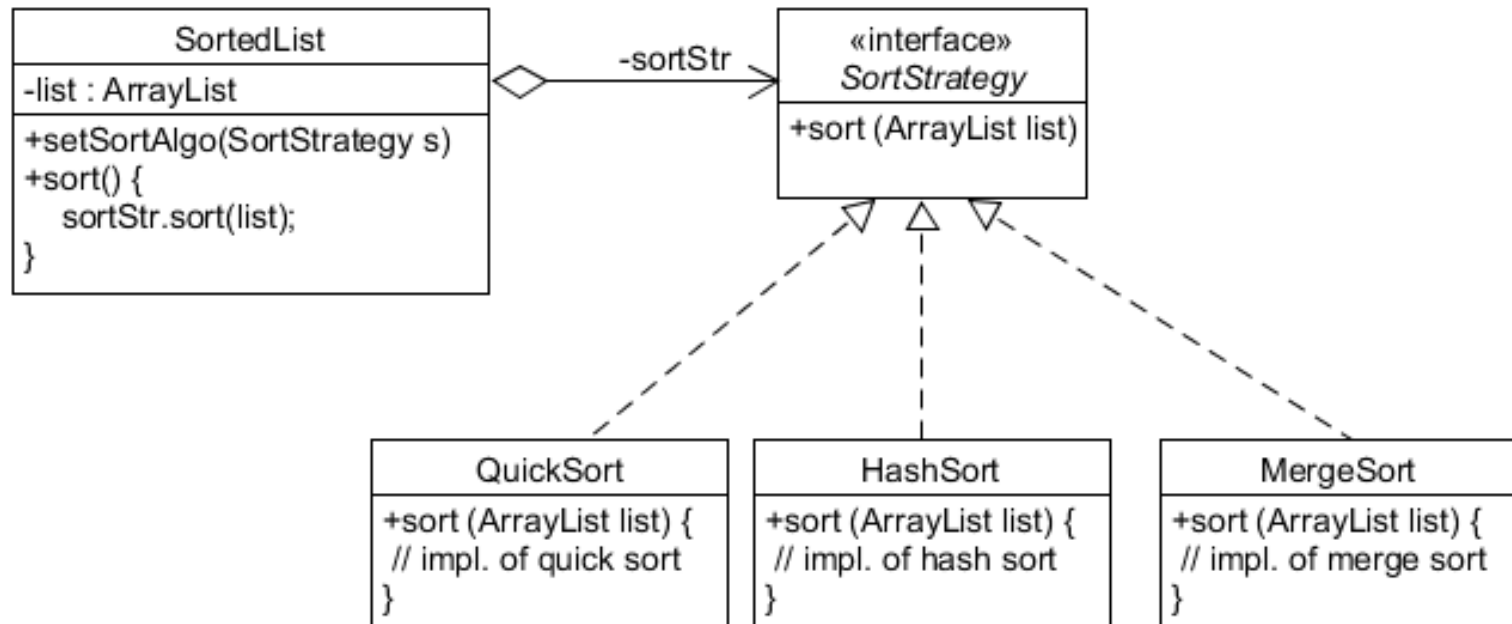
Sorting Example

- **Requirement:** we want to sort a list of integers using different sorting algorithms, e.g. quick sort, selection sort, insertion sort, etc.
- One way to solve this problem is to write a function for each sorting algorithm, e.g.
 - `quicksort(int[] in, int[] res)`
 - `insertionsort(int[] in, int[] res)`
 - `mergesort(int[] in, int[] res)`

A better way is to use the Strategy pattern

Sorting Example

```
void MethodOfClient(ArrayList a1) {  
    SortedList s1 = new SortedList(a1);  
    s1.setSortAlgo( new QuickSort() );  
    s1.sort();  
}
```



Hierarchy of Pattern Knowledge

Design Pattern

Strategy pattern
defines a family of algorithms,
encapsulates each one, and makes
them interchangeable. Strategy lets
the algorithm vary independently
from clients using it

OO Principles

Encapsulate what varies
Favor composition over inheritance
Program to interface, not implementations

OO Basics

Abstraction
Encapsulation
Polymorphism
Inheritance

Design Patterns

- Knowing the OO basics does not make you a good OO designer
- Good OO design are reusable, extensible and maintainable
- Patterns don't give you code, they give you general solutions to design problems
- Most patterns and principles address issues of change in software.

Way to Learn Design Patterns

- Learning how such a pattern was **discovered**
 - When people begin to look at design patterns, they often focus on the solutions the patterns offer. This seems reasonable because they are advertised as providing good solutions to the problems at hand.
 - However, this is starting at the wrong end. When you learn patterns by focusing on the solutions they present, it makes it hard to determine the situations in which a pattern applies. This only tells us *what to do* **but not when to use it or why to do it.**

Summary

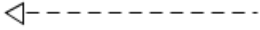
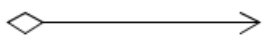
- Design Principles

- Encapsulate what varies
- Program to interface, not implementations
- Favor composition over inheritance

- Strategy pattern

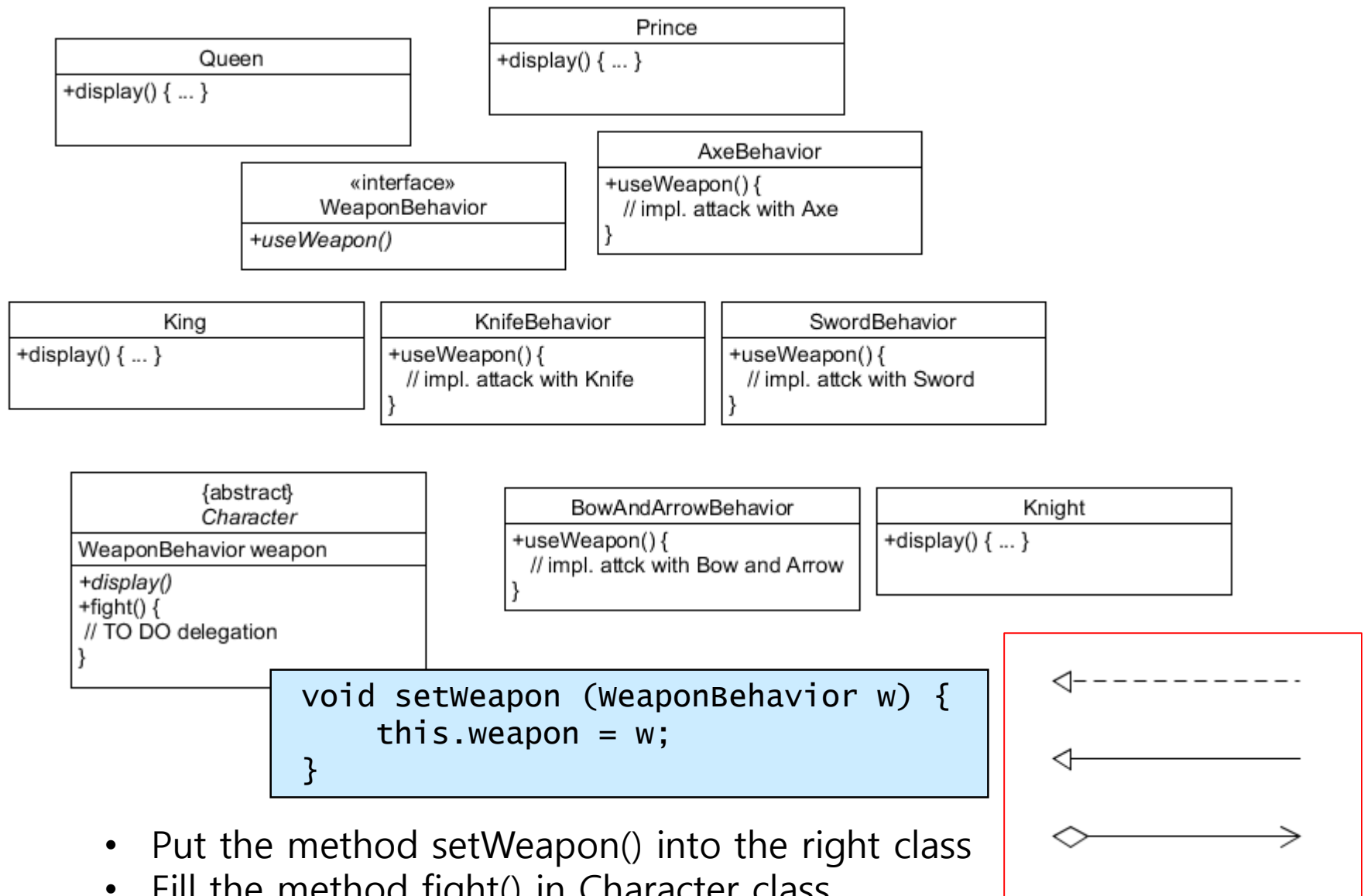
- Lets the algorithm vary independently from clients that use it
- Uses composition & delegation mechanism

Strategy Pattern Quiz: Problem

- ① Arrange the classes.
- ② Identify one abstract class, one interface and eight classes.
- ③ Draw arrows between classes.
 - a. Draw this kind of arrow for inheritance("extends"). 
 - b. Draw this kind of arrow for interface("implements"). 
 - c. Draw this kind of arrow for "HAS-A". 
- ④ Put the method setWeapon() into the right class.

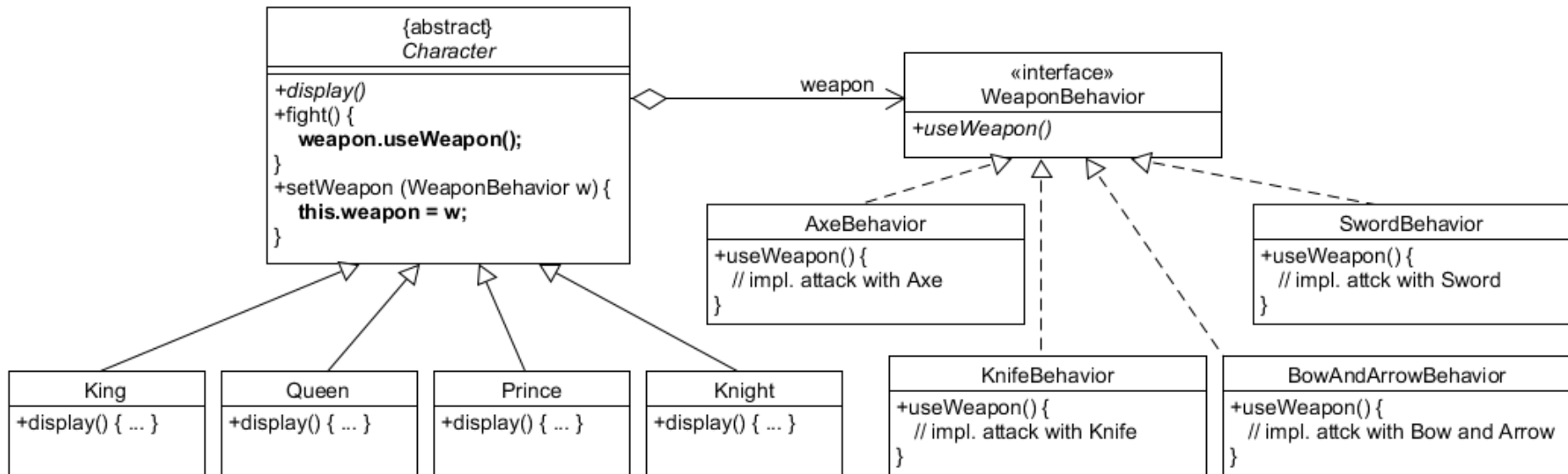
```
void setWeapon (weaponBehavior w) {  
    this.weapon = w;  
}
```
- ⑤ Fill the method fight() in Character class.

Strategy Pattern Quiz: Problem



- Put the method `setWeapon()` into the right class
- Fill the method `fight()` in **Character** class.

Strategy Pattern Quiz: Answer



6. OBSERVER PATTERN

2020. 04. 16

중앙대학교 소프트웨어학부

이찬근 교수

Contents

- Intro. to the problem
- Structure of observer pattern
- Applying observer pattern
- Observer pattern in Java

Observer Pattern

- **Also known as**

- Publish/Subscribe model

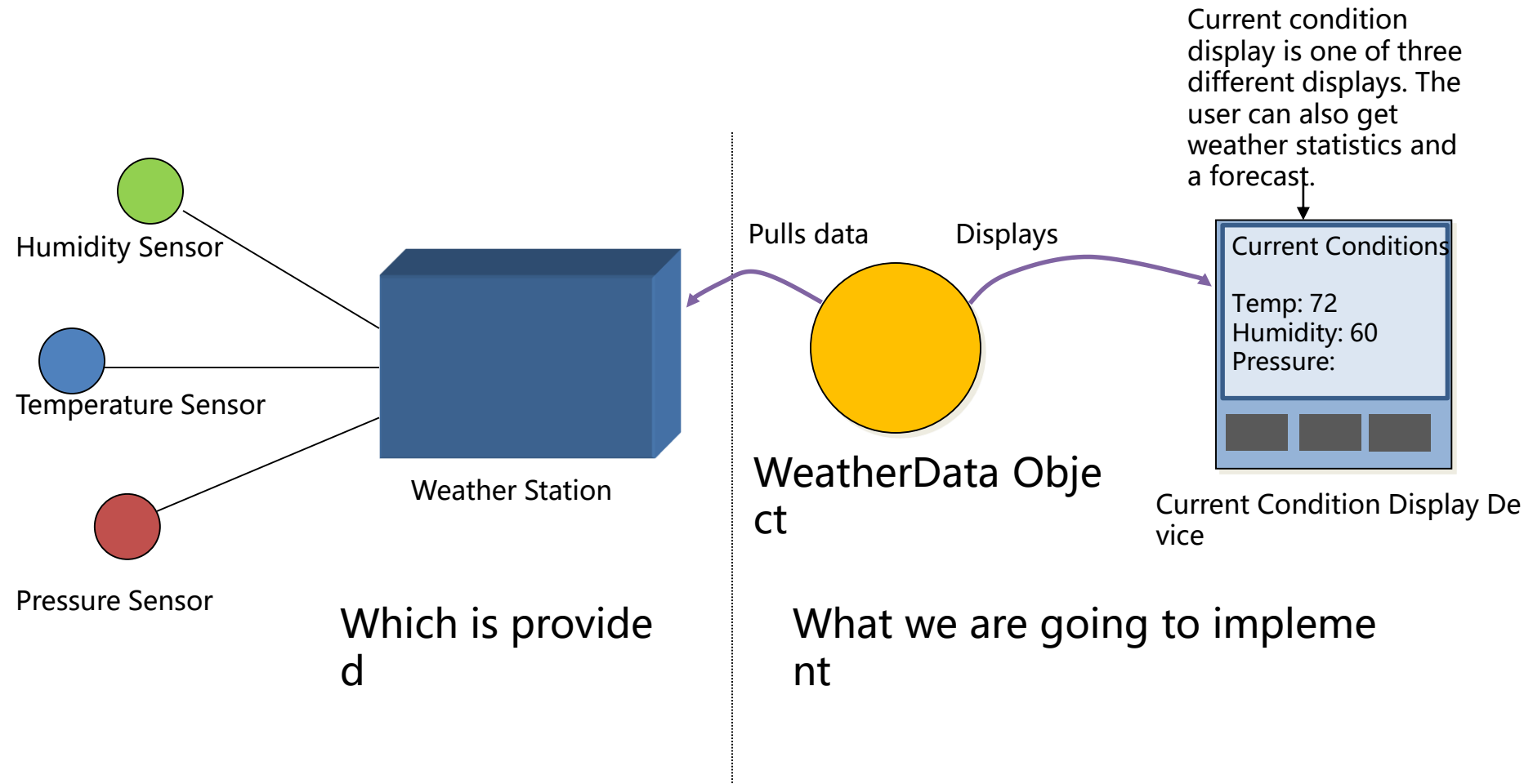
- **Purpose**

- Lets one or more objects be notified of state changes in other objects within the system.

- **Use When**

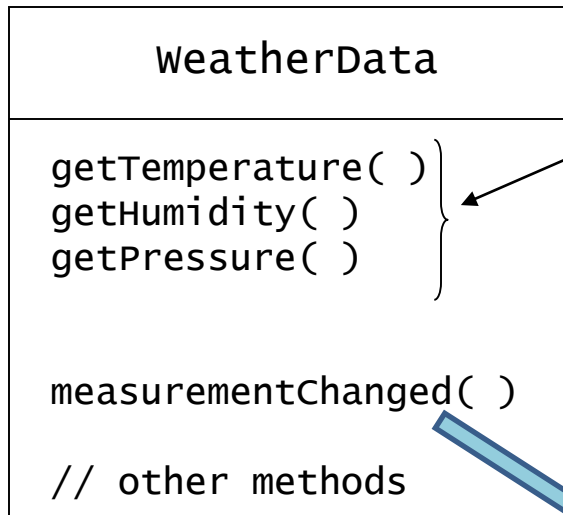
- Loose coupling is needed for communications
 - State changes in one or more objects should trigger behavior in other objects
 - Broadcasting capabilities are required.
 - An understanding exists that objects will be blind to the expense of notification.

The Application Overview



Create an application using the WeatherData object to update the displays.

The WeatherData class



These three methods return the most recent weather measurements for temperature, humidity, and pressure respectively.

We don't care how these variables are set; the `WeatherData` object knows how to get updated information from the Weather Station

```
/* This method gets called whenever the  
 * measurements have been updated. * /
```

```
public void measurementsChanged( ) {  
    // Your code goes here  
}
```

What do we know?

- Display elements that use the weather data:
 - Current conditions display
 - Statistics display
 - Forecast display
- They must be updated each time WeatherData has new measurements
- System must be expandable
 - other developers can create new custom display elements and users can add or remove as many display elements as they want to the application.
 - Currently, we know about only the initial three display types: current conditions, statistics, and forecast

The First Attempt

```
public class WeatherData {  
    // instance variable declarations  
  
    public void measurementsChanged() {  
        float temp = getTemperature();  
        float humidity = getHumidity();  
        float pressure = getPressure();  
  
        currentConditionDisplay.update(temp, humidity, pressure);  
        statisticsDisplay.update(temp, humidity, pressure);  
        forecastDisplay.update(temp, humidity, pressure);  
    }  
  
    // other WeatherData methods here  
}
```

The First Attempt

```
public class WeatherData {  
    // instance variable declarations  
  
    public void measurementsChanged() {  
        float temp = getTemperature();  
        float humidity = getHumidity();  
        float pressure = getPressure();  
  
        currentConditionDisplay.update(temp, humidity, pressure);  
        statisticsDisplay.update(temp, humidity, pressure);  
        forecastDisplay.update(temp, humidity, pressure);  
    }  
  
    // other WeatherData methods here  
}
```

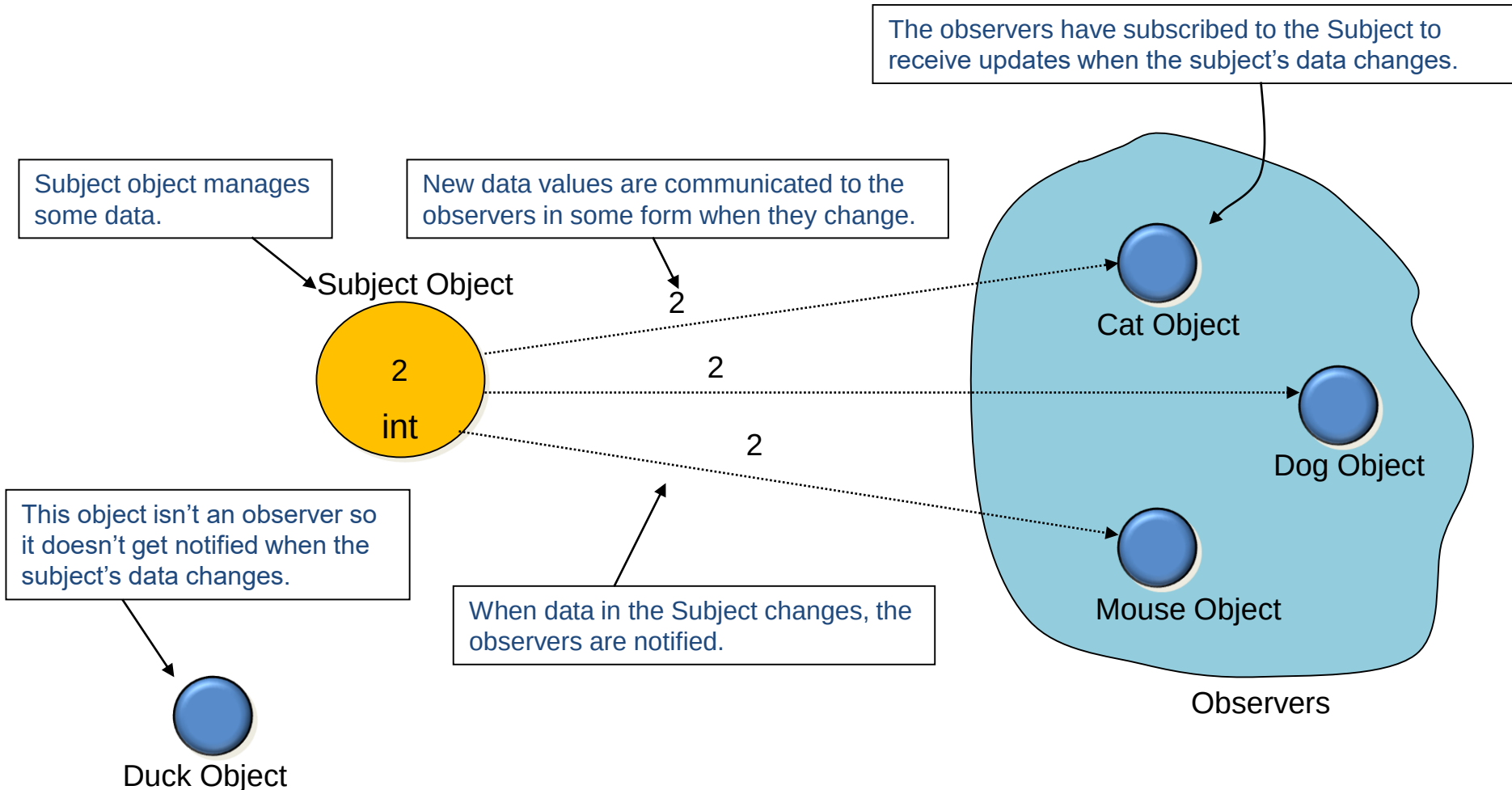
By coding to concrete implementations we have no way to add or remove other display elements without making changes to the program.

Meet the Observer Pattern

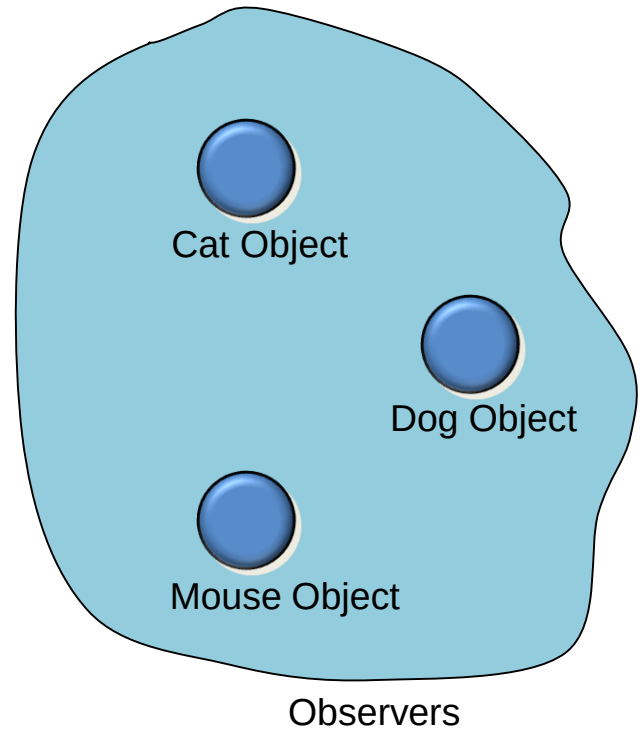
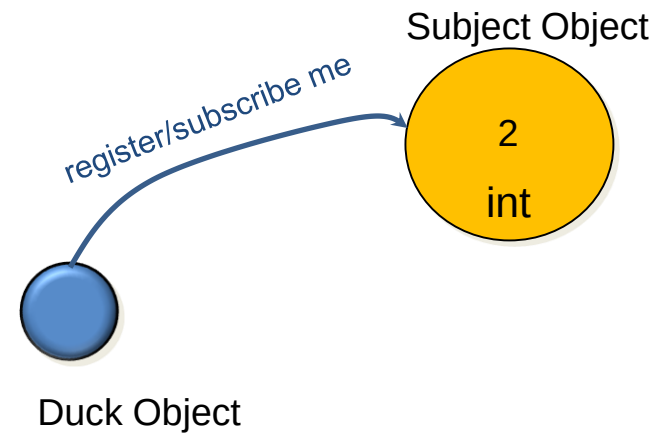
- A news paper **publisher** goes into business and begins publishing newspapers
- You **subscribe** to a particular publisher
 - Every time there's new edition it gets delivered to you. As long as you remain a subscriber; you get new newspapers
- You **unsubscribe** when you don't want papers anymore
 - They stop delivery
- While the publisher remains in business, people, hotels, airlines and other business **constantly subscribe and unsubscribe** to the newspaper.

Observer Pattern

- Publisher & Subscriber Model
 - Publisher : Subscriber = Subject : Observer

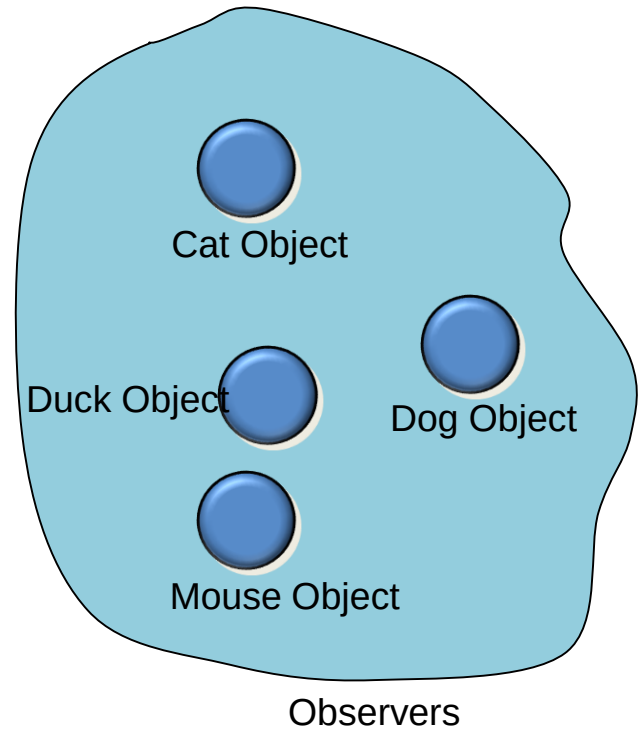
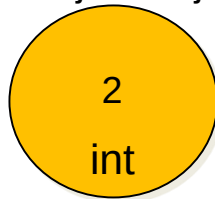


Duck wants to subscribe

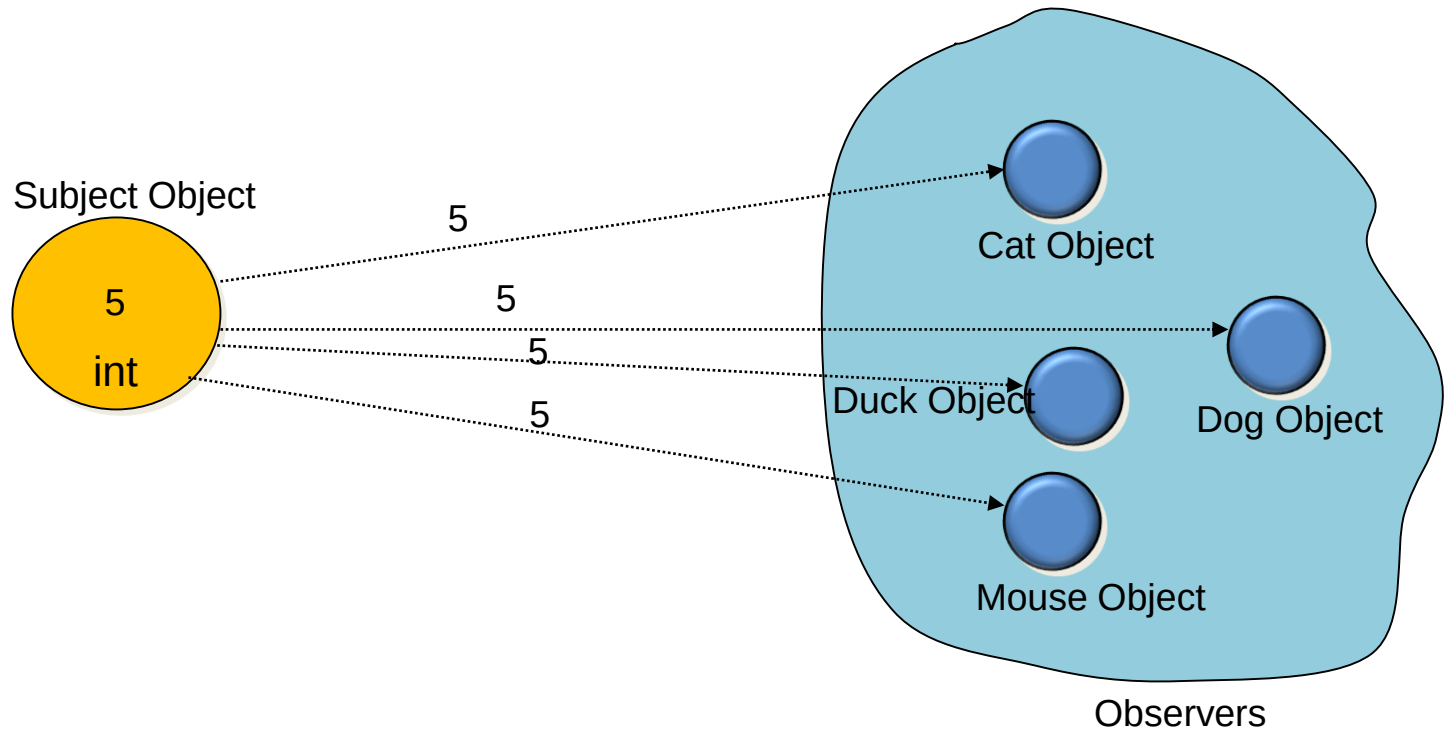


Duck is now an observer, too

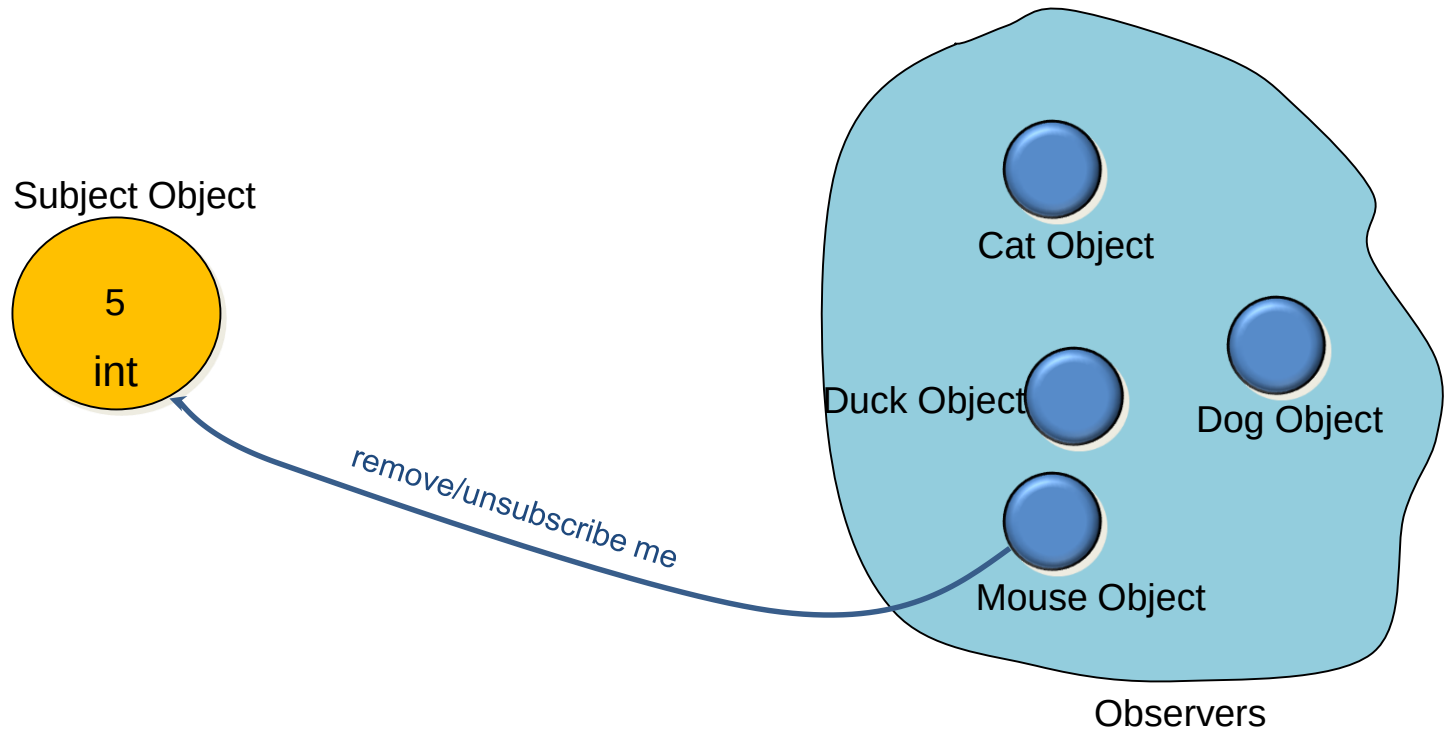
Subject Object



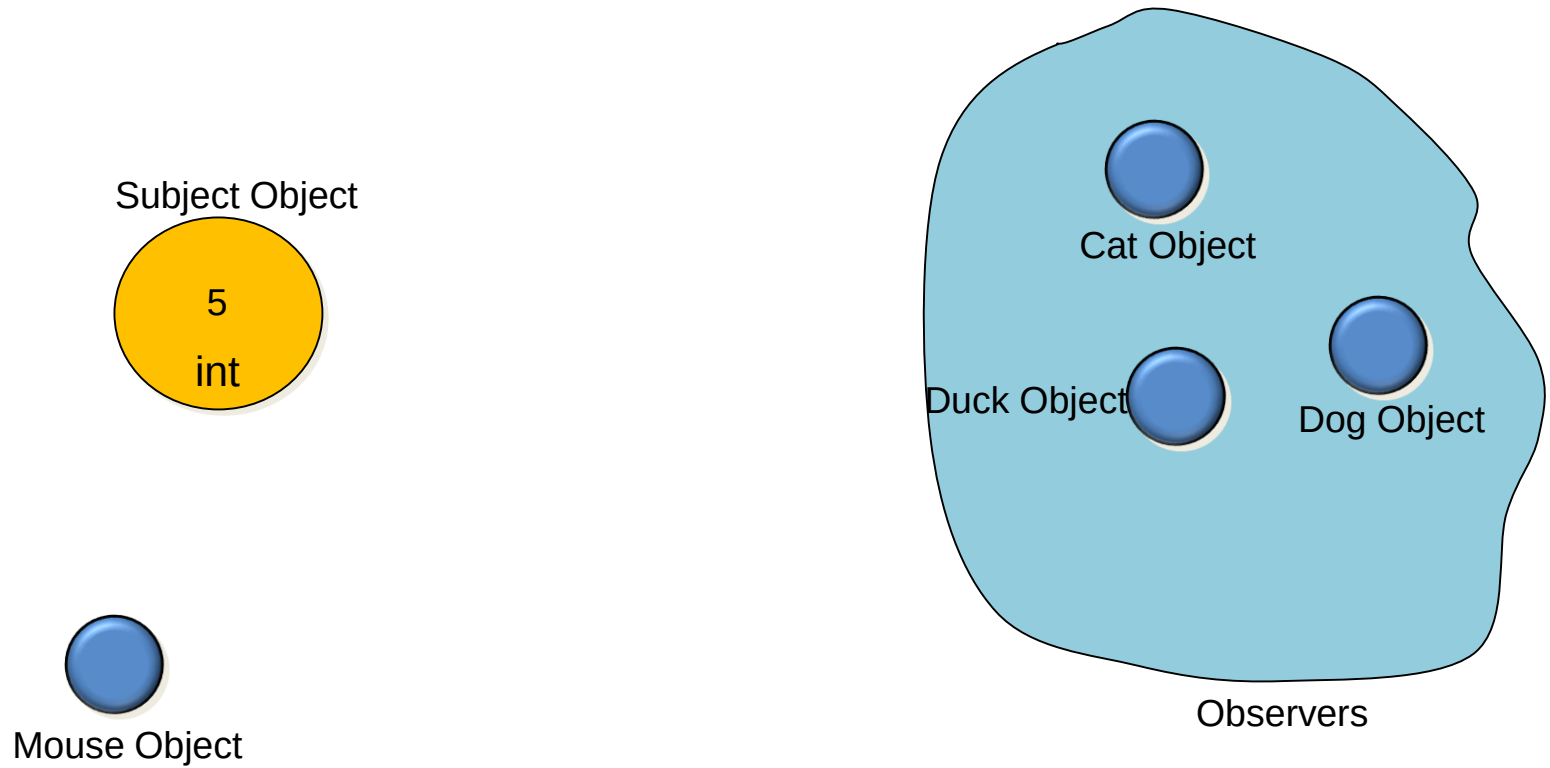
Notifying the observers



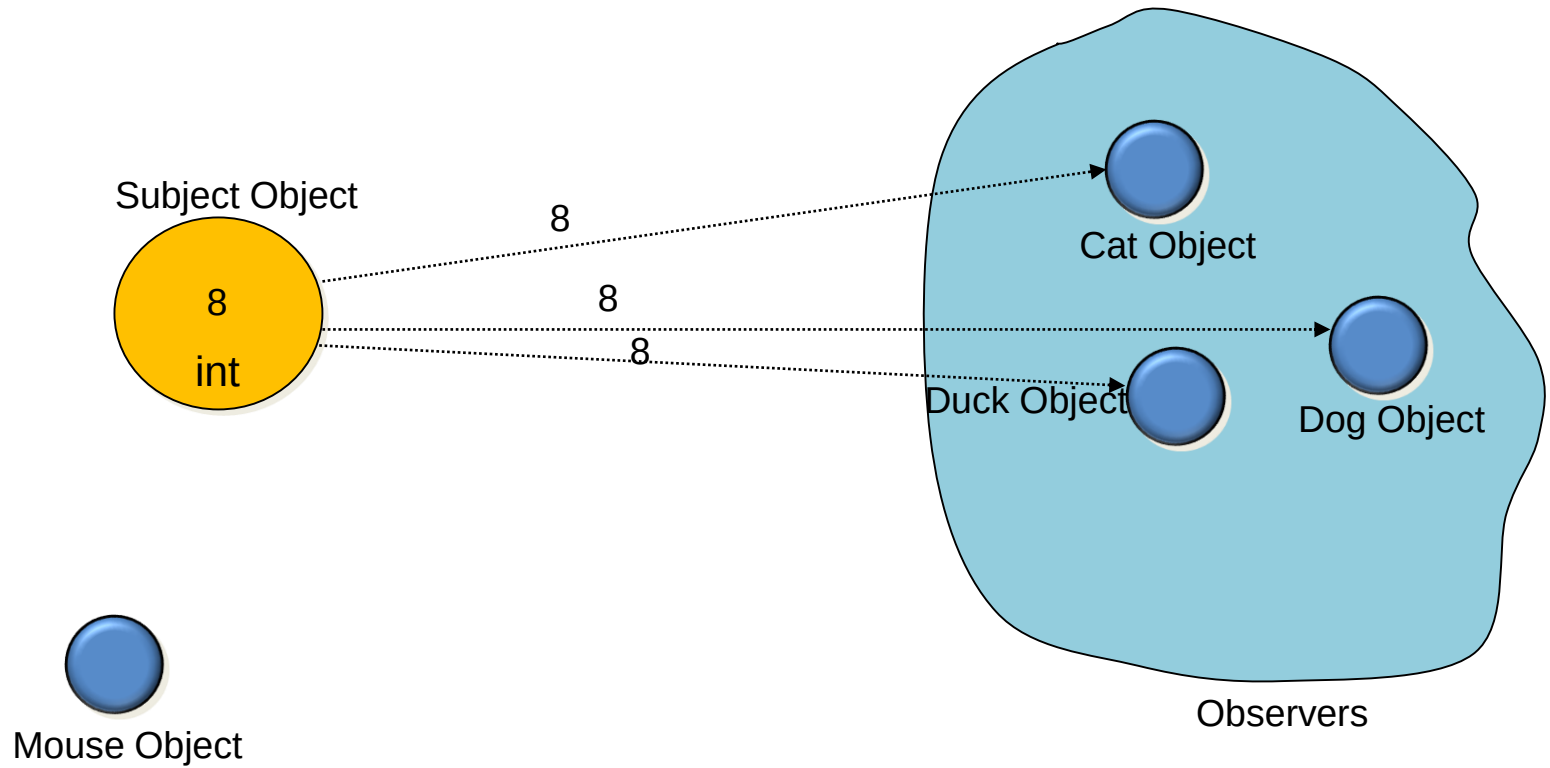
Mouse wants to unsubscribe



Mouse wants to unsubscribe



Notifying the observers

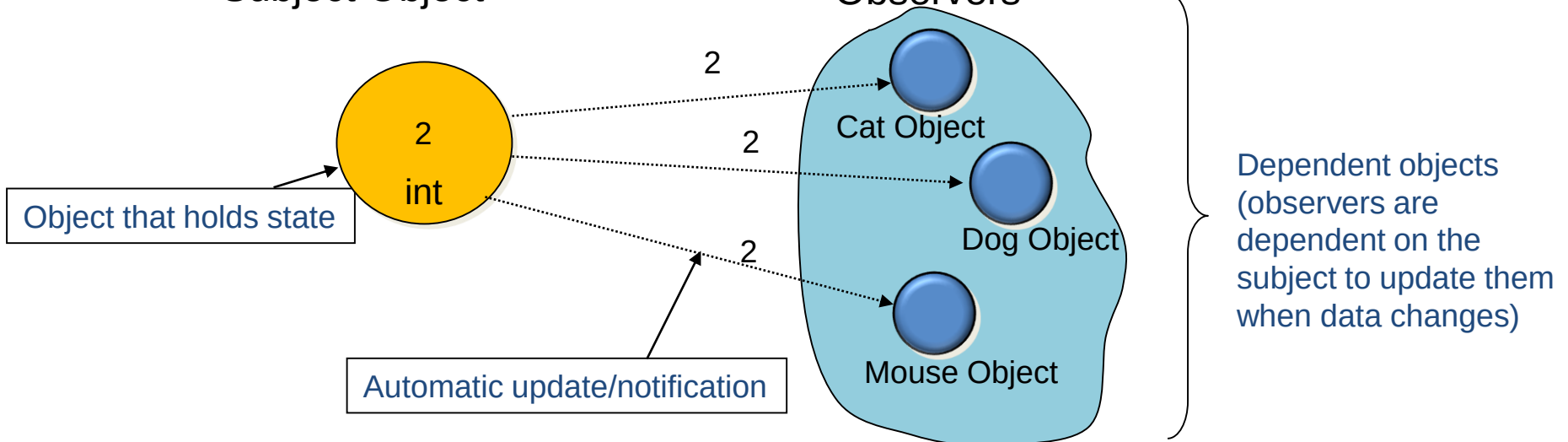


The Observer Pattern Defined

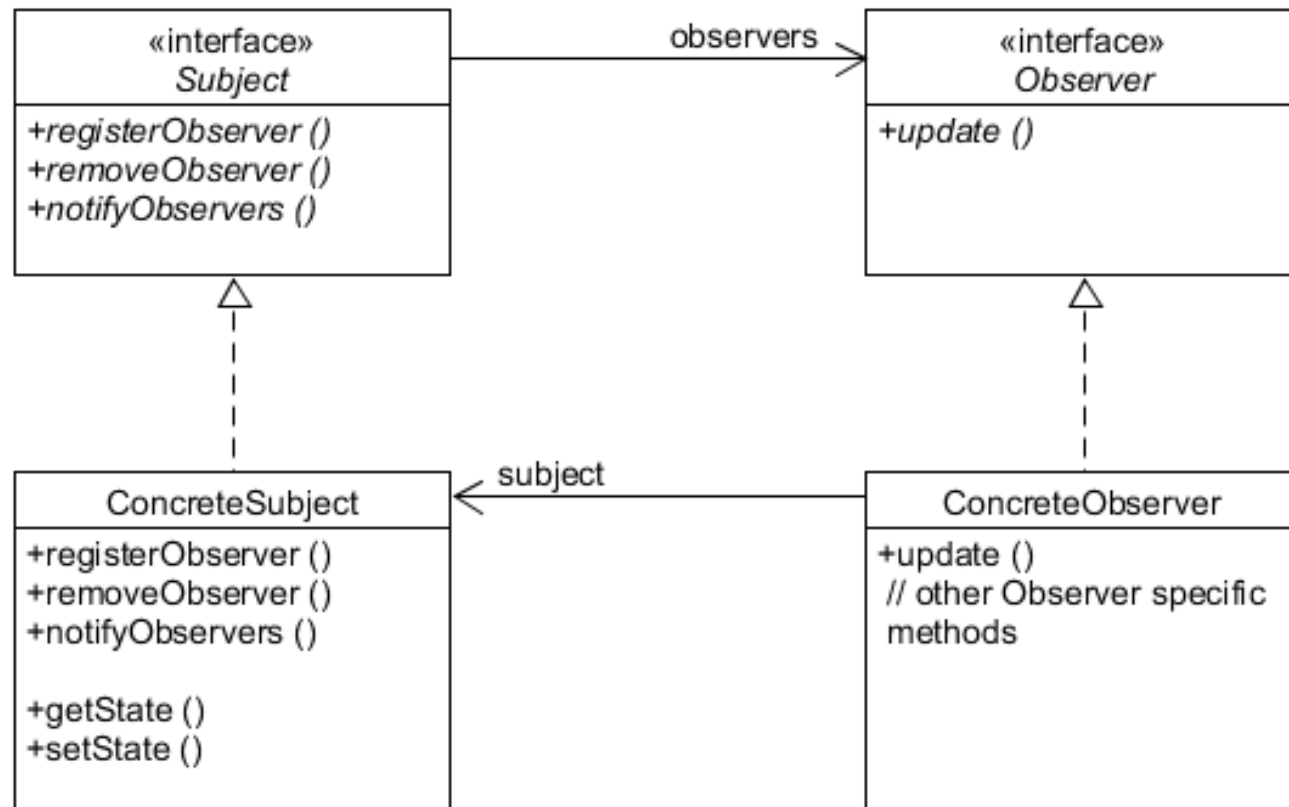
The **Observer Pattern** defines a one-to-many dependency between objects so that when one object changes state, all of its dependents are notified and updated automatically.

One to many relationship (Subject can have many observers)

Subject Object → Observers



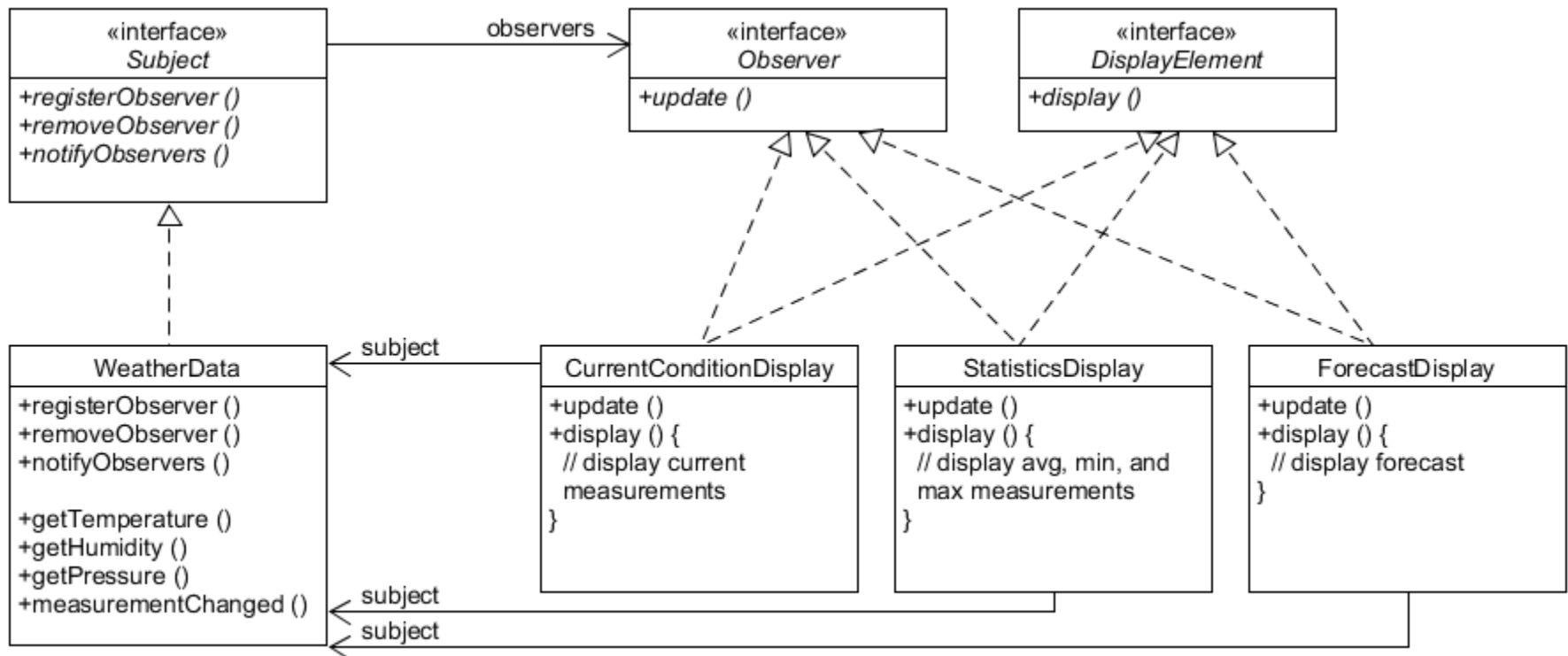
Class Diagram for Observer Pattern



The Power of Loose Coupling

- Principle of loose coupling
 - Strive for loosely coupled designs between object that interact
- When two objects are **loosely coupled**, they can interact, but have very **little knowledge of each other**
- Observer Pattern provides an object design where subjects and observers are loosely coupled

Designing the Weather Station



Preparing Interfaces

```
public interface Subject {  
    public void registerObserver(Observer o);  
    public void removeObserver(Observer o);  
    public void notifyObservers();  
}  
  
public interface Observer {  
    public void update(float temp, float humidity, float pressure);  
}  
  
public interface DisplayElement {  
    public void display();  
}
```

Implementing WeatherData

```
public class WeatherData implements Subject {
    private ArrayList observers;
    private float temperature;
    private float humidity;
    private float pressure;

    public WeatherData() {
        observers = new ArrayList();
    }

    public void registerObserver(Observer o) {
        observers.add(o);
    }

    public void removeObserver(Observer o) {
        int i = observers.indexOf(o);
        if(i >= 0) observers.remove(i);
    }

    public void notifyObservers() {
        for(int i = 0; i < observers.size(); i++) {
            Observer observer = (Observer)observers.get(i);
            observer.update(temperature, humidity, pressure);
        }
    }
}
```

Implementing WeatherData (cont')

```
public void measurementsChanged() {  
    notifyObservers();  
}  
  
public void setMeasurements(float temperature, float humidity,  
                             float pressure) {  
    this.temperature = temperature;  
    this.humidity = humidity;  
    this.pressure = pressure;  
    measurementsChanged();  
}  
  
// other WeatherData methods here  
}
```

Implementing CurrentConditionDisplay

```
public class CurrentConditionDisplay implements Observer, DisplayElement {
    private float temperature;
    private float humidity;
    private Subject weatherData;

    public CurrentConditionDisplay(Subject weatherData) {
        this.weatherData = weatherData;
        weatherData.registerObserver(this);
    }

    public void update(float temperature, float humidity, float pressure) {
        this.temperature = temperature;
        this.humidity = humidity;
        display();
    }

    public void display() {
        System.out.println("Current conditions: " + temperature
            + "F degrees and" + humidity + "% humidity");
    }
}
```

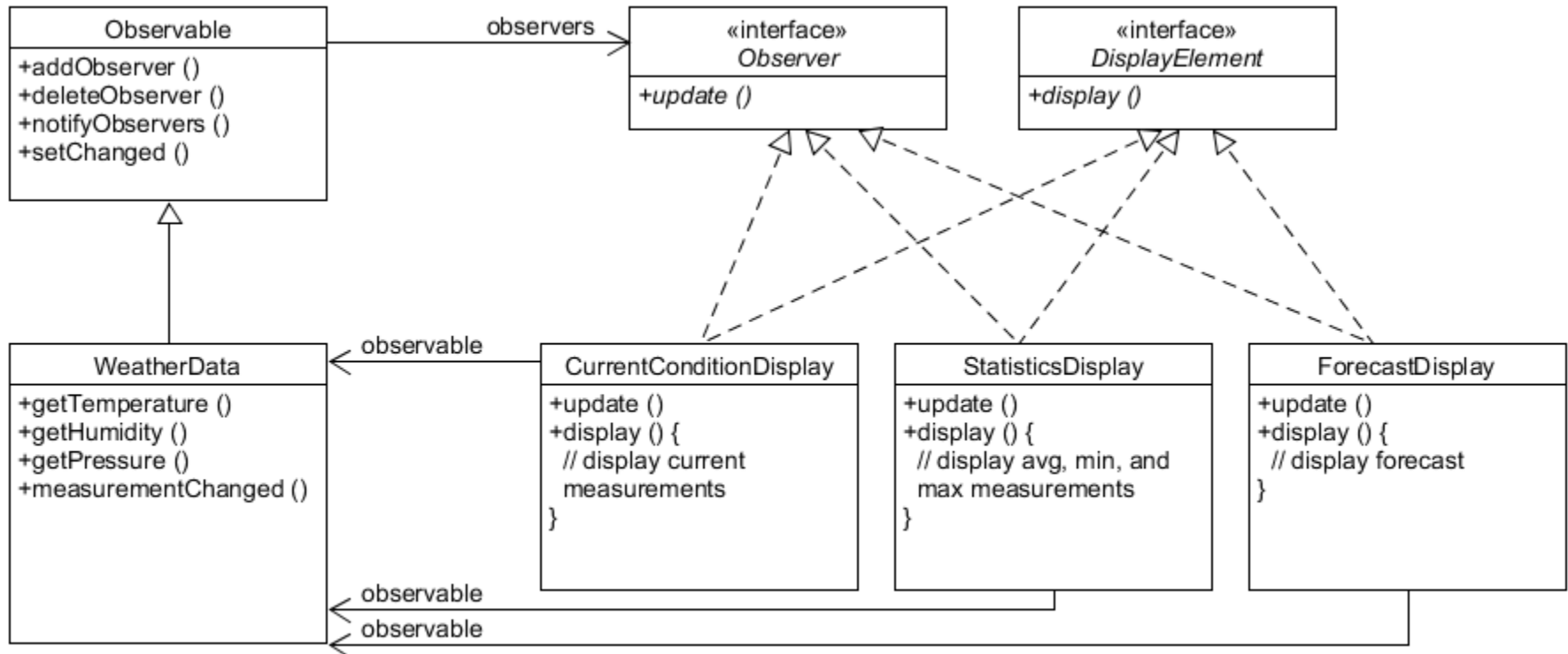
Implementing the Weather Station and Output

```
public class WeatherStation {  
    public static void main(String[] args) {  
        WeatherData weatherData = new WeatherData();  
  
        CurrentConditionsDisplay currentDisplay =  
            new CurrentConditionsDisplay(weatherData);  
        StatisticsDisplay statisticsDisplay =  
            new StatisticsDisplay(weatherData);  
        ForecastDisplay forecastDisplay =  
            new ForecastDisplay(weatherData);  
  
        weatherData.setMeasurements(80, 65, 30.4f);  
        weatherData.setMeasurements(82, 70, 29.2f);  
        weatherData.setMeasurements(78, 90, 29.2f);  
    }  
}
```

Current conditions: 80.0F degrees and 65.0% humidity
Avg/Max/Min temperature = 80.0/80.0/80.0
Forecast: Improving weather on the way!

Current conditions: 82.0F degrees and 70.0% humidity
Avg/Max/Min temperature = 81.0/82.0/80.0
Forecast: Watch out for cooler, rainy weather
Current conditions: 78.0F degrees and 90.0% humidity
Avg/Max/Min temperature = 80.0/82.0/78.0
Forecast: More of the same

Using Java's Official Observable Class



How to Use Observable Class

- Observable (Publisher) side
 - To send notifications
 - Call `SetChanged()`
 - Call either `notifyObservers()` or `notifyObservers(Object arg)`

- Observer (Subscriber) side
 - To become an observer
 - Implement `java.util.Observer` interface
 - To subscribe
 - Call `addObserver()` on any Observable object
 - To unsubscribe
 - Call `deleteObserver()`
 - To get updated
 - Implement `update(Observable o, Object arg)`

Internals of setChanged()

```
setChanged() {  
    changed = true;  
}  
  
notifyObservers(Object arg) {  
    if (changed) {  
        for every observer on the list {  
            call update (this, arg);  
        }  
        changed = false;  
    }  
}  
  
notifyObservers() {  
    notifyObservers(null);  
}
```

Why is the `setChanged ( )` necessary?

- `setChanged ( )` method is there to give you more flexibility
 - Optimize the notifications
 - Example:
 - If the sensor reading is very sensitive, you may not want to update the observers with every minor fluctuation
 - `setChanged ( )` allows you to control the notification points
- Other relevant methods in `Observable`
 - `clearChanged ( )`
 - `hasChanged ( )`

Observable side

```
import java.util.Observable;
import java.util.Observer;

public class WeatherData extends Observable {
    private float temperature;
    private float humidity;
    private float pressure;

    public WeatherData() {}
    public void measurementsChanged() {
        setChanged();
        notifyObservers();
    }
    public void setMeasurements(float temperature, float humidity, float pressure) {
        this.temperature = temperature;
        this.humidity = humidity;
        this.pressure = pressure;
        measurementsChanged();
    }
    public float getTemperature() {
        return temperature;
    }
    public float getHumidity() {
        return humidity;
    }
    public float getPressure() {
        return pressure;
    }
}
```

Observer side

```
import java.util.Observable;
import java.util.Observer;

public class CurrentConditionsDisplay implements Observer, DisplayElement {
    Observable observable;
    private float temperature;
    private float humidity;

    public CurrentConditionsDisplay(Observable observable) {
        this.observable = observable;
        observable.addObserver(this);
    }

    public void update(Observable obs, Object arg) {
        if(obs instanceof WeatherData) {
            WeatherData weatherData = (WeatherData)obs;
            this.temperature = weatherData.getTemperature();
            this.humidity = weatherData.getHumidity();
            display();
        }
    }

    public void display() {
        System.out.println("Current conditions: " + temperature
            + "F degrees and" + humidity + "% humidity");
    }
}
```

Summary

- Principle of loose coupling
 - Strive for loosely coupled designs between objects that interact
- Observer Pattern
 - Observer defines a one-to-many dependency between objects so that when one object changes state, all its dependents are notified and updated automatically

Observer Pattern Quiz : Problem

- Observer 패턴에서 약한 커플링(loose coupling)이 어떻게 달성되는지 간략히 설명하십시오.
- Observer 패턴을 사용할 때, Publisher 역할을 수행하는 객체가 여러 개 있을 수 있는가?

Observer Pattern Quiz : Answer

- Observer 패턴에서 약한 커플링(loose coupling)이 어떻게 달성되는지 간략히 설명하십시오.
- When **two objects know very little of each other** but are able to interact, they are considered **loosely coupled**. The observer pattern is a good example of loose coupling.
- **The only thing the subject knows about observers is that they implement a certain interface**. This is good because changes to either the subject or the observer will not affect the other. It is possible to add new types of observers at any time while never making any changes to the subject.

Observer Pattern Quiz : Answer

- Observer 패턴을 사용할 때, Publisher 역할을 수행하는 객체가 여러 개 있을 수 있는가?

- 가능함. Observer 패턴에 참여하고 있는 복수개의 클래스들이 Publisher의 역할을 할 수 있으며, Subscriber들은 원하는 복수개의 Publisher에 구독 신청을 할 수 있다. 또한 어떤 클래스는 Publisher이면서 다른 Publisher의 Subscriber가 될 수도 있다.

- Example

```
public class Publisher_A extends Observable implements Observer {  
    public void update(Observable obs, Object arg) { ... }  
  
    ...  
}
```

7. TEMPLATE METHOD PATTERN

2020. 04. 16

중앙대학교 소프트웨어학부

이찬근 교수

Contents

- Template method pattern in real-life
- Structure of template method pattern

Template Method Pattern

- **Purpose**

- Identifies the framework of an algorithm, allowing implementing classes to define the actual behavior.

- **Use When**

- A single abstract implementation of an algorithm is needed.
 - Common behavior among subclasses should be localized to a common class.
 - Parent classes should be able to uniformly invoke behavior in their subclasses.
 - Most or all subclasses need to implement the behavior.

Class Coffee

```
public class Coffee {  
    void prepareRecipe() {  
        boilWater();  
        brewCoffeeGrinds();  
        pourInCup();  
        addSugarAndMilk();  
    }  
    public void boilWater() {  
        System.out.println("Boiling water");  
    }  
    public void brewCoffeeGrinds() {  
        System.out.println("Dripping Coffee through filter");  
    }  
    public void pourInCup() {  
        System.out.println("Pouring into cup");  
    }  
    public void addSugarAndMilk() {  
        System.out.println("Adding Sugar and Milk");  
    }  
}
```

Class Tea

```
public class Tea {  
    void prepareRecipe() {  
        boilWater();  
        steepTeaBag();  
        pourInCup();  
        addLemon();  
    }  
    public void boilWater() {  
        Sysetm.out.println("Boiling water");  
    }  
    public void steepTeaBag() {  
        Sysetm.out.println("Steeping the tea");  
    }  
    public void pourInCup() {  
        System.out.println("Pouring into cup");  
    }  
    public void addLemon() {  
        System.out.println("Adding Lemon");  
    }  
}
```

Problems with the original design

- Code is duplicated across the classes – code changes would have to be made in more than one place.
- Adding a new beverage would result in further duplication.
- Knowledge of the algorithm and implementation is distributed over classes.

Abstracting prepareRecipe()

```
public abstract class CaffeineBeverage {  
    final void prepareRecipe() {  
        boilWater();  
        brew();  
        pourInCup();  
        addCondiments();  
    }  
    abstract void brew();  
    abstract void addCondiments();  
  
    public void boilWater() {  
        System.out.println("Boiling water");  
    }  
    public void pourInCup() {  
        System.out.println("Pouring into cup");  
    }  
}
```

Coffee

```
void prepareRecipe() {  
    boilWater();  
    brewCoffeeGrinds();  
    pourInCup();  
    addSugarAndMilk();  
}
```

Tea

```
void prepareRecipe() {  
    boilWater();  
    steepTeaBag();  
    pourInCup();  
    addLemon();  
}
```



Rewriting Coffee and Tea

```
public class Coffee extends CaffeineBeverage {
    public void brew() {
        Sysetm.out.println("Dripping Coffee through filter");
    }
    public void addCondiments() {
        System.out.println("Adding Sugar and Milk");
    }
}

public class Tea extends CaffeineBeverage {
    public void brew() {
        Sysetm.out.println("Steeping the tea");
    }
    public void addCondiments() {
        System.out.println("Adding Lemon");
    }
}
```

More General Approach

- Changes

- Both subclasses inherit a **general algorithm**.
- Some methods in the algorithm are **concrete**, i.e. methods that perform the **same actions for all subclasses**.
- Other methods in the algorithm are **abstract**, i.e. methods that perform **class-specific actions**.

- Advantages

- A **single class protects and controls the algorithm**, namely, CaffeineBeverage.
- The **superclass facilitates reuse** of methods.
- **Code changes** will occur in **only one place**.
- Other beverages can be **easily added**.

Template Method

```
public abstract class CaffeineBeverage {
```

```
    void final prepareRecipe() {  
        boilWater();  
        brew();  
        pourInCup();  
        addCondiments();  
    }
```

```
        abstract void brew();  
        abstract void addCondiments();
```

```
    void boilWater() {  
        // implementation  
    }
```

```
    void pourInCup () {  
        // implementation  
    }
```

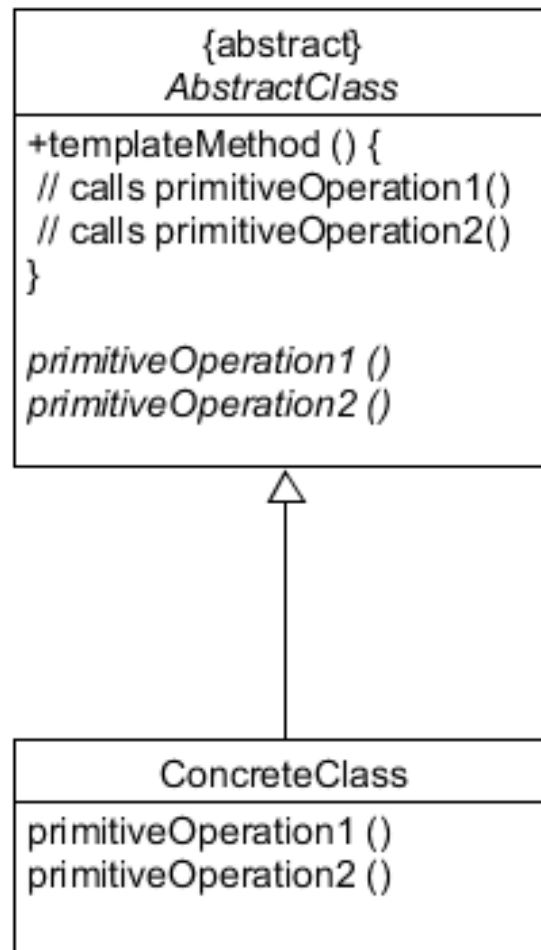
```
}
```

Template
Method

Template Method Pattern

- *prepareRecipe()* implements the **template method pattern**.
 - serves as a **template for an algorithm**, namely that for making a caffeinated beverage.
 - In the template, **each step is represented by a method**.
 - Some methods are implemented in the superclass.
 - **Other method must be implemented by the subclass and are declared abstract**.
- The **template pattern** defines the steps of an algorithm and **allows the subclasses to implement** one or more of the **steps**.

Template Method Pattern



Template Method Pattern

- **Encapsulates** an algorithm by **creating a template** for it.
- Defines the **skeleton of an algorithm** as a **set of steps**.
- Some methods of the algorithm **have to be implemented by the subclasses** – these are abstract methods in the super class.
- The **subclasses** can **redefine certain steps** of the algorithm **without changing the algorithm's structure**.
- Some steps of the algorithm are **concrete methods** defined in the super class.

Hook Method

- A **hook** is a method that is declared in the abstract class, but only given an empty or default implementation.
 - Gives the subclasses the ability to “*hook into*” the algorithm at various points, if they wish; they can ignore the hook as well.

```
public abstract class CaffeineBeverage {
    final void prepareRecipe() {
        boilWater();
        brew();
        pourInCup();
        if (customerWantsCondiments()) addCondiments();
    }
    abstract void brew();
    abstract void addCondiments();
    public void boilWater() {
        System.out.println("Boiling water");
    }
    public void pourInCup() {
        System.out.println("Pouring into cup");
    }
    boolean customerWantsCondiments() {
        return true;
    }
}
```


Using the hook in the derived class

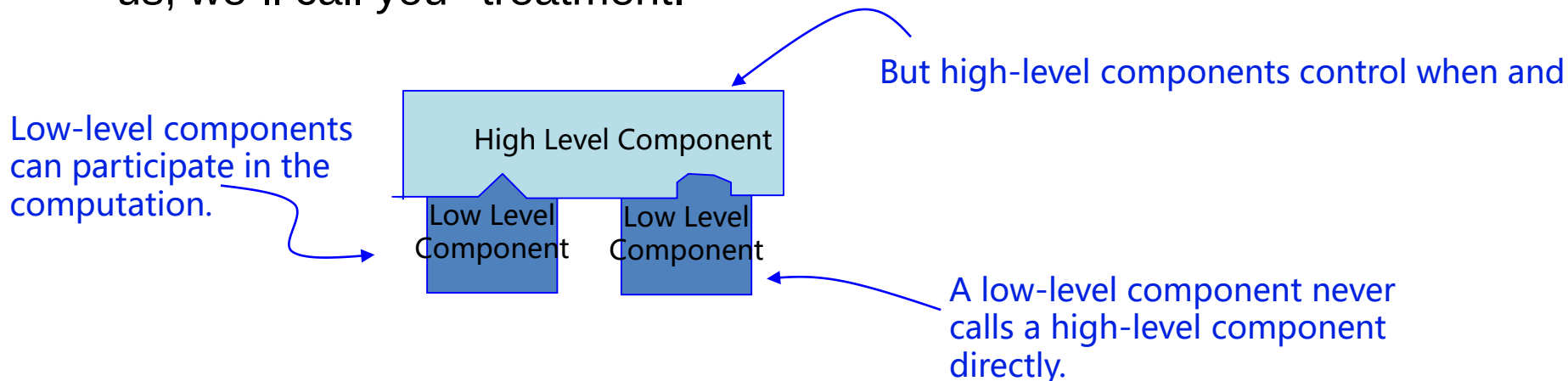
```
public class Coffee extends CaffeineBeverage {
    public void brew() {
        System.out.println("Dripping Coffee through filter");
    }
    public void addCondiments() {
        System.out.println("Adding Sugar and Milk");
    }
    public boolean customerWantsCondiments() {
        String answer = getUserInput();
        if (answer.toLowerCase().startsWith("y_"))
            return true;
        else
            return false;
    }
}
```

Examples of Hooks in the Java API

- JFrame hooks
 - paint()
- Applet hooks
 - init()
 - repaint()
 - start()
 - stop()
 - destroy()
 - paint()

Design Principle: Hollywood Principle

- The Hollywood Principle: Don't call us, we'll call you!
 - It prevents “Dependency rot”
 - Dependency rot: high-level components depend on low-level components, and vice versa.
- With the Hollywood principle
 - We allow low level components to hook themselves into a system
 - But high level components determine when they are needed and how.
 - High level components give the low-level components a “don't call us, we'll call you” treatment.



Related Patterns

- Template Method uses inheritance to vary part of an algorithm.
- Strategy uses delegation to vary the entire algorithm.
- Factory Method is a specialization of Template Method

Summary

- Hollywood Principle

- Don't call us, we'll call you

- Template Method Pattern

- Define the skeleton of an algorithm in an operation, deferring some steps to subclasses.
 - Template Method lets subclasses redefine certain steps of an algorithm without changing the algorithm's structure.

Template Method Pattern Quiz : Problem

- Template Method 패턴은 class scope의 패턴인가? 아니면 object scope의 패턴인가?
- Template Method 패턴과 Strategy 패턴 모두 알고리즘의 다양성을 높 이는데 활용될 수 있는데, 어떤 차이가 있는가?

Template Method Pattern Quiz : Answer

- Template Method 패턴은 class scope의 패턴인가? 아니면 object scope의 패턴인가?

		Purpose		
		Defer object creation to another class or object	Describe ways to assemble objects	Address problems of assigning responsibilities
		Creational	Structural	Behavioral
Scope	Class	Factory Method	Adapter	Interpreter Template
	Object	Abstract Factory Builder Prototype Singleton	Adapter Bridge Composite Decorator Façade Flyweight Proxy	CoR Command Iterator Mediator Memento Observer State Strategy Visitor

Template Method Pattern Quiz : Answer

- Template Method 패턴과 Strategy 패턴 모두 알고리즘의 다양성을 높이는데 활용될 수 있는데, 어떤 차이가 있는가?
- **Template Method** uses inheritance to vary part of an algorithm.
- **Strategy** uses delegation to vary the entire algorithm.

8. ITERATOR PATTERN

2020. 04. 16

중앙대학교 소프트웨어학부

이찬근 교수

Contents

- Iterator pattern discovery
- Structure of Iterator pattern
- Iterator in Java

The Iterator Pattern

- **Also Known As**

- Cursor

- **Purpose**

An *aggregate object* is an object that contains other objects for the purpose of grouping those objects as a unit.

It is also called a *container* or a *collection*.

- Allows for access to the elements of an aggregate object without allowing access to its underlying representation.

- **Use When**

- Access to elements is needed without access to the entire representation.
 - Multiple or concurrent traversals of the elements are needed.
 - A uniform interface for traversal is needed.
 - Subtle differences exist between the implementation details of various iterators.

Breaking news.....

- The Objectville Diner and Pancake House have merged!
- Menus must be merged. Owners agree on the implementation of the MenuItem.

```
public class MenuItem {  
    String name;  
    String description;  
    boolean vegetarian;  
    double price;  
    public MenuItem (String name, String description, boolean vegetarian,  
                     double price ) {  
        // code here  
    }  
    // set of getter methods to get access to the fields.  
}
```

Implementation of PancakeHouseMenu

```
public class PancakeHouseMenu {
    ArrayList menuItems;
    public PancakeHouseMenu ( ) {
        menuItems = new ArrayList ();
        addItem ("Regular Pancake Breakfast",
                  "Pancakes with fried eggs, sausage", false, 2.99);
        addItem ("Blueberry pancakes",
                  "Pancakes made with fresh blueberries, true, 3.49);
        //other items
    }
    public void addItem (String name, String description, boolean vegetarian, double price) {
        MenuItem menuItem = new MenuItem (name, description, vegetarian, price);
        menuItems.add(menuItem);
    }
    public ArrayList getMenuItems ( ) {
        return menuItems;
    }
    // other methods
}
```

Implementation of DinerMenu

```
public class DinerMenu {
    static final int MAX_ITEMS = 6; int numberOfItems = 0;
    MenuItem[] menuItems;
    public DinerMenu ( ) {
        menuItems = new MenuItem[MAX_ITEMS];
        addItem ("Vegetarian BLT", "Bacon with Lettuce & tomato on whole wheat", true, 2.99);
        addItem ("Soup of the Day", "Soup of the Day with potato salad", false, 3.29);
        // other items
    }
    public void addItem (String name, String description, boolean vegetarian, double price) {
        MenuItem menuItem = new MenuItem (name, description, vegetarian, price);
        if (numberOfItems >= MAX_ITEMS)
            System.err.println("Sorry menu is full! Can't add any more items");
        else {
            menuItems[numberOfItems] = menuItem;
            numberOfItems = numberOfItems + 1;
        }
    }
    public MenuItem[] getMenuItems ( ) {
        return menuItems;
    }
    // other methods
}
```

Implementing printMenu()

- It prints every item on the menu

```
PencakeHouseMenu pancakeHouseMenu = new PancakeHouseMenu();
ArrayList breakfastItems = pancakeHouseMenu.getMenuItems();
DinerMenu dinerMenu = new DinerMenu();
MenuItem[] lunchItems = dinerMenu.getMenuItems();

for (int i = 0; i < breakfastItems.size(); i++){
    MenuItem menuItem = (MenuItem)breakfastItems.get(i);
    System.out.print (menuItem.getName() + " ");
    System.out.println(menuItem.getPrice() + " ");
    System.out.println(menuItem.getDescription() + " ");
}
for (int i = 0; i < lunchItems.size(); i++){
    MenuItem menuItem = lunchItems[i];
    System.out.print (menuItem.getName() + " ");
    System.out.println(menuItem.getPrice() + " ");
    System.out.println(menuItem.getDescription() + " ");
}
```

Problems

1. We are coding to the concrete implementations, not to an interface
2. If we decided to switch from using DinerMenu to another type of menu implemented with a Hashtable, we'd have to modify a lot of code
3. The waitress needs to know how each menu represents its internal collection of menu items.
4. We have duplicate code: the printMenu()

Two Approaches

// The Original Approach

```
for (int i = 0; i < breakfastItems.size(); i++)
    MenuItem menuItem = (MenuItem) breakfastItems.get(i);

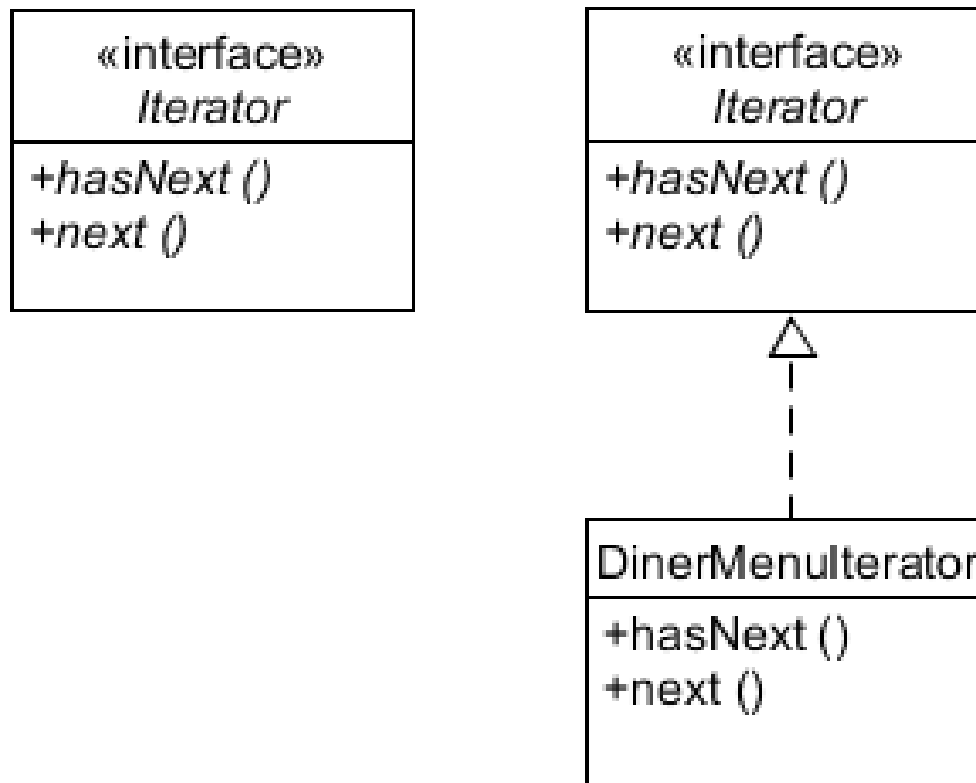
for (int i = 0; i < lunchItems.length; i++)
    MenuItem menuItem = lunchItems[i];
```

// The Second Approach using Iterator

```
Iterator iterator = breakfastMenu.createIterator();
while (iterator.hasNext())
    MenuItem menuItem = (MenuItem) iterator.next();

iterator = lunchMenu.createIterator();
while (iterator.hasNext())
    MenuItem menuItem = (MenuItem) iterator.next();
```

Implementing Iterator Interface



Defining Iterator and DinerMenuIterator

```
public interface Iterator {
    boolean hasNext();
    Object next();
}

public class DinerMenuIterator implements Iterator {
    MenuItem[] items;
    int position = 0;
    public DinerMenuIterator (MenuItem[] items) {
        this.items = items;
    }
    public Object next() {
        MenuItem menuItem = items[position];
        position ++;
        return menuItem;
    }
    public boolean hasNext() {
        if (position >= items.length) return false;
        else return true;
    }
}
```

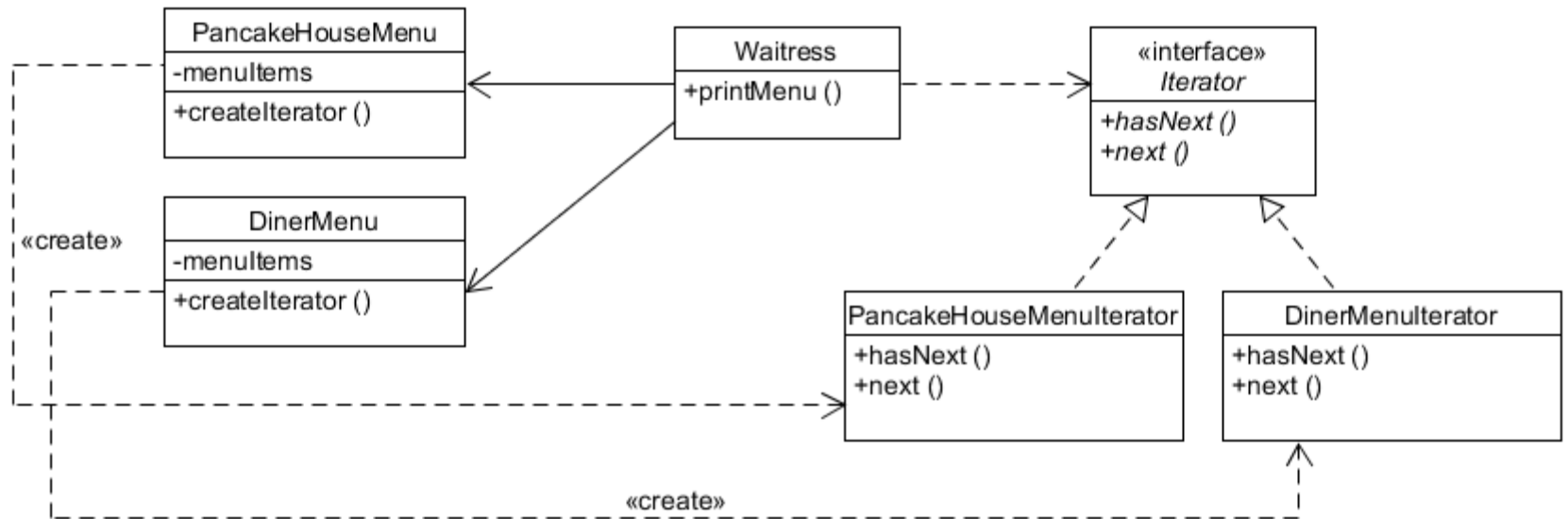
Reworking the DinerMenu with Iterator

```
public class DinerMenu {  
    // constructor  
    // addItem  
    MenuItem[] menuItems;  
  
    public menuItem[] getMenuItems() {  
        return menuItems;  
    }  
  
    public Iterator createIterator() {  
        return new DinerMenuIterator(menuItems);  
    }  
}
```

Fixing up the Waitress code

```
public class waitress {
    PancakeHouseMenu pancakeHouseMenu;
    DinerMenu dinerMenu;
    public waitress(PancakeHouseMenu pancakeHouseMenu, DinerMenu dinerMenu) {
        this.pancakeHouseMenu = pancakeHouseMenu;
        this.dinerMenu = dinerMenu;
    }
    public void printMenu() {
        Iterator pancakeIterator = pancakeHouseMenu.createIterator();
        Iterator dinerIterator = dinerMenu.createIterator();
        System.out.println("Menu\n---\nBREAKFAST");
        printMenu(pancakeIterator);
        System.out.println("\nLUNCH");
        printMenu(dinerIterator);
    }
    private void printMenu(Iterator iterator) {
        while (iterator.hasNext()) {
            MenuItem menuItem = (MenuItem) iterator.next();
            System.out.println(menuItem.getName() + ", ");
            ..
        }
    }
}
```

Current Design



Using Java's iterator

- The java.util.Iterator interface:

```
public interface Iterator {  
    boolean hasNext();  
    Object next();  
    void remove();  
}
```

Rewriting DinerMenuIterator

```
import java.util.Iterator;

public class DinerMenuIterator implements Iterator {
    MenuItem[] list;

    public Object next() {
        // the same
    }

    public boolean hasNext() {
        // the same
    }

    public void remove() {
        if (position <= 0 )
            throw new IllegalStateException("You can't remove blah blah");

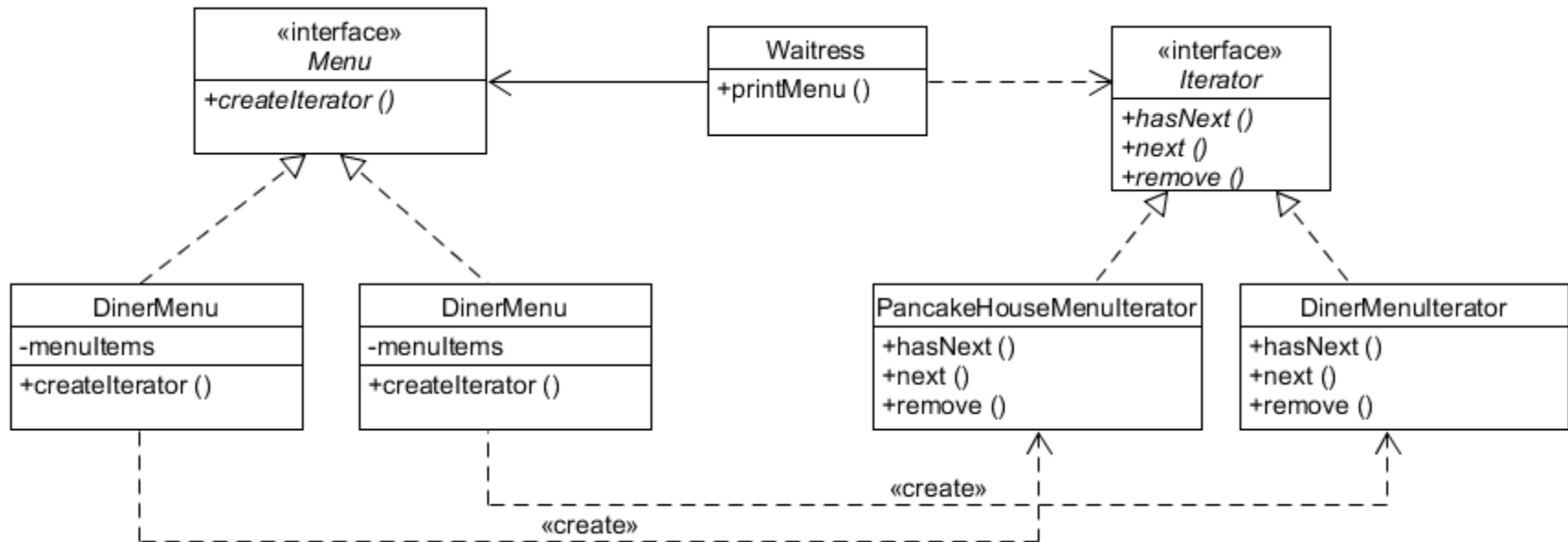
        // code for removing an item and shifting the rest
    }
}
```


Using Menu Interface

```
public class waitress {
    Menu pancakeHouseMenu;
    Menu dinerMenu;
    public waitress(Menu pancakeHouseMenu, Menu dinerMenu) {
        this.pancakeHouseMenu = pancakeHouseMenu;
        this.dinerMenu = dinerMenu;
    }
    public void printMenu() {
        Iterator pancakeIterator = pancakeHouseMenu.createIterator();
        Iterator dinerIterator = dinerMenu.createIterator();
        System.out.println("Menu\n----\nBREAKFAST");
        printMenu(pancakeIterator);
        System.out.println("\nLUNCH");
        printMenu(dinerIterator);
    }
    private void printMenu() {
        while (iterator.hasNext()) {
            MenuItem menuItem = (MenuItem) iterator.next();
            System.out.println(menuItem.getName() + ", ");
            ..
        }
    }
}
```

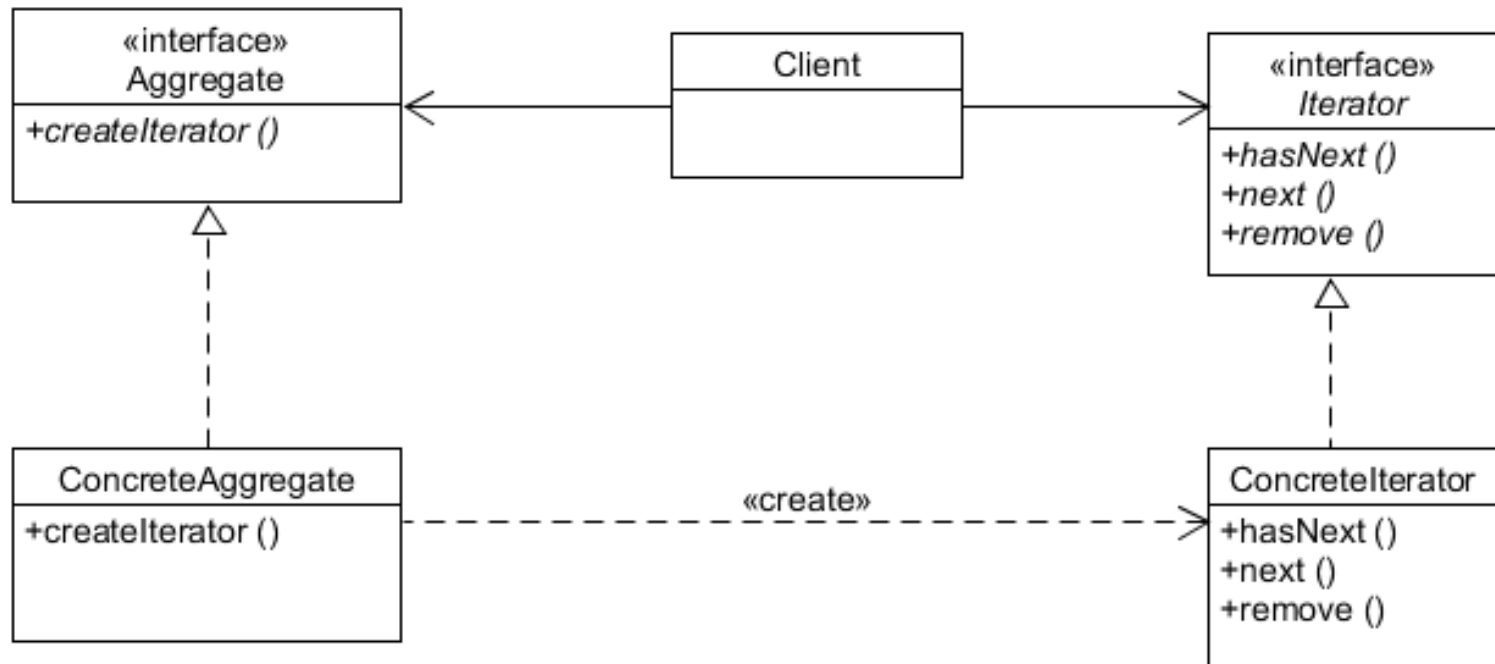
```
public interface Menu {
    public Iterator createIterator();
}
```

New Design



Iterator Pattern

- Provides a way to access the elements of an aggregate object sequentially without exposing its underlying representation.



Single Responsibility

- A class should have only one reason to change
 - Aggregate and Iteration – two different responsibilities
 - Cohesion
 - measure of how closely a class or a module supports a single purpose or responsibility
 - High cohesion – designed around a set of related functions
 - Low cohesion – designed around a set of unrelated functions

Menu with Hashtable

```
public class CafeMenu implements Menu {  
    Hashtable menuItems = new Hashtable();  
    public CafeMenu() {  
        // constructor  
    }  
    public void addItem(String name, String desc, boolean vegetarian,  
        double price) {  
        ..  
    }  
    public Iterator createIterator() {  
        return menuItems.values().iterator();  
    }  
}
```

Data Structures

- Collections
 - ArrayList, Hashtable, Vector, LinkedList
 - `HashTable.values().iterator()`
- Built-in Array
 - We implemented iterator for it

Collection Interface

«interface» Collection
<i>+add()</i> <i>+addAll()</i> <i>+clear()</i> <i>contains(Object o)</i> <i>containsAll(Collection c)</i> <i>equals(Object o)</i> <i>hashCode()</i> <i>isEmpty()</i> <i>iterator()</i> <i>remove(Object o)</i> <i>removeAll(Collection c)</i> <i>retainAll(Collection c)</i> <i>size()</i> <i>toArray()</i>

List and ListIterator Interface

```
public interface List extends Collection {  
    Object get(int index);  
    Object set(int index, Object element);  
    void add(int index, Object element);  
    Object remove(int index);  
    boolean addAll(int index, Collection c);  
    int indexOf(Object o);  
    int lastIndexOf(Object o);  
    ListIterator listIterator();  
    ListIterator listIterator(int index);  
    List subList(int from, int to);  
}
```

```
public interface ListIterator extends Iterator {  
    boolean hasNext();  
    Object next();  
    void remove();  
    boolean hasPrevious();  
    Object previous();  
    int nextIndex();  
    int previousIndex();  
    void set(Object o);  
    void add(Object o);  
}
```


Using ListIterator

```
// Forward traverse
List list = new LinkedList();
...
ListIterator iterator = list.listIterator();
while (iterator.hasNext()) {
    Object item = iterator.next();
    // Code here to process item.
}

// Backward traverse
List list = new LinkedList();
ListIterator iterator = list.listIterator(list.size());
...
while (iterator.hasPrevious()) {
    Object item = iterator.previous();
    // Code here to process item.
}
```

Iterators and Collections in Java5

- For/In (similar to foreach in C#)

```
for (Object obj: collection ) {...}
```

- Example

```
ArrayList items = new ArrayList();  
...  
for (MenuItem item: items)  
    System.out.println("item = " + item);
```

Related Patterns

- Iterator can traverse a Composite. Visitor can apply an operation over a Composite.
- Polymorphic Iterators rely on Factory Methods to instantiate the appropriate Iterator subclass.
- Memento is often used in conjunction with Iterator. An Iterator can use a Memento to capture the state of an iteration. The Iterator stores the Memento internally.

Summary

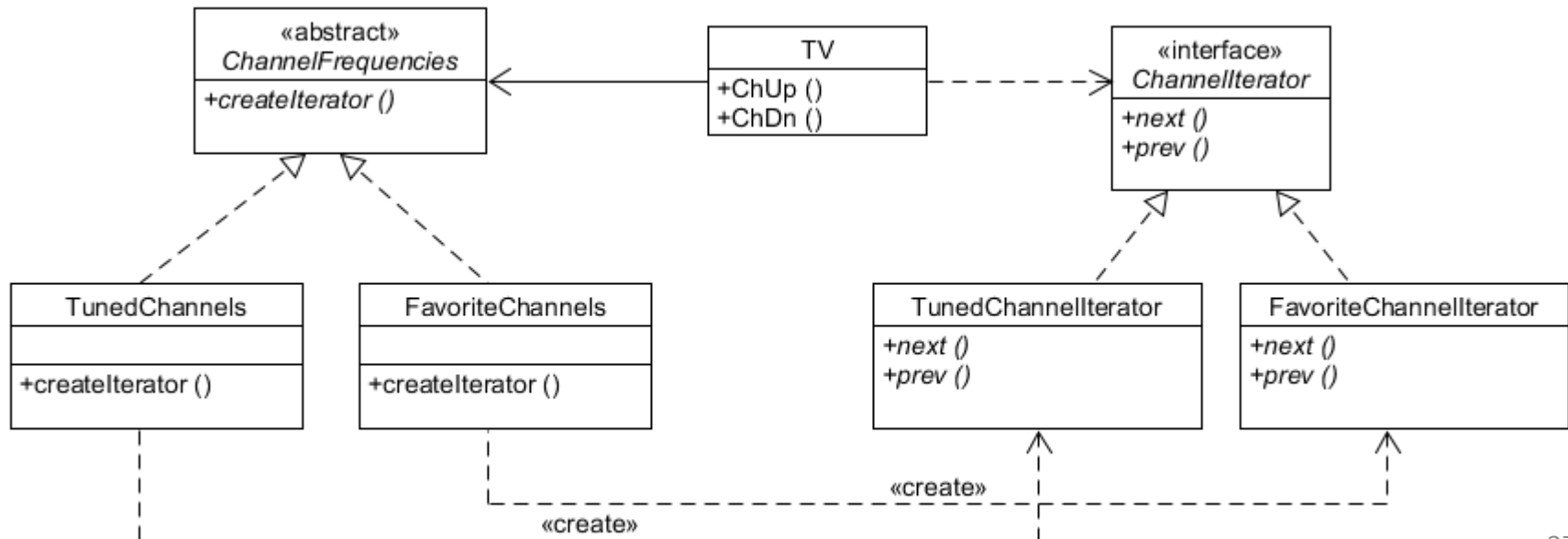
- Single Responsibility Principle
 - A class should have only one reason to change
- Iterator Pattern
 - Provide a way to access the elements of an aggregate object sequentially without exposing its underlying representation

Iterator Pattern Quiz : Problem

- TV의 채널 collection을 TunedChannels과 FavoriteChannels 클래스로 구별하여 관리하고자 한다. TunedChannels 클래스에는 순방향 혹은 역방향으로 순회할 수 있는 TunedChannelliterator를 생성하는 메소드가 제공된다. 이와 유사하게 FavoriteChennelliterator가 FavoriteChannels 클래스를 위해 제공된다. TunedChannelliterator와 FavoriteChannelliterator 클래스에는 역방향 순회용 prev(), 순방향 순회용 next() 메소드가 제공된다.
- Iterator 패턴을 기반으로 상기 모델을 TV 클래스와 함께 클래스 다이어그램으로 그리시오

Iterator Pattern Quiz : Answer

- TV의 채널 collection을 TunedChannels과 FavoriteChannels 클래스로 구별하여 관리하고자 한다. TunedChannels 클래스에는 순방향 혹은 역방향으로 순회할 수 있는 TunedChannelIterator를 생성하는 메소드가 제공된다. 이와 유사하게 FavoriteChennelIterator가 FavoriteChannels 클래스를 위해 제공된다. TunedChannelIterator와 FavoriteChennelIterator 클래스에는 역방향 순회용 prev(), 순방향 순회용 next() 메소드가 제공된다.
- Iterator 패턴을 기반으로 상기 모델을 TV 클래스와 함께 클래스 다이어그램으로 그리시오



9. STATE PATTERN

2020. 04. 16

중앙대학교 소프트웨어학부

이찬근 교수

Contents

- State pattern discovery
- Structure and run-time mechanism of State pattern
- Similarity with Strategy pattern

State Pattern

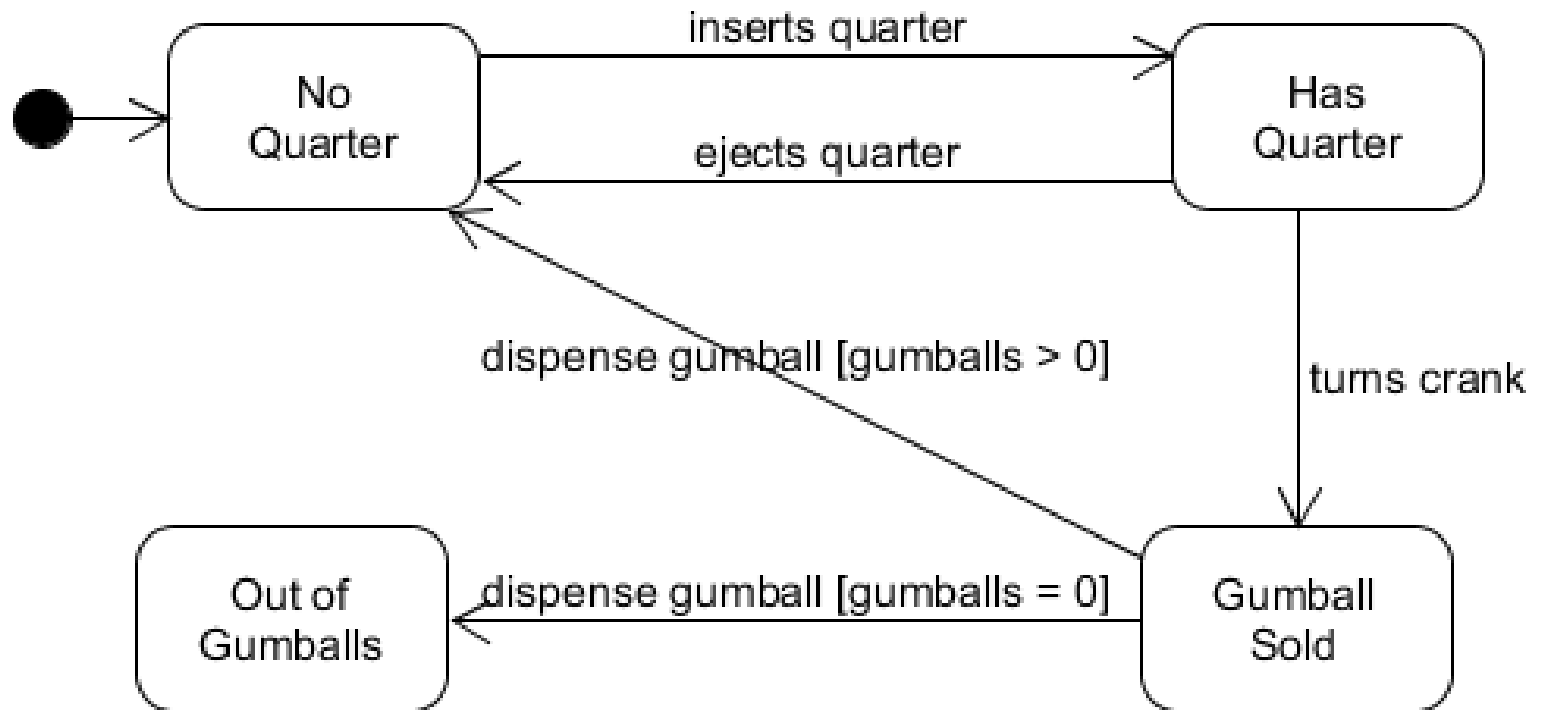
- **Purpose**

- Ties object circumstances to its behavior, allowing the object to behave in different ways based upon its internal state.

- **Use When**

- The behavior of an object should be influenced by its state.
 - Complex conditions tie object behavior to its state.
 - Transitions between states need to be explicit.

States & Actions to Implement



Coding the States

```
final static int SOLD_OUT = 0;  
final static int NO_QUARTER = 1;  
final static int HAS_QUARTER = 2;  
final static int SOLD = 3;  
int state = SOLD_OUT;
```

Writing the Code

```
public class GumballMachine {  
    final static int SOLD_OUT = 0;  
    final static int NO_QUARTER = 1;  
    final static int HAS_QUARTER = 2;  
    final static int SOLD = 3;  
  
    int state = SOLD_OUT;  
    int count = 0;  
  
    public GumballMachine(int count) {  
        this.count = count;  
        if (count > 0) state = NO_QUARTER;  
    }  
  
    public void insertQuarter() {  
        if (state == HAS_QUARTER) {  
            ::  
            ::  
        }  
  
    public void ejectQuarter() {  
        if (state == HAS_QUARTER) {  
            ::  
            ::  
        }  
    }  
}
```

Coding per Action

```
public void insertQuarter() {  
    if (state == HAS_QUARTER)  
        System.out.println("You can't insert another quarter");  
  
    else if (state == SOLD_OUT)  
        System.out.println("You can't insert a quarter");  
        System.out.println("The machine is sold out");  
  
    else if (state == SOLD)  
        System.out.println("Please wait, already giving you a gumball");  
  
    else if (state == NO_QUARTER) {  
        state = HAS_QUARTER;  
        System.out.println("You inserted a quarter");  
    }  
}
```

Test Drive

```
public class GumballMachineTestDrive {  
    public static void main(String[] args) {  
        GumballMachine gumballMachine = new GumballMachine(5);  
  
        System.out.println(gumballMachine);  
  
        gumballMachine.insertQuarter();  
        gumballMachine.turnCrank();  
  
        System.out.println(gumballMachine);  
  
        gumballMachine.insertQuarter();  
        gumballMachine.ejectQuarter();  
        gumballMachine.turnCrank();  
  
        System.out.println(gumballMachine);  
  
        ...  
    }  
}
```

A Change Request!

Be a Winner!
One in Ten
get a FREE
GUMBALL!



We think that by turning "gumball buying" into a game we can significantly increase our sales. We're going to put one of these stickers on every machine.

We're so glad we've got Java in the machines because this is going to be easy, right?



Messy state of things...

```
final static int SOLD_OUT = 0;
final static int NO_QUARTER = 1;
final static int HAS_QUARTER = 2;
final static int SOLD = 3;

public void insertQuarter() {
}

public void ejectQuarter() {
}

public void turnCrank() {
}

public void dispense() {
}
```


New Idea

- Encapsulate what varies

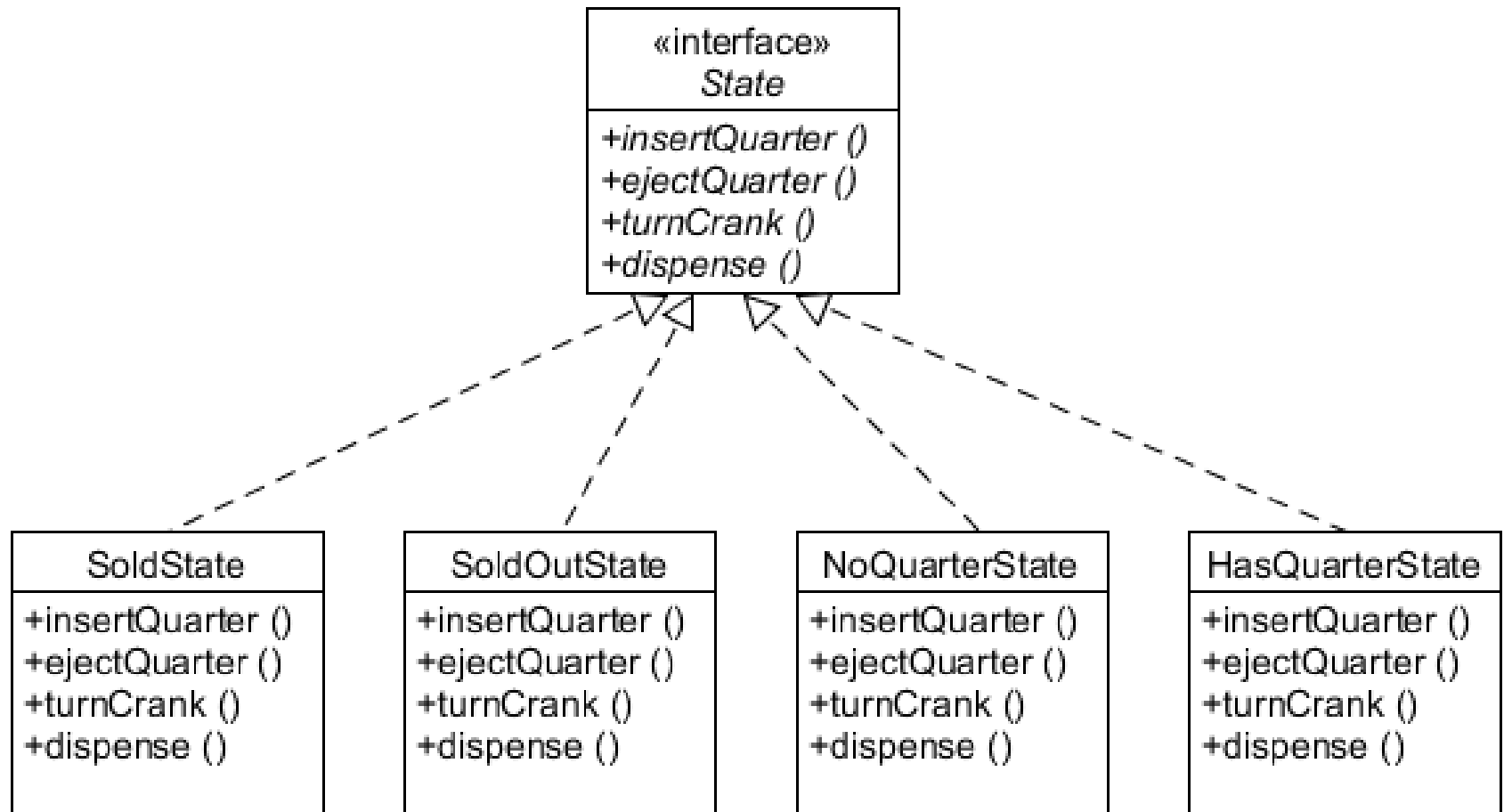
- Put each state's behavior in its own class, then every state just implements its own actions
- The Gumball Machine can delegate to the state object that represents the current state

- Favor composition over inheritance

New Design

- Define a **State interface** that contains a **method for every action** in the Gumball Machine
- Implement a **State class** for every state of the machine.
 - responsible for **behavior** of the machine when it is in the **corresponding state**.
- **Get rid of** all of our **conditional code** and instead **delegate to the state object** to do the work for us

Defining State Interface and classes



Implementing NoQuarterState

```
public class NoQuarterState implements State {
    GumballMachine gumballMachine;

    public NoQuarterState(GumballMachine gumballMachine) {
        this.gumballMachine = gumballMachine;
    }

    public void insertQuarter() {
        System.out.println("You inserted a quarter");
        gumballMachine.setState(gumballMachine.getHasQuarterState());
    }

    public void ejectQuarter() {
        System.out.println("You haven't inserted a quarter");
    }

    public void turnCrank() {
        System.out.println("You turned, but there's no quarter");
    }

    public void dispense() {
        System.out.println("No Gumball dispensed");
    }
}
```

Reworking the Gumball Machine

```
public class GumballMachine {  
    State soldOutState;  
    State noQuarterState;  
    State hasQuarterState;  
    State soldState;  
  
    State state = soldOutState;  
    int count = 0;  
  
    public GumballMachine(int numberGumballs) {  
        soldOutState = new SoldOutState(this);  
        noQuarterState = new NoQuarterState(this);  
        hasQuarterState = new HasQuarterState(this);  
        soldState = new SoldState(this);  
  
        this.count = numberGumballs;  
        if (numberGumballs > 0) {  
            state = noQuarterState;  
        }  
    }  
}
```

Reworking the Gumball Machine

```
public void insertQuarter() {
    state.insertQuarter();
}

public void ejectQuarter() {
    state.ejectQuarter();
}

public void turnCrank() {
    state.turnCrank();
}

public void dispense() {
    state.dispense();
}

void setState(State state) {
    this.state = state;
}

void releaseBall() {
    System.out.println("A gumball comes rolling out the slot...");
    if (count != 0)
        count = count - 1;
}
}
```

Implementing More States

```
public class HasQuarterState implements State {
    GumballMachine gumballMachine;

    public HasQuarterState(GumballMachine gumballMachine) {
        this.gumballMachine = gumballMachine;
    }
    public void insertQuarter() {
        System.out.println("You can't insert another quarter");
    }
    public void ejectQuarter() {
        System.out.println("Quarter returned");
        gumballMachine.setState(gumballMachine.getNoQuarterState());
    }
    public void turnCrank() {
        System.out.println("You turned...");
        gumballMachine.setState(gumballMachine.getSoldState());
        gumballMachine.dispense();
    }
    public void dispense() {
        System.out.println("No Gumball dispensed");
    }
}
```

Implementing More States

```
public class SoldState implements State {
    GumballMachine gumballMachine;

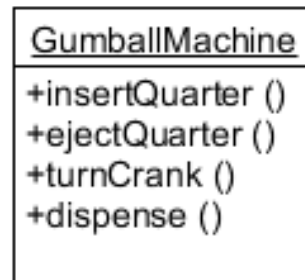
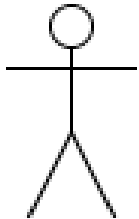
    public SoldState(GumballMachine gumballMachine) {
        this.gumballMachine = gumballMachine;
    }

    public void insertQuarter() {
        System.out.println("Please wait, we're already giving you a gumball");
    }
    public void ejectQuarter() {
        System.out.println("Sorry, you already turned the crank");
    }
    public void turnCrank() {
        System.out.println("Turning twice doesn't get you another gumball!");
    }
    public void dispense() {
        gumballMachine.releaseBall();
        if (gumballMachine.getCount() > 0) {
            gumballMachine.setState(gumballMachine.getNoQuarterState());
        } else {
            System.out.println("Oops, out of gumballs!");
            gumballMachine.setState(gumballMachine.getSoldOutState());
        }
    }
}
```


What We've Done So Far

- Structurally quite different from the original version, but functionally the same!
- Changes
 - Localized the **behavior of each state** into its own class
 - **Removed** all the troublesome **conditional statements** that would have been difficult to maintain
 - **Closed each state for modification**, and yet left the Gumball Machine **open to extension by adding new state classes**
 - Created a code base and class structure that maps much more closely to the Mighty Gumball diagram and is **easier to read and understand**

How It is Working



NoQuarterState

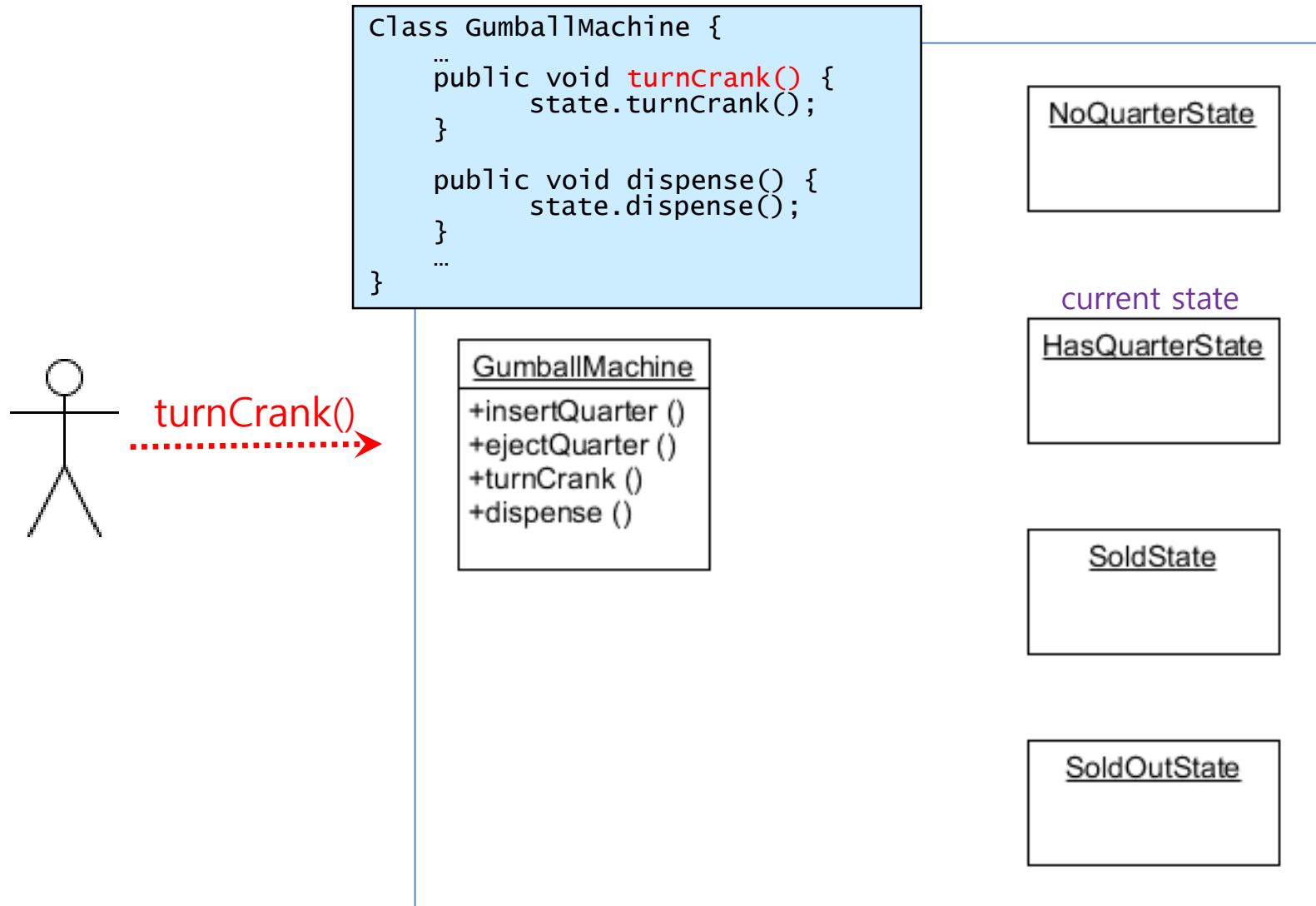
current state

HasQuarterState

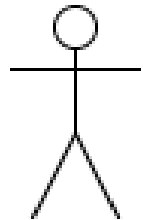
SoldState

SoldOutState

How It is Working



How It is Working



```
Class GumballMachine {  
    ...  
    public void turnCrank() {  
        state.turnCrank();  
    }  
    public void dispense() {  
        state.dispense();  
    }  
    ...  
}
```

GumballMachine
+insertQuarter ()
+ejectQuarter ()
+turnCrank ()
+dispense ()

turnCrank()

NoQuarterState

current state

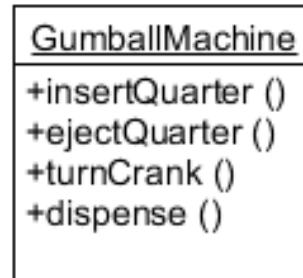
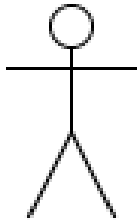
HasQuarterState

SoldState

SoldOutState

```
public class HasQuarterState implements State {  
    GumballMachine gumballMachine;  
    ...  
    public void turnCrank() {  
        System.out.println("You turned...");  
        gumballMachine.setState(gumballMachine.getSoldState());  
        gumballMachine.dispense();  
    }  
    ...  
}
```

How It is Working



NoQuarterState

HasQuarterState

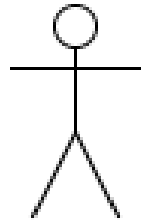
current state

SoldState

SoldOutState

```
public class HasQuarterState implements State {
    GumballMachine gumballMachine;
    ...
    public void turnCrank() {
        System.out.println("You turned...");
        gumballMachine.setState(gumballMachine.getSoldState());
        gumballMachine.dispense();
    }
    ...
}
```

How It is Working



```
Class GumballMachine {  
    ...  
    public void turnCrank() {  
        state.turnCrank();  
    }  
    public void dispense() {  
        state.dispense();  
    }  
    ...  
}
```

GumballMachine

- +insertQuarter ()
- +ejectQuarter ()
- +turnCrank ()
- +dispense ()

```
public class HasQuarterState implements State {  
    GumballMachine gumballMachine;  
    ...  
    public void turnCrank() {  
        System.out.println("You turned...");  
        gumballMachine.setState(gumballMachine.getSoldState());  
        gumballMachine.dispense();  
    }  
    ...  
}
```

NoQuarterState

HasQuarterState

current state

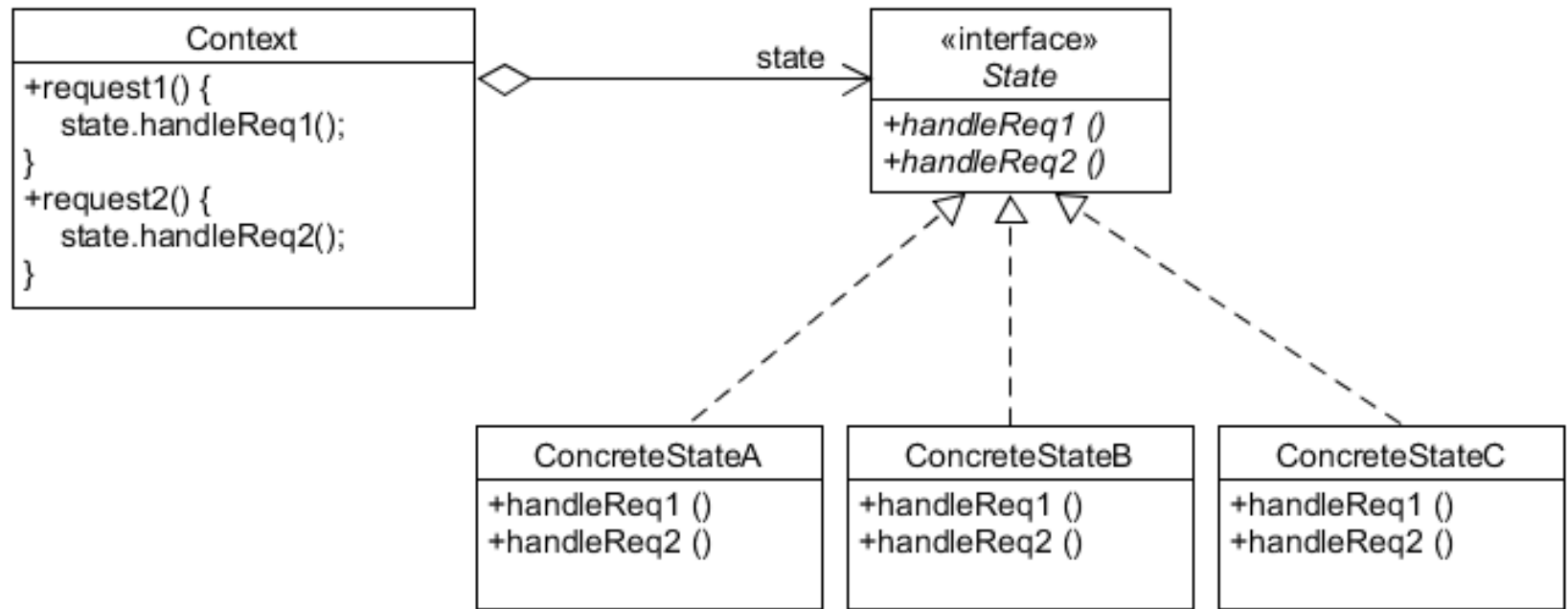
SoldState

SoldOutState

dispense()

The State Pattern

- The State Pattern **allows an object to alter its behavior when its internal state changes**. The object will appear to change its class.



Applicability of the State Pattern

- Use the State pattern when
 - An object's **behavior depends on its state**, and it **must change its behavior at run-time depending on that state**
 - Operations have large, multipart conditional statements that depend on the object's state. The State pattern puts each branch of the conditional in a separate class.

Consequences of the State Pattern

- Benefits

- Puts all behavior associated with a state into one object
- Allows state transition logic to be incorporated into a state object rather than in a monolithic if or switch statement
- Helps avoid inconsistent states since state changes occur using just the one state object and not several objects or attributes

- Liabilities

- Increased number of objects

State v.s. Strategy

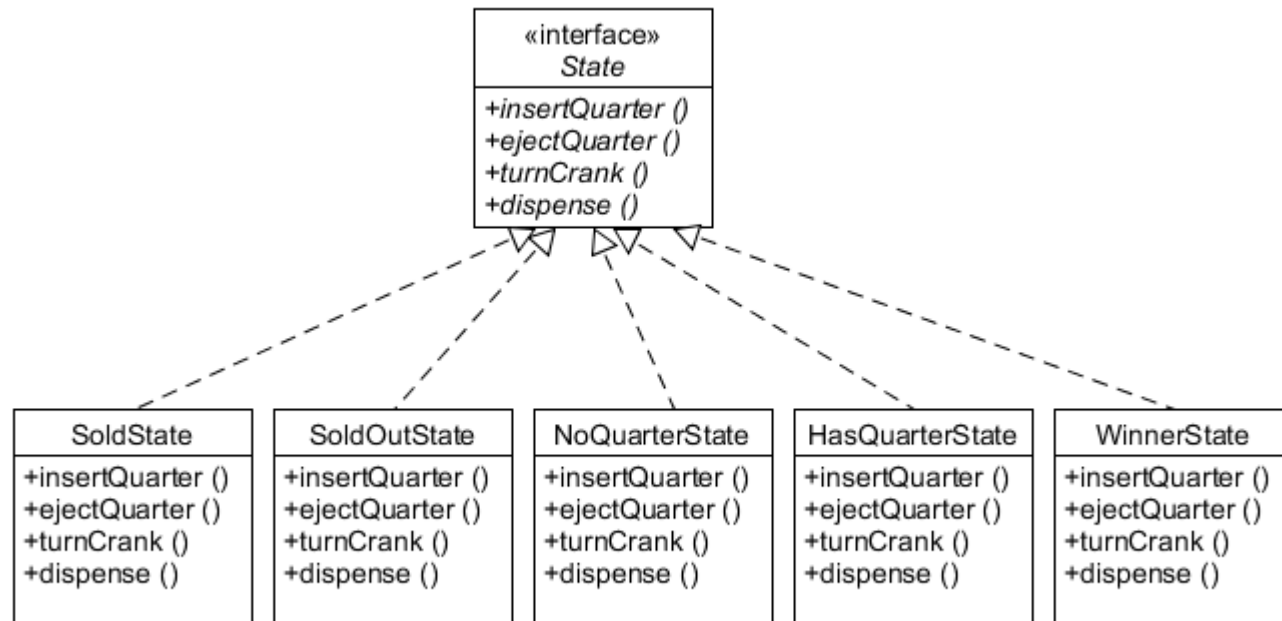
- Note the similarities between the State and Strategy patterns!
 - The difference is one of intent.
- A State object encapsulates a state-dependent behavior (and possibly state transitions)
 - The context's behavior changes over time
 - An alternative to putting lots of conditionals in the context
- A Strategy object encapsulates an algorithm
 - Often, there is a strategy object that is most appropriate for a context object
 - A flexible alternative to subclassing
- They are both examples of Composition with Delegation!

Implementation Issues

- Who defines the state transitions?
 - The Context class: okay for simple situations
 - The ConcreteState classes: generally more flexible, but causes implementation dependencies between the ConcreteState classes. In our example ConcreteState classes define the state transitions. Try to use getter methods.
- When are the ConcreteState objects created?
 - Create ConcreteState objects as needed
 - Create all ConcreteState objects once and have the Context object keep references to them

Finishing Gumball 1 in 10 game

```
public class GumballMachine {  
  
    State soldOutState;  
    State noQuarterState;  
    State hasQuarterState;  
    State soldState;  
    State winnerState;  
  
    State state = soldOutState;  
    int count = 0;  
}
```



Coding the WinnerState

```
public class WinnerState implements State {
    GumballMachine gumballMachine;
    public WinnerState(GumballMachine gumballMachine) {
        this.gumballMachine = gumballMachine;
    }
    public void insertQuarter() {
        System.out.println("Please wait, we're already giving you a Gumball");
    }
    ..
    public void dispense() {
        System.out.println("YOU'RE A WINNER! You get two gumballs!");
        gumballMachine.releaseBall();
        gumballMachine.releaseBall();
        if (gumballMachine.getCount() > 0)
            gumballMachine.setState(gumballMachine.getNoQuarterState());
        else
            gumballMachine.setState(gumballMachine.getSoldOutState());
    }
}
```

Implementing 1 in 10 in the HasQuarterState

```
public class HasQuarterState implements State {
    Random randomWinner = new Random(System.currentTimeMillis());

    GumballMachine gumballMachine;

    public HasQuarterState(GumballMachine gumballMachine) {
        this.gumballMachine = gumballMachine;
    }

    public void turnCrank() {
        System.out.println("You turned...");
        int winner = randomWinner.nextInt(10);
        if ((winner == 0) && (gumballMachine.getCount() > 1))
            gumballMachine.setState(gumballMachine.getWinnerState());
        else
            gumballMachine.setState(gumballMachine.getSoldState());
        gumballMachine.dispense();
    }
    ...
}
```

Related Patterns

- The implementation of the State pattern builds on the Strategy pattern. The difference between State and Strategy is in the intent.
 - Strategy: the choice of algorithm is fairly stable.
 - State: a change in the state of the "context" object causes it to select from its "palette" of State objects.

Summary

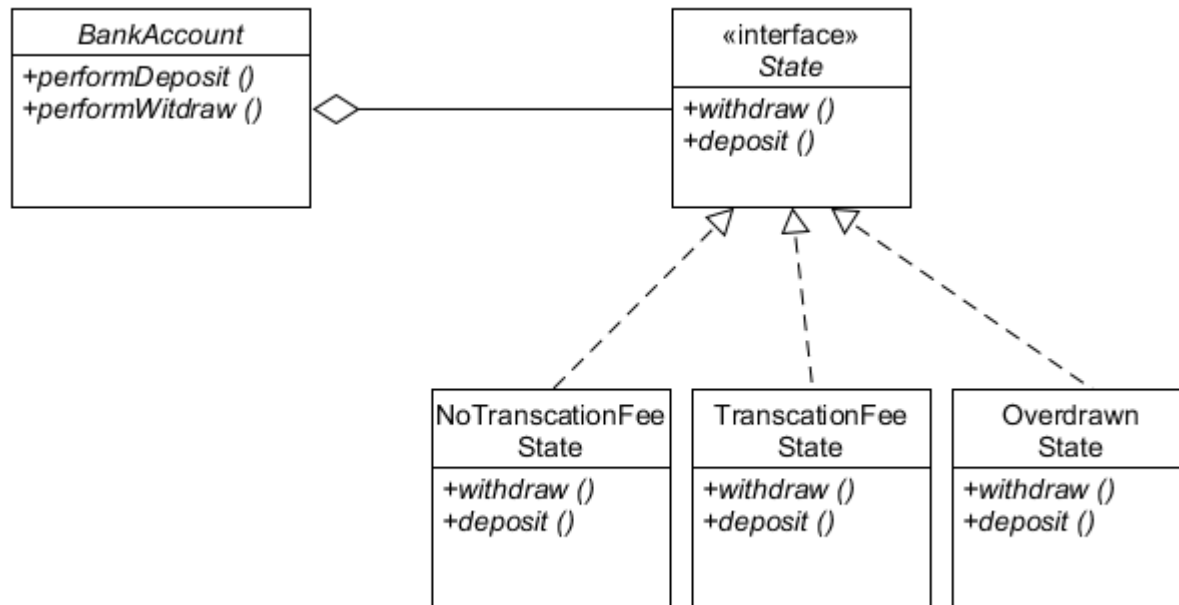
- State Pattern
 - Encapsulate state-based behavior and delegate behavior to the current state
- Strategy Pattern
 - Encapsulate interchangeable behaviors and use delegation to decide which behavior to use
- Template Method
 - Subclasses decide how to implement steps in an algorithm

State Pattern Quiz : Problem

- 은행계좌(BankAccount)에는 다음과 같은 세가지의 상태가 있고, 입금(deposit) 혹은 출금(withdraw)에 따라 상태의 전환이 된다.
 - NoTransactionFeeState
 - TransactionFeeState
 - OverdrawnState
- 해당 문제를 State 패턴을 이용하여 설계하고 BankAccount 클래스의 핵심적인 코드 골격을 제시하시오. (Object Composition과 Delegation 반드시 표시)

From	To	전이(transition) 조건
NoTransactionFeeState	TransactionFeeState	잔고가 마이너스가 아니지만 최소 유지금액 이하로 만드는 출금
	OverdrawnState	잔고를 마이너스로 만드는 출금
TransactionFeeState	NoTransactionFeeState	잔고를 최소 유지금액 이상으로 만드는 입금
	OverdrawnState	잔고를 마이너스로 만드는 출금
OverdrawnState	NoTransactionFeeState	잔고를 최소 유지금액 이상으로 만드는 입금
	TransactionFeeState	잔고가 마이너스가 아니지만 최소 유지금액 이하로 만드는 입금

State Pattern Quiz : Answer



```
public BankAccount {
    State currState;
    public performDeposit(int amount) {
        currState.deposit(amount);
    }
    public performWithdraw(int amount) {
        currState.withdraw(amount);
    }
}
```

10. MEDIATOR PATTERN

2020. 04. 16

중앙대학교 소프트웨어학부

이찬근 교수

Contents

- Mediator pattern in real-life
- Structure and run-time mechanism of Mediator pattern
- Comparison with Observer pattern

Mediator Pattern

- **Purpose**

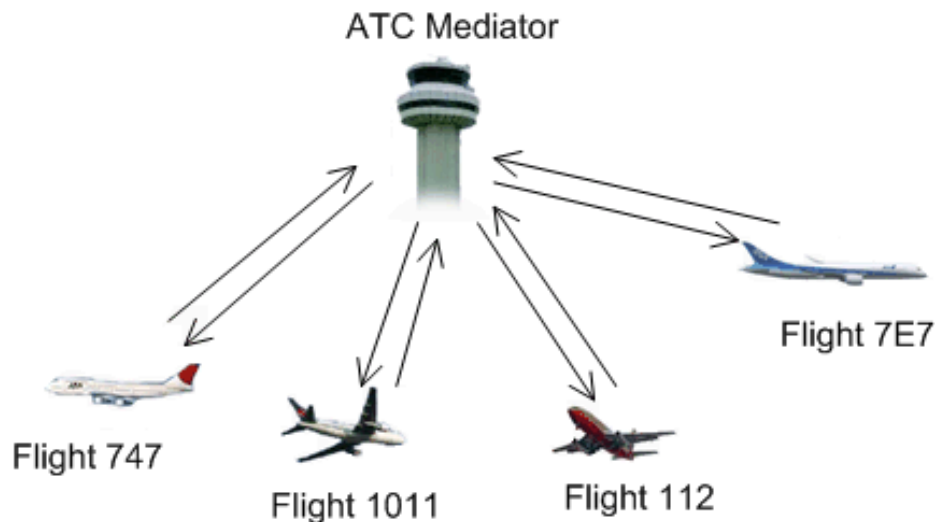
- Allows loose coupling by encapsulating the way disparate sets of objects interact and communicate with each other.

- **Use When**

- Communication between sets of objects is well defined and complex.
 - Too many relationships exist and common point of control or communication is needed.

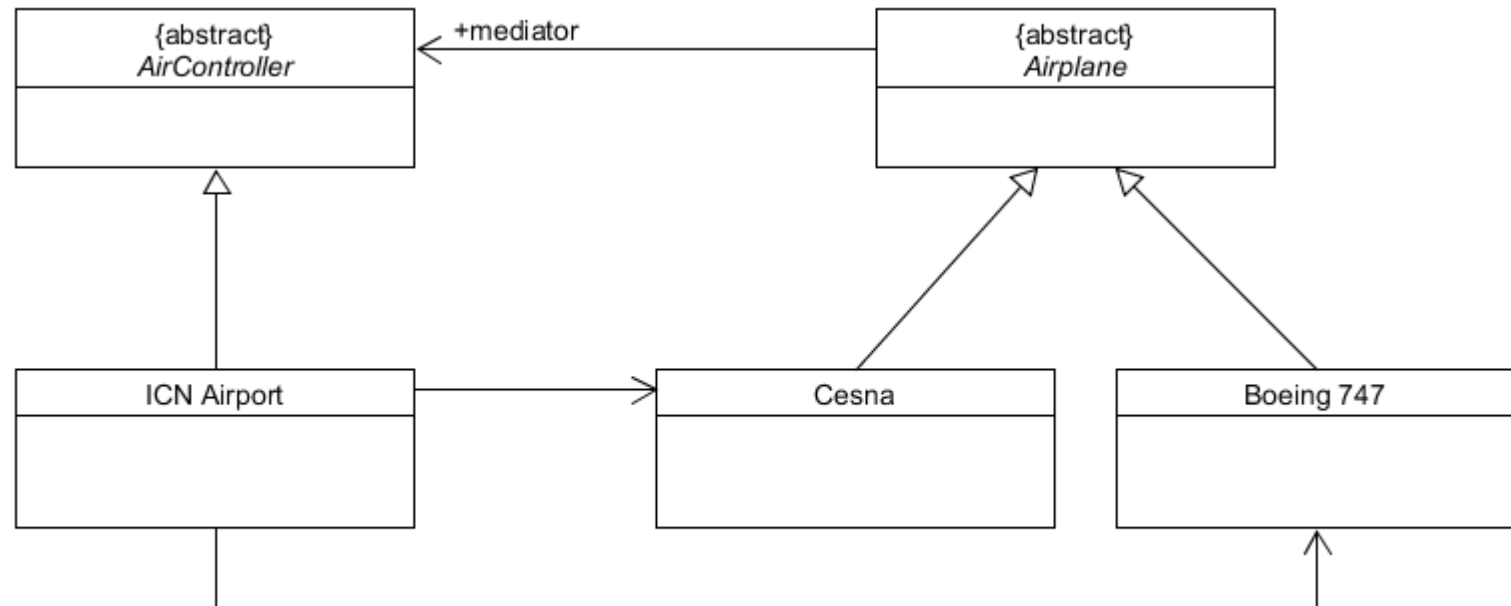
Battling Class Complexity

- Consider an air traffic controller
 - Many planes circle an airport
 - If they communicated with each other the skies above an airport would be a chaos
 - Accepted solution: have the planes communicate directly with an air traffic controller tower for permission to land
 - The planes do not even have to know about each other
 - This information is kept with the tower

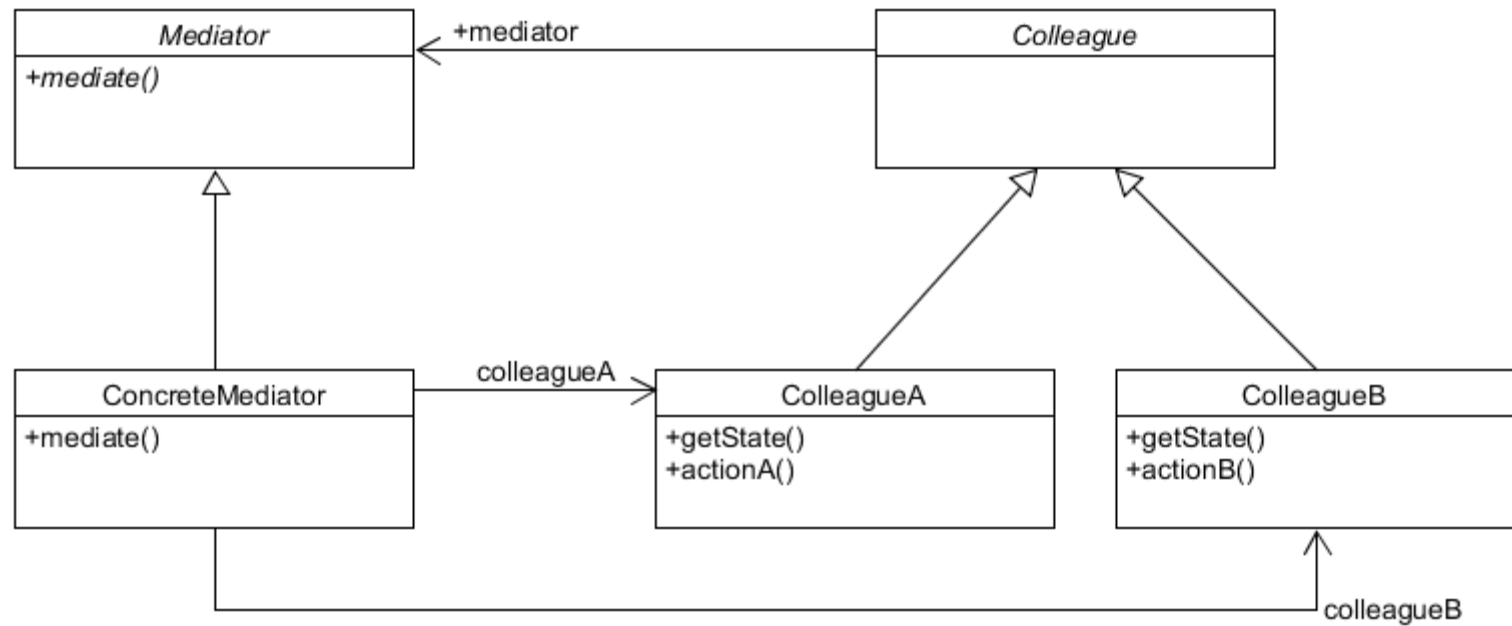


Battling Class Complexity

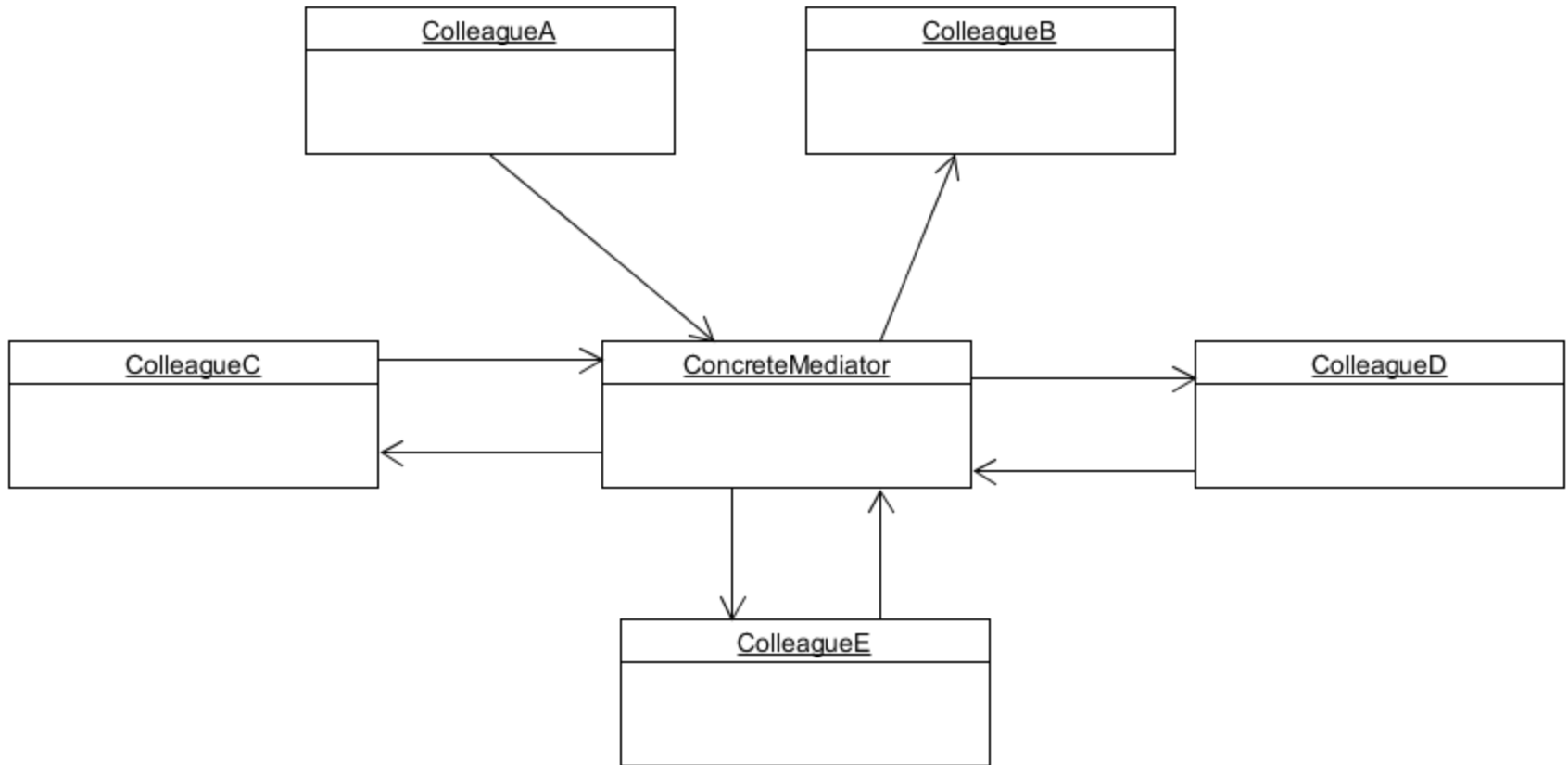
- Let's abstract this to classes and their objects in a program
 - When objects are allowed to **communicate directly with each other**, then **they become too tightly coupled**
 - When one object wishes to send a message to another then we need the equivalent of an air traffic **controller to forward the message** to the recipient
 - Keep the **dispatching information inside the new controller**
 - Call this coordinating object a **mediator**



Class Diagram for Mediator



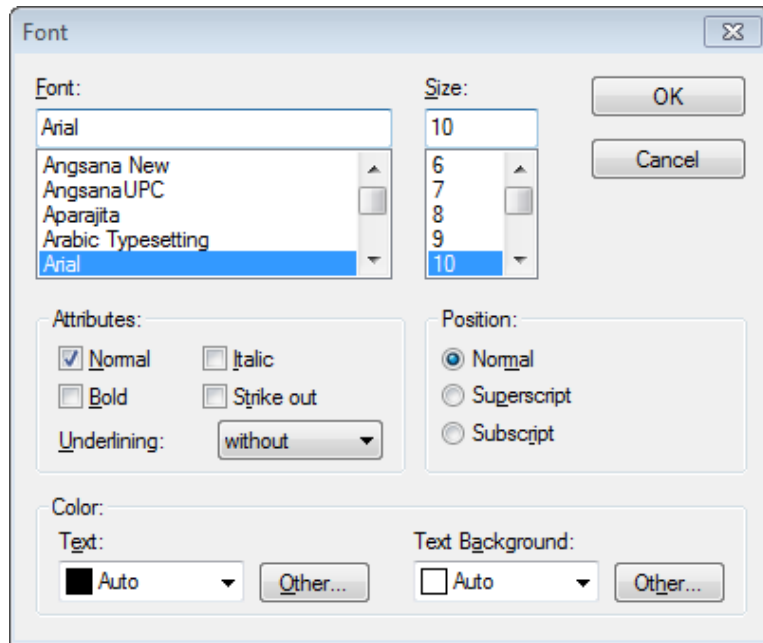
Object Diagram for Mediator



Mediator Pattern

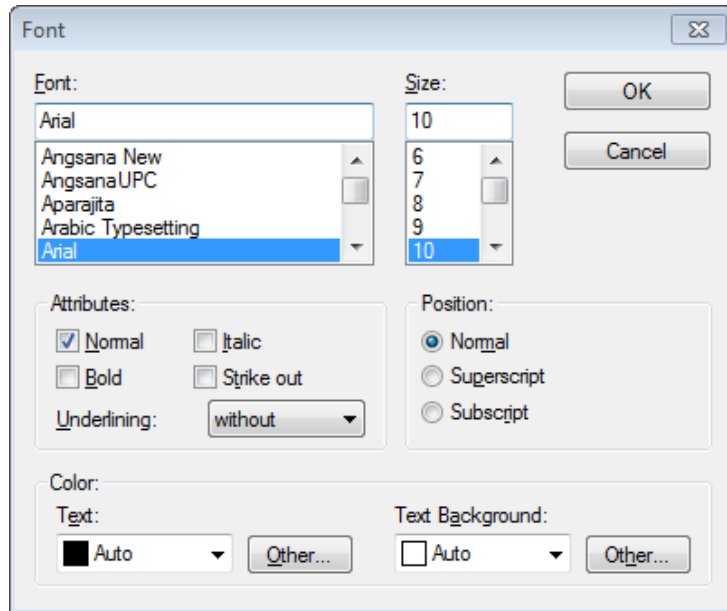
- Encapsulates interconnects between objects into **Mediator**
 - communications hub
 - Responsible for coordinating and controlling colleague interaction
 - Promotes loose coupling between classes
 - By preventing from referring to each other explicitly
 - Mediator is commonly used to coordinate related GUI components
- (-) mediators are hardly ever reusable
- (+) easy to understand the flow of communication

Mediator Example: FontDialog



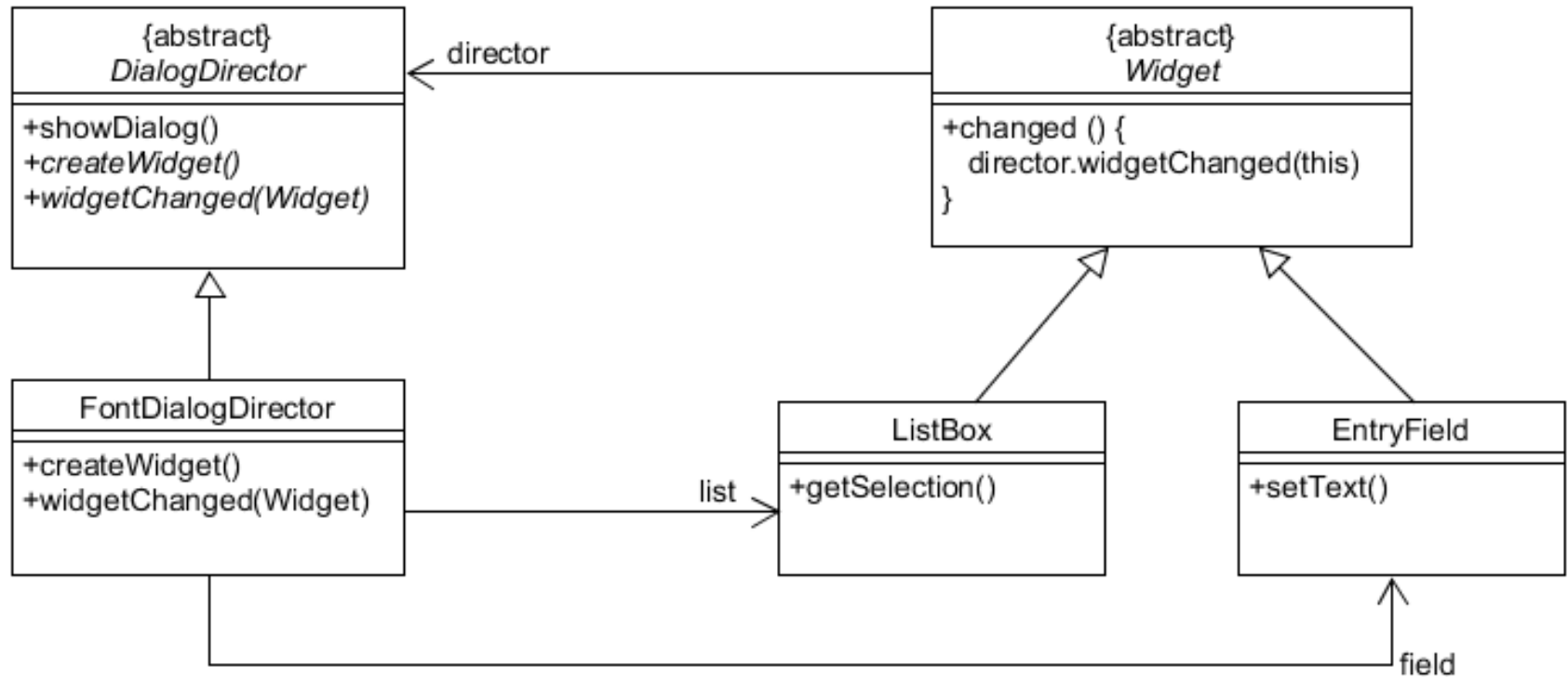
- Often there are dependencies between the widgets in the dialog.
- For example, a button gets disabled when a certain entry field is empty. Selecting an entry in a list of choices called a **list box** might change the contents of an entry field.
- Conversely, typing text into the entry field might automatically select one or more corresponding entries in the list box.
- Once text appears in the entry field, other buttons may become enabled that let the user do something with the text, such as changing or deleting the thing to which it refers.

Mediator Example: FontDialog

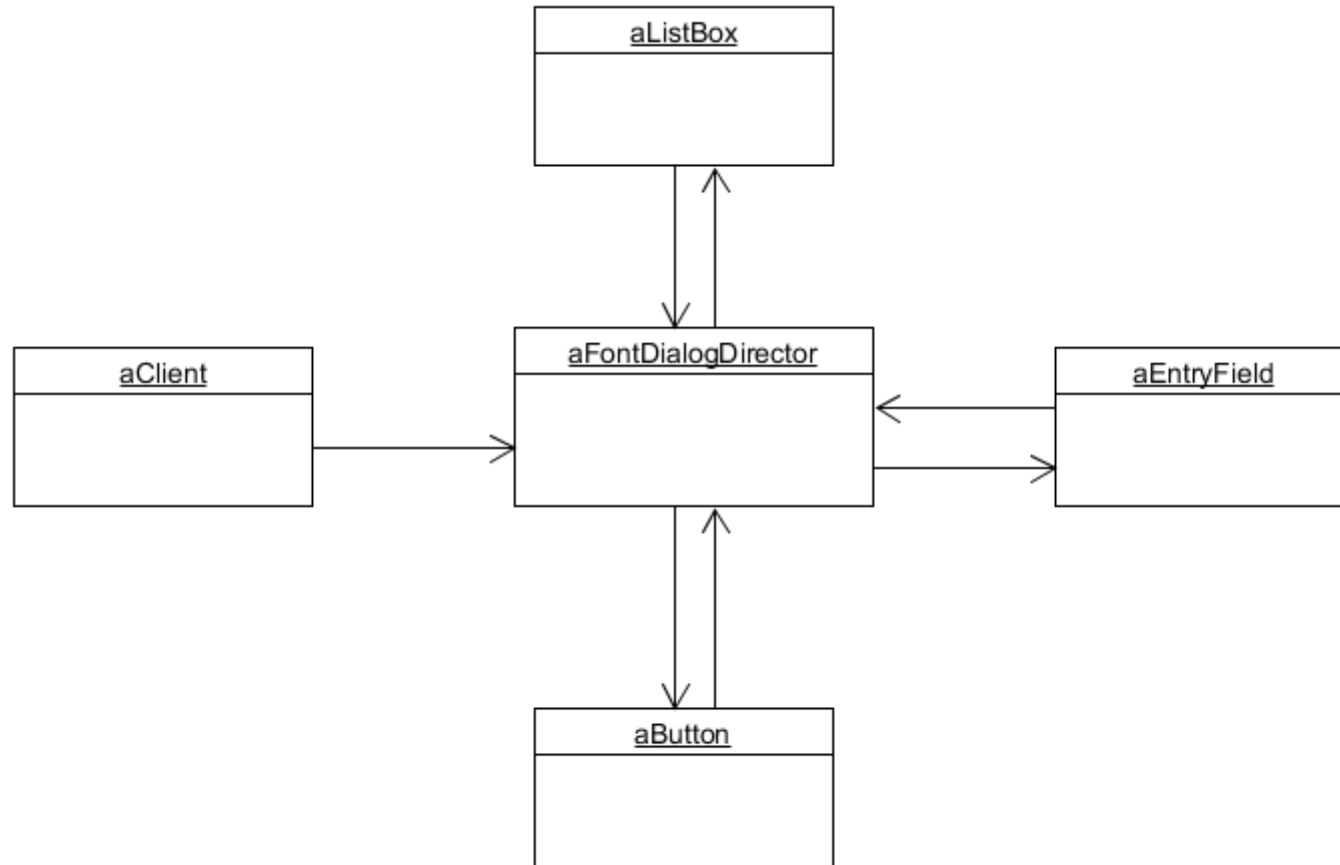


- Different dialog boxes will have different dependencies between widgets.
- So even though dialogs display the same kinds of widgets, they can't simply reuse stock widget classes; **they have to be customized to reflect dialog-specific dependencies.**
- Customizing them individually by subclassing will be tedious, since many classes are involved.
- **You can avoid these problems by encapsulating collective behavior in a separate **mediator** object.**

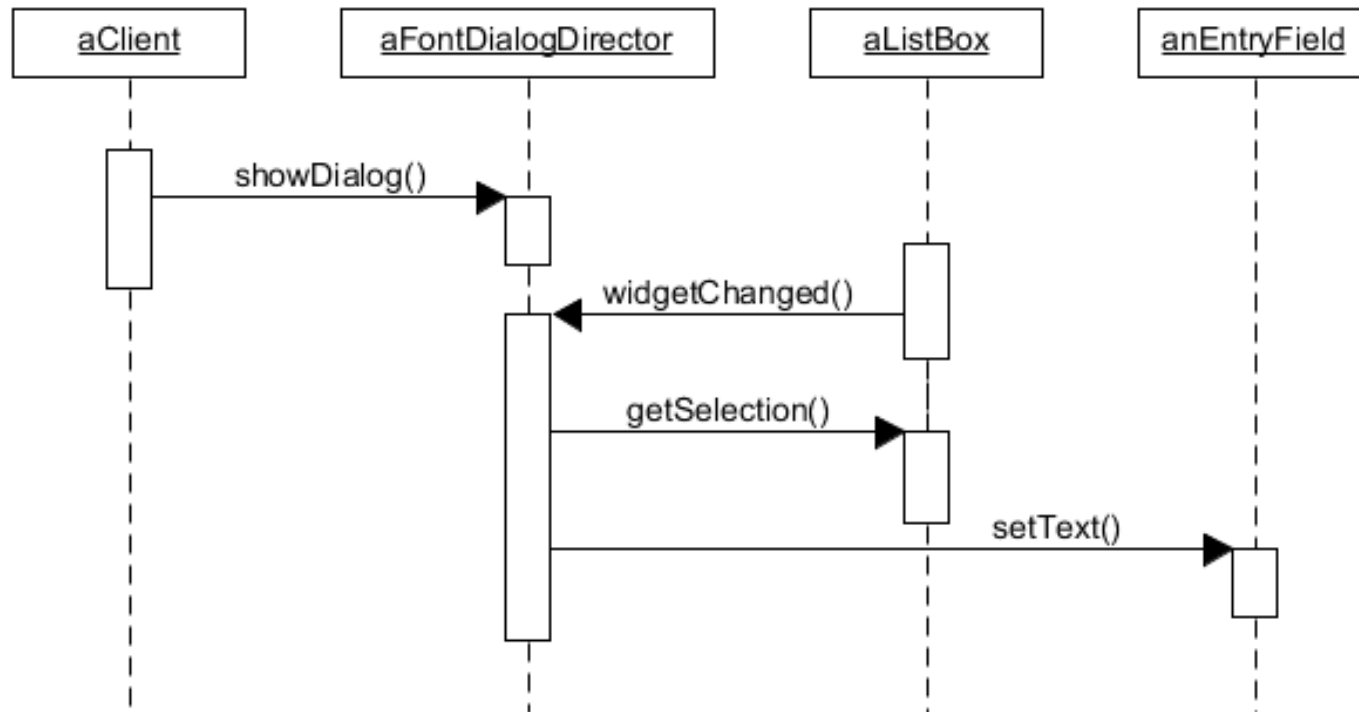
Structure of FontDialog



Object Diagram of FontDialog



Sequence diagram of FontDialog



1. The list box tells its director that it's changed.
2. The director gets the selection from the list box.
3. The director passes the selection to the entry field.
4. Now that the entry field contains some text, the director enables button(s) for initiating an action (e.g., "demibold," "oblique").

Related Patterns and Summary

- Observer

- Communication distributed by using observer and subject objects
- reusable
- can be hard to understand the multiple flows of communication

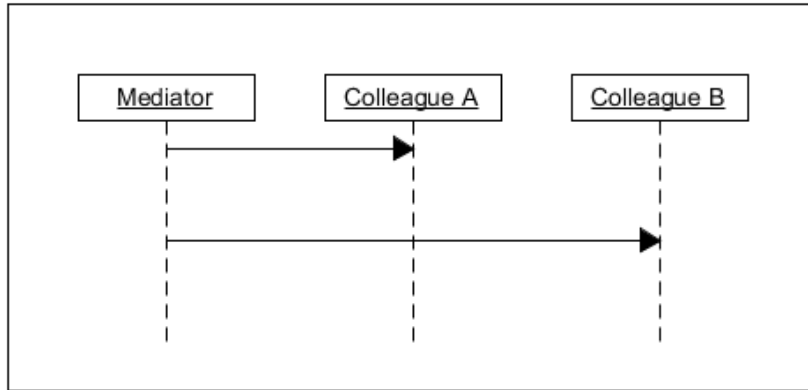
- Mediator

- Mediator encapsulates the communication
- mediators are hardly ever reusable
- easy to understand the flow of communication

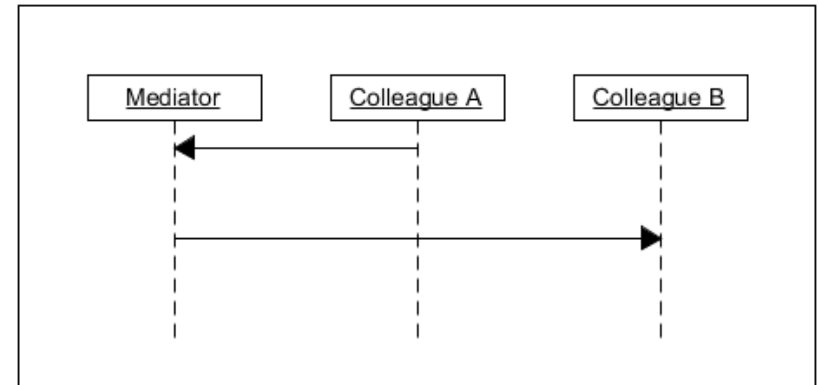
Mediator Pattern Quiz : Problem

- 다음 중 Mediator 패턴의 시퀀스 다이어그램으로 잘못된 것은?

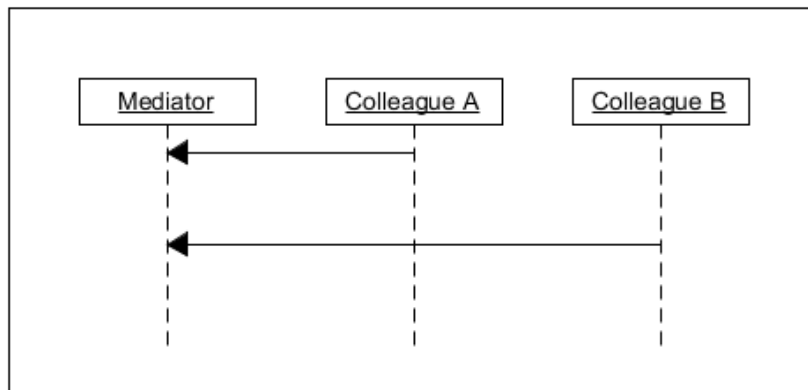
(a)



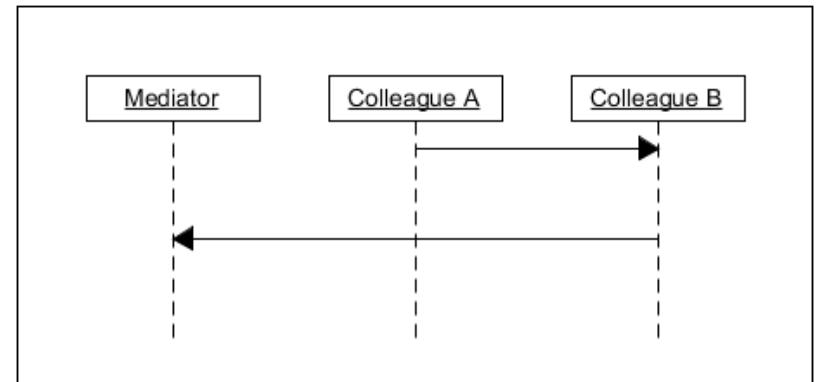
(b)



(c)



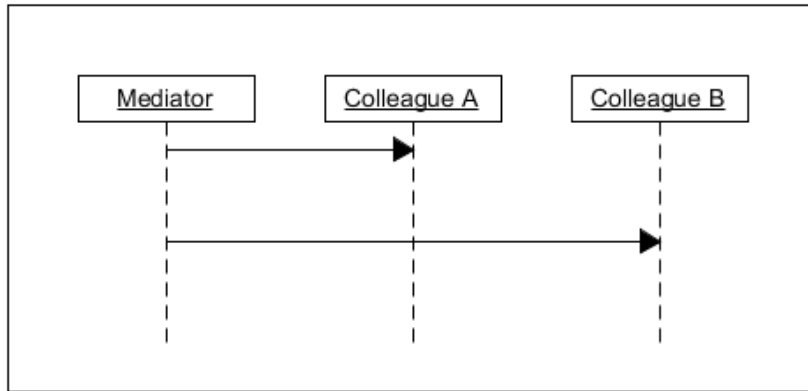
(d)



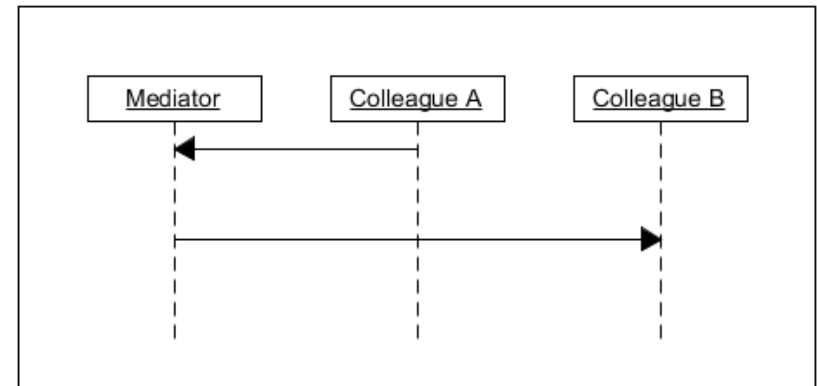
Mediator Pattern Quiz : Answer

- 다음 중 Mediator 패턴의 시퀀스 다이어그램으로 잘못된 것은?

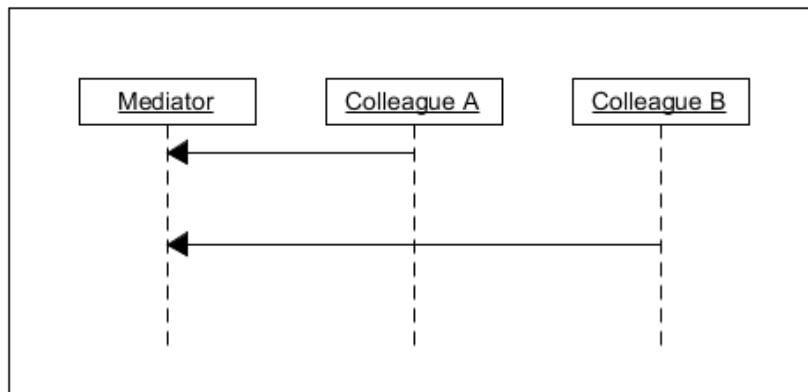
(a)



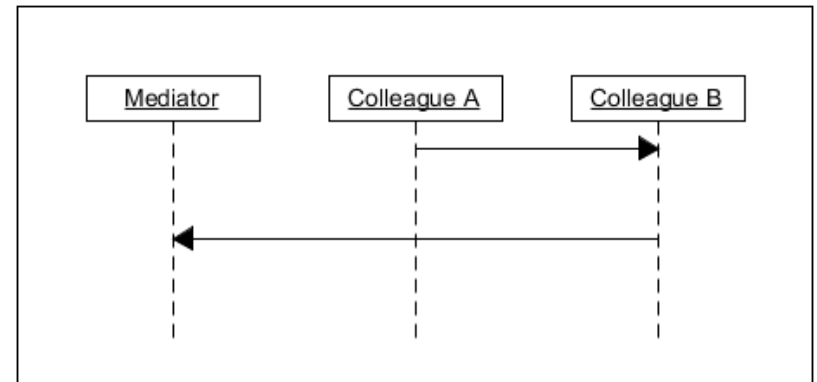
(b)



(c)



(d)



11. FACTORY METHOD PATTERN

ABSTRACT FACTORY PATTERN

2020. 04. 16

중앙대학교 소프트웨어학부

이찬근 교수

Contents

- Motivation of creational patterns
- Intro. to the Pizza Business
- Simple Factory (Not official)
- Factory Method Pattern
- Abstract Factory Method Pattern

Creating Objects

- Coding to an interface!

```
Duck duck = new MallardDuck();
```

- Duck – interface
- MallardDuck – concrete class

- Needs different ducks depending on the condition!

```
Duck duck;  
if (picnic)  
    duck = new MallardDuck();  
else if (hunting)  
    duck = new DecoyDuck();  
else if (inBathTub)  
    duck = new RubberDuck();
```

- But “new” operator needs Concrete Class!

Factory Patterns

- Creational patterns
 - Allow to creating new objects **without explicitly using the new operator**
 - We can instantiate different objects without modifying client code!
 - Examples: Factory Method, Abstract Factory, Singleton, Builder, Prototype
- Factory Method
 - Uses inheritance to decide the object to be instantiated
- Abstract Factory
 - Delegates object creation to a factory object

Original Code

```
Pizza orderPizza(String type) {  
    Pizza pizza;  
  
    if (type.equals("cheese"))  
        pizza = new CheesePizza();  
    else if (type.equals("greek"))  
        pizza = new GreekPizza();  
    else if (type.equals("pepperoni"))  
        pizza = new PepperoniPizza();  
  
    pizza.prepare();  
    pizza.bake();  
    pizza.cut();  
    pizza.box();  
    return pizza;  
}
```

Pizza types are changing!

```
Pizza orderPizza(String type) {  
    Pizza pizza;  
  
    if (type.equals("cheese"))  
        pizza = new CheesePizza();  
    else if (type.equals("greek"))  
        pizza = new GreekPizza();  
    else if (type.equals("pepperoni"))  
        pizza = new PepperoniPizza();  
  
    pizza.prepare();  
    pizza.bake();  
    pizza.cut();  
    pizza.box();  
  
    return pizza;  
}
```

```
Pizza orderPizza(String type) {  
    Pizza pizza;  
  
    if (type.equals("cheese"))  
        pizza = new CheesePizza();  
    else if (type.equals("clam"))  
        pizza = new ClamPizza();  
    else if (type.equals("pepperoni"))  
        pizza = new PepperoniPizza();  
  
    pizza.prepare();  
    pizza.bake();  
    pizza.cut();  
    pizza.box();  
  
    return pizza;  
}
```


Building a simple factory

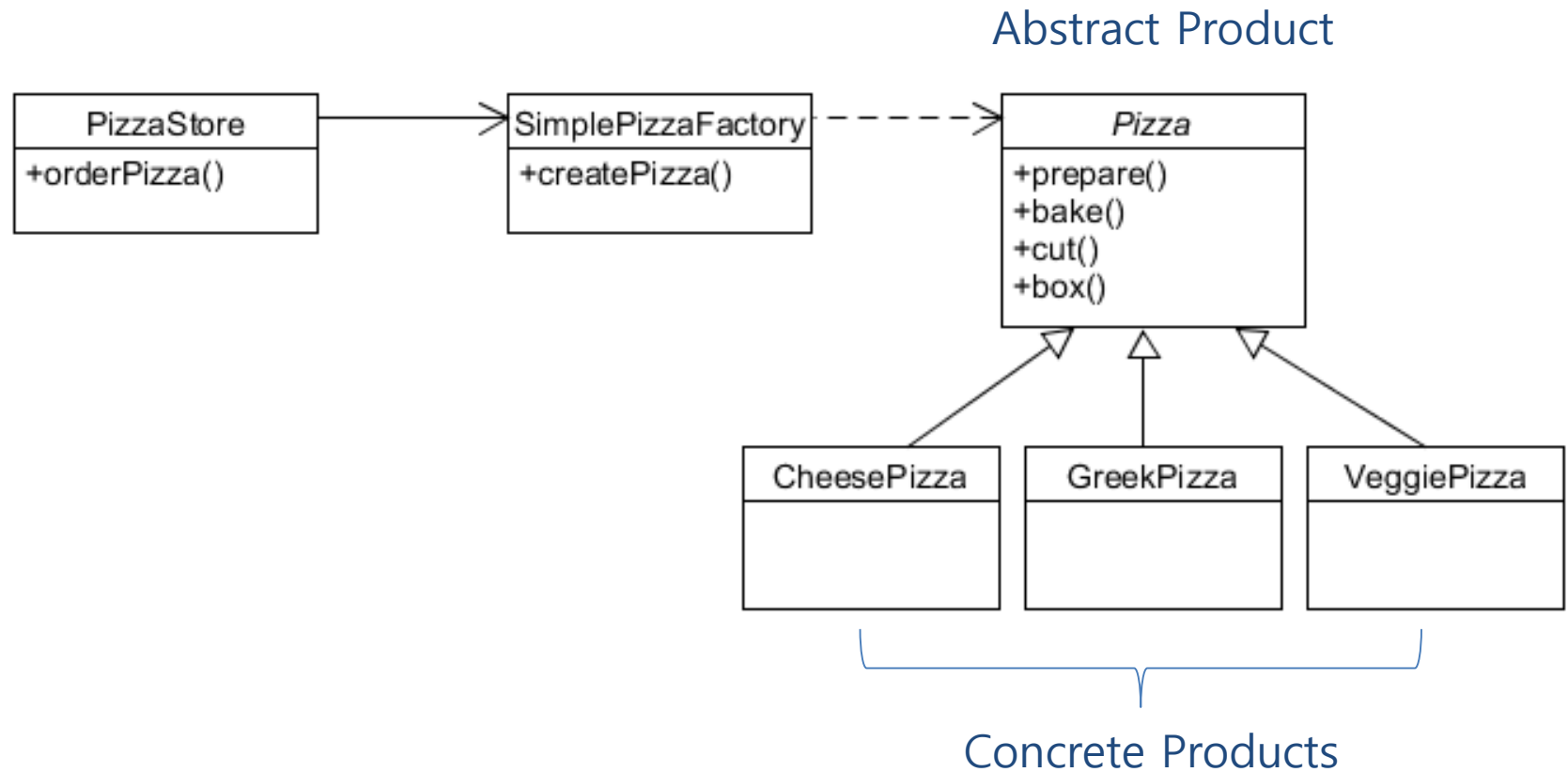
```
public class SimplePizzaFactory {  
    public Pizza createPizza(String type) {  
        Pizza pizza;  
  
        if (type.equals("cheese"))  
            pizza = new CheesePizza();  
        else if (type.equals("greek"))  
            pizza = new GreekPizza();  
        else if (type.equals("pepperoni"))  
            pizza = new PepperoniPizza();  
  
        return pizza;  
    }  
}
```

Revising PizzaStore class using the Simple Factory

```
public class PizzaStore {  
    SimplePizzaFactory factory;  
    public PizzaStore(SimplePizzaFactory factory) {  
        this.factory = factory;  
    }  
  
    public Pizza orderPizza(String type) {  
        Pizza pizza;  
        pizza = factory.createPizza(type);  
  
        pizza.prepare();  
        pizza.bake();  
        pizza.cut();  
        pizza.box();  
        return pizza;  
    }  
}
```

Q) It looks like we are just pushing the problem off to another object

Simple Factory (not an official pattern)



Franchising the pizza store

- Different styles
 - NY, Chicago, California, etc
- Different pizza factories
 - NYPizzaFactory, ChicagoPizzaFactory, CaliforniaPizzaFactory

```
NYPizzaFactory nyFactory = new NYPizzaFactory();  
PizzaStore nyStore = new PizzaStore(nyFactory);  
nyStore.order("Veggie");
```

```
ChicagoPizzaFactory chicagoFactory = new ChicagoPizzaFactory();  
PizzaStore chicagoStore = new PizzaStore(chicagoFactory);  
chicagoStore.order("Veggie");
```

Factory Method Pattern

- **Purpose**

- Exposes a method for creating objects, allowing subclasses to control the actual creation process.

- **Use When**

- A class will not know what classes it will be required to create.
 - Subclasses may specify what objects should be created.
 - Parent classes wish to defer creation to their subclasses.

Requirement Change

- Now we need want to make a framework which ties the store and the pizza creation

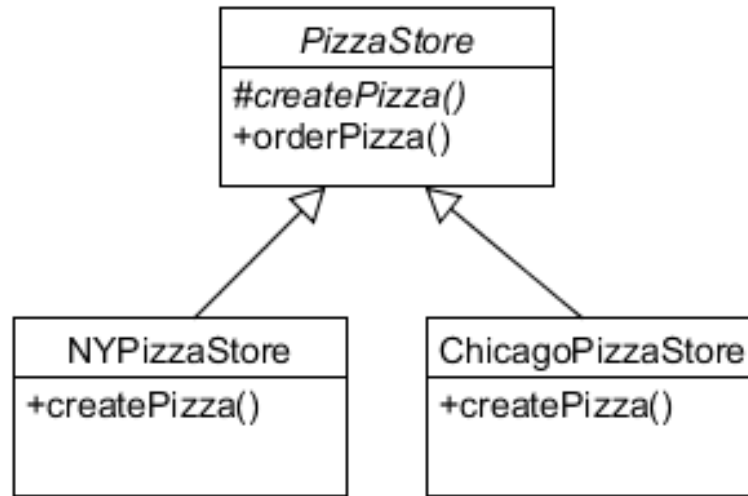
```
public abstract class PizzaStore {  
    public Pizza orderPizza(String type) {  
        Pizza pizza;  
        pizza = createPizza(type);  
  
        pizza.prepare();  
        pizza.bake();  
        pizza.cut();  
        pizza.box();  
        return pizza;  
    }  
    protected abstract Pizza createPizza(String type);  
}
```

Factory Method

- The example is a OO framework providing service “prepare pizza”.
 - The framework invokes the createPizza() factory method to create a pizza that it can prepare using a well-defined, consistent process.
 - A “client” of the framework will subclass this class and provide an implementation of the createPizza() method.
 - Any dependencies on concrete “product” classes are encapsulated in the subclass.

```
public abstract class PizzaStore {  
    public Pizza orderPizza(String type) {  
        Pizza pizza;  
        pizza = createPizza(type);  
  
        pizza.prepare();  
        pizza.bake();  
        pizza.cut();  
        pizza.box();  
        return pizza;  
    }  
    protected abstract Pizza createPizza(String type);  
}
```

Allowing the subclasses to decide



```
public Pizza createPizza(String type) {
    Pizza pizza;
    if (type.equals("cheese")) {
        pizza = new NYStyleCheesePizza();
    } else if (type.equals("veggie")) {
        pizza = new NYStyleVeggiePizza();
    }
    return pizza;
}
```

```
public Pizza createPizza(String type) {
    Pizza pizza;
    if (type.equals("cheese")) {
        pizza = new ChicagoStyleCheesePizza();
    } else if (type.equals("veggie")) {
        pizza = new ChicagoStyleVeggiePizza();
    }
    return pizza;
}
```


Ordering a pizza using the Factory Method

1. Need an instance of PizzaStore

```
PizzaStore nyPizzaStore = new NYPizzaStore();
```

2. Call orderPizza()

```
nyPizzaStore.orderPizza("cheese");
```

3. orderPizza() calls createPizza()

```
Pizza pizza = createPizza("cheese");
```

4. orderPizza() now prepare, bake, cut and box the pizza

```
pizza.prepare();
```

```
pizza.bake();
```

```
pizza.cut();
```

```
pizza.box();
```

Implementing Pizza class

```
public abstract class Pizza {
    String name;
    String dough;
    String sauce;
    ArrayList toppings = new ArrayList();
    void prepare(){
        System.out.println("Preparing " + name);
        System.out.println("Tossing dough...");
        System.out.println("Adding sauce...");
        System.out.println("Adding toppings: ");
        for (int i = 0; i < toppings.size(); i++)
            System.out.println("  " + toppings.get(i));
    }
    void bake(){
        System.out.println("Bake for 25 minutes at 350");
    }
    void cut(){
        System.out.println("Cutting the pizza into diagonal slices");
    }
    void box(){
        System.out.println("Place pizza in official PizzaStore box");
    }
}
```

Subclasses of Pizza

```
public class NYStyleCheesePizza extends Pizza {
    public NYStyleCheesePizza(){
        name = "NY Style Sauce and Cheese Pizza";
        dough = "Thin Crust Dough";
        sauce = "Marinara Sauce";
        toppings.add("Grated Reggiano Cheese");
    }
}

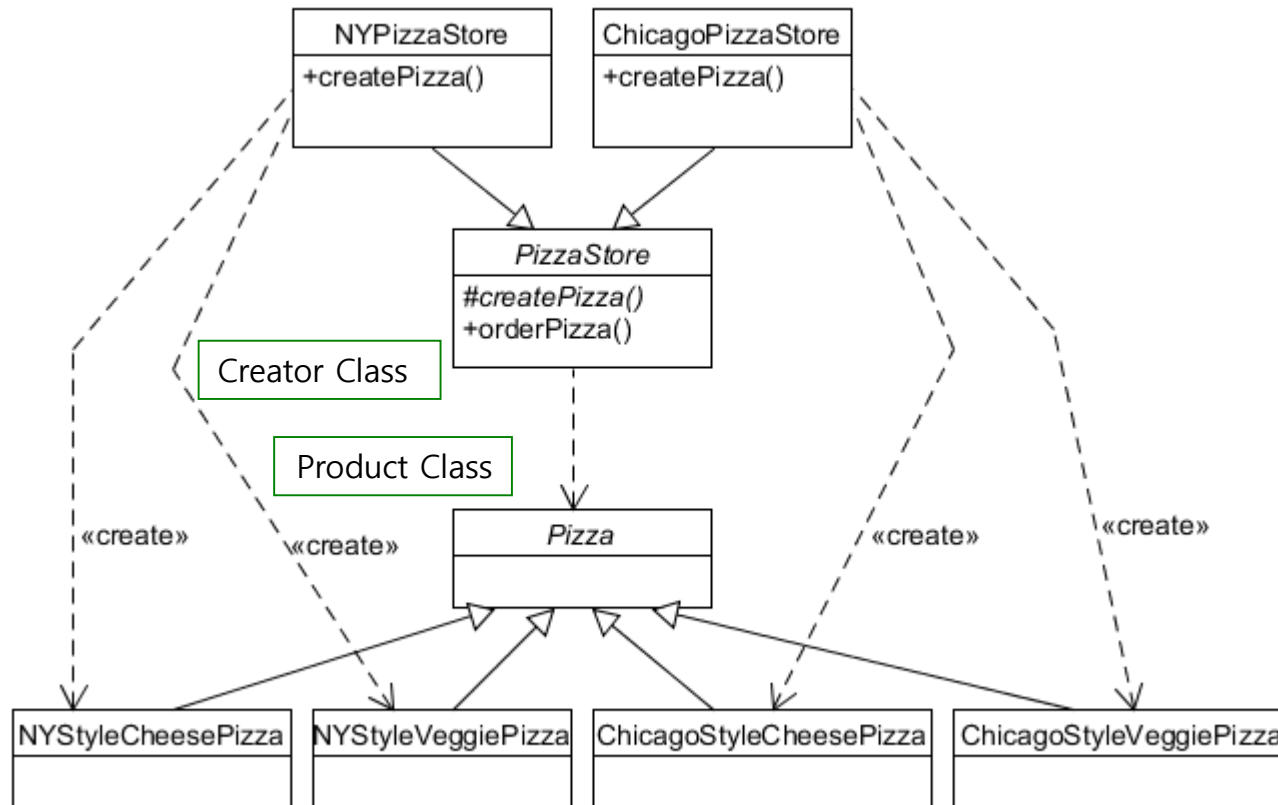
public class ChicagoStyleCheesePizza extends Pizza {
    public ChicagoStyleCheesePizza(){
        name = "Chicago Style Deep Dish and Cheese Pizza";
        dough = "Extra Thick Crust Dough";
        sauce = "Plum Tomato Sauce";
        toppings.add("Shredded Mozzarella Cheese");
    }
    void cut(){
        System.out.println("Cutting the pizza into square slices");
    }
}
```

Test Drive

```
public class PizzaTestDrive {  
    public static void main(String[] args){  
        PizzaStore nyStore = new NYPizzaStore();  
        PizzaStore chicagoStore = new ChicagoPizzaStore();  
  
        Pizza pizza = nyStore.orderPizza("cheese");  
        System.out.println("Ethan ordered a " + pizza.getName());  
        Pizza pizza = chicagoStore.orderPizza("cheese");  
        System.out.println("Joel ordered a " + pizza.getName());  
    }  
}
```

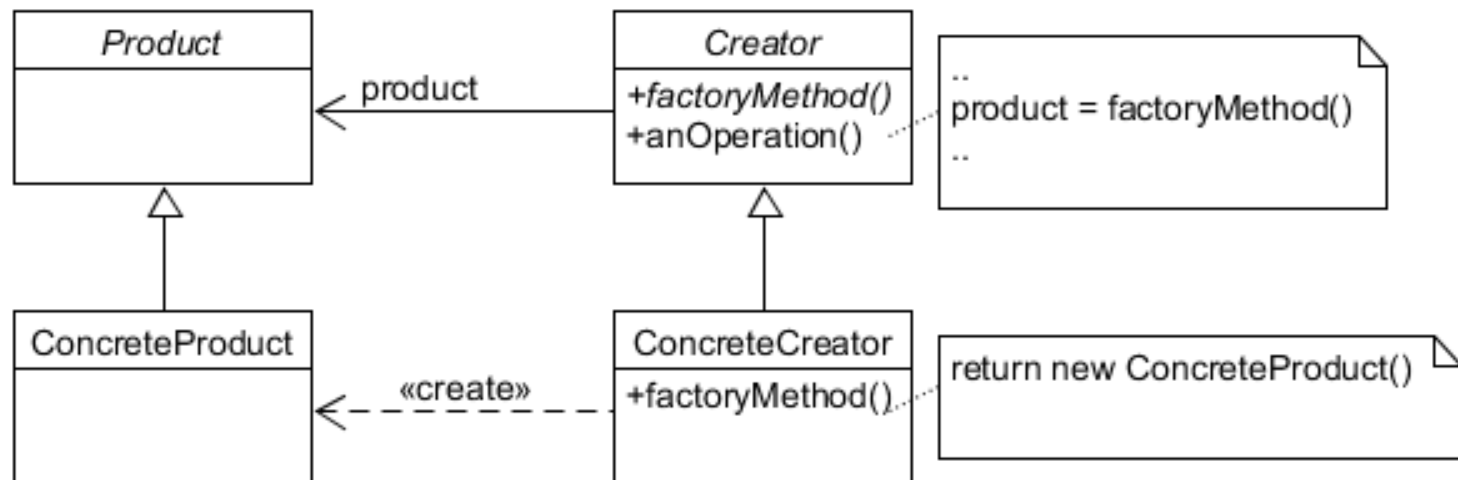
Applying Factory Method Pattern

- The Factory Method pattern encapsulates object creation by letting subclasses decide what objects to create



Factory Method Pattern

- Defines an interface for creating an object, but lets subclasses decide which class to instantiate.
- Factory Method lets a class defer instantiation to subclasses.



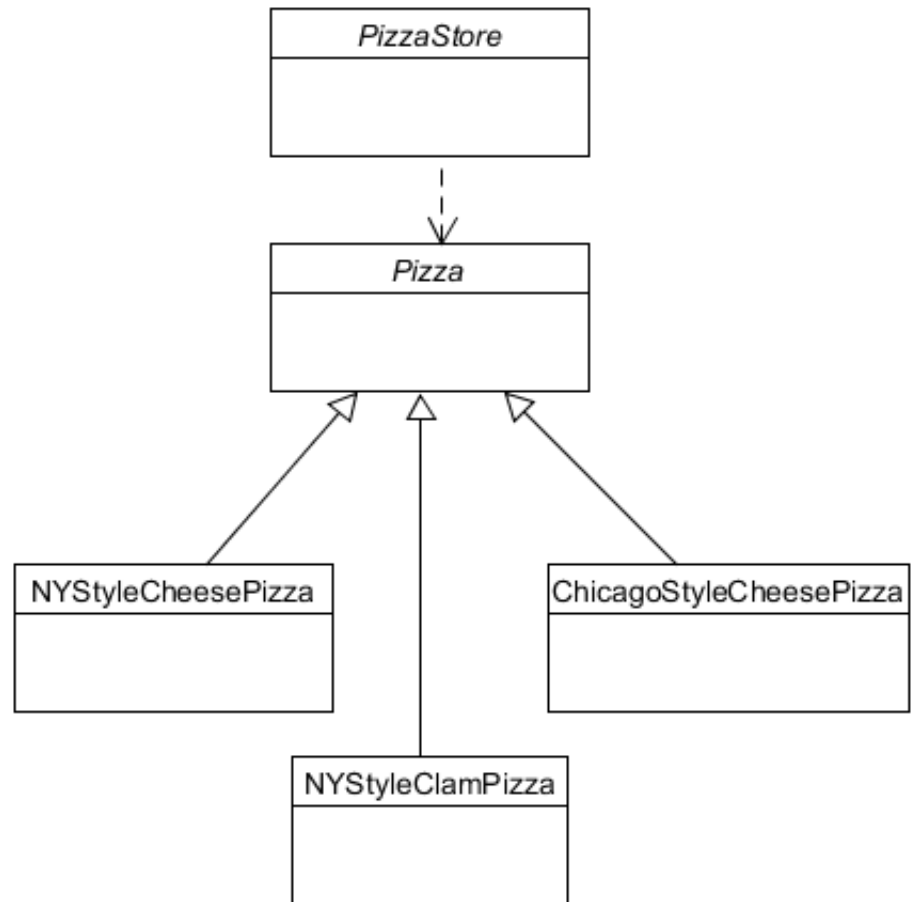
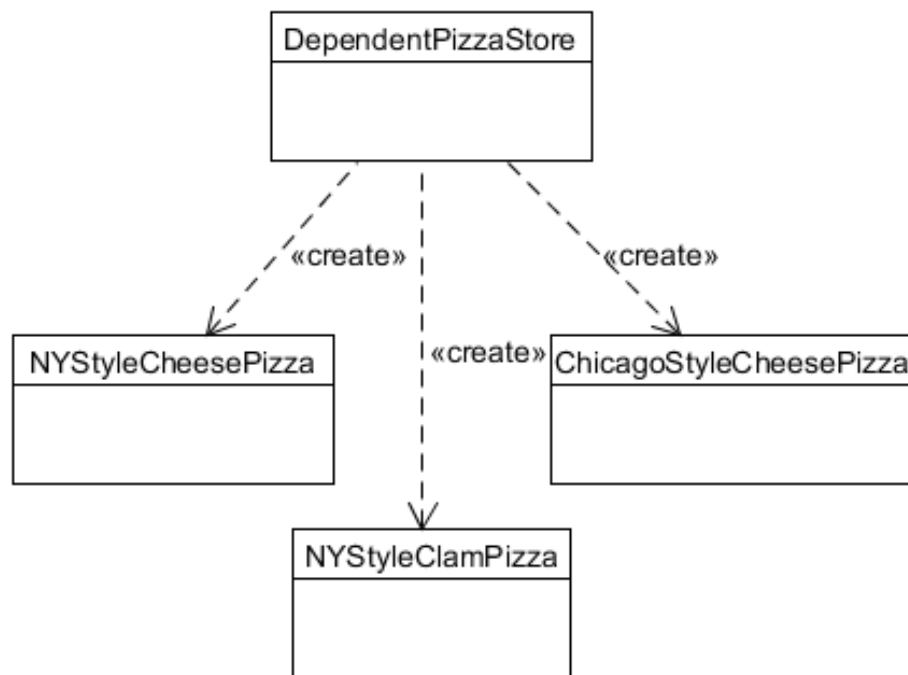
A very dependent PizzaStore (bad design)

```
public class DependentPizzaStore {
    public Pizza createPizza(String style, String type) {
        Pizza pizza = null;
        if (style.equals("NY")) {
            if (type.equals("cheese")) {
                pizza = new NYStyleCheesePizza();
            } else if (type.equals("veggie")) {
                pizza = new NYStyleVeggiePizza();
            }
        } else if (style.equals("Chicago")) {
            if (type.equals("cheese")) {
                pizza = new ChicagoStyleCheesePizza();
            } else if (type.equals("veggie")) {
                pizza = new ChicagoStyleVeggiePizza();
            }
        } else {
            System.out.println("Error: invalid type of pizza");
            return null;
        }
        pizza.prepare();
        pizza.bake();
        pizza.cut();
        pizza.box();
        return pizza;
    }
}
```

Design Principle: Dependency Inversion Principle

- Dependency Inversion Principle
 - Depend upon abstractions. Do not depend upon concrete classes.
 - High-level components should not depend on low-level components; rather, they should both depend on abstractions
- Factory Method is one way of following the dependency inversion principle

Dependent PizzaStore VS Factory Method



Factory Method Pattern

- What exactly does it mean by saying that “the Factory Method Pattern lets subclasses decide which class to instantiate?”
 - **Creator class** is written in such a fashion that it does not know what actual Concrete Product class will be instantiated. The **Concrete Product** class to be instantiated is determined solely by which **Concrete Creator** subclass is instantiated and used by the application.
 - It does *not* mean that the subclass decides at runtime which ConcreteProduct class to create

Abstract Factory Pattern

▪ Purpose

- Provide an interface that delegates creation calls to one or more concrete classes in order to deliver specific objects.

▪ Use When

- The creation of objects should be independent of the system utilizing them.
- Systems should be capable of using multiple families of objects.
- Families of objects must be used together.
- Libraries must be published without exposing implementation details.
- Concrete classes should be decoupled from clients.

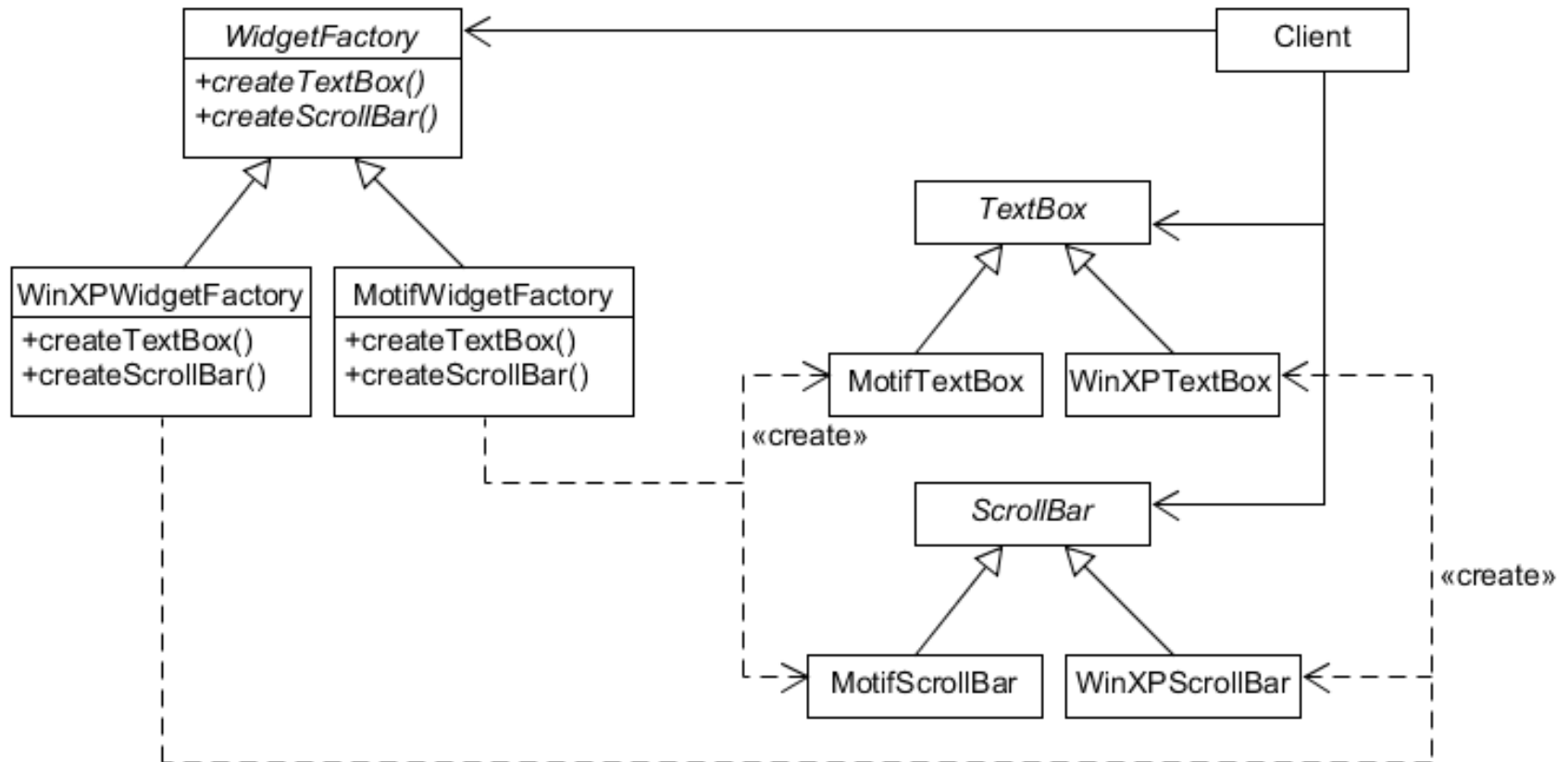
Factory Patterns

- Abstract factory pattern is one level of abstraction higher than the factory method pattern
- To delegate or defer the responsibility of object instantiation to another object or class
 - **Abstract Factory** pattern uses **composition**
 - **Factory Method** pattern uses **inheritance**

Example

- Building a user interface toolkit that supports multiple look and feel standards
 - Windows XP, MAC OS X, Motif, X Window
- Have different appearances and behavior for a large set of subclasses
 - scroll bars, windows, buttons, etc.

Abstract Factory Example

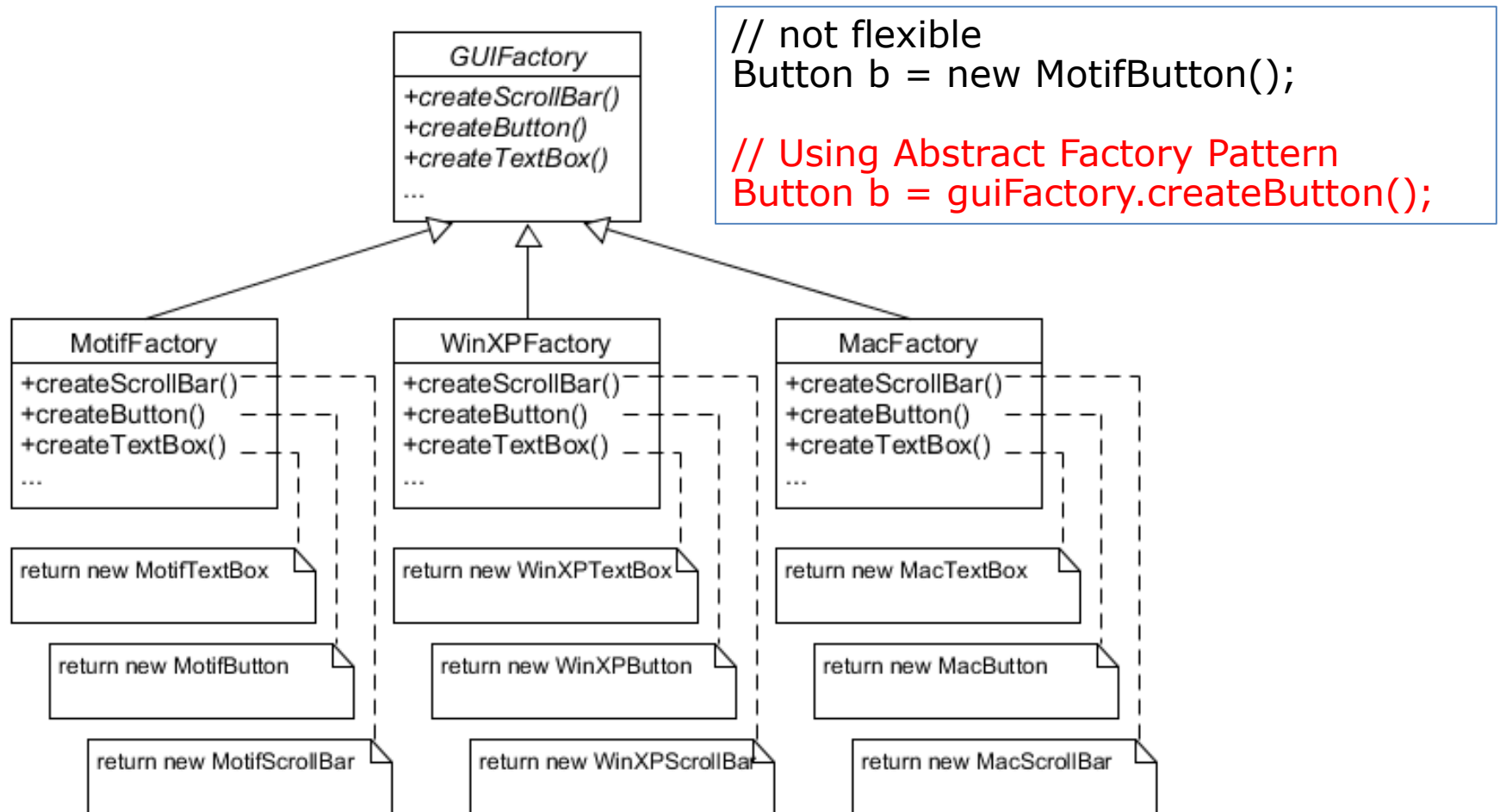


Factory Object Set-up and Delegation

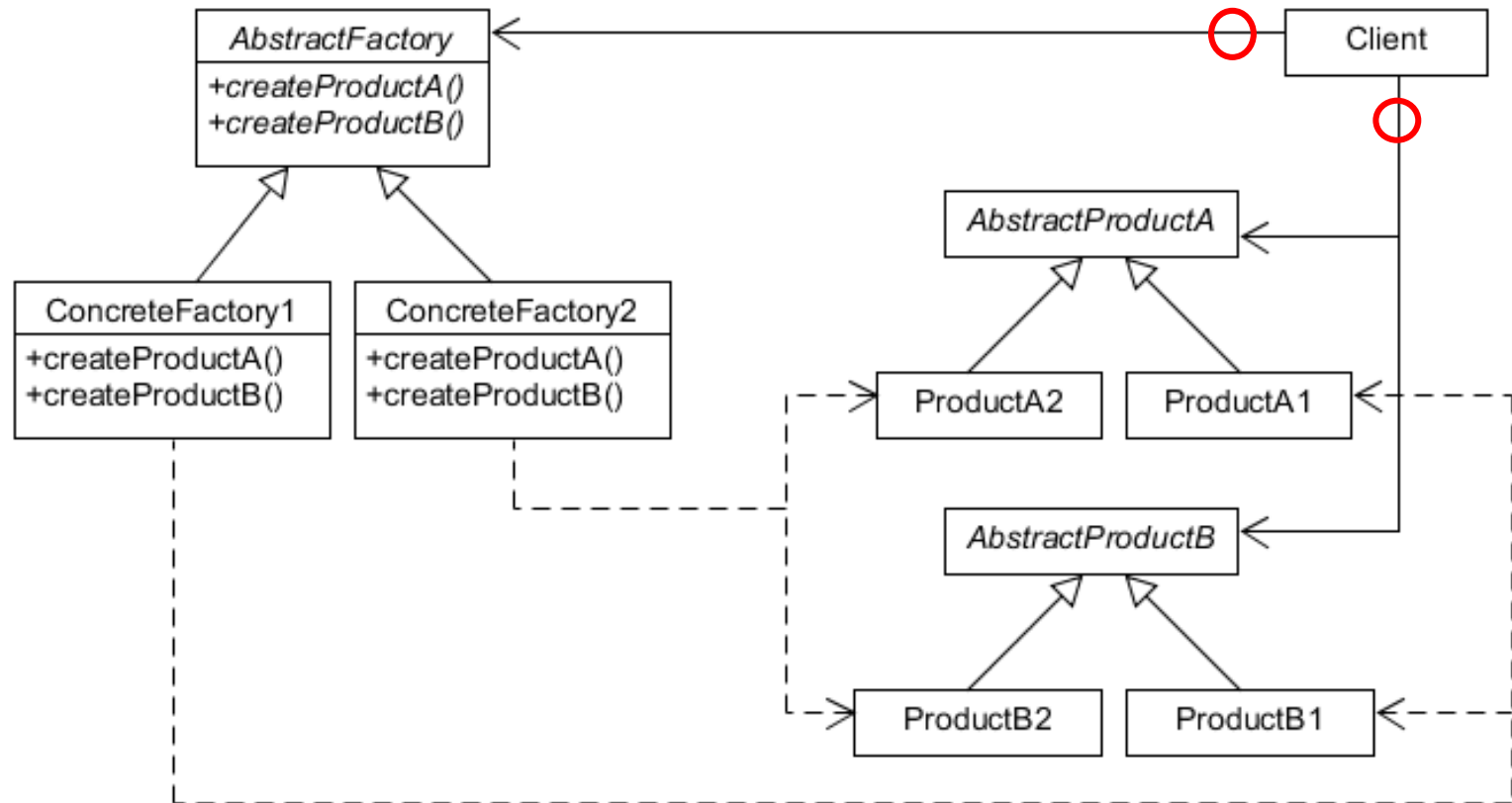
```
public class UIToolkit {  
    widgetFactory wfactory;  
  
    public UIToolkit (widgetFactory wf)  
        this.wfactory = wf;  
    }  
  
    public void PopUpPrinterDialog() {  
        Dialogwindow d= wfactory.createDialogwindow("Printer Setup");  
        d.add ( wfactory.createButton(OK_Button) );  
        d.add ( wfactory.createButton(Cancel_Button) );  
        d.showWindow();  
    }  
}
```

```
public class DriverForMotif {  
    // setup a specific widget factory  
    widgetFactory wf = new winXPWidgetFactory();  
  
    // start a client program  
    UIToolkit ui = new UIToolkit(wf);  
  
    // rest of code  
}
```

Class Diagram for GUIFactory



Class Diagram for Abstract Factory Pattern



Participants

- **AbstractFactory**
 - Declares an interface for operations that create abstract product objects
- **ConcreteFactory**
 - Implements the operations to create concrete product objects
- **AbstractProduct**
 - Declares an interface for a type of product object
- **ConcreteProduct**
 - Defines a product object to be created by the corresponding concrete factory
 - Implements the AbstractProduct interface
- **Client**
 - **Uses only interfaces** declared by AbstractFactory and AbstractProduct classes

Making factories for ingredients (Abstract Factory)

```
public interface PizzaIngredientFactory {  
    public Dough createDough();  
    public Sauce createSauce();  
    public Cheese createCheese();  
    public Veggies[] createVeggies();  
    public Pepperoni createPepperoni();  
    public Clams createClam();  
}
```

Concrete PizzaIngredientFactory (Factory Object)

```
public class NYPizzaIngredientFactory implements PizzaIngredientFactory {  
    public Dough createDough() {  
        return new ThinCrustDough();  
    }  
    public Sauce createSauce() {  
        return new MarinaraSauce();  
    }  
    public Cheese createCheese() {  
        return new ReggianoCheese();  
    }  
    public Veggies[] createVeggies() {  
        Veggies veggies[] = {new Garlic(), new Onion(), new Mushroom(),  
                               new RedPepper() };  
        return veggies;  
    }  
    public Pepperoni createPepperoni() {  
        return new SlicedPepperoni();  
    }  
    public Clams createClam() {  
        return new FreshClams();  
    }  
}
```

Pizza product class

```
public abstract class Pizza {
    String name;
    Dough dough;
    Sauce sauce;
    Veggies veggies[];
    Cheese cheese;

    abstract public void prepare();
    public void bake() {
        System.out.println("Bake for 25 minutes at 350");
    }
    public void cut() {
        System.out.println("Cutting the pizza into diagonal slices");
    }
    public void box() {
        System.out.println("Place pizza in official PizzaStore box");
    }
    public void setName(String name) {
        this.name = name;
    }
    public String getName() {
        return name;
    }
    public String toString() {
        return this.getName();
    }
}
```

```
public class NYPizzaStore extends PizzaStore {
    protected Pizza createPizza(String item) {
        Pizza pizza = null;
        PizzaIngredientFactory ingredientFactory = new NYPizzaIngredientFactory();
        if (item.equals("cheese")) {
            pizza = new CheesePizza(ingredientFactory);
            pizza.setName("New York Style Cheese Pizza");
        } else if (item.equals("veggie")) {
            pizza = new VeggiePizza(ingredientFactory);
            pizza.setName("New York Style Veggie Pizza");
        } else if (item.equals("clam")) {
            pizza = new ClamPizza(ingredientFactory);
            pizza.setName("New York Style Clam Pizza");
        } else if (item.equals("pepperoni")) {
            pizza = new PepperoniPizza(ingredientFactory);
            pizza.setName("New York Style Pepperoni Pizza");
        }
        return pizza;
    }
}
```

```
public class CheesePizza extends Pizza {
    PizzaIngredientFactory ingredientFactory;
    public CheesePizza(PizzaIngredientFactory ingredientFactory) {
        this.ingredientFactory = ingredientFactory;
    }
    void prepare() {
        System.out.println("Preparing " + name);
        dough = ingredientFactory.createDough();
        sauce = ingredientFactory.createSauce();
        cheese = ingredientFactory.createCheese();
    }
}
```

What has changed?

1. Instantiating PizzaStore

```
PizzaStore nyPizzaStore = new NYPizzaStore();
```

2. Order

```
nyPizzaStore.orderPizza("cheese");
```

3. orderPizza() calls createPizza()

```
Pizza pizza = createPizza("cheese");
```

/ BEGIN of Changed Flow for Ingredient Factory */**

4. createPizza() now needs ingredientFactory

```
Pizza pizza = new CheesePizza(nyIngredientFactory);
```

5. prepare() creates ingredients using ingredient factory

```
void prepare(){  
    dough = ingredientFactory.createDough();  
    sauce = ingredientFactory.createSauce();  
    cheese = ingredientFactory.createCheese();  
}
```

/ END of Changed Flow for Ingredient Factory */**

6. Now it is ready. orderPizza() calls bake(), cut(), box()

When to Use Abstract Factory Pattern?

- Common to creational patterns
 - A system must be independent of how its products are created, composed, and represented
 - A class cannot anticipate the class of objects it must create
- Specific to Abstract Factory pattern
 - A system should use one of a set of **families of products**
 - A **family of related product objects** is designed to be used together, and the **constraint** needs to be enforced

Abstract Factory Pattern

■ Consequences

(+) Isolates concrete classes

- Factory encapsulates responsibility and process of creating parts
- Isolates clients from implementation classes

(+) Exchanging product families easy

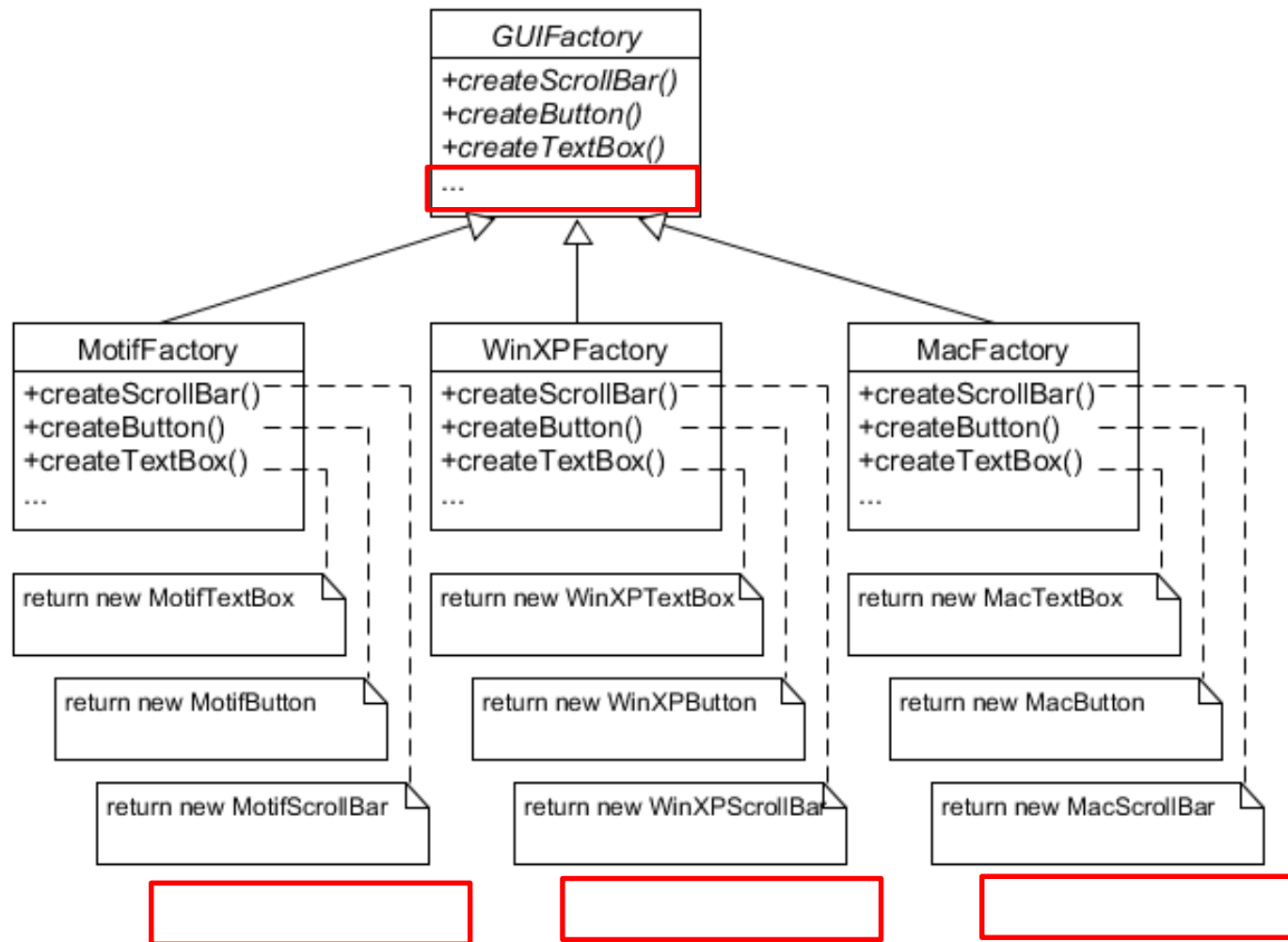
- Concrete factory appears once where it is instantiated

(+) Promotes consistency among products

(-) Supporting new kinds of products is difficult

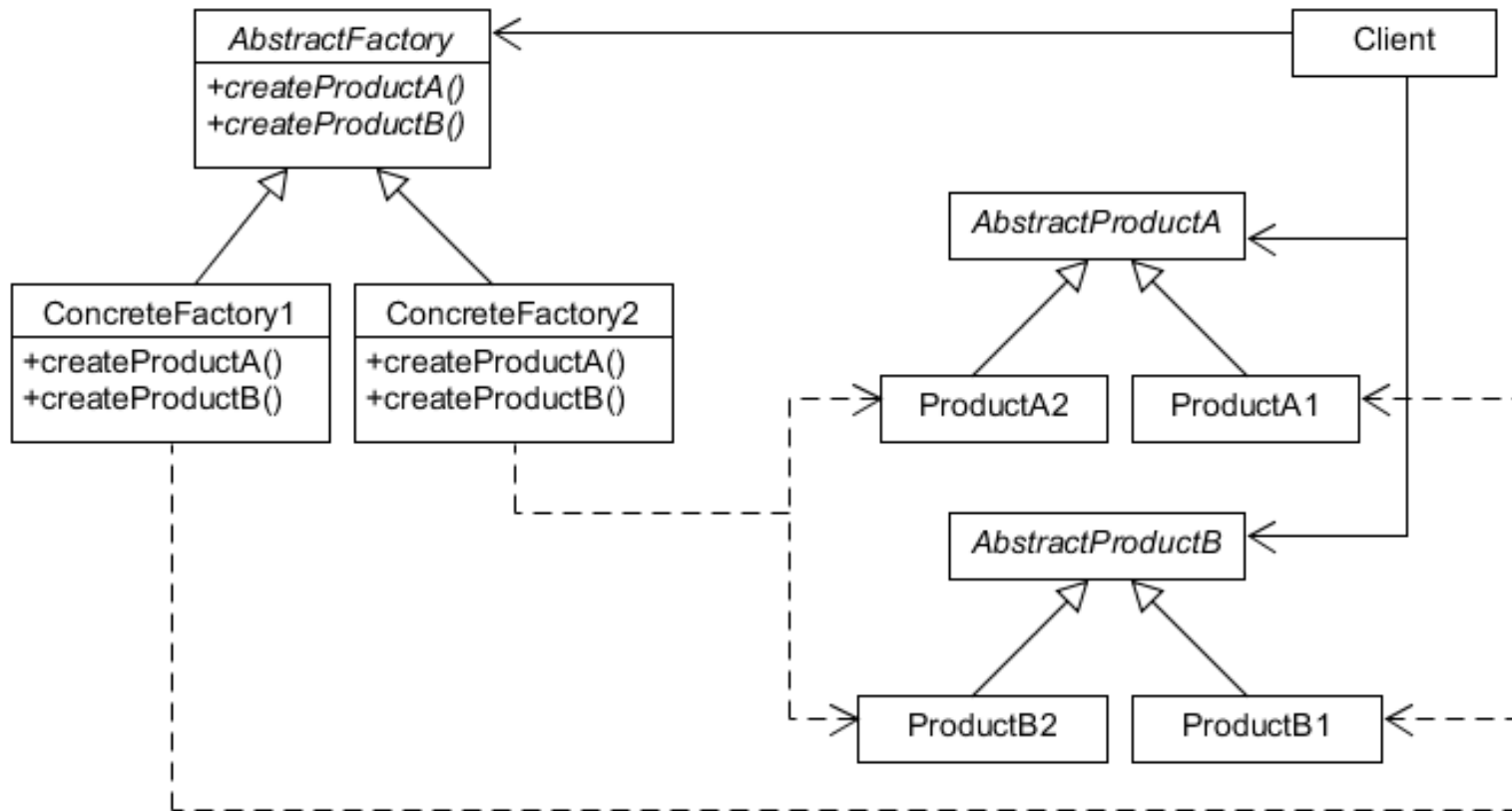
- Fixes set of parts to be created

Abstract Factory Pattern



Supporting new kinds of products is difficult (not impossible, but costly)

Abstract Factory Pattern



Supporting additional factory object is easy

Related Patterns

- Abstract Factory
 - classes are often implemented with Factory Methods
 - but they can also be implemented using Prototype
- Easy start → Factory Method
 - less complicated, more customizable, subclasses proliferate
 - Later evolve toward Abstract Factory, Prototype, or Builder when more flexibility is needed.

Summary

- Dependency Inversion Principle (DIP)
 - Depend on abstractions. Do not depend on concrete classes
- Factory Method
 - Define an **interface for creating an object**, but **let subclasses decide** which class to instantiate.
 - Class-scope pattern (uses only inheritance)
- Abstract Factory
 - Provide an interface for creating **families of** related or dependent **objects** without specifying their concrete classes
 - Object-scope pattern (uses object composition & delegation)

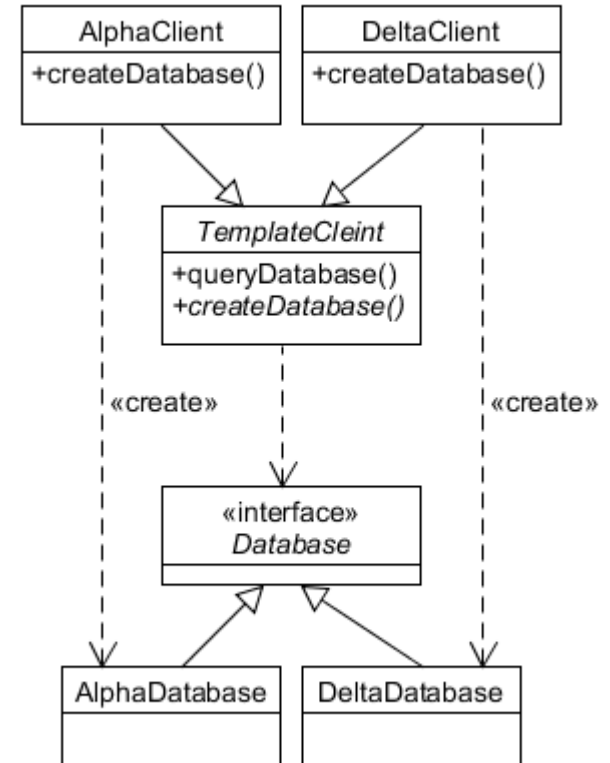
Factory Method Pattern Quiz : Problem

```
public interface Database {}  
public class AlphaDatabase implements Database {}  
public class DeltaDatabase implements Database {}  
  
public class BadClient {  
    public Map queryDatabase(String queryString) {  
        Database db = new AlphaDatabase();  
        checkQueryIsValid(queryString);  
        return db.find(queryString);  
    }  
}
```

- Apply the Factory Method Pattern and revise the above code
- Hint: Database is a product

Factory Method Pattern Quiz : Answer

```
public abstract class TemplateClient {  
    public Map queryDatabase(String queryString) {  
        Database db = createDatabase ();  
        checkQueryIsValid(queryString);  
        return db.find(queryString);  
    }  
    protected abstract Database createDatabase ();  
}  
  
public class AlphaClient extends TemplateClient {  
    public Database createDatabase () {  
        return new AlphaDatabase();  
    }  
}  
  
public class DeltaClient extends TemplateClient {  
    public Database createDatabase () {  
        return new DeltaDatabase();  
    }  
}
```



12. BUILDER PATTERN

2020. 04. 16

중앙대학교 소프트웨어학부

이찬근 교수

Contents

- Structure of Builder pattern
- Builder for Aircraft creation example
- Builder for TextConverter example
- Related patterns

Builder Pattern

▪ Purpose

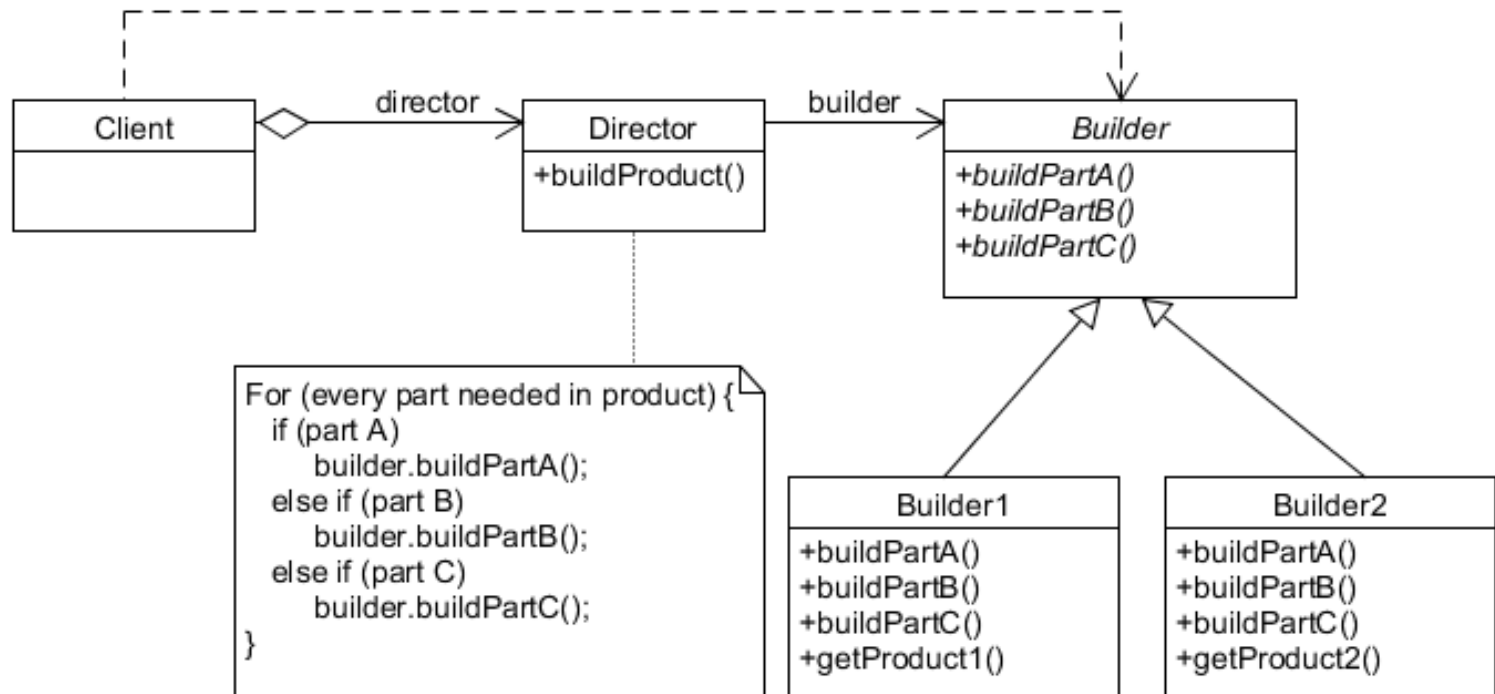
- Allows for the dynamic **creation** of objects based upon **easily interchangeable** algorithms.

▪ Use When

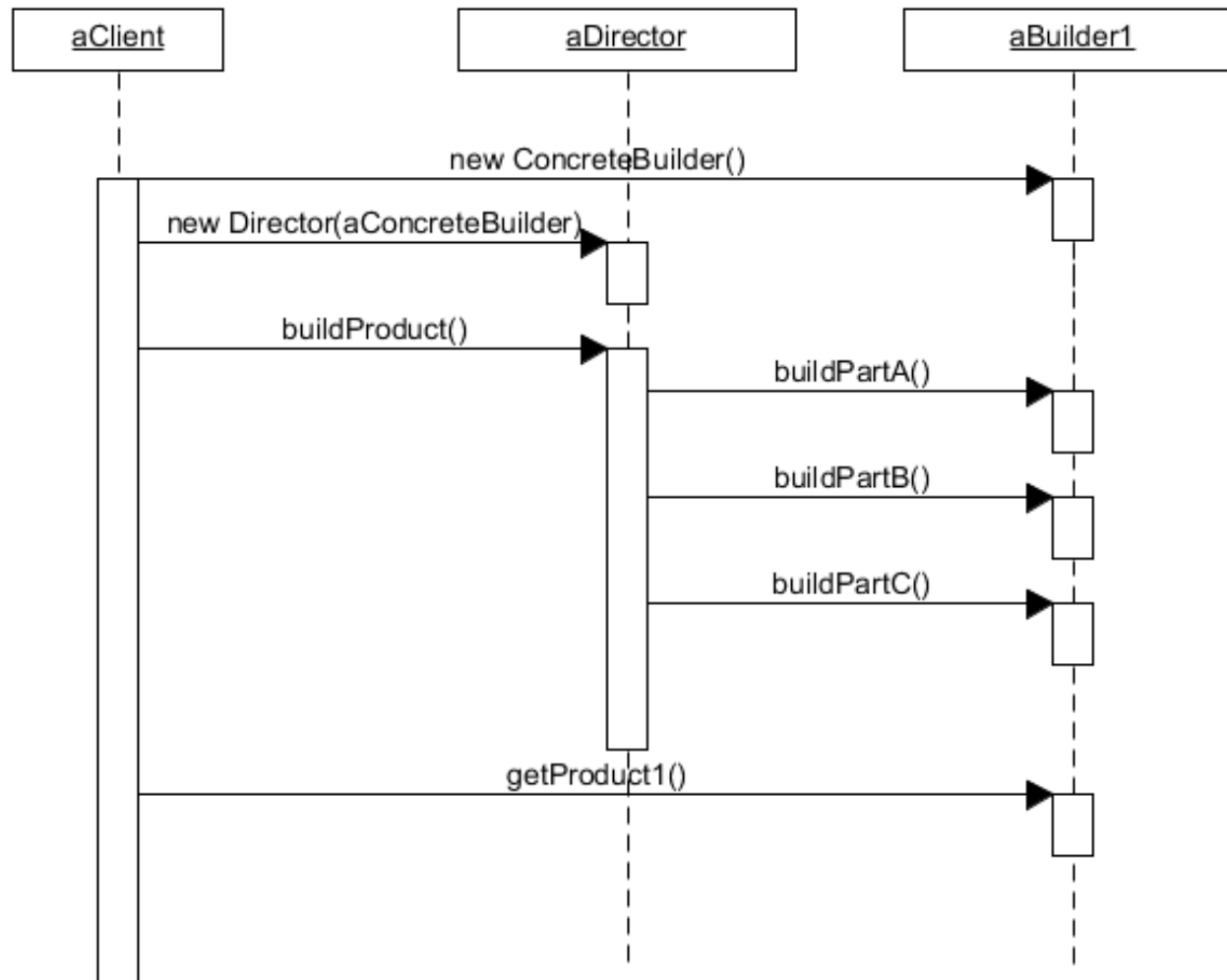
- Runtime control over the creation process is required.
- Multiple representations of creation algorithms are required.
- Object creation algorithms should be decoupled from the system.
- The addition of new creation functionality without changing the core code is necessary.

Builder - Structure

- **Director** knows **what parts are needed** for the final product
- **Concrete builder** knows **how to produce the part** and **add it** to the final product



Builder - Collaborations



Builder - Participants

- Client
 - selects director and concrete builder to build the product
 - asks concrete builder to return final constructed product
- Director
 - knows **what steps** it takes to build a product
 - but it **does not know how each step** is to be carried out
- Builder
 - specifies an abstract interface for creating parts of a Product object
- Concrete Builder
 - constructs and assembles parts of the product by implementing the Builder interface
 - **defines and keeps track of the representation** it creates
 - provides an **interface for retrieving the product**
- Product
 - represents the **complex object** under construction

Example: Building Airplanes

- **AerospaceEngineer**: director
- **AirplaneBuilder**: abstract builder
- **Airplane**: product
- Sample concrete builders:
 - **CropDuster**
 - **FighterJet**
 - **Glider**

Director

```
public class AerospaceEngineer {  
    private AirplaneBuilder airplaneBuilder;  
  
    public void setAirplaneBuilder(AirplaneBuilder ab) {  
        airplaneBuilder = ab;  
    }  
  
    public Airplane getAirplane() {  
        return airplaneBuilder.getAirplane();  
    }  
  
    public void constructAirplane() {  
        airplaneBuilder.createNewAirplane();  
        airplaneBuilder.buildwings();  
        airplaneBuilder.buildPowerplant();  
        airplaneBuilder.buildAvionics();  
        airplaneBuilder.buildSeats();  
    }  
}
```

Abstract Builder

```
public abstract class AirplaneBuilder {  
  
    protected Airplane airplane;  
    protected String customer;  
    protected String type;  
  
    public Airplane getAirplane() {  
        return airplane;  
    }  
  
    public void createNewAirplane() {  
        airplane = new Airplane(customer, type);  
    }  
  
    public abstract void buildwings();  
    public abstract void buildPowerplant();  
    public abstract void buildAvionics();  
    public abstract void buildSeats();  
  
}
```


Product

```
public class Airplane {
    private String type;
    private float wingspan;
    private String powerplant;
    private int crewSeats;
    private int passengerSeats;
    private String avionics;
    private String customer;

    Airplane (String customer, String type) {
        this.customer = customer;
        this.type = type; }

    public void setwingspan(float w) { this.wingspan = w; }
    public void setPowerplant(String p) { this.powerplant = p; }
    public void setAvionics(String a) { this.avionics = a; }
    public void setNumberSeats(int crewSeats, int passengerSeats) {
        this.crewSeats = crewSeats;
        this.passengerSeats = passengerSeats;
    }
    public String getCustomer() { return customer; }
    public String getType() { return type; }
}
```

Concrete Builder 1

```
public class CropDuster extends AirplaneBuilder {  
  
    CropDuster (String customer){  
        super.customer = customer;  
        super.type = "Crop Duster v3.4";  
    }  
  
    public void buildwings() {  
        airplane.setWingspan(9f);  
    }  
  
    public void buildPowerplant() {  
        airplane.setPowerplant("single piston");  
    }  
  
    public void buildAvionics() {}  
  
    public void buildSeats() {  
        airplane.setNumberSeats(1,1);  
    }  
  
}
```

Concrete Builder 2

```
public class FighterJet extends AirplaneBuilder {  
    FighterJet (String customer){  
        super.customer = customer;  
        super.type = "F-35 Lightning II";  
    }  
  
    public void buildwings() {  
        airplane.setwingspan(35.0f);  
    }  
  
    public void buildPowerplant() {  
        airplane.setPowerplant("dual thrust vectoring");  
    }  
  
    public void buildAvionics() {  
        airplane.setAvionics("military");  
    }  
  
    public void buildSeats() {  
        airplane.setNumberSeats(1,0);  
    }  
}
```

Concrete Builder 3

```
public class Glider extends AirplaneBuilder {  
  
    Glider (String customer) {  
        super.customer = customer;  
        super.type = "Glider v9.0";  
    }  
  
    public void buildwings() {  
        airplane.setwingspan(57.1f);  
    }  
  
    public void buildPowerplant() { }  
  
    public void buildAvionics() { }  
  
    public void buildSeats() {  
        airplane.setNumberSeats(1,0);  
    }  
  
}
```

Client Application

```
public class BuilderExample {  
    public static void main(String[] args) {  
        // instantiate the director (hire the engineer)  
        AerospaceEngineer aero = new AerospaceEngineer();  
  
        // instantiate each concrete builder (take orders)  
        AirplaneBuilder crop = new CropDuster("Farmer Joe");  
        AirplaneBuilder fighter = new FighterJet("The Navy");  
        AirplaneBuilder glider = new Glider("Tim Rice");  
  
        // build a CropDuster  
        aero.setAirplaneBuilder(crop);  
        aero.constructAirplane();  
        Airplane completedCropDuster = aero.getAirplane();  
        System.out.println(completedCropDuster.getType() +  
            " is completed and ready for delivery to " +  
            completedCropDuster.getCustomer());  
    }  
}
```

Client Application output

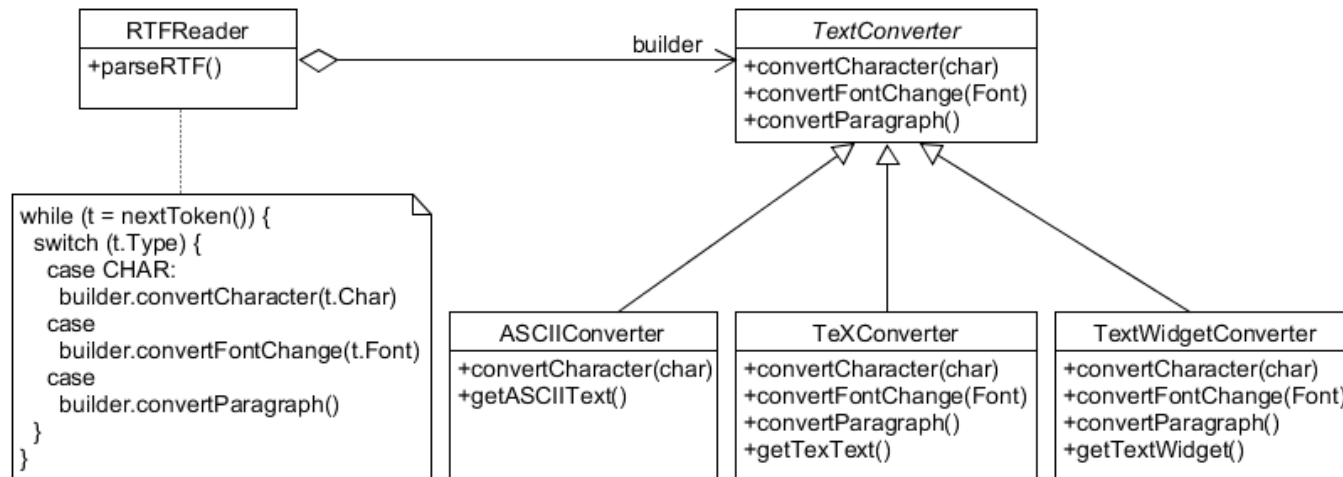
Crop Duster v3.4 is completed and ready for delivery to Farmer Joe

F-35 Lightning II is completed and ready for delivery to the Navy

Glider v9.0 is completed and ready for delivery to Tim Rice

Real-life Example

- RTF reader should be able to convert RTF to many text format
- Adding new conversions without modifying the reader should be easy



- **Separate** the **construction of a complex object** from its representation so that the same construction process can create different representations.

When a Builder Shouldn't Be Used

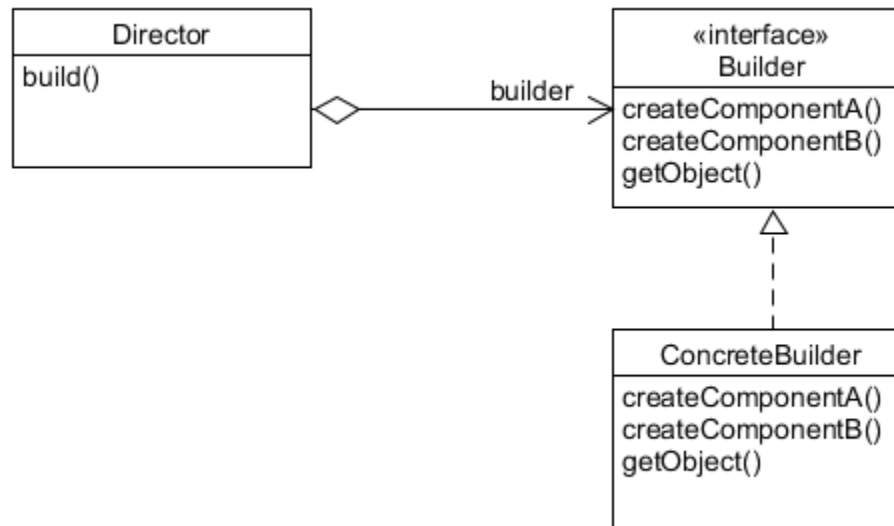
- If the interface is not stable the Builder has few benefits
 - Every interface change requires a change to the Controller and impacts the abstract base class or its objects
 - A new method would require changing the base class and all concrete classes that will need to override the new method
 - A specific method interface change would require all concrete classes supporting the old method to change

Related Patterns and Summary

- A Composite is what the Builder often builds
- A Builder is a Strategy that is specialized to create a composite object or data structure
- Comparison with Abstract Factory
 - Builder constructs the object **step-by-step** and the **result is requested at a later stage**
 - Abstract factory returns the requested object immediately
 - Abstract factory does not have an abstract builder; application calls the factory methods directly

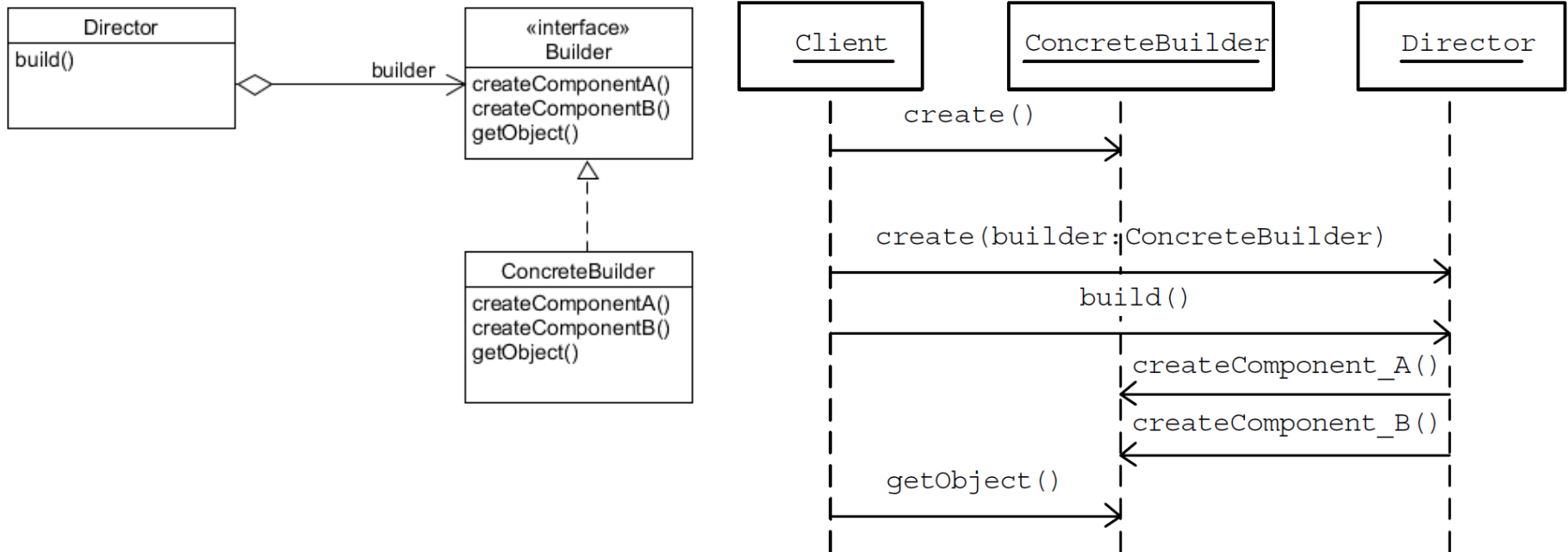
Builder Pattern Quiz : Problem

- 다음과 같이 빌더 패턴이 적용된 설계에 대해, 클라이언트가 완성된 복합 객체를 얻어내는 과정을 UML 시퀀스 다이어그램으로 표현하시오.



Builder Pattern Quiz : Answer

- 다음과 같이 빌더 패턴이 적용된 설계에 대해, 클라이언트가 완성된 복합 객체를 얻어내는 과정을 UML 시퀀스 다이어그램으로 표현하시오.



13. SINGLETON PATTERN

2019. 04. 16

중앙대학교 소프트웨어학부

이찬근 교수

Contents

- Usage of Singleton pattern
- Singleton pattern implementation for single-thread
- Singleton pattern implementation for multi-thread
- Related patterns

Singleton Pattern

- **Purpose**

- Ensures that only one instance of a class is allowed within a system.

- **Use When**

- Exactly one instance of a class is required.
 - Controlled access to a single object is necessary.

One of a Kind Objects

- One and only one
 - Window Manager, Printer Spooler, Thread Pool Manager, Caches, Logging, Factory
 - We want to instantiate them only once
- Why is it difficult after all?
 - How about global variables?
 - Multi-threading issue

Usage of Singleton

```
public class AnyClientProgram {  
    // Singleton s = new Singleton();  
    // the above triggers compilation error  
    Singleton s = Singleton.getInstance();  
}
```

Clients are not allowed to use new operator!

The Skeleton of Singleton

```
public class Singleton {  
    private static Singleton uniqueInstance;  
  
    // other useful instance variables  
  
    private Singleton() {}  
  
    public static Singleton getInstance() {  
        if (uniqueInstance == null) {  
            uniqueInstance = new Singleton();  
        }  
        return uniqueInstance;  
    }  
  
    // other useful methods  
}
```

Beware of Concurrency!!

Design Points

- Make the constructor be private
 - `private Singleton() {}`
 - Private constructor ?
 - Otherwise ...
- Provide a `getInstance()` method
 - `public static Singleton getInstance()`
 - Should it be static?
 - Why public ?
- Remember the instance once you have created it
 - `private static Singleton uniqueInstance`
 - `if (uniqueInstance == null)`
 `uniqueInstance = new Singleton();`

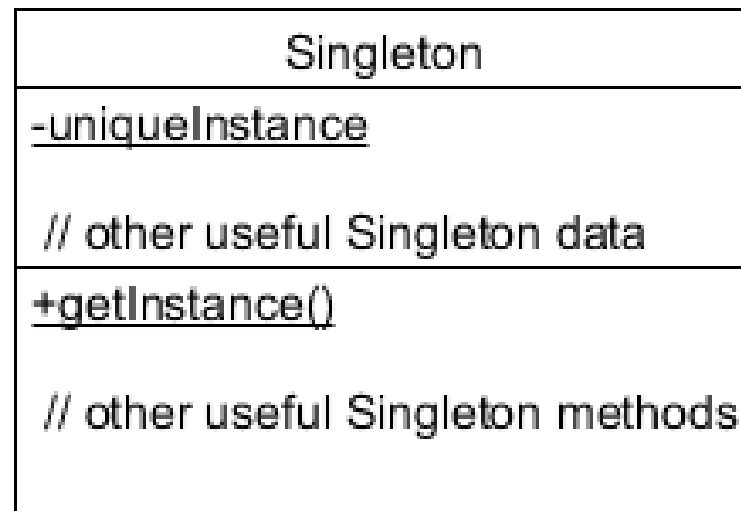
Exercise: Applying Singleton

```
public class ChocolateBoiler {  
    private boolean empty;  
    private boolean boiled;  
      
      
    ChocolateBoiler() {  
        empty = true;  
        boiled = false;  
    }  
      
      
    public void fill() {  
        if(isEmpty()) {  
            empty = false;  
            boiled = false;  
            // fill the boiler with a milk/chocolate mixture  
        }  
    }  
    // rest of ChocolateBoiler code...  
}
```

Solution

```
public class ChocolateBoiler {  
    private boolean empty;  
    private boolean boiled;  
    private static ChocolateBoiler uniqueInstance;  
  
    private ChocolateBoiler() {  
        empty = true;  
        boiled = false;  
    }  
  
    public static ChocolateBoiler getInstance() {  
        if (uniqueInstance == null) {  
            uniqueInstance = new ChocolateBoiler();  
        }  
        return uniqueInstance;  
    }  
  
    ..  
}
```

Class Diagram



Using Singleton on Multi-threads

- Thread One and Thread Two executes

```
ChocolateBoiler boiler = ChocolateBoiler.getInstance();  
boiler.fill();  
boiler.boil();  
boiler.drain();
```

What happened?

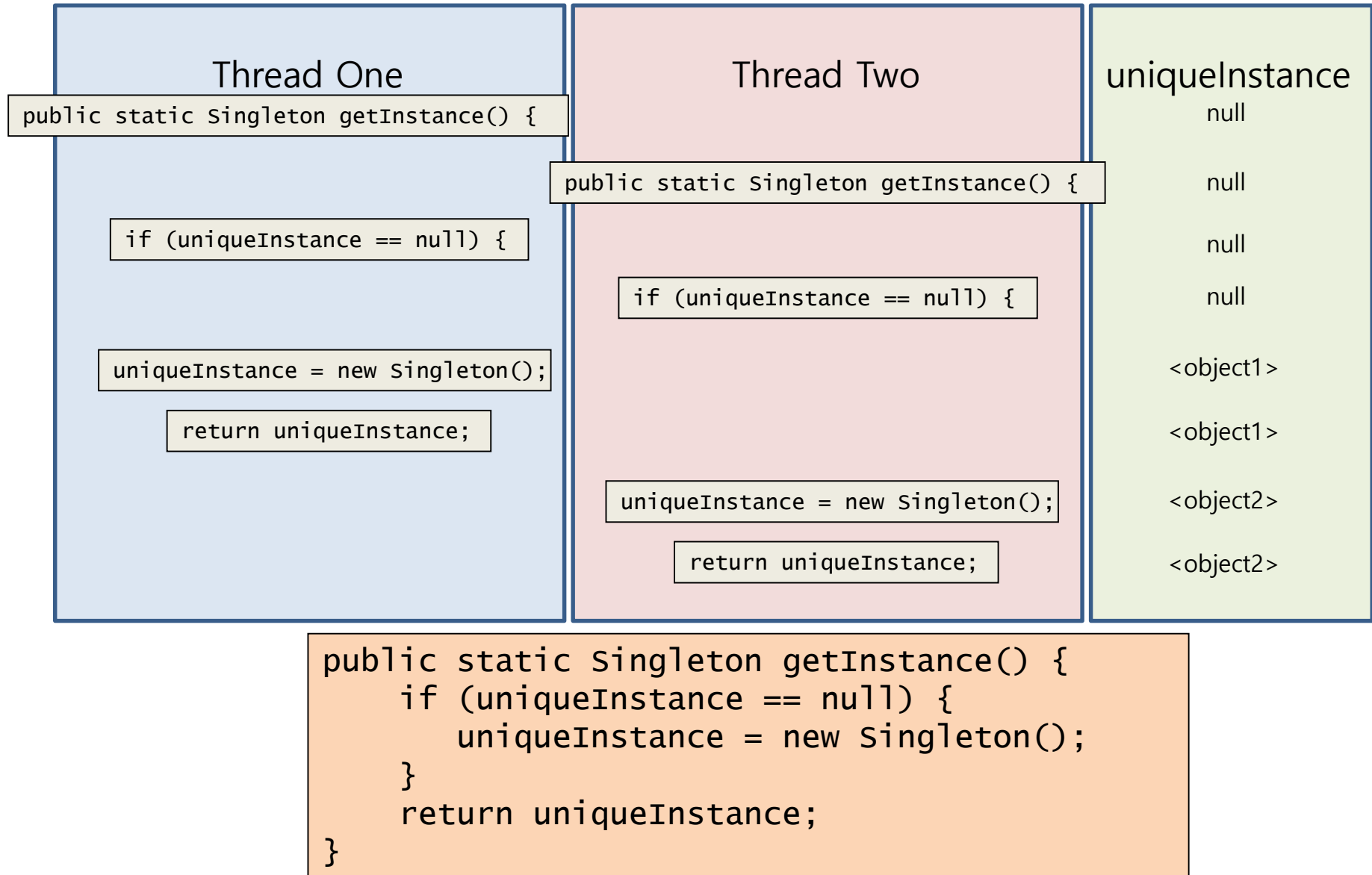
```
public static Singleton getInstance() {  
    if (uniqueInstance == null) {  
        uniqueInstance = new Singleton();  
    }  
    return uniqueInstance;  
}
```

Thread One

Thread Two

uniqueInstance

What happened?



Solving the Problem (Option 1)

```
public class Singleton {  
    private static Singleton uniqueInstance;  
  
    // other useful instance variables  
  
    private Singleton() {}  
  
    public static synchronized Singleton getInstance() {  
        if (uniqueInstance == null) {  
            uniqueInstance = new Singleton();  
        }  
        return uniqueInstance;  
    }  
  
    // other useful methods  
}
```

Isn't this too much locking?

Solving the Problem (Option2)

```
public class Singleton {  
    private static Singleton uniqueInstance = new Singleton();  
  
    // other useful instance variables  
  
    private Singleton() {}  
  
    public static Singleton getInstance() {  
        return uniqueInstance;  
    }  
  
    // other useful methods  
}
```

Remind the overhead of global variables

Let's revisit the solution for the single thread

```
public class Singleton {  
    private static Singleton uniqueInstance;  
  
    // other useful instance variables  
  
    private Singleton() {}  
  
    public static Singleton getInstance() {  
        if (uniqueInstance == null) {  
            uniqueInstance = new Singleton();  
        }  
        return uniqueInstance;  
    }  
  
    // other useful methods  
}
```

Is this correct?

```
public class Singleton {
    private static Singleton uniqueInstance = null;

    // other useful instance variables

    private Singleton() {}

    public static Singleton getInstance() {
        if (uniqueInstance == null) {
            synchronized(Singleton.class) {
                uniqueInstance = new Singleton();
            }
        }
        return uniqueInstance;
    }

    // other useful methods
}
```

Developing Idea

1. Check that the variable is initialized (without obtaining the lock). If it is initialized, return it immediately.
2. Obtain the lock.
3. **Double-check** whether the variable has already been initialized: if another thread acquired the lock first, it may have already done the initialization. If so, return the initialized variable.
4. Otherwise, initialize and return the variable.

Then, how about this?

```
public class Singleton {
    private static Singleton uniqueInstance = null;

    // other useful instance variables

    private Singleton() {}

    public static Singleton getInstance() {
        if (uniqueInstance == null) {
            synchronized(Singleton.class) {
                if (uniqueInstance == null)
                    uniqueInstance = new Singleton();
            }
        }
        return uniqueInstance;
    }
    // other useful methods
}
```

Double-Checked Locking

- At first glance, the previous algorithm seems like an efficient solution to the problem. However, this technique has many subtle problems and should usually be avoided.
- In a scenario, thread *A* notices that the value is not initialized, so it obtains the lock and begins to initialize the value.
- Due to the semantics of some programming languages, the code generated by the compiler is allowed to update the shared variable to point to a partially constructed object before *A* has finished performing the initialization.
- Thread *B* notices that the shared variable has been initialized (or so it appears), and returns its value. Because thread *B* believes the value is already initialized, it does not acquire the lock. **If the variable is used before *A* finishes initializing it, the program will likely crash.**

Double-Checked Locking

- One of the dangers of using double-checked locking in J2SE 1.4 and earlier versions is that it will often appear to work.
- Depending on the compiler, the interleaving of threads by the scheduler and the nature of other concurrent system activities, failures resulting from an incorrect implementation of double-checked locking may only occur intermittently.
- Reproducing the failures can be difficult.

Corrected Double-Checked Locking (Option 3)

```
public class Singleton {
    private volatile static Singleton uniqueInstance = null;
    // other useful instance variables

    private Singleton() {}

    public static Singleton getInstance() {
        if (uniqueInstance == null) {
            synchronized(Singleton.class) {
                if (uniqueInstance == null)
                    uniqueInstance = new Singleton();
            }
        }
        return uniqueInstance;
    }
    // other useful methods
}
```

Using volatile keyword

- Thread *A* notices that the value is not initialized, so it obtains the lock and begins to initialize the value.
- ~~Due to the semantics of some programming languages, the code generated by the compiler is allowed to update the shared variable to point to a partially constructed object before *A* has finished performing the initialization.~~
- Only after complete construction of the object, the shared variable is set to point to the object.
- Thread *B* notices that the shared variable has been initialized, and returns its value. Because thread *B* believes the value is already initialized, it does not acquire the lock.
- Using the value in thread *B* does not trigger any problem since the object pointed by the value is completely constructed.

Reviewing the Options

- Synchronize the getInstance() method (Option 1):
 - A straightforward technique that is guaranteed to work. It causes small impact on run-time performance due to frequent locking.
- Use eager instantiation (Option 2):
 - In case we are always going to instantiate the class, then statically initializing the instance would cause no concerns.
- Double checked locking (Option 3):
 - A perfect solution w.r.t performance. However, double-checked locking may be overkill in case we have no performance concerns. In addition, we'd have to ensure that we are running at least Java 5

Related Patterns and Summary

- Abstract Factory, Builder, and Prototype can use Singleton in their implementation.
- Facade objects are often Singletons because only one Facade object is required.
- State objects are often Singletons.
- Singleton pattern
 - creates at most one instance
 - Implementation: beware of multi-threaded issue

Singleton Pattern Quiz : Problem

SingletonA 클래스와 SingletonB 클래스가 각각 싱글톤 패턴으로 구현되었다고 가정할 때, 다음 코드의 수행결과를 쓰시오.

```
SingletonA aRef1, aRef2;  
SingletonB bRef1, bRef2;  
aRef1 = SingletonA.getInstance( );  
aRef2 = SingletonA.getInstance( );  
if (aRef1 == aRef2) System.out.print("1");  
bRef1 = SingletonB.getInstance( );  
bRef2 = SingletonB.getInstance( );  
if (bRef1 == bRef2) System.out.print("2");  
if (aRef1 == bRef1) System.out.print("3");
```

Singleton Pattern Quiz : Answer

SingletonA 클래스와 SingletonB 클래스가 각각 싱글톤 패턴으로 구현되었다고 가정할 때, 다음 코드의 수행결과를 쓰시오.

```
SingletonA aRef1, aRef2;  
SingletonB bRef1, bRef2;  
aRef1 = SingletonA.getInstance();  
aRef2 = SingletonA.getInstance();  
if (aRef1 == aRef2) System.out.print("1");  
bRef1 = SingletonB.getInstance();  
bRef2 = SingletonB.getInstance();  
if (bRef1 == bRef2) System.out.print("2");  
if (aRef1 == bRef1) System.out.print("3");
```

14. DECORATOR PATTERN

2019. 04. 16

중앙대학교 소프트웨어학부

이찬근 교수

Contents

- Intro. to the problem
- Discovery of Decorator pattern
- Structure and Run-time mechanism of Decorator pattern
- Decorator in Java I/O

Decorator Pattern

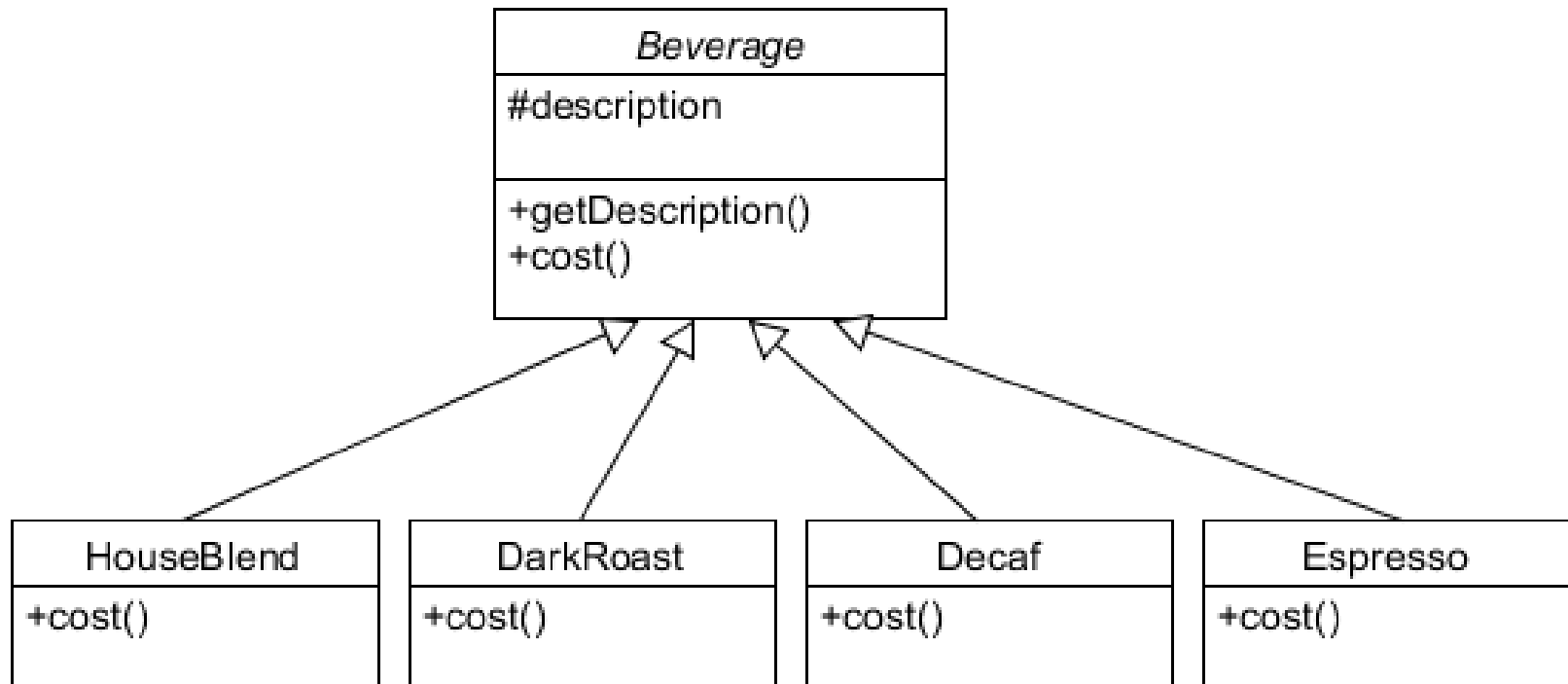
▪ Purpose

- Allows for the **dynamic wrapping** of objects in order to **modify their existing responsibilities** and behaviors.

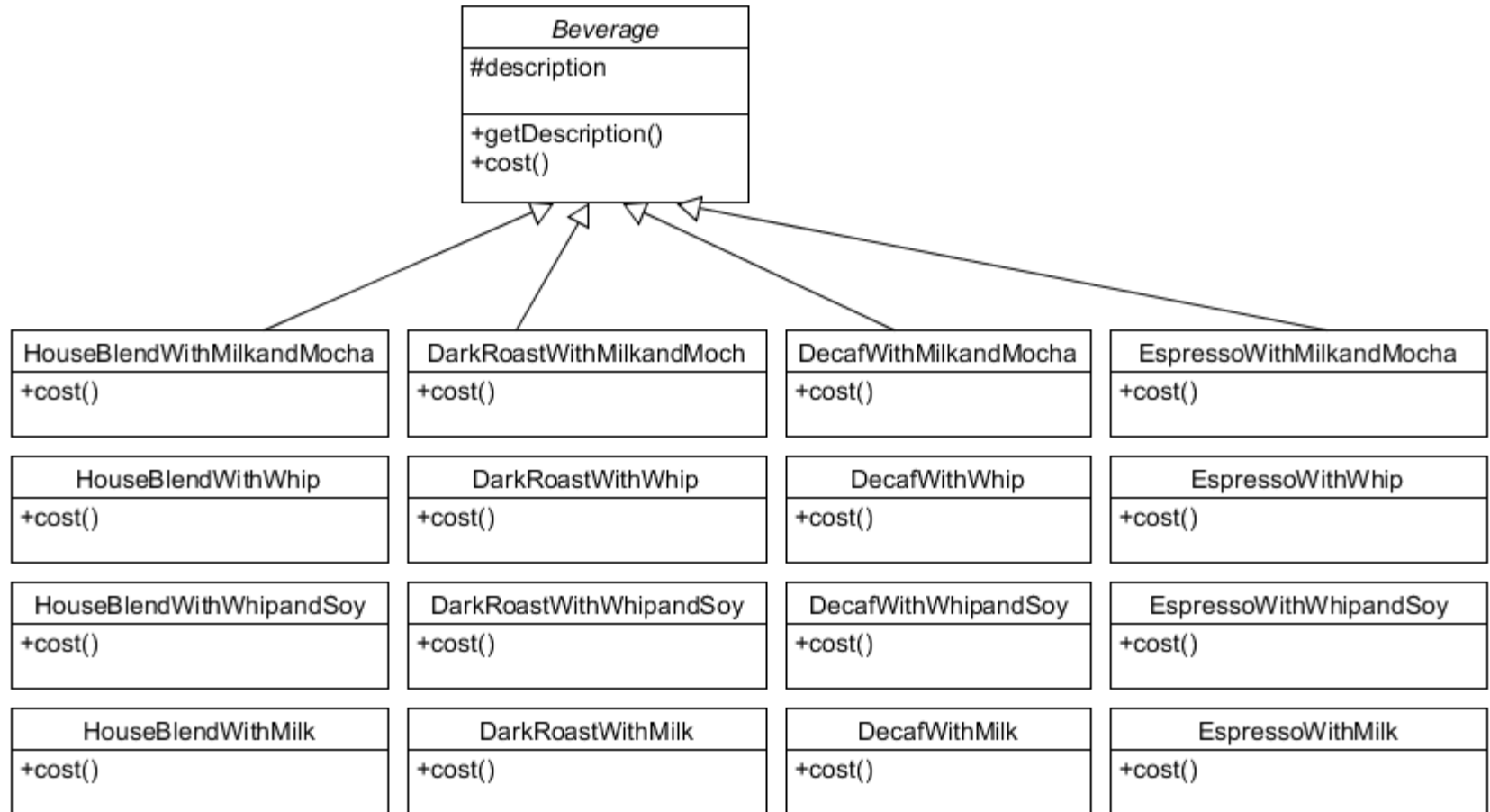
▪ Use When

- Object responsibilities and behaviors should be dynamically modifiable.
- Concrete implementations should be decoupled from responsibilities and behaviors.
- Subclassing to achieve modification is impractical or impossible.
- Specific functionality should not reside high in the object hierarchy.
- A lot of little objects surrounding a concrete implementation is acceptable.

Original Design

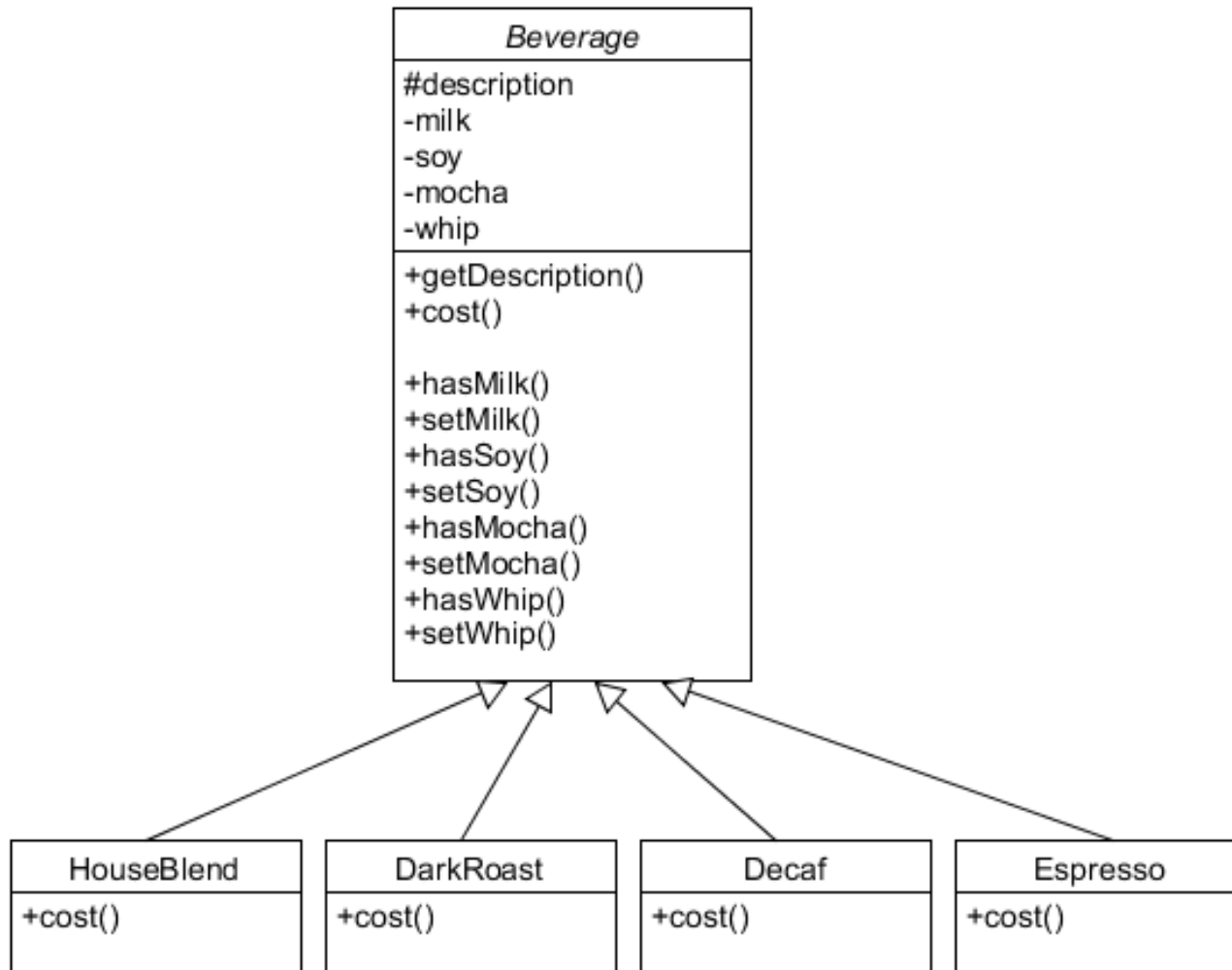


Adding Condiments



Class number explosion!

Adding Boolean Variables



Writing cost() method

```
public class Beverage {
    protected String description;
    boolean milk, soy, mocha, whip;
    public float cost () {
        float condimentCost = 0.0;
        if (hasMilk())
            condimentCost += milkCost;
        if (hasSoy())
            condimentCost += soyCost;
        if (hasMocha())
            condimentCost += mochaCost;
        if (hasWhip())
            condimentCost += whipCost;
        return condimentCost;
    }
}

public class DarkRoast extends Beverage {
    public DarkRoast () {
        description = "Excellent Dark Roast";
    }
    public float cost () {
        return 1.99 + super.cost();
    }
}
```

Expected Changes Impacting the Design

```
public class Beverage {  
    protected String description;  
    boolean milk, soy, mocha, whip;  
    public float cost () {  
        float condimentCost = 0.0;  
        if (hasMilk())  
            condimentCost += milkCost;  
        if (hasSoy())  
            condimentCost += soyCost;  
        if (hasMocha())  
            condimentCost += mochaCost;  
        if (hasWhip())  
            condimentCost += whipCost;  
        return condimentCost;  
    }  
}
```

- Price changes for condiments will force us to alter existing code
- New condiments will force us to add new methods and alter the cost method in the superclass
- We may have new beverages. For some of these beverages the condiments may not be appropriate
- What if a customer wants double mocha?

Design Principle: OCP

- Open Closed Principle
 - Classes should be open for extension, but closed for modification
 - Allow classes to be **easily extended** to incorporate new behavior **without modifying existing code**.
 - resilient to change and flexible enough to take on new functionality to meet changing requirements.
- Caution: **Don't try applying** the Open-Closed Principle (OCP) to **every single case**. **Keep simple designs if possible!**

Meet the Decorator Pattern

We start with a beverage and “decorate” it with the condiments at runtime. If a customer wants a Dark Roast with Mocha and Whip we do the following:

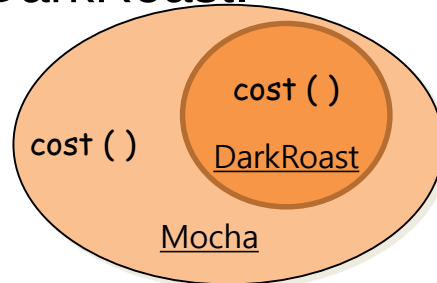
1. Take a `DarkRoast` object
2. Decorate it with a `Mocha` object
3. Decorate it with a `Whip` object
4. Call the `cost ( )` method and rely on delegation to add on the condiment costs.

Constructing a drink order with Decorators

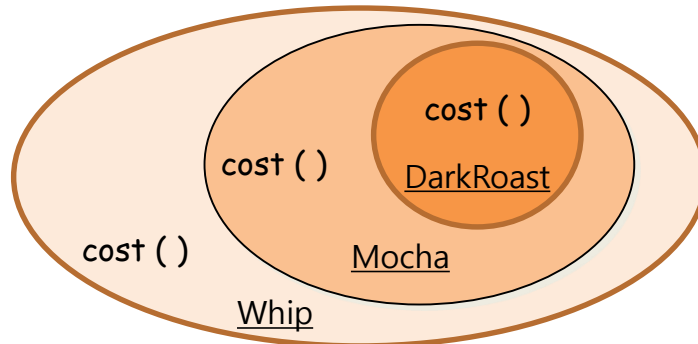
1. Start with the DarkRoast object



2. Customer wants Mocha, so we create a Mocha object and wrap it around the DarkRoast.

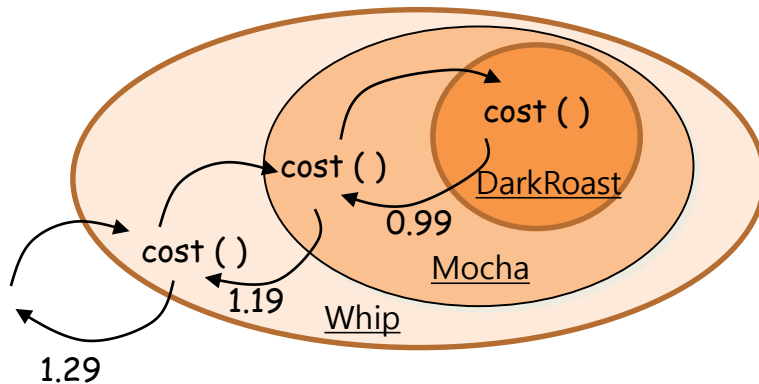


3. The customer also wants Whip, so we create a Whip decorator and wrap Mocha with it.



Constructing a drink order with Decorators

4. Now its time to compute the cost for the customer. Do this by calling `cost()` on the outermost decorator, Whip, and Whip is going to delegate computing cost to the objects it decorates. Once it gets a cost, it will add on the cost of the Whip.



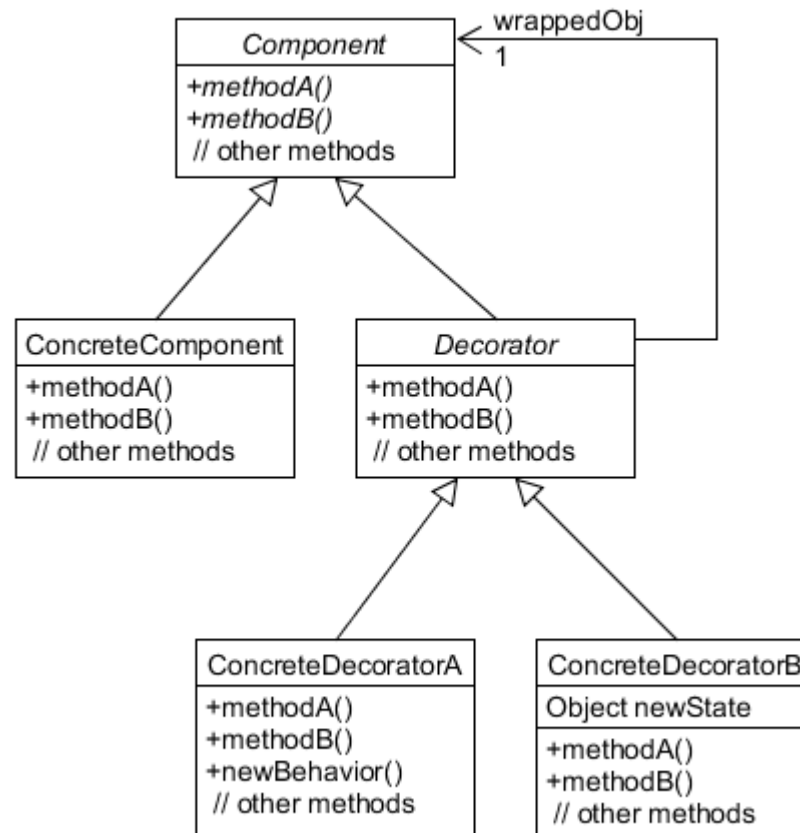
Ingredient	Price
DarkRoast	<u>\$0.99</u>
Mocha	\$0.2
Whip	\$0.1

Review of Decorator Idea

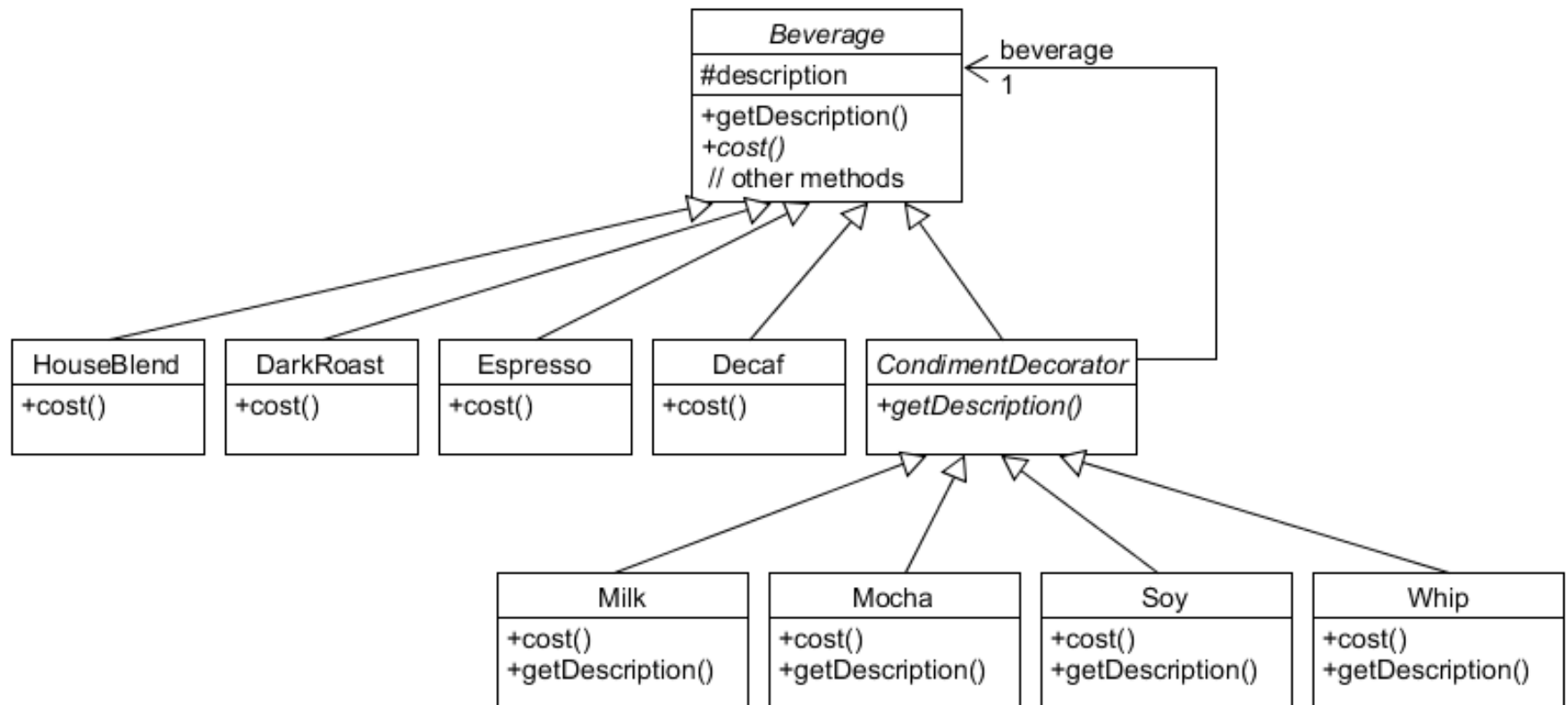
- The decorator adds its own behavior.
- You can use one or more decorators to wrap an object.
- We can pass around a decorated object in place of the original (wrapped) object.
- Decorators have the same super type as the objects they decorate.
- We can decorate objects dynamically at runtime with as many decorators as we want.

The Definition of the Decorator

The **Decorator Pattern** attaches additional responsibilities to an object dynamically. Decorators provide a flexible alternative to subclassing for extending functionality.



Applying to Our Example



Beverage class and Concrete Beverage class

```
public abstract class Beverage {
    protected String description = "Unknown Beverage";

    public String getDescription() {
        return description;
    }

    public abstract double cost();
}

public class Espresso extends Beverage {
    public Espresso() {
        description = "Espresso";
    }

    public double cost() {
        return 1.99;
    }
}
```

Coding Condiments

```
public abstract class CondimentDecorator extends Beverage {
    protected Beverage beverage;
    public abstract String getDescription();
}

public class Mocha extends CondimentDecorator {
    public Mocha(Beverage beverage) {
        this.beverage = beverage;
    }

    public String getDescription() {
        return beverage.getDescription() + ", Mocha";
    }

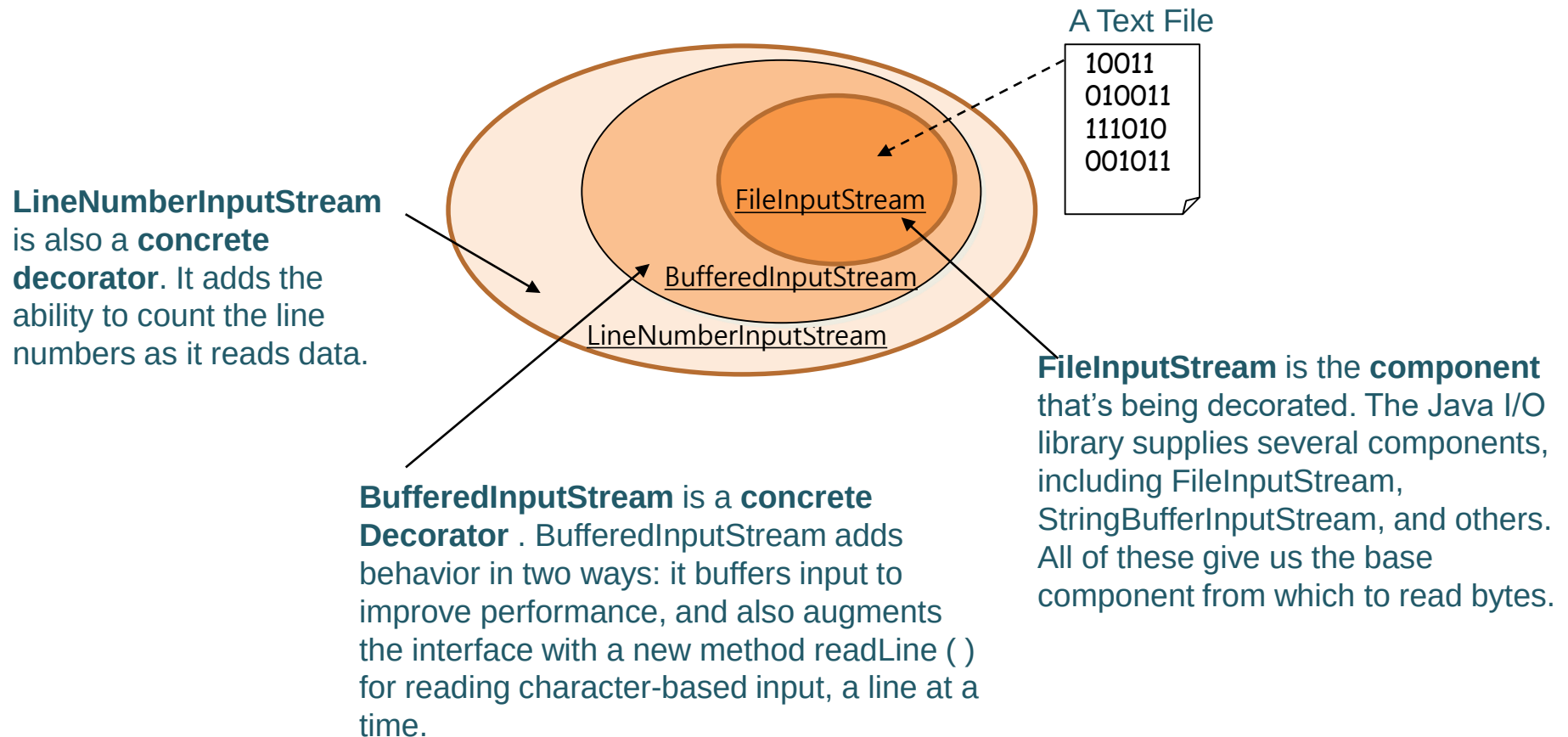
    public double cost() {
        return .20 + beverage.cost();
    }
}
```

Decorator Test Drive

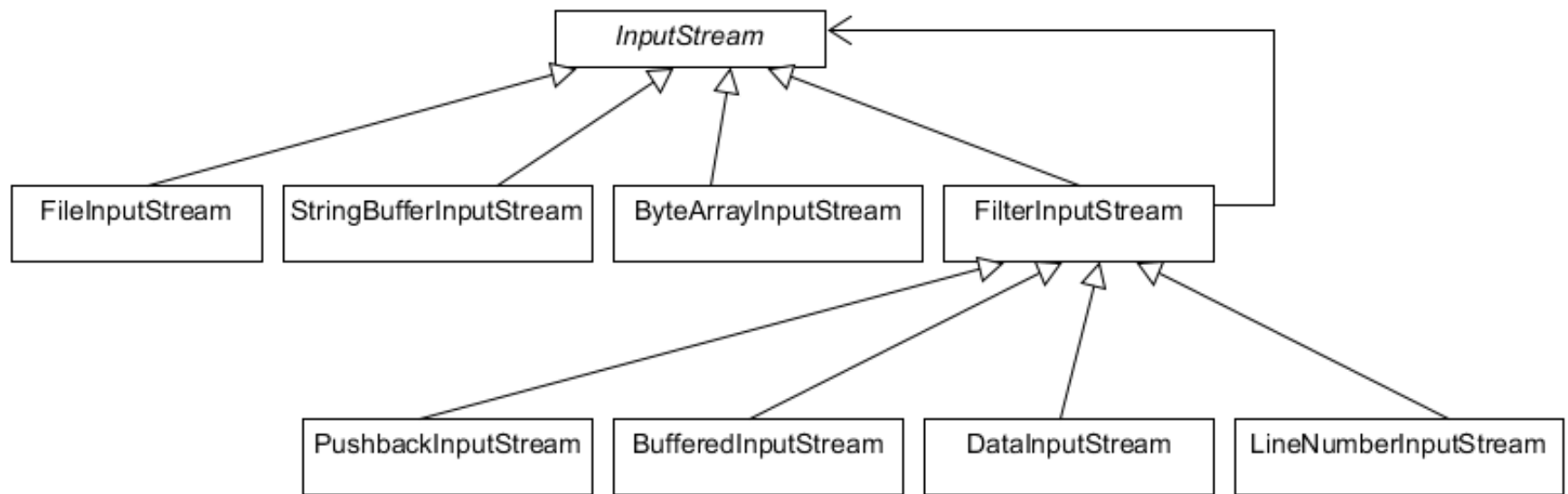
```
public class StarbuzzCoffee {  
  
    public static void main(String args[]) {  
        Beverage beverage = new Espresso();  
        System.out.println(beverage.getDescription()  
            + " $" + beverage.cost());  
  
        Beverage beverage2 = new DarkRoast();  
        beverage2 = new Mocha(beverage2);  
        beverage2 = new Mocha(beverage2);  
        beverage2 = new Whip(beverage2);  
        System.out.println(beverage2.getDescription()  
            + " $" + beverage2.cost());  
  
        Beverage beverage3 = new HouseBlend();  
        beverage3 = new Soy(beverage3);  
        beverage3 = new Mocha(beverage3);  
        beverage3 = new Whip(beverage3);  
        System.out.println(beverage3.getDescription()  
            + " $" + beverage3.cost());  
    }  
}
```


Using Decorator Pattern in Java I/O

What is the typical set of objects that use decorators to add functionality to reading data from a file?



java.io classes



A Sample of Java I/O Decorator

```
Public class LowerCaseInputStream extends FilterInputStream {
    public LowerCaseInputStream(InputStream in) {
        super(in);
    }

    public int read() throws IOException {
        int c = super.read();
        return (c == -1 ? C : Character.toLowerCase((char)c));
    }

    public int read(byte[] b, int offset, int len) throws IOException {
        int result = super.read(b, offset, len);
        for (int i = offset; i < offset+result; i++) {
            b[i] = (byte)Character.toLowerCase((char)b[i]);
        }
        return result;
    }
}
```

Run our Java I/O decorator

```
public class InputTest {  
    public static void main(String[] args) throws IOException {  
        int c;  
        try {  
            InputStream in =  
                new LowerCaseInputStream(  
                    new BufferedInputStream(  
                        new FileInputStream("test.txt")));  
  
            while((c = in.read()) >= 0) {  
                System.out.println((char)c);  
            }  
            in.close();  
        } catch (IOException e) {  
            e.printStackTrace();  
        }  
    }  
}
```

Related Patterns

- Regarding the interfaces
 - **Adapter** provides a **different interface** to its subject
 - **Proxy** provides the **same interface**
 - **Decorator** provides an **enhanced interface**

Summary

- Design Principle: Open-Closed Principle (OCP)
- Advantages
 - attaches **additional responsibilities** to an object **dynamically**
 - **flexible alternative** to **subclassing for extending functionality**
- Key mechanisms
 - Uses object composition and delegation
 - Decorator class mirrors the type of components they are decorating
 - We can wrap a component with any number of decorators
- Disadvantage
 - can generate a lot of small classes(e.g. Java I/O)
 - hard to understand if not familiar with the pattern

Decorator Pattern Quiz : Problem

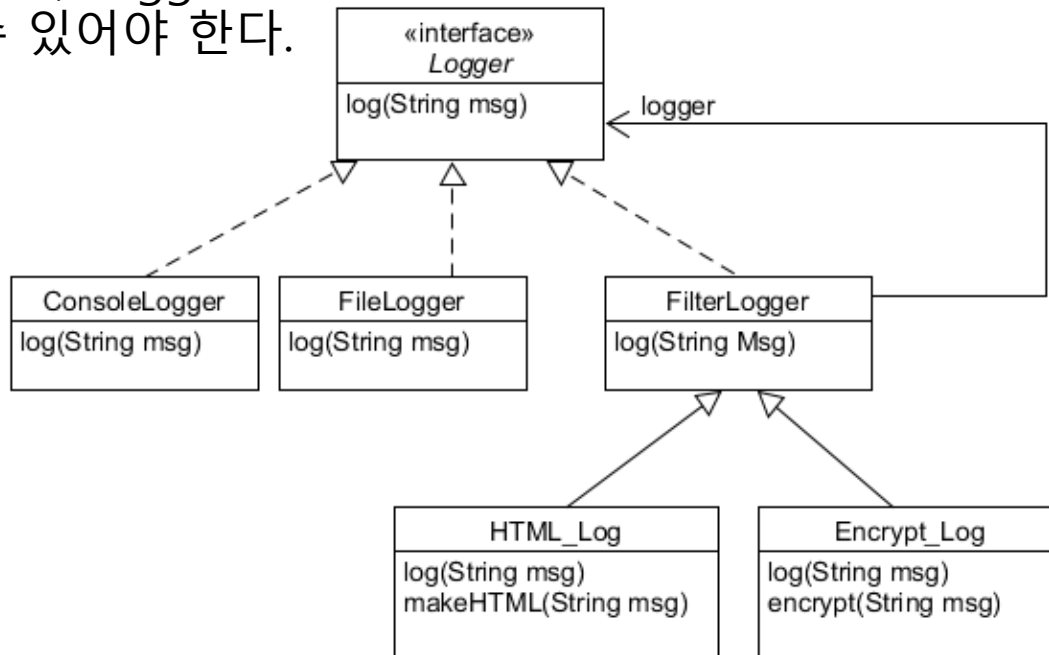
- Logger의 종류에는 ConsoleLogger와 FileLogger가 있다. 이들은 모두 `log(String msg)`의 메소드를 구현하고 있다.
- 로깅되는 메시지를 HTML로 변환해주는 `HTML_Log` 클래스가 있으며 이 클래스는 `makeHTML(String msg)` 메소드를 제공한다.
- 로깅되는 메시지를 암호화해주는 `Encrypt_Log` 클래스가 있으며 이 클래스는 `encrypt(String msg)` 메소드를 제공한다.

상기 요구사항을 설계한 결과를 클래스 다이어그램으로 나타내시오. 단 `HTML_Log` 및 `Encrypt_Log`는 개별적으로 혹은 함께 어느 하나의 `Logger`에 적용될 수 있으며, `Logger`의 종류와 메시지를 가공하는 클래스의 종류도 향후 쉽게 추가될 수 있어야 한다.

Decorator Pattern Quiz : Answer

- Logger의 종류에는 ConsoleLogger와 FileLogger가 있다. 이들은 모두 log(String msg)의 메소드를 구현하고 있다.
- 로깅되는 메시지를 HTML로 변환해주는 HTML_Log 클래스가 있으며 이 클래스는 makeHTML(String msg) 메소드를 제공한다.
- 로깅되는 메시지를 암호화해주는 Encrypt_Log 클래스가 있으며 이 클래스는 encrypt(String msg) 메소드를 제공한다.

상기 요구사항을 설계한 결과를 클래스 다이어그램으로 나타내시오. 단 HTML_Log 및 Encrypt_Log는 개별적으로 혹은 함께 어느 하나의 Logger에 적용될 수 있으며, Logger의 종류와 메시지를 가공하는 클래스의 종류도 향후 쉽게 추가될 수 있어야 한다.



15. ADAPTER PATTERN

2019. 04. 16

중앙대학교 소프트웨어학부

이찬근 교수

Contents

- Concept of Adapter
- Adapter pattern definition
- Class Adapter and Object Adapter
- Example Adapters for Enumeration/Iterator interface

Adapter Pattern

- **Also known as**

- Wrapper

- **Purpose**

- Permits classes with **different interfaces** to work together by creating a common object by which they may communicate and interact.

- **Use When**

- A class to be used doesn't meet interface requirements.

Adapters

- Real world is full of them!



- Adapters in SW
 - You have an existing software system
 - You need to work a new vendor library into, but the new vendor designed their interfaces differently than the last vendor
 - Write a class that adapts the new vendor interface into the one you're expecting

Typical working process and Motivation

■ Mechanism

- A **client** makes a request to the adapter by **calling a method** on it using the **target interface**
- The **adapter translates the request** into one or more calls on the adaptee using the **adaptee interface**
- The **client receives the results** of the call and never knows there is an adapter doing the translation

■ Motivation

- A toolkit or class library may have an interface which is incompatible with an application's interface we want to integrate.
- It is possible that we do not have access to the source code of the toolkit or library.
- Even if the source code is available, we may want to minimize the change

Simplified Duck and Turkey

```
public interface Duck {
    public void quack ();
    public void fly ();
}

public class MallardDuck implements Duck {
    public void quack () {
        System.out.println("Quack");
    }
    public void fly ( ) {
        System.out.println ("I am flying");
    }
}

public interface Turkey {
    public void gobble ();
    public void fly ( );
}

public class wildTurkey implements Turkey {
    public void gobble ( ) {
        System.out.println("Gobble Gobble");
    }
    public void fly ( ){
        System.out.println("I'm flying a short distance");
    }
}
```

Turkey Adapter

```
public class TurkeyAdapter implements Duck {
    Turkey turkey;

    public TurkeyAdapter (Turkey turkey) {
        this.turkey = turkey;
    }

    public void quack () {
        turkey.gobble ();
    }

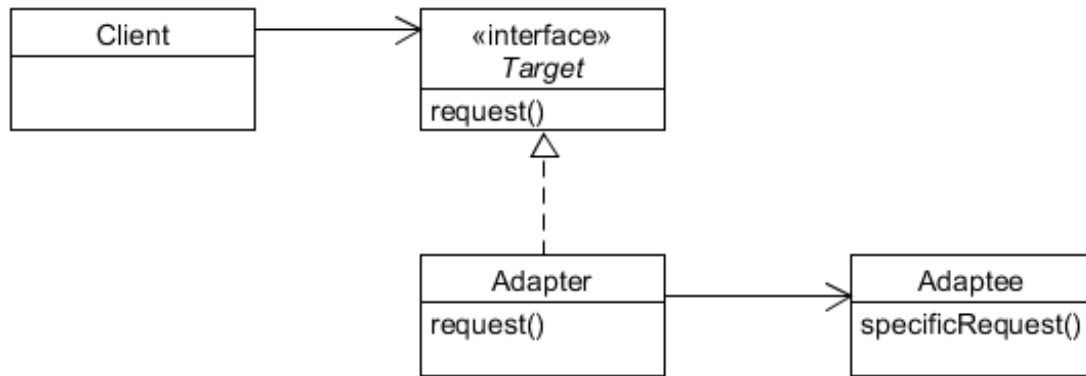
    public void fly () {
        for (int i= 0; i < 5; i++)
            turkey.fly ();
    }
}
```

TestDrive

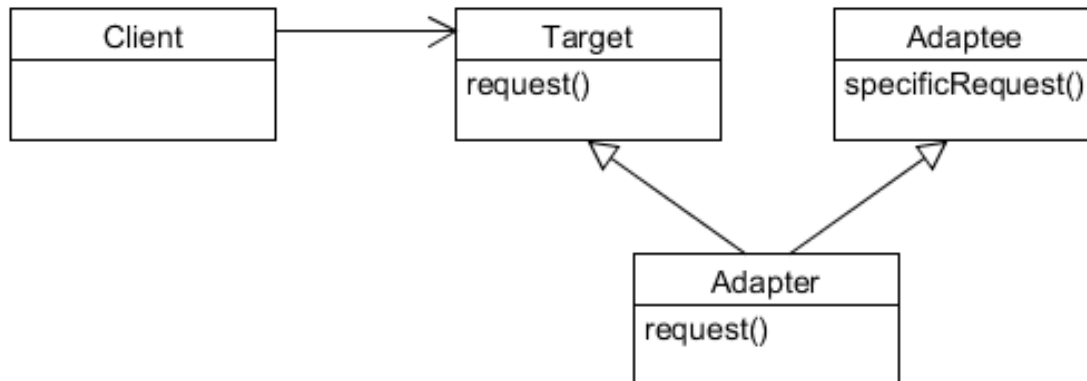
```
public class DuckTestDrive {  
    public static void main(String[] args) {  
        MallardDuck duck = new MallardDuck();  
  
        wildTurkey turkey = new wildTurkey();  
        Duck turkeyAdapter = new TurkeyAdapter(turkey);  
  
        System.out.println("The Turkey says...");  
        turkey.gobble();  
        turkey.fly();  
  
        System.out.println("\nThe Duck says...");  
        testDuck(duck);  
  
        System.out.println("\nThe TurkeyAdapter says...");  
        testDuck(turkeyAdapter);  
    }  
  
    static void testDuck(Duck duck) {  
        duck.quack();  
        duck.fly();  
    }  
}
```


Object Adapter v.s. Class Adapter

- Object Adapter: uses object composition and delegation



- Class Adapter: uses inheritance



Implementation Issues

- How much adaptation?
 - Simple and straightforward interface conversion such as
 - Changing method names
 - Changing the order of arguments
 - Totally different set of operations
- Two-way transparency?
 - A *two-way adapter* supports both the Target and the Adaptee interface. It allows an adapted object (Adapter) to appear as an Adaptee object or a Target object

Enumerators v.s. Iterator

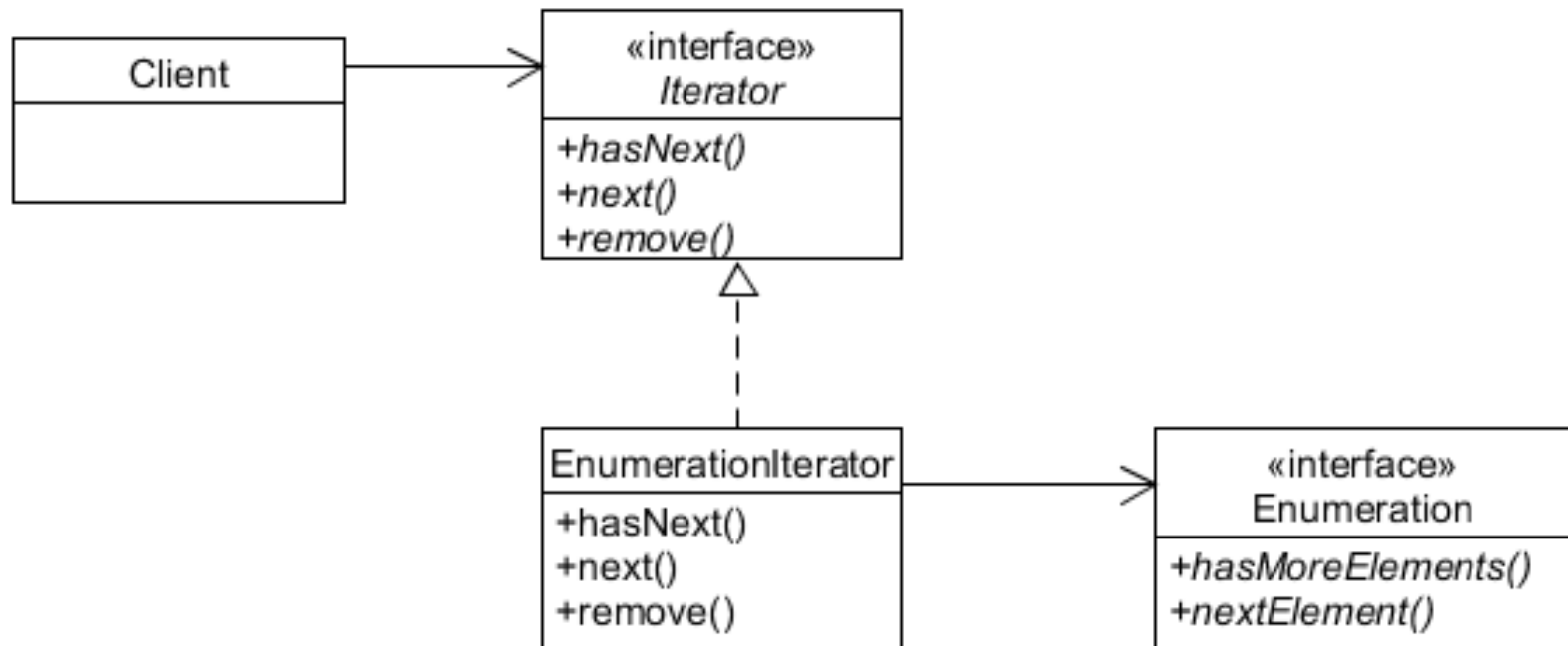
- Enumeration

- Java's old collection types such as Vector, Stack, HashTable, etc
- `hasMoreElements()`
- `nextElement()`

- Iterator

- `hasNext()`
- `next()`
- `remove()`

Adapting Enumeration to Iterator (EnumerationIterator)



EnumerationIterator Adapter

```
public class EnumerationIterator implements Iterator {
    Enumeration enum;
    public EnumerationIterator(Enumeration enum) {
        this.enum = enum;
    }
    public boolean hasNext() {
        return enum.hasMoreElements();
    }
    public Object next() {
        return enum.nextElement();
    }
    public void remove() {
        throw new UnsupportedOperationException();
    }
}
```

Summary

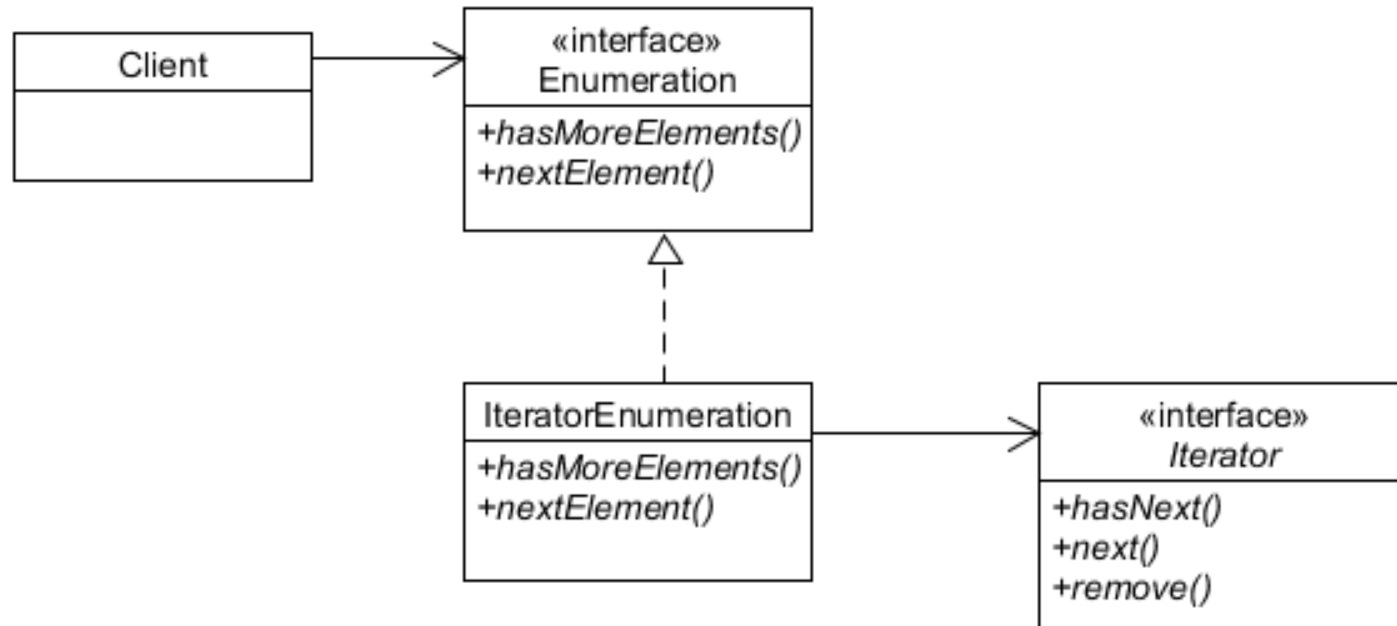
- Adapter pattern
 - Converts the interface of a class into another interface clients expect.
 - Lets classes work together that couldn't otherwise because of incompatible interfaces
 - Class adapter and object adapter

Adapter Pattern Quiz : Problem

앞서 설계된 EnumerationIterator의 반대 작용을 하는 Adapter를 만드시오. 즉, 클라이언트는 Enumeration을 target interface로 기대하고 있으나 실제 벤더의 구현은 Iterator 인터페이스를 따를 때, 이를 위한 Adapter의 클래스 다이어그램을 제시하시오.

Adapter Pattern Quiz : Answer

앞서 설계된 EnumerationIterator의 반대 작용을 하는 Adapter를 만드시오. 즉, 클라이언트는 Enumeration을 target interface로 기대하고 있으나 실제 벤더의 구현은 Iterator 인터페이스를 따를 때, 이를 위한 Adapter의 클래스 다이어그램을 제시하시오.



16. COMPOSITE PATTERN

2019. 04. 16

중앙대학교 소프트웨어학부

이찬근 교수

Contents

- Idea Developing for Hierarchical Information
- Structure of Composite Pattern
- Working with Iterator pattern

Composite Pattern

- **Purpose**

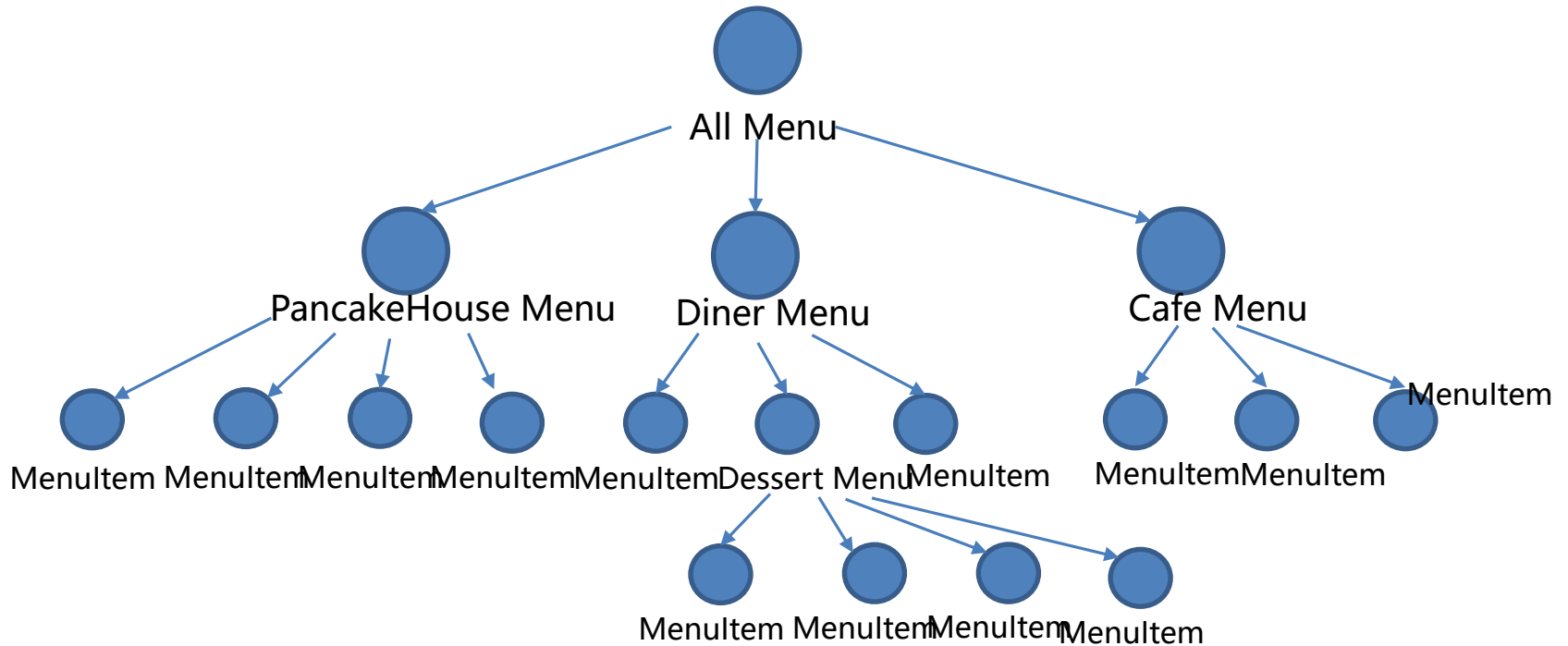
- Facilitates the creation of **object hierarchies** where each object can be treated independently or as a set of nested objects through the same interface.

- **Use When**

- **Hierarchical** representations of objects are needed.
 - **Objects** and **compositions of objects** should be treated uniformly.

Problems

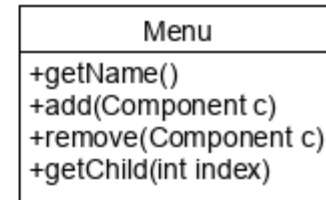
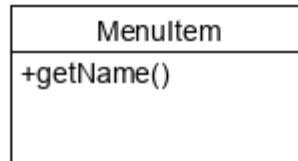
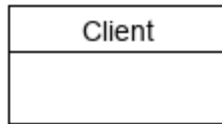
(1) Representing hierarchical structure :



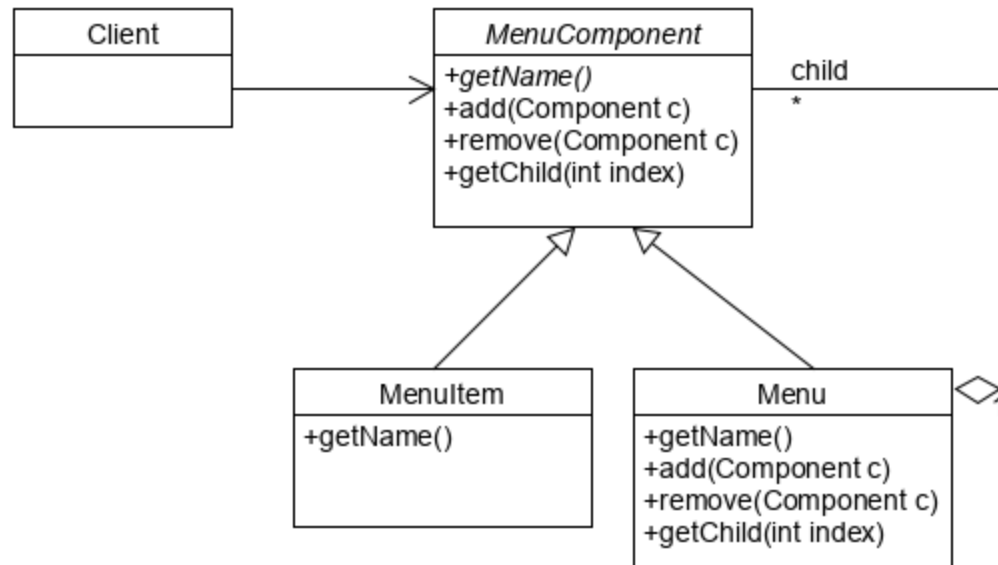
(2) Uniform access not like this:

```
if ( current instanceof(MenuItem))  
    // ... handle Menu Item in a way  
else if (o instanceof(Menu))  
    // ... handle Menu in another way
```

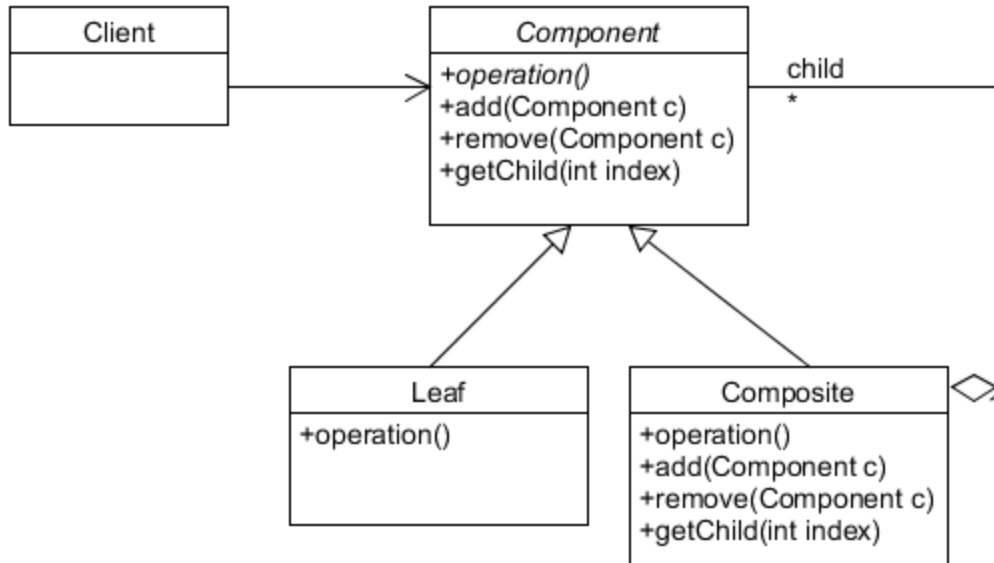
Developing Idea (Intermediate)



Developing Idea (Final)



Composite Pattern Diagram



Component Class

```
public class MenuComponent {  
    public void add(MenuComponent menuComponent) {  
        throw new UnsupportedOperationException();  
    }  
    public void remove(MenuComponent menuComponent) {  
        throw new UnsupportedOperationException();  
    }  
    public void getChild(int i) {  
        throw new UnsupportedOperationException();  
    }  
    public String getName() {  
        throw new UnsupportedOperationException();  
    }  
    public String getDescription() {  
        throw new UnsupportedOperationException();  
    }  
    public double getPrice() {  
        throw new UnsupportedOperationException();  
    }  
    public double isVegetarian() {  
        throw new UnsupportedOperationException();  
    }  
    public double print() {  
        throw new UnsupportedOperationException();  
    }  
}
```


Leaf Class

```
public class MenuItem extends MenuComponent {
    String name;
    String description;
    boolean vegetarian;
    double price;
    public MenuItem(String name, String description, boolean vegetarian,
        double price) {
        ...
    }
    public String getName() {
        return name;
    }
    public String getDescription() {
        return description;
    }
    public double getPrice() {
        return price;
    }
    public boolean isVegetarian() {
        return vegetarian;
    }
    public void print() {
        System.out.print("  " + getName());
        if (this.isVegetarian()) System.out.println("(v)");
        System.out.println(", " + getPrice());
        System.out.println("  --" + getDescription());
    }
}
```

Composite Class

```
public class Menu extends MenuComponent {
    ArrayList menuComponents = new ArrayList();
    String name;
    String description;
    public Menu(String name, String description) {
        this.name = name;
        this.description = description;
    }
    public void add(MenuComponent menuComponent) {
        menuComponents.add(menuComponent);
    }
    public void remove(MenuComponent menuComponent) {
        menuComponents.remove(menuComponent);
    }
    public MenuComponent getChild(int i) {
        return (MenuComponent)menuComponents.get(i);
    }
    public String getName() { return name; }
    public String getDescription () { return description; }
    public void print() {
        Iterator iterator = menuComponents.iterator();
        while (iterator.hasNext()) {
            MenuComponent menuComponent = (MenuComponent)iterator.next();
            menuComponent.print();
        }
    }
}
```

Using Composite Pattern

```
public class MenuTestDrive {
    public static void main(String args[]) {

        // mid-level Menu
        MenuComponent pancakeHouseMenu = new Menu("PANCAKE HOUSE MENU", "Breakfast");
        MenuComponent dinerMenu = new Menu("DINER MENU", "Lunch");
        MenuComponent cafeMenu = new Menu("CAFE MENU", "Dinner");

        // adding mid-level Menus to root-level Menu
        MenuComponent allMenus = new Menu("ALL MENUS", "All menus combined");
        allMenus.add(pancakeHouseMenu);
        allMenus.add(dinerMenu);
        allMenus.add(cafeMenu);

        // adding bottom-level Menu to mid-level Menu
        MenuComponent dessertMenu = new Menu("DESSERT MENU", "Dessert of course!");
        dinerMenu.add(dessertMenu);

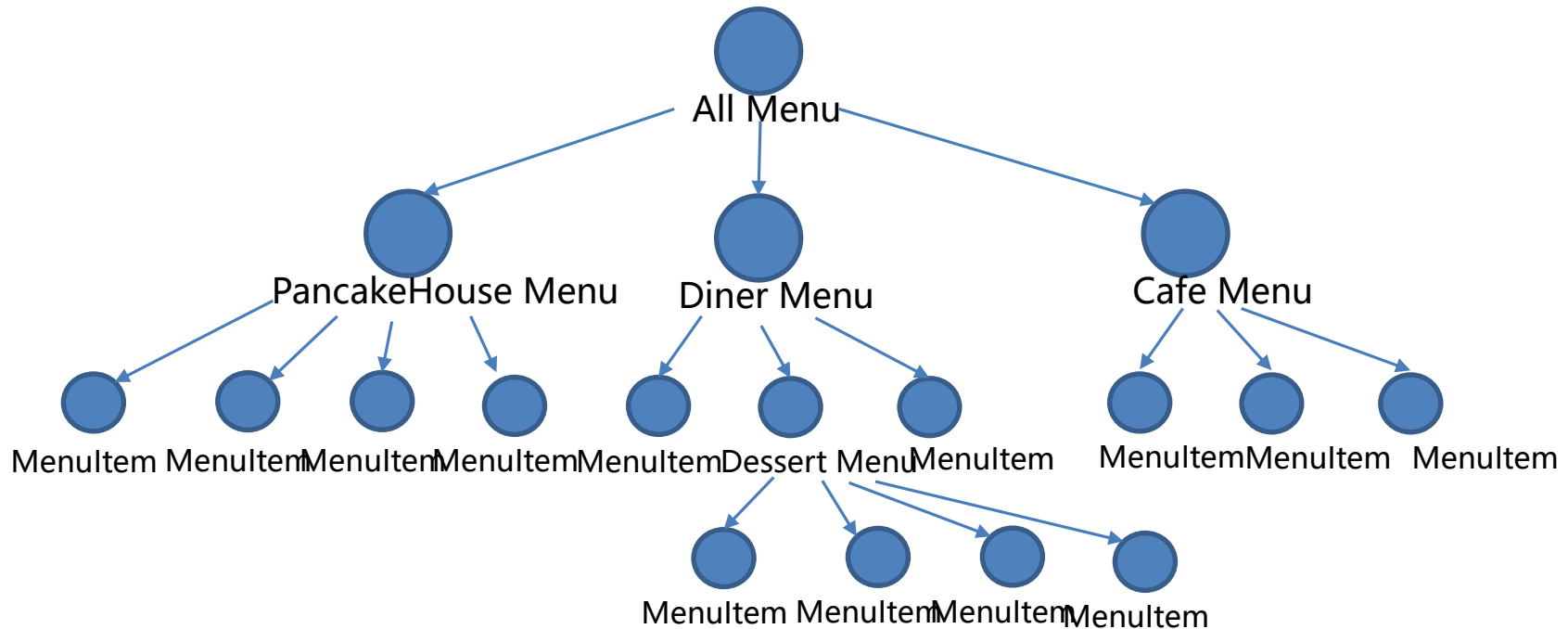
        // adding MenuItem to Menus
        pancakeHouseMenu.add(new MenuItem( "K&B's Pancake Breakfast",
            "Pancakes with scrambled eggs, and toast", true, 2.99));
        pancakeHouseMenu.add(new MenuItem( "Regular Pancake Breakfast",
            "Pancakes with fried eggs, sausage", false, 2.99));

        ... adding more MenuItem ...

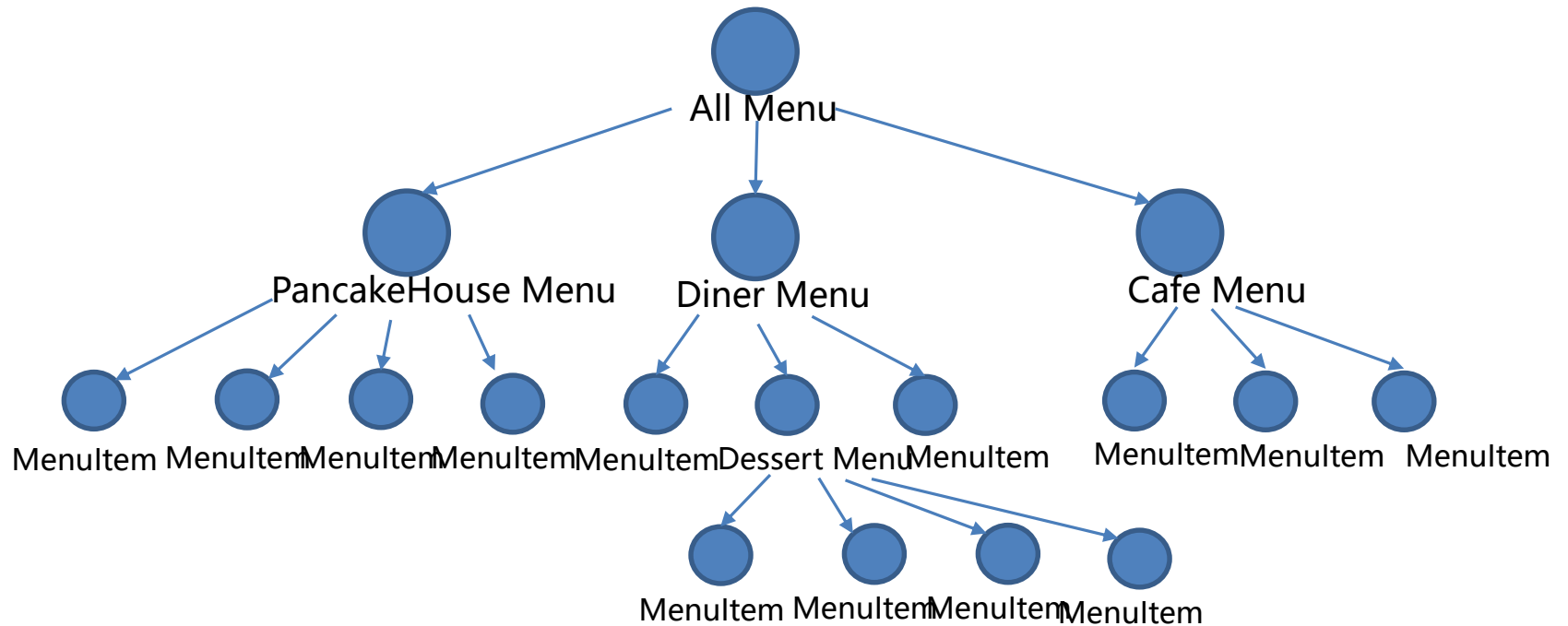
        waitress waitress = new waitress(allMenus);
        waitress.printVegetarianMenu();
    }
}
```

Waitress Class (Client using Composite Pattern)

```
public class waitress {  
    MenuComponent allMenus;  
  
    public waitress(MenuComponent allMenus) {  
        this.allMenus = allMenus;  
    }  
  
    public void printMenu() {  
        allMenus.print();  
    }  
}
```



Iterator for Tree?



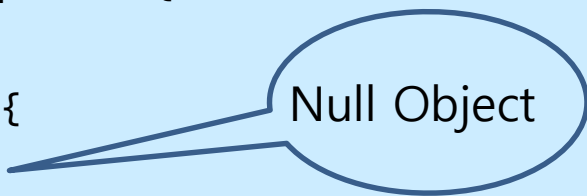
Iterator is useful for Trees! (and for Graphs, etc)

```
public class waitress {
    MenuComponent allMenus;
    public waitress(MenuComponent allMenus) {
        this.allMenus = allMenus;
    }
    public void printMenu() {
        allMenus.print();
    }
    public void printVegetarianMenu() {
        Iterator iterator = allMenus.createIterator();
        System.out.println("VEGETARIAN MENU-----");
        int counter = 0;
        while (iterator.hasNext()) {
            MenuComponent menuComponent = (MenuComponent)iterator.next();
            if (menuComponent.isVegetarian()) {
                menuComponent.print();
                counter++;
            }
        }
    }
}
```

Extending Iterator to support Composite Traversal

```
public class Menu extends MenuComponent {
    Iterator iterator = null;
    // other code here doesn't change
    public Iterator createIterator() {
        if (iterator == null)
            iterator = new CompositeIterator(menuComponents.iterator());
        return iterator;
    }
}

public class MenuItem extends MenuComponent {
    // other code here doesn't change
    public Iterator createIterator() {
        return new NullIterator();
    }
}
```



Null Object

Iterator for Composite

```
public class CompositeIterator extends Iterator {
    Stack stack = new Stack();
    public CompositeIterator(Iterator iterator) {
        stack.push(iterator);
    }
    public Object next() {
        if (hasNext()) {
            Iterator iterator = (Iterator)stack.peek();
            MenuComponent component = (MenuComponent)iterator.next();
            if (component instanceof Menu)
                stack.push(component.createIterator());
            return component;
        }
        else return null;
    }
    public boolean hasNext() {
        if (stack.empty()) return false;
        Iterator iterator = (Iterator)stack.peek();
        if (!iterator.hasNext()) {
            stack.pop();
            return hasNext();
        } else return true;
    }
    public void remove() {
        throw new UnsupportedOperationException();
    }
}
```

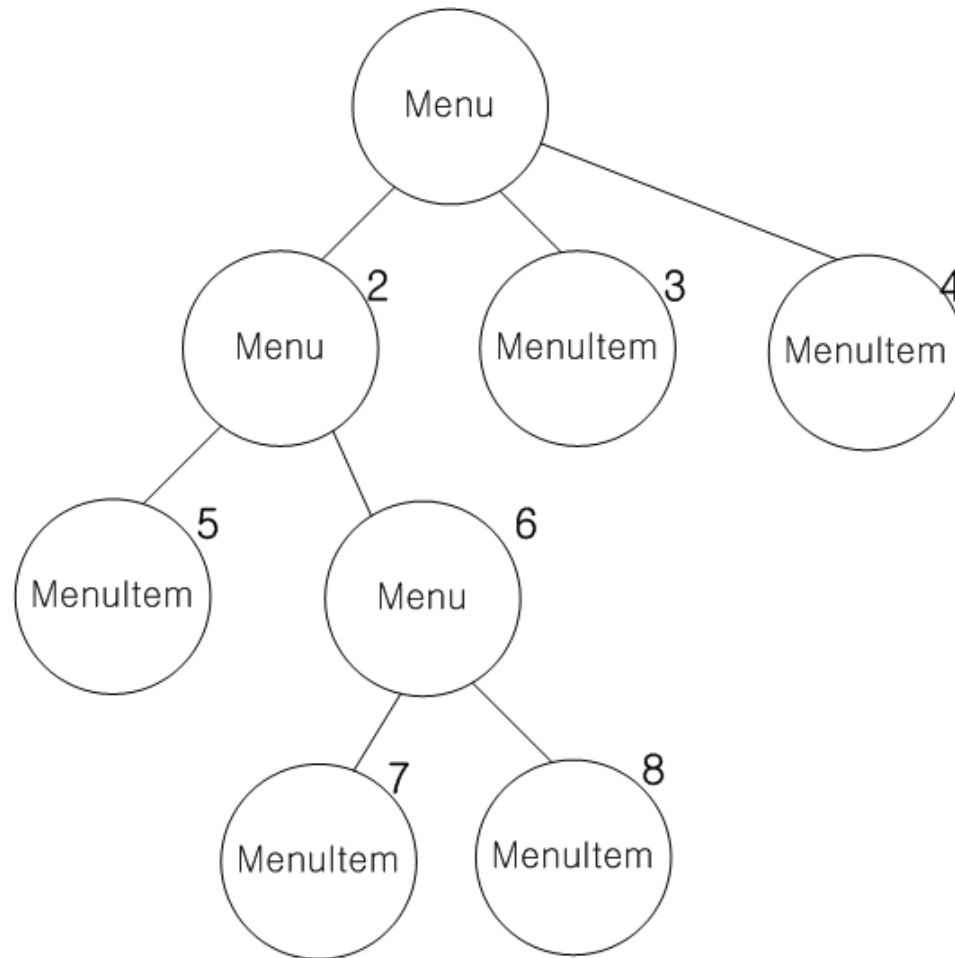

Null Iterator

```
public class NullIterator implements Iterator {  
    public Object next() {  
        return null;  
    }  
    public boolean hasNext() {  
        return false;  
    }  
    public void remove() {  
        throw new UnsupportedOperationException();  
    }  
}
```

Printing the Vegetarian Menu

```
public class waitress {
    MenuComponent allMenus;
    public waitress(MenuComponent allMenus) {
        this.allMenus = allMenus;
    }
    public void printMenu() {
        allMenus.print();
    }
    public void printVegetarianMenu() {
        Iterator iterator = allMenus.createIterator();
        System.out.println("\nVEGETARIAN MENU\n-----");
        while (iterator.hasNext()) {
            MenuComponent menuComponent = (MenuComponent)iterator.next();
            try {
                if (menuComponent.isVegetarian()) menuComponent.print();
            } catch (UnsupportedOperationException e) {}
        }
    }
}
```

Simulate the Algorithm

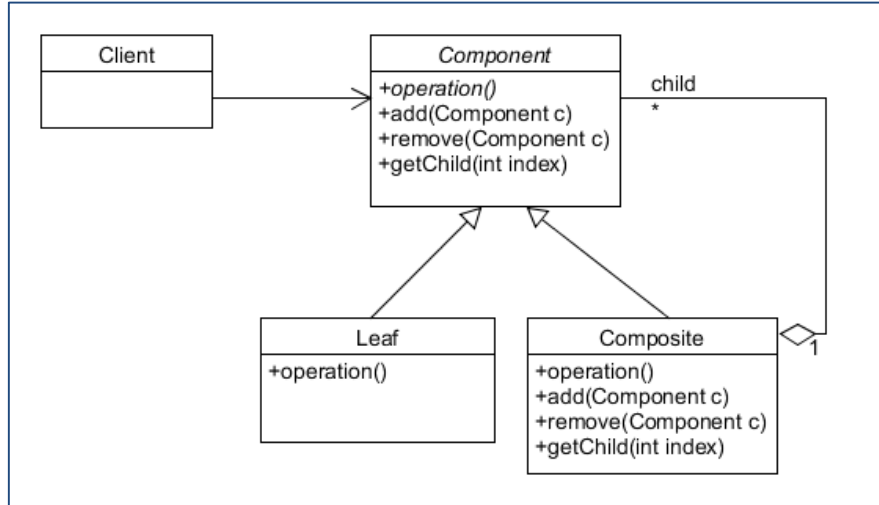


Visited order: 2, 5, 6, 7, 8, 3, 4

Things to Consider

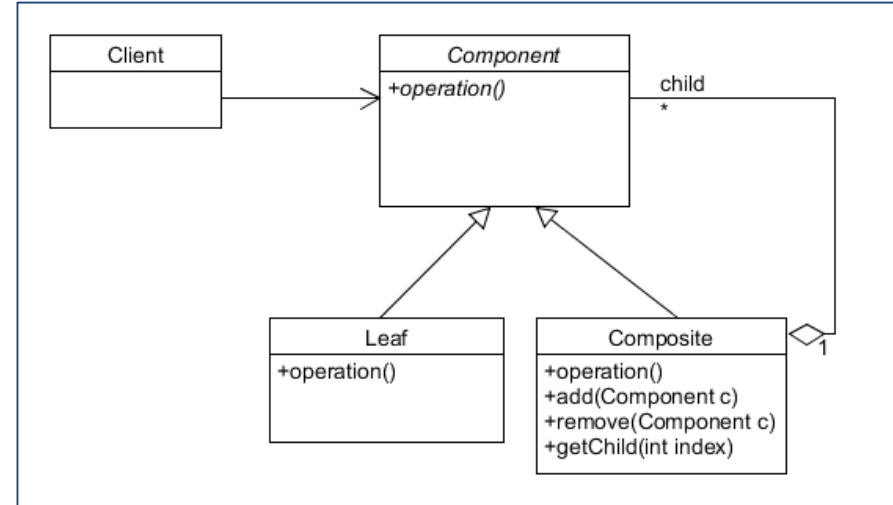
- A composite object stores the information about its contained components, i.e., its children. Should each component maintain **a reference to its parent component**?
 - It depends on applications. Having these references supports the Chain of Responsibility pattern
- **Where** do we need to declare the methods such as add(), remove(), and getChild() for **managing children**?
 - Implementation for transparency or safety (see next slide)

Where to place Child Mgt. Interface?



Placing in **Component** class gives
Transparency

since all components can be treated the same. However, it is not safe. Clients can try to do meaningless things to leaf components at run-time.



Placing in **Composite** class gives
Safety

since any attempt to perform a child operation on a leaf component will be caught at compile-time. However, we lose transparency because now leaf and composite components have different interfaces.

Related patterns

- Composite vs Decorator
 - Both have similar structure diagrams
 - **recursive composition** to organize an open-ended number of objects
 - Different intentions
 - **Decorator** lets you **add responsibilities** to objects without subclassing
 - **Composite**'s focus is not on embellishment but on **representation**
 - They are complementary; hence, Composite and Decorator are often used in concert
- Iterator
 - Provide a way to access the elements of an **aggregate object** (=typically uses composite pattern) sequentially without exposing its underlying representation

Summary

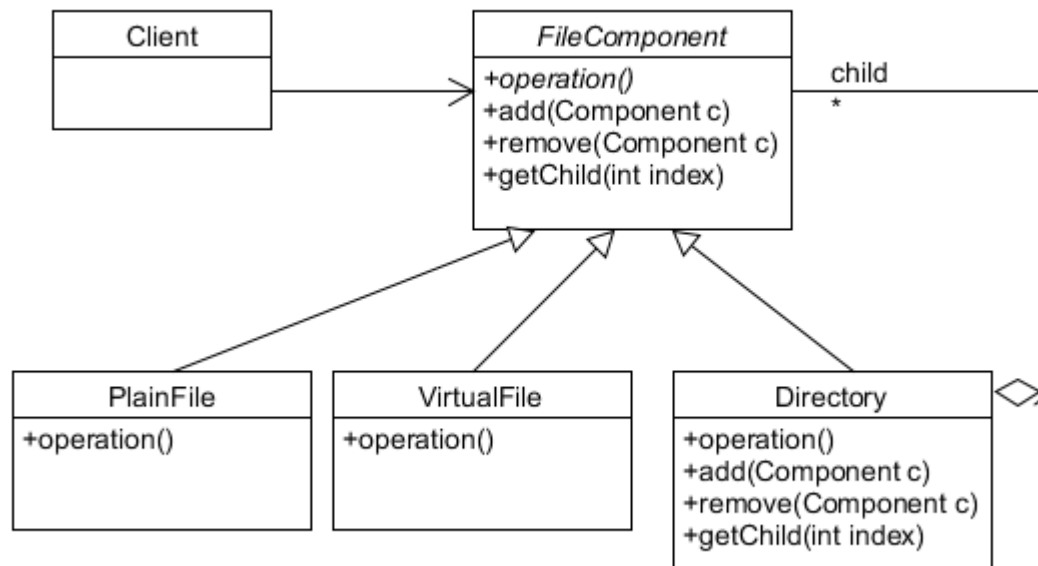
- Composite Pattern
 - composes objects into **tree** structures to represent **whole-part hierarchies**.
 - lets clients **treat** individual objects and compositions of objects **uniformly**

Composite Pattern Quiz : Problem

파일 시스템의 컴포넌트의 종류에는 PlainFile, VirtualFile, 그리고 Directory가 있다. Directory에는 다른 PlainFile, VirtualFile 혹은 다른 Directory를 포함할 수 있다. 클라이언트는 이들 파일 시스템의 컴포넌트를 일관된 방식으로 다루고자 한다. 이를 위해 파일 시스템의 컴포넌트 체계를 Composite 패턴을 사용하여 표현하시오.

Composite Pattern Quiz : Answer

파일 시스템의 컴포넌트의 종류에는 PlainFile, VirtualFile, 그리고 Directory가 있다. Directory에는 다른 PlainFile, VirtualFile 혹은 다른 Directory를 포함할 수 있다. 클라이언트는 이들 파일 시스템의 컴포넌트를 일관된 방식으로 다루고자 한다. 이를 위해 파일 시스템의 컴포넌트 체계를 Composite 패턴을 사용하여 표현하시오.



17. BRIDGE PATTERN

2019. 04. 16

중앙대학교 소프트웨어학부

이찬근 교수

Contents

- Intro. to the problem
- Discovery of bridge pattern
- Structure and run-time mechanism of bridge pattern
- Comparison with other pattern

Bridge Pattern

- **Purpose**

- Defines an **abstract object structure** independently of the **implementation object structure** in order to limit coupling.

- **Use When**

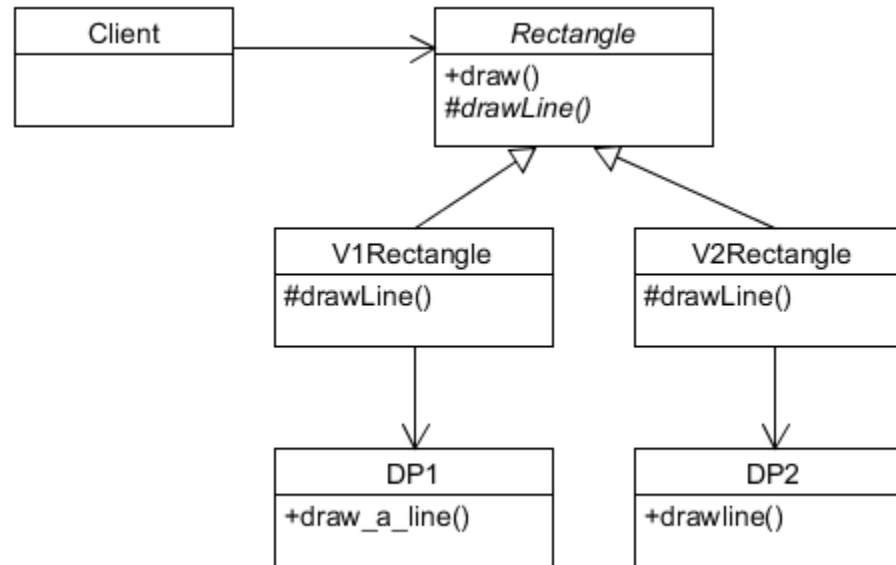
- Abstractions and implementations should not be bound at compile time.
 - Abstractions and implementations should be independently extensible.
 - Changes in the implementation of an abstraction should have no impact on clients.
 - Implementation details should be hidden from the client.

Initial Problem

- Suppose a simple drawing program
 - draw rectangles
 - using either of two drawing library
 - configuration decides to use either drawing program 1 (DP1) or drawing program 2 (DP2)

	DP1	DP2
Line	draw_a_line(x1, y1, x2, y2)	drawline(x1, x2, y1, y2)

Solution for Initial Problem



By having an **abstract class Rectangle**, we take advantage of the fact that the only difference between the different types of Rectangle are how they implement the **drawLine method**.

Solution for Initial Problem

```
abstract class Rectangle {
    public void draw() {
        drawLine(_x1, _y1, _x2, _y1);
        drawLine(_x2, _y1, _x2, _y2);
        drawLine(_x2, _y2, _x1, _y2);
        drawLine(_x1, _y2, _x1, _y1);
    }
    abstract protected void drawLine(double x1, double y1,
        double x2, double y2);
}

class V1Rectangle extends Rectangle {
    protected void drawLine(double x1, double y1, double x2, double y2) {
        DP1.draw_a_line(x1, x2, y1, y2);
    }
}

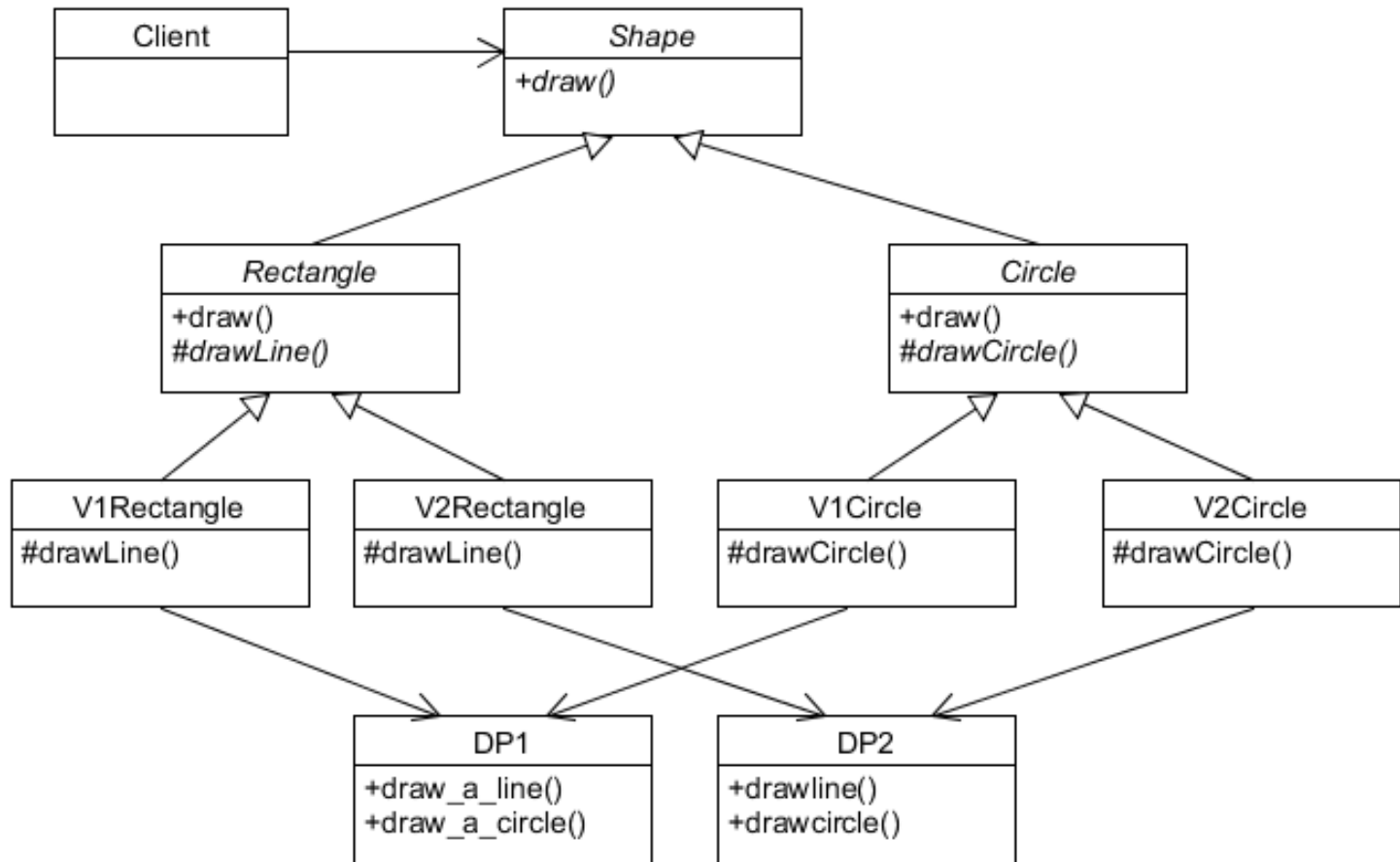
class V2Rectangle extends Rectangle {
    protected void drawLine(double x1, double y1, double x2, double y2) {
        DP2.drawLine(x1, x2, y1, y2);
    }
}
```

Requirement Changes

- Now, we should support another type of shape, Circle
- The collection object does not want to know the difference between Rectangles and Circles.
- Simply extend the previous approach by adding another level to the class hierarchy.
 - We only need to add a new class, called **Shape**, from which we will derive the **Rectangle** and **Circle** classes.
 - This way, the client object can just refer to Shape objects without worrying about what kind of Shape it has been given.

	DP1	DP2
Line	draw_a_line(x1, y1, x2, y2)	drawline(x1, x2, y1, y2)
Circle	draw_a_circle(x, y, r)	drawcircle(x, y, r)

A More Complex Situation



Source code for Shape and Circle

```
abstract class Shape {  
    abstract public void draw();  
}  
  
abstract class Circle extends Shape {  
    public void draw() {  
        drawCircle(x, y, r);  
    }  
    abstract protected void drawCircle(double x, double y, double r);  
}  
  
class V1Circle extends Circle {  
    protected void drawCircle() {  
        DP1.draw_a_circle(x, y, r);  
    }  
}  
  
class V2Circle extends Circle {  
    protected void drawCircle() {  
        DP2.drawcircle(x, y, r);  
    }  
}
```

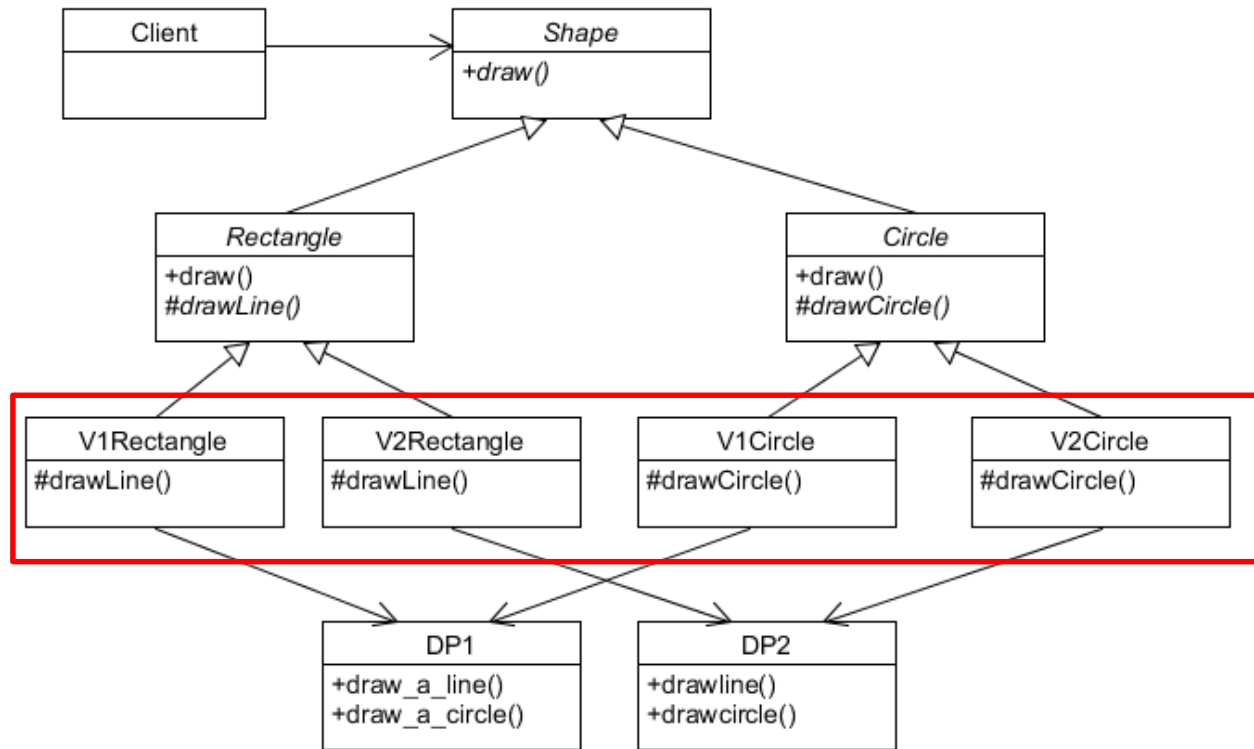
Source code for Rectangle

```
abstract class Rectangle extends Shape {
    public void draw() {
        drawLine(_x1, _y1, _x2, _y1);
        drawLine(_x2, _y1, _x2, _y2);
        drawLine(_x2, _y2, _x1, _y2);
        drawLine(_x1, _y2, _x1, _y1);
    }
    abstract protected void drawLine(double x1, double y1,
        double x2, double y2);
}

class V1Rectangle extends Rectangle {
    protected void drawLine(double x1, double y1, double x2, double y2) {
        DP1.draw_a_line(x1, y1, x2, y2);
    }
}

class V2Rectangle extends Rectangle {
    protected void drawLine(double x1, double y1, double x2, double y2) {
        DP2.draw_line(x1, x2, y1, y2);
    }
}
```

The combinatorial problem !



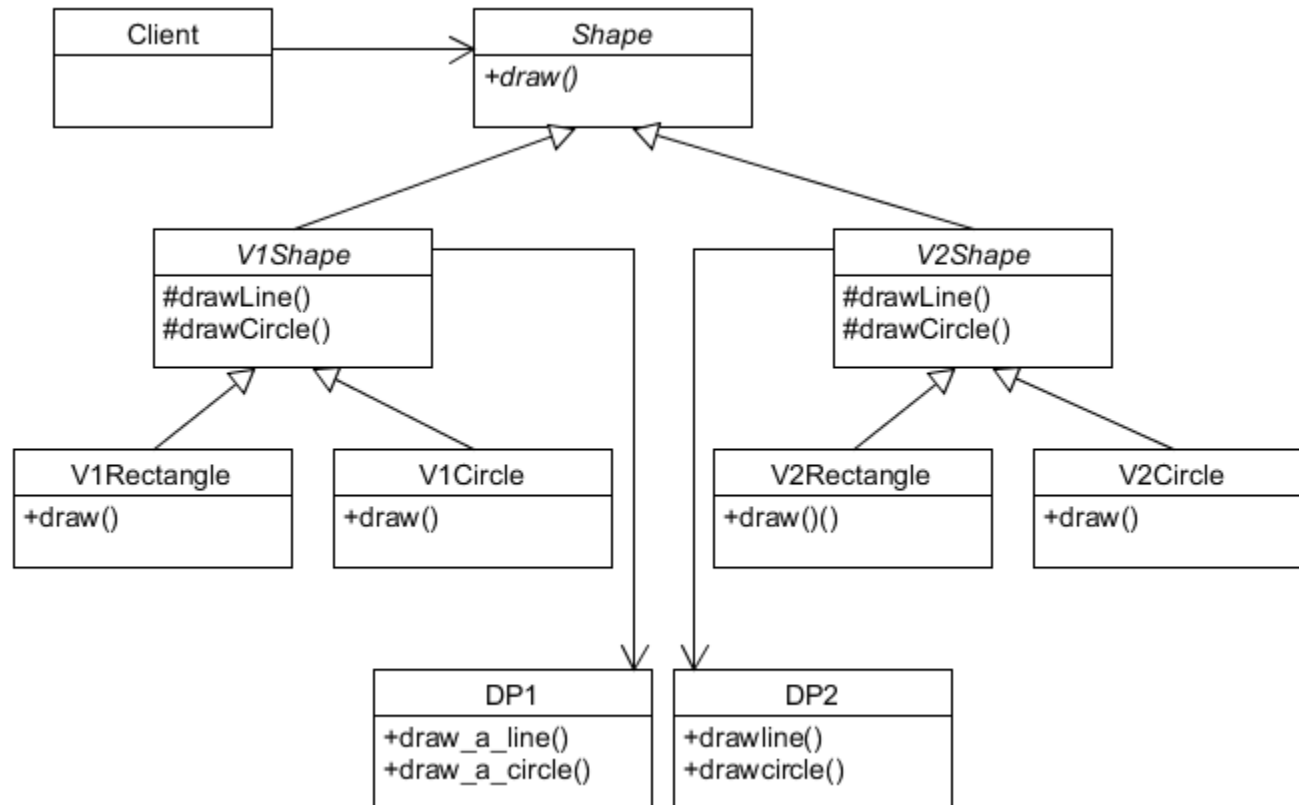
- What if I get another drawing program: variation in implementation
 - 2 shapes times 3 implementations = 6
- What if I get another type of shape: variation in concept
 - 3 shapes times 3 implementations = 9

Problem Analysis

- Problem: the class explosion, because
 - the **coupling** of the **abstraction** (the kinds of shapes) and the **implementation** (the drawing programs) is **tight**
 - each type of shape must know what type of drawing program it is using.
- Solution
 - Need to **separate** the **variations in abstraction** from the **variations in implementation** so that the number of classes only grows linearly

Another Approach with Inheritance Only

- May be we used the wrong type of inheritance hierarchy?
Is this one any better?



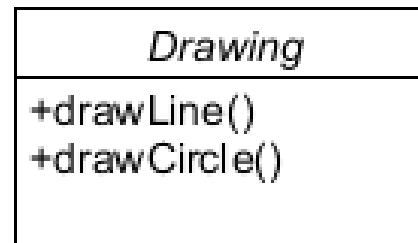
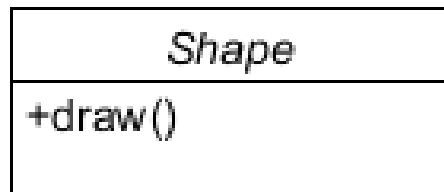
Still the same combinatorial problem!

Analyzing the problem

- You have to do two things first:
 - Commonality analysis
 - Variability analysis
- The common concepts will be represented by abstract classes, variations will be implemented by concrete classes.

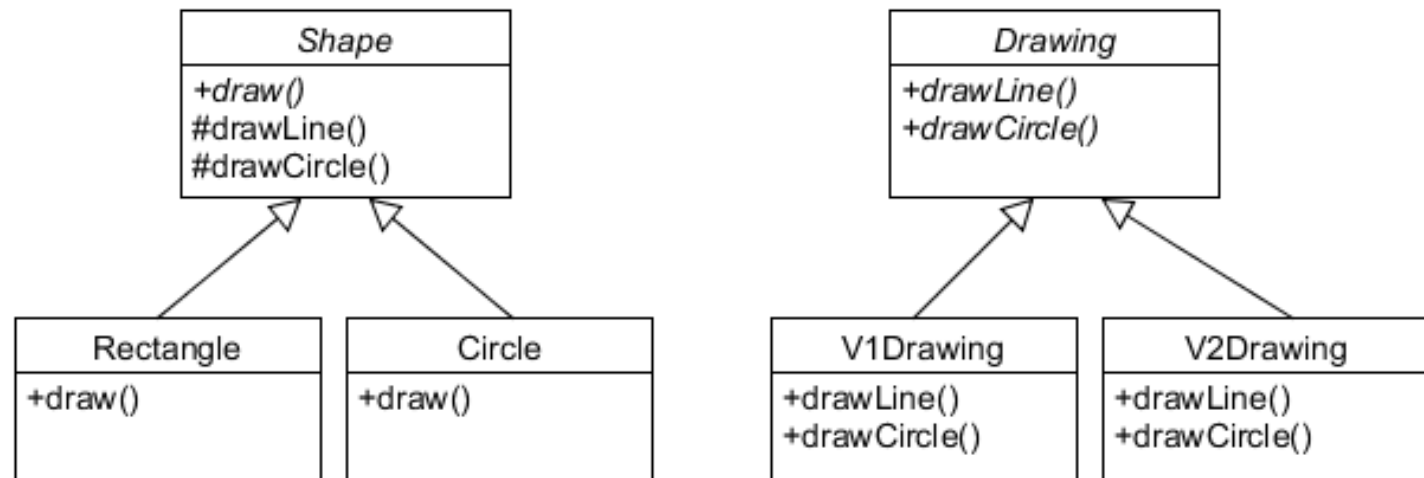
Commonality Analysis

- In this case we have different types of shapes and different type of drawing programs. The common concepts are **Shape** and **Drawing**. So we encapsulate them.



Variability Analysis

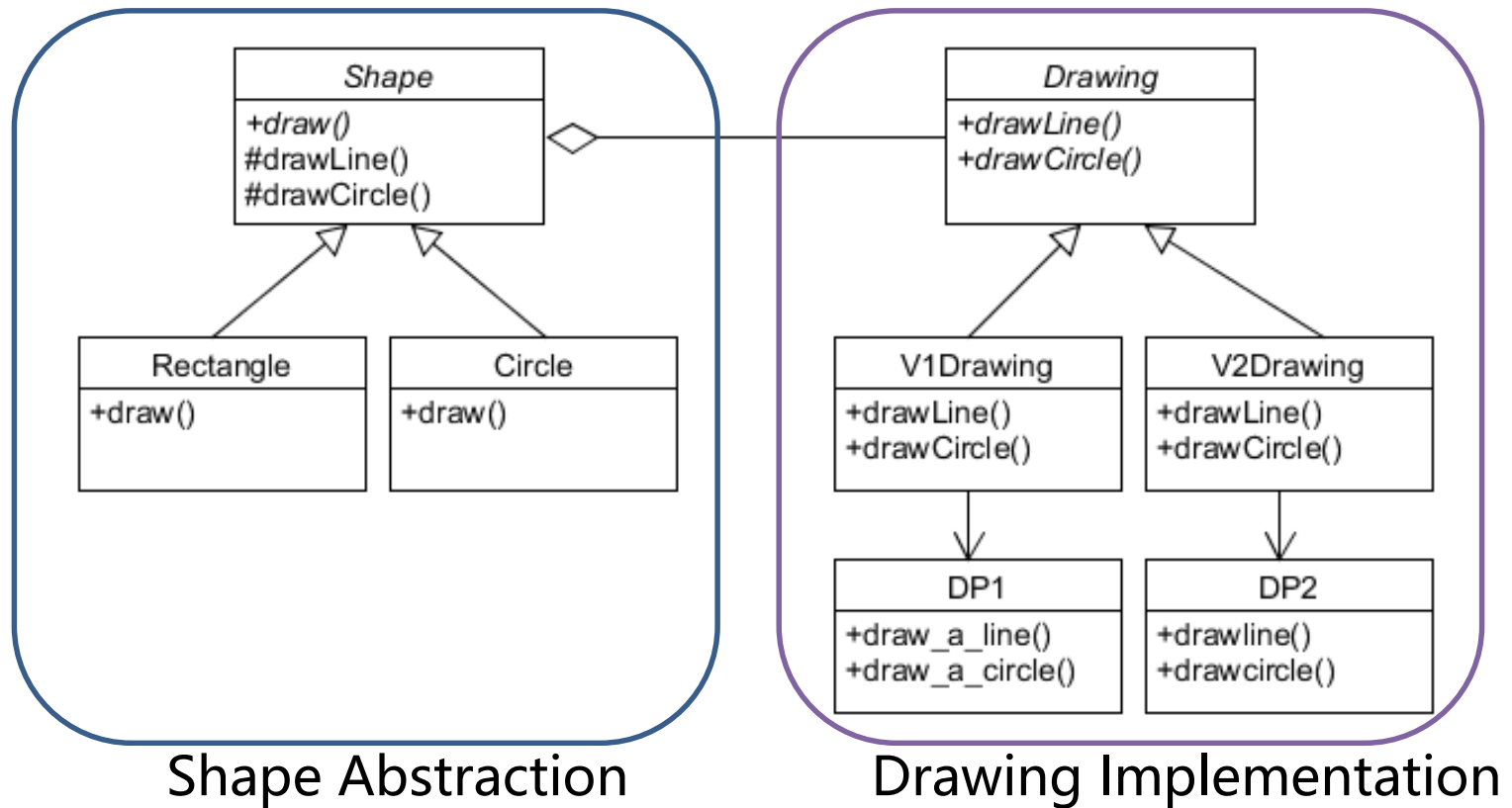
- The next step is to represent the specific variations that are present. For Shape, rectangles and circles. For drawing programs, DP1 and DP2 respectively.



Relating each other

- Relating the classes by having one use the other
- Option 1: Drawing programs uses Shape
 - If drawing programs could draw shapes directly, then they would have to know some things about shapes in general: what they are, what they look like. But this violates a fundamental principle of objects: an object should only be responsible for itself.
 - It also violates encapsulation. Drawing objects would have to know specific information about the Shapes (the kind of Shape) in order to draw them.
- Option 2: Shape uses the Drawing programs
 - Shapes use Drawing objects to draw themselves.
 - Shapes wouldn't need to know what type of Drawing object it used since we could have Shapes refer to the Drawing class.

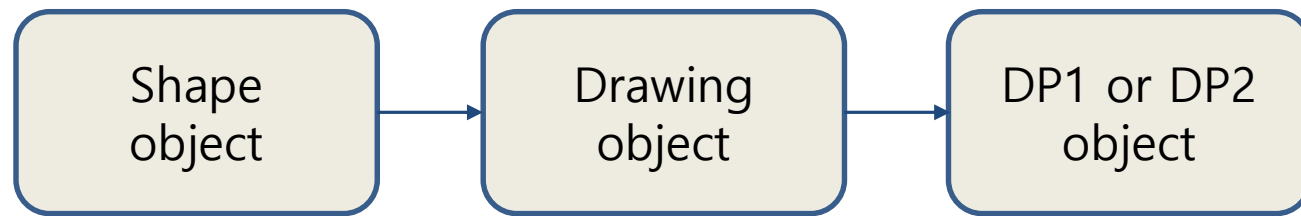
Solution using Bridge



- Find what varies and encapsulate it
- Favor object composition over class inheritance
 - Variations in abstractions of a concept and Variations in how these concepts are implemented.

Visualizing the Solution

There are only three objects to deal with at any one time, even though there are several classes



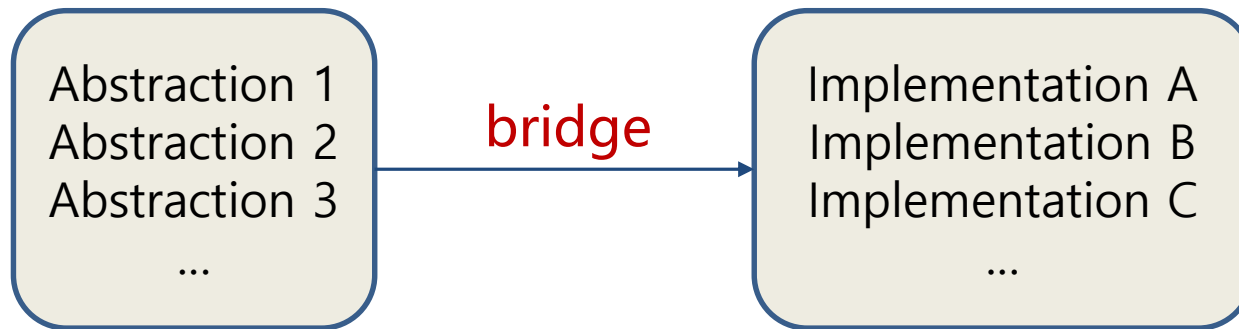
This is actually a Rectangle or a Circle, but the client cannot tell which, since they share the same interface

This is actually a V1Drawing or a V2Drawing, but the Shape cannot tell which, since they share the same interface

This must be the correct type of object, but the Drawing object using it will know which it is

Introducing Bridge Pattern

- Intent of the Bridge pattern
 - decouples an abstraction from its implementation so that the two can vary independently.



Solution Code

```
class Client {  
    public static void main (String argv[]) {  
        Shape r1, r2;  
        Drawing dp;  
  
        dp = new V1Drawing();  
        r1 = new Rectangle( dp, 1, 1, 2, 2 );  
  
        dp = new V2Drawing();  
        r2 = new Circle( dp, 2, 2, 3 );  
  
        r1.draw();  
        r2.draw();  
    }  
}
```

Solution Code

```
abstract class Shape {  
    abstract public void draw();  
    private Drawing _dp;  
  
    Shape (Drawing dp) {  
        _dp = dp;  
    }  
    public void drawLine (double x1, double y1, double x2, double y2) {  
        _dp.drawLine(x1,y1,x2,y2);  
    }  
  
    public void drawCircle (double x, double y, double r) {  
        _dp.drawCircle(x,y,r);  
    }  
}
```

```
abstract public class Drawing {  
    abstract public void drawLine(double x1, double y1,double x2,  
        double y2);  
  
    abstract public void drawCircle (double x, double y, double r);  
}
```

Solution Code

```
class V1Drawing extends Drawing {
    public void drawLine(double x1, double y1, double x2, double y2) {
        DP1.draw_a_line(x1,y1,x2,y2);
    }
    public void drawCircle(double x, double y, double r) {
        DP1.draw_a_circle(x,y,r);
    }
}

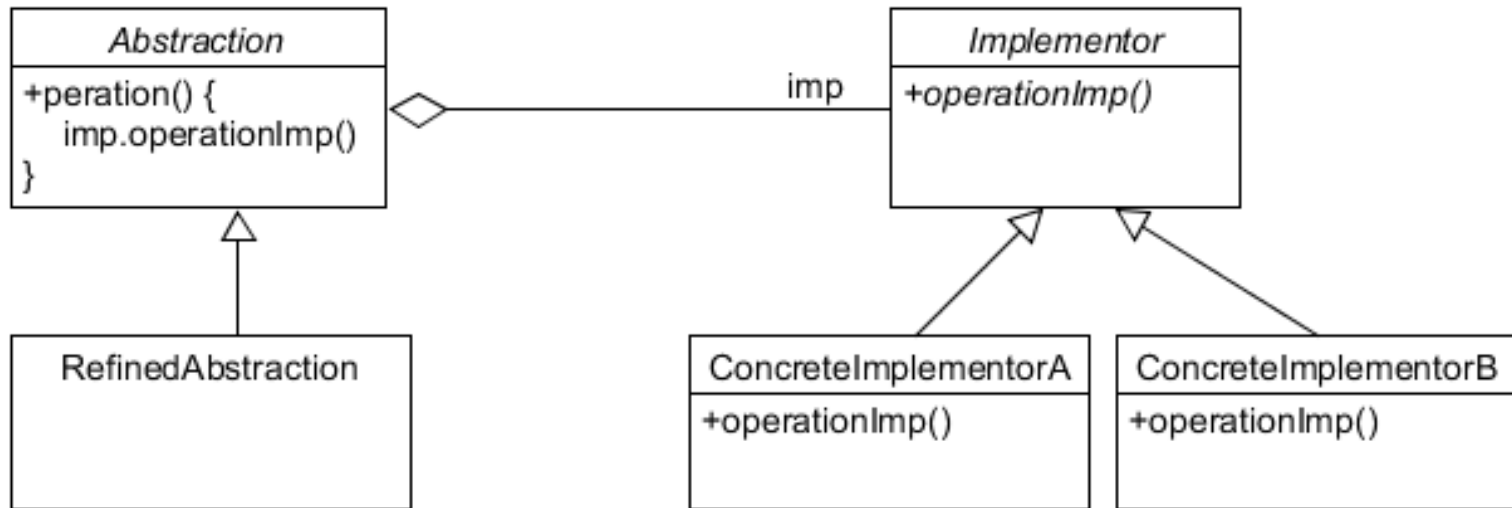
class V2Drawing extends Drawing {
    public void drawLine(double x1, double y1, double x2, double y2) {
        DP2.drawline(x1,x2,y1,y2);
    }
    public void drawCircle (double x, double y, double r) {
        DP2.drawcircle(x,y,r);
    }
}
```


Solution Code

```
class Rectangle extends Shape {
    public Rectangle (Drawing dp, double x1, double y1, double x2, double y2) {
        super(dp);
        _x1 = x1; _x2 = x2; _y1 = y1; _y2 = y2;
    }
    public void draw() {
        drawLine(_x1,_y1,_x2,_y1);
        drawLine(_x2,_y1,_x2,_y2);
        drawLine(_x2,_y2,_x1,_y2);
        drawLine(_x1,_y2,_x1,_y1);
    }
}

class Circle extends Shape {
    public Circle (Drawing dp, double x, double y, double r) {
        super(dp);
        _x = x; _y = y; _r = r;
    }
    public void draw() {
        drawCircle(_x,_y,_r);
    }
}
```

Structure of Bridge Pattern



- Abstraction (and RefinedAbstraction) is a high-level control layer for some entity
- This layer isn't supposed to do any real work on its own.
- It should delegate the work to the implementation layer (also called platform).

Note that we're not talking about "abstract classes" from the programming language. These aren't the same things!

Participants & Collaborations

- **Abstraction**

- defines the abstraction's interface
- maintains a reference to the Implementor
- forwards requests to the Implementor (collaboration)

- **RefinedAbstraction**

- extends abstraction interface

- **Implementor**

- defines interface for implementations

- **ConcreteImplementor**

- implements Implementor interface, i.e. defines an implementation

Comparison with Adapter

- Similarities:
 - Both used to hide the details of the underlying implementation.
- Difference:
 - The adapter pattern is geared towards making unrelated components work together
 - Applied to systems **after they're designed** (reengineering, interface engineering).
 - The bridge pattern is used **up-front in a design** to let abstractions and implementations vary independently.
 - Designing “extensible system”
 - New “beasts” can be added to the “object zoo”, even if these are not known at analysis or system design time.
 - Structural difference
 - *Bridge* can abstract a complex entity from its implementation
 - *Adapter* only abstracts a single interface

Summary

- Bridge pattern
 - lets you split a large class or a set of closely related classes into two separate hierarchies—**abstraction and implementation**—which can be developed independently of each other.

Bridge Pattern Quiz : Problem

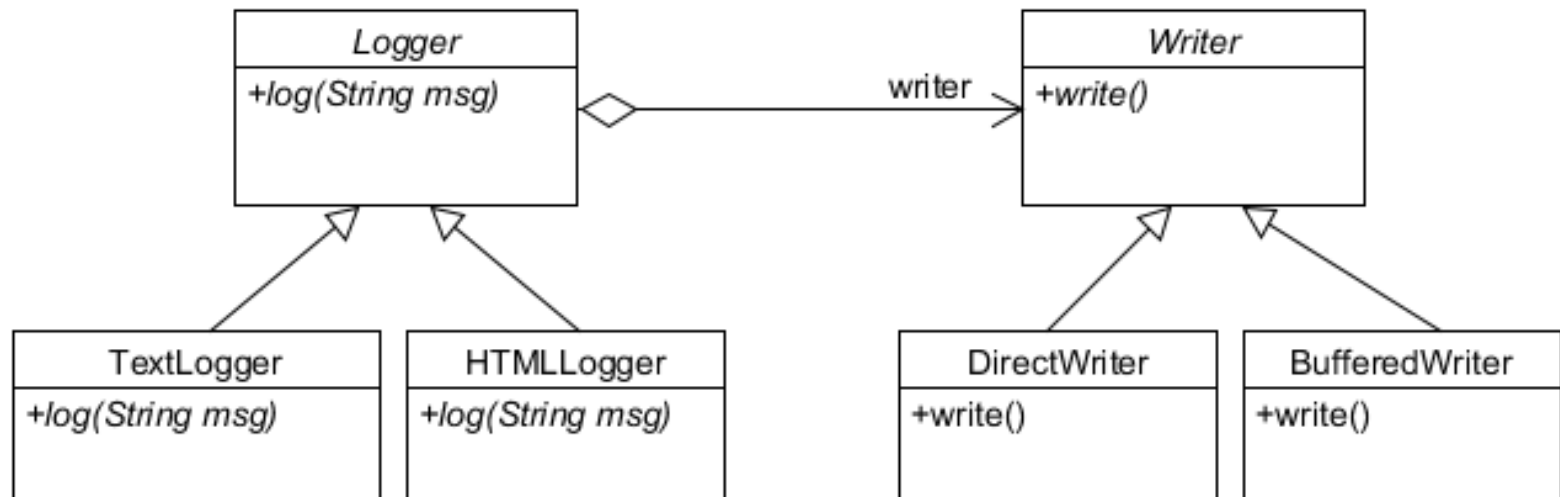
Logger에는 일반 Text 메시지로 로깅하는 TextLogger와 HTML 메시지로 로깅하는 HTMLLogger가 있다. Text 메시지를 쓰는(Write) 라이브러리에는 DirectWriter 클래스와 BufferedWriter 클래스가 있으며, Logger는 이중 어떤 것이라도 채택하여 사용할 수 있다. 추후 Logger와 Writer의 변종들이 늘어날 수 있다.

위의 요구사항에 대한 해결책을 Bridge 패턴을 사용하여 설계하시오.

Bridge Pattern Quiz : Answer

Logger에는 일반 Text 메시지로 로깅하는 TextLogger와 HTML 메시지로 로깅하는 HTMLLogger가 있다. Text 메시지를 쓰는(Write) 라이브러리에는 DirectWriter 클래스와 BufferedWriter 클래스가 있으며, Logger는 이중 어떤 것이라도 채택하여 사용할 수 있다. 추후 Logger와 Writer의 변종들이 늘어날 수 있다.

위의 요구사항에 대한 해결책을 Bridge 패턴을 사용하여 설계하시오.



18. MVC PATTERN

2020. 04. 16

중앙대학교 소프트웨어학부

이찬근 교수

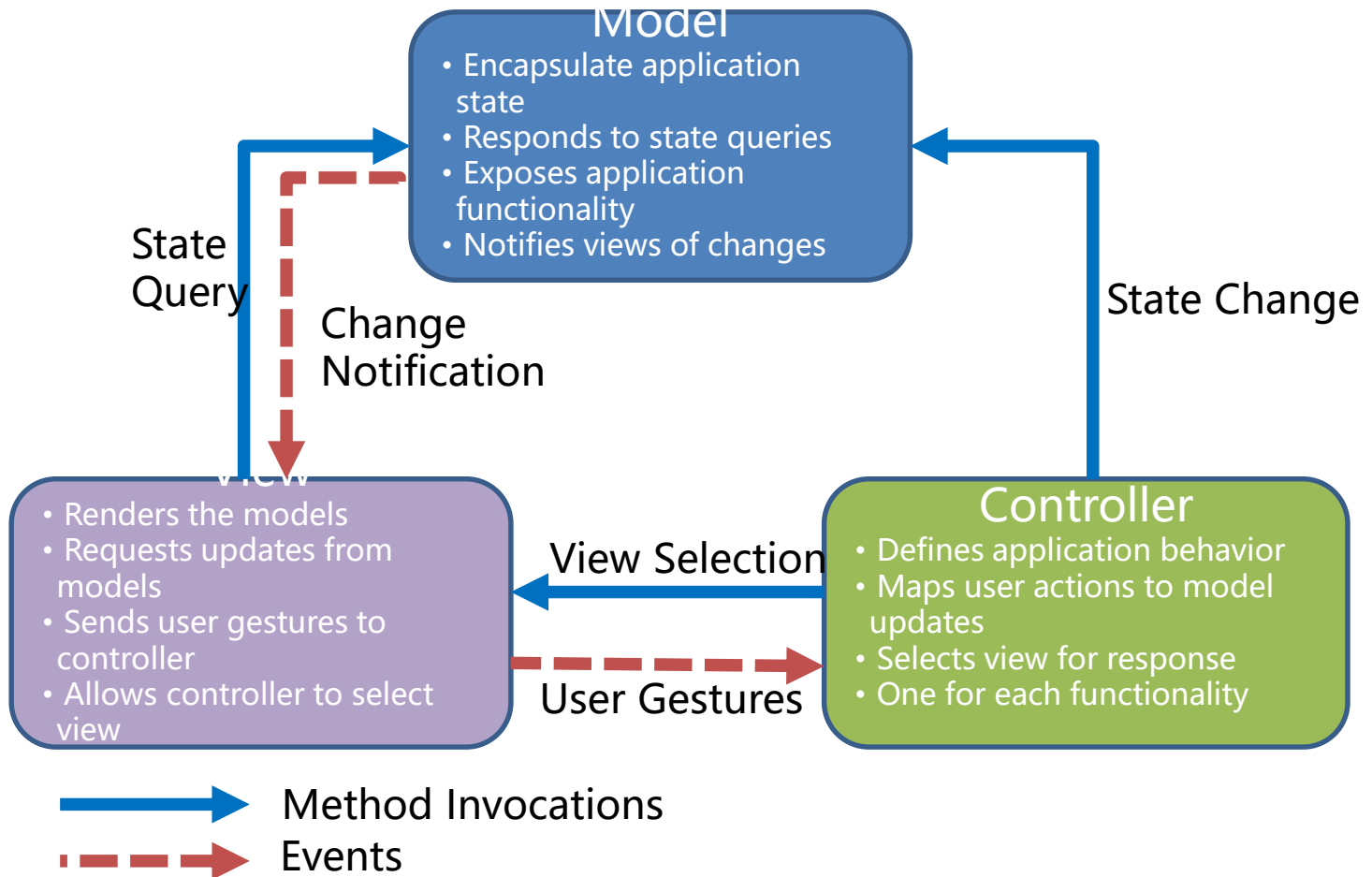
Contents

- Structure of MVC
- Motivation of MVC
- MVC Example with Bouncing Ball
- MVC Example with Java DJ View

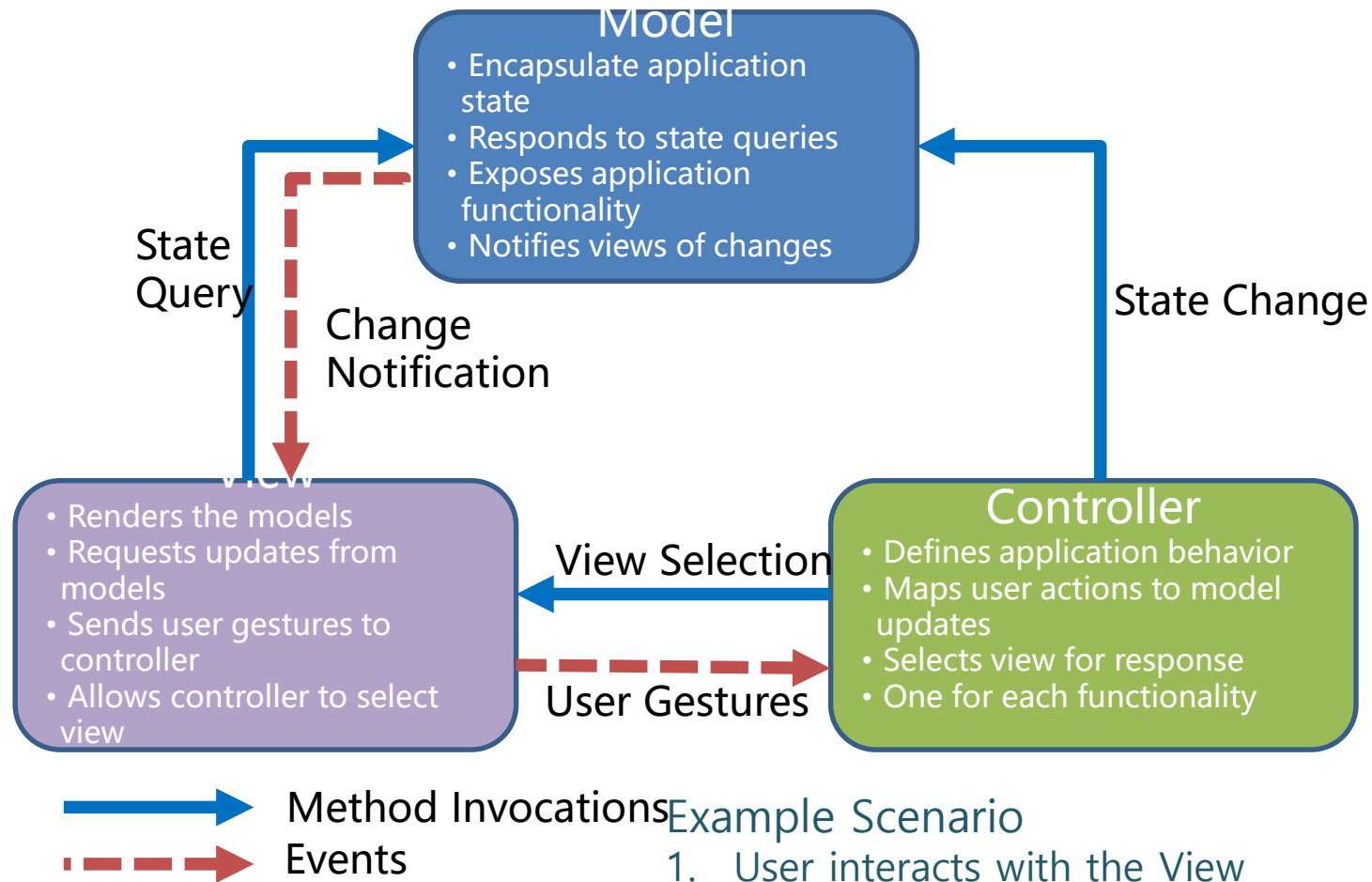
Introduction to Model-View-Controller Pattern

- Compound Pattern
 - A compound pattern combines two or more patterns into a solution that solves a recurring or general problem.
 - MVC uses different patterns
 - Strategy pattern
 - Observer pattern
 - Composite pattern
- MVC consists of three kinds of objects:
 - **Model** – the application object
 - **View** – UI (screen presentation)
 - **Controller** – defines the way the UI reacts to user inputs

MVC Structure



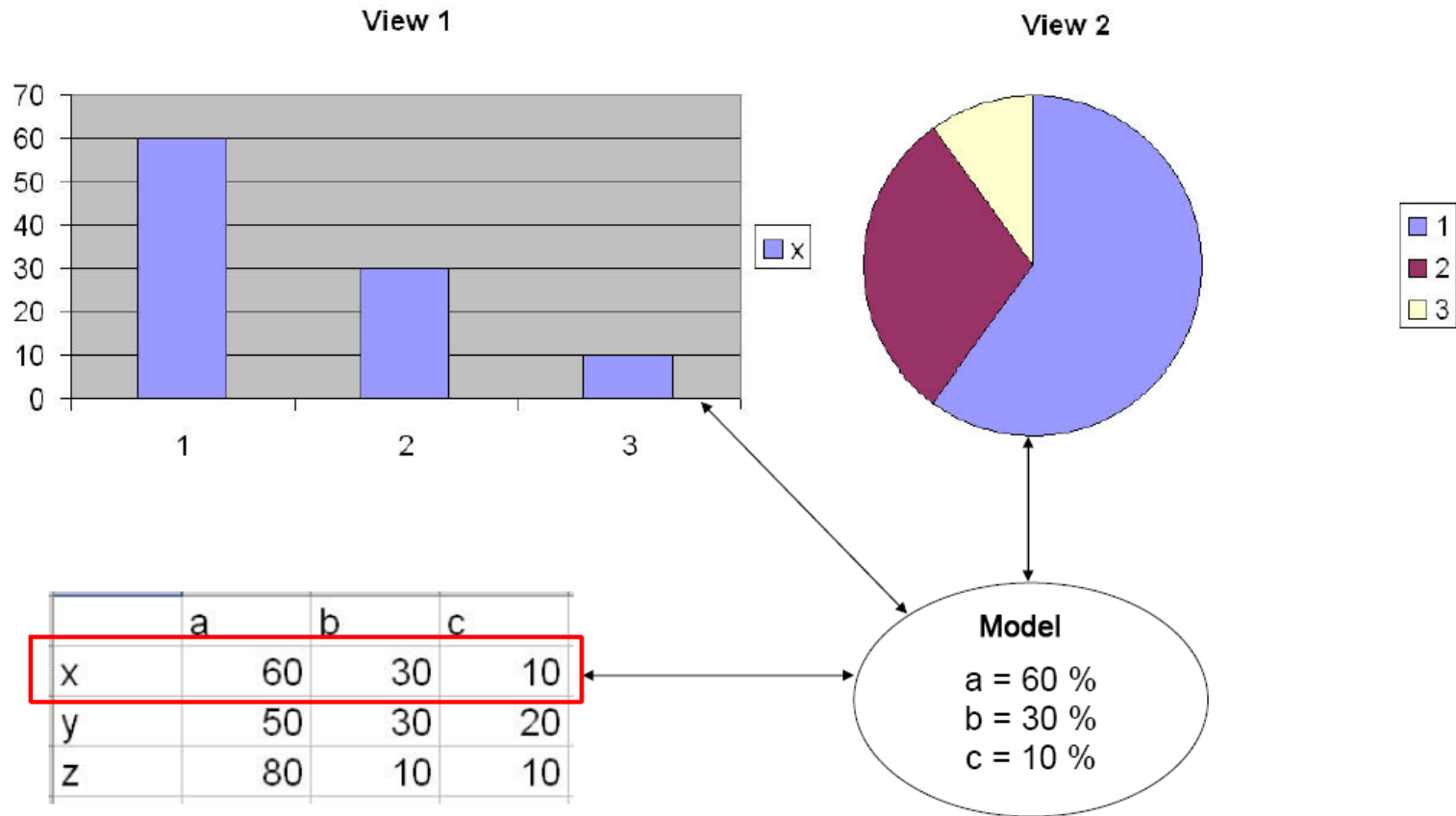
MVC Structure



Example Scenario

1. User interacts with the View
2. The controller handles the user input (often a callback function attached to UI elements)
3. The controller updates the model
4. The model notifies the view and it renders the changes
5. The view waits for user interaction

MVC Example



Motivations

- The same enterprise data needs to be accessed when **presented in different views**
 - e.g. HTML, JFC/swing, XML
- The same enterprise data needs to be **updated through different interactions**
- Supporting **multiple types of views and interactions should not impact** the components that provide the **core functionality** of the enterprise application

Solution

- **Separating responsibilities**

- **Separate core business model** functionality from the **presentation** and **control logic**
- Allows **multiple views** to share the same enterprise data model
- Makes supporting multiple clients **easier to implement, test, and maintain**

- **Model**

- Enterprise **data** and the business **rules**

- **View**

- **Renders** the contents of a model
- Accesses data through the **model**

- **Controller**

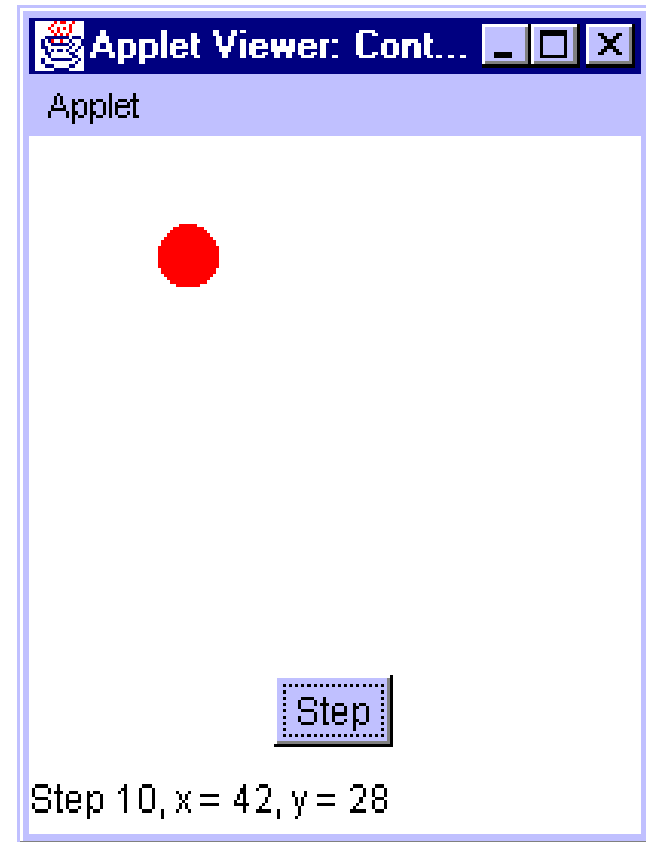
- **Translates** interactions with the **view** into actions to be performed by the **model**

Patterns in MVC

- Decouples views from models:
 - **Observer pattern**
 - Decoupling objects so that changes to one can affect any number of others without requiring the object to know details of the others
- Nested view widgets:
 - **Composite pattern**
 - Grouping primitive and composite objects into new objects with identical interfaces: Composite View objects
- View delegates the gesture interpretation to controllers:
 - **Strategy pattern**
 - Controllers can be replaced to support different interpretations
- MVC uses **Factory** and **Decorator patterns** as well

First Example: Bouncing Ball Applet

- Each click of the Step button advances the ball a small amount
- The step number and ball position are displayed in the status line



The Ball Applet: Model

- The Ball Applet shows a ball bouncing in a window
- The Model controls the motion of the ball
- The Model must know the size of the window
 - so it knows when the ball should be made to bounce
- The Model does not need to know anything else about the GUI
- The Model is a Observable
 - `java.util` provides an `Observer` *interface* and an `Observable` *class*
 - An `Observable` is an object that can be “observed”
 - An `Observer` is “notified” when an object that it is observing announces a change

Model code

```
class Model extends Observable {
    final int BALL_SIZE = 20;
    int xPosition = 0;
    int yPosition = 0;
    int xLimit, yLimit;
    int xDelta = 6;
    int yDelta = 4;

    void makeOneStep() {
        xPosition += xDelta;
        if (xPosition < 0 || xPosition >= xLimit) {
            xDelta = -xDelta;
            xPosition += xDelta;
        }
        yPosition += yDelta;
        if (yPosition < 0 || yPosition >= yLimit) {
            yDelta = -yDelta;
            yPosition += yDelta;
        }
        setChanged();
        notifyObservers();
    } // end of makeOneStep method
} // end of Model class
```

The Ball Applet: View

- Needs access to the ball's state
 - in this case, its x-y location
- Should draw a ball
- The View is a Observer
- The View displays what is going on in the model
 - It should never display what *should* be going on in the model
 - For example, if you ask to save a file, the View shouldn't itself tell you that the file has been saved—it should tell you what the model reports

View code

```
class View extends Canvas implements Observer {
    Controller controller;
    Model model;
    int stepNumber = 0;

    View (Model model) {
        this.model = model;
    }
    public void paint(Graphics g) {
        g.setColor(Color.red);
        g.fillOval(model.xPosition, model.yPosition, model.BALL_SIZE,
            model.BALL_SIZE);
        controller.showStatus("Step " + (stepNumber++) +
            ", x = " + model.xPosition +
            ", y = " + model.yPosition);
    } // end paint method

    public void update(Observable obs, Object arg) {
        repaint();
    }
} // end class
```

The Ball Applet: Controller

- Controller tells the Model what to do
- Controller tells the View when it needs to refresh the display
- Controller doesn't need to know the inner workings of the Model
- Controller doesn't need to know the inner workings of the View

Controller code

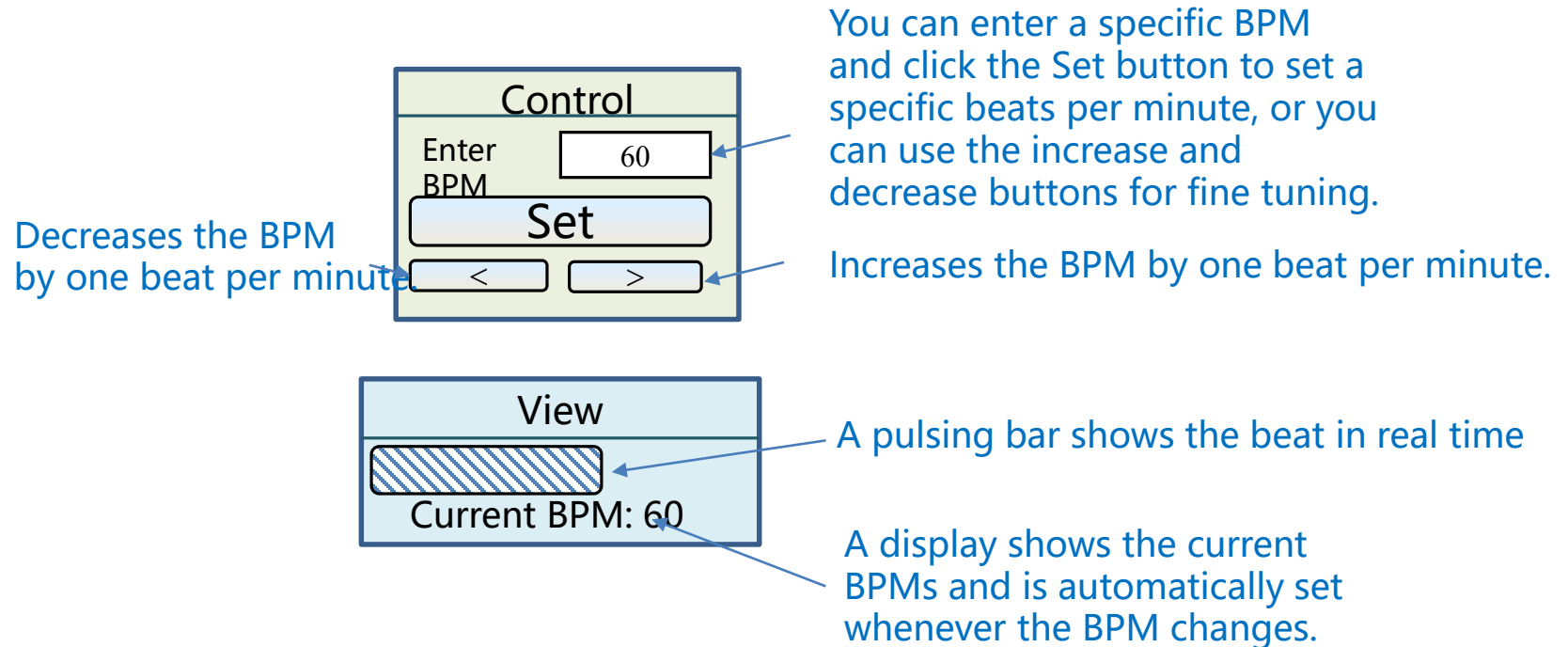
```
public class Controller extends JApplet {
    JPanel buttonPanel = new JPanel();
    JButton stepButton = new JButton("Step");
    Model model = new Model();
    View view = new View();

    public void init() {
        setLayout(new BorderLayout());
        buttonPanel.add(stepButton);
        this.add(BorderLayout.SOUTH, buttonPanel);
        this.add(BorderLayout.CENTER, view);
        stepButton.addActionListener(new ActionListener() {
            public void actionPerformed(ActionEvent event) {
                model.makeOneStep();
            }
        });
        model.addObserver(view);
        view.controller = this;
    }

    public void start() {
        model.xLimit = view.getSize().width - model.BALL_SIZE;
        model.yLimit = view.getSize().height - model.BALL_SIZE;
        repaint();
    }
}
```

Second Example: Java DJ View

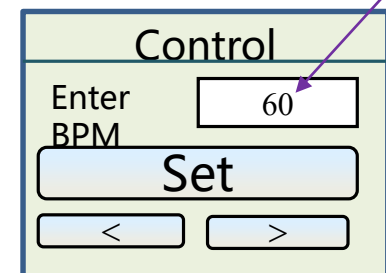
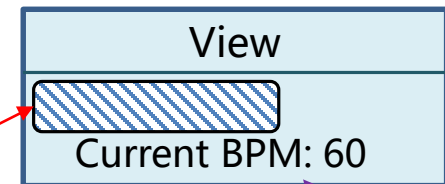
- Time to be a DJ and start mixing with a beat!
- The Java DJ view: It has two parts, the part for viewing the state of the model and the part for controlling things.



BeatModel Interface

```
public interface BeatModelInterface {  
    void initialize ();  
    void on ();  
    void off ();  
    void setBPM (int bpm);  
    int getBPM ();  
    void registerObserver (BeatObserver o);  
    void removeObserver (BeatObserver o);  
    void registerObserver (BPMObserver o);  
    void removeObserver (BPMObserver o);  
}
```

We've split this into two kinds of observers:
observers that want to be notified on every beat, and
observers that just want to be notified with the beats per
minute changes.



Concrete BeatModel class

```
public class BeatModel implements BeatModelInterface, MetaEventListener {
    Sequencer sequencer;
    ArrayList beatObservers = new ArrayList();
    ArrayList bpmObservers = new ArrayList();
    int bpm = 90;
    // other instance variables
    public void initialize() { setupMidi(); buildTrackAndStart(); }
    public void on() { sequencer.start(); setBPM(90); }
    public void off() { setBPM(0); sequencer.stop(); }

    public void setBPM (int bpm) {
        this.bpm = bpm;
        sequencer.setTempoInBPM(getBPM());
        notifyBPMObservers();
    }
    public int getBPM() { return bpm; }

    void beatEvent () {
        notifyBeatObservers();
    }
    // code to register and notify observers
    // Lots of MIDI code to handle the beat.
}
```

Implementing the View (just an outline!)

```
public class DJView implements ActionListener, BeatObserver, BPMObserver {
    BeatModelInterface model;
    ControllerInterface controller;
    // lots of Swing components...
    public DJView (ControllerInterface cont, BeatModelInterface model) {
        this.controller = cont;
        this.model = model;
        model.registerObserver ((BeatObserver) this);
        model.registerObserver ((BPMObserver) this);
    }
    public void createView () { // create all Swing components here }
    public void updateBPM () {
        if (model.getBPM () == 0) bpmOutputLabel.setText ("offline");
        else bpmOutputLabel.setText ("Current BPM: " + model.getBPM ( ));
    }
    public void updateBeat () { beatBar.setValue (100); }

    public void actionPerformed (ActionEvent event) {
        if (event.getSource ( ) == setBPMButton) {
            int bpm = Integer.parseInt (bpmTextField.getText ( ) );
            controller.setBPM (bpm);
        } else if (event.getSource ( ) == increaseBPMButton) {
            controller.increaseBPM ( ) ;
        } else if ...
        // other action processings
    }
}
```

Controller Interface

- Remember - the controller is the strategy that we plug into the view.
- What does a Strategy pattern look like? What do we need?

```
public interface ControllerInterface {  
    void start ( );  
    void stop ( );  
    void increaseBPM ( );  
    void decreaseBPM ( );  
    void setBPM (int bpm);  
}
```

Concrete Controller

```
public class BeatController implements ControllerInterface {
    BeatModelInterface model;
    DJView view;
    public BeatController(BeatModelInterface model) {
        this.model = model;
        view = new DJView(this, model);
        view.createView();
        view.createControls();
        view.disableStopMenuItem();
        view.enableStartMenuItem();
        model.initialize();
    }
    public void start() {
        model.on();
        view.disableStartMenuItem();
        view.enableStopMenuItem();
    }
    public void stop() {
        model.off();
        view.disableStopMenuItem(); view.enableStartMenuItem();
    }
    public void setBPM (int bpm) { model.setBPM (bpm); }
    public void increaseBPM() { model.setBPM(model.getBPM() + 1); }
    public void decreaseBPM() { model.setBPM(model.getBPM() - 1); }
}
```

Putting it all together...

```
public class DJTestDrive {  
    public static void main (String[] args) {  
        BeatModelInterface model = new BeatModel ();  
        ControllerInterface controller = new BeatController (model);  
    }  
}
```

Notes

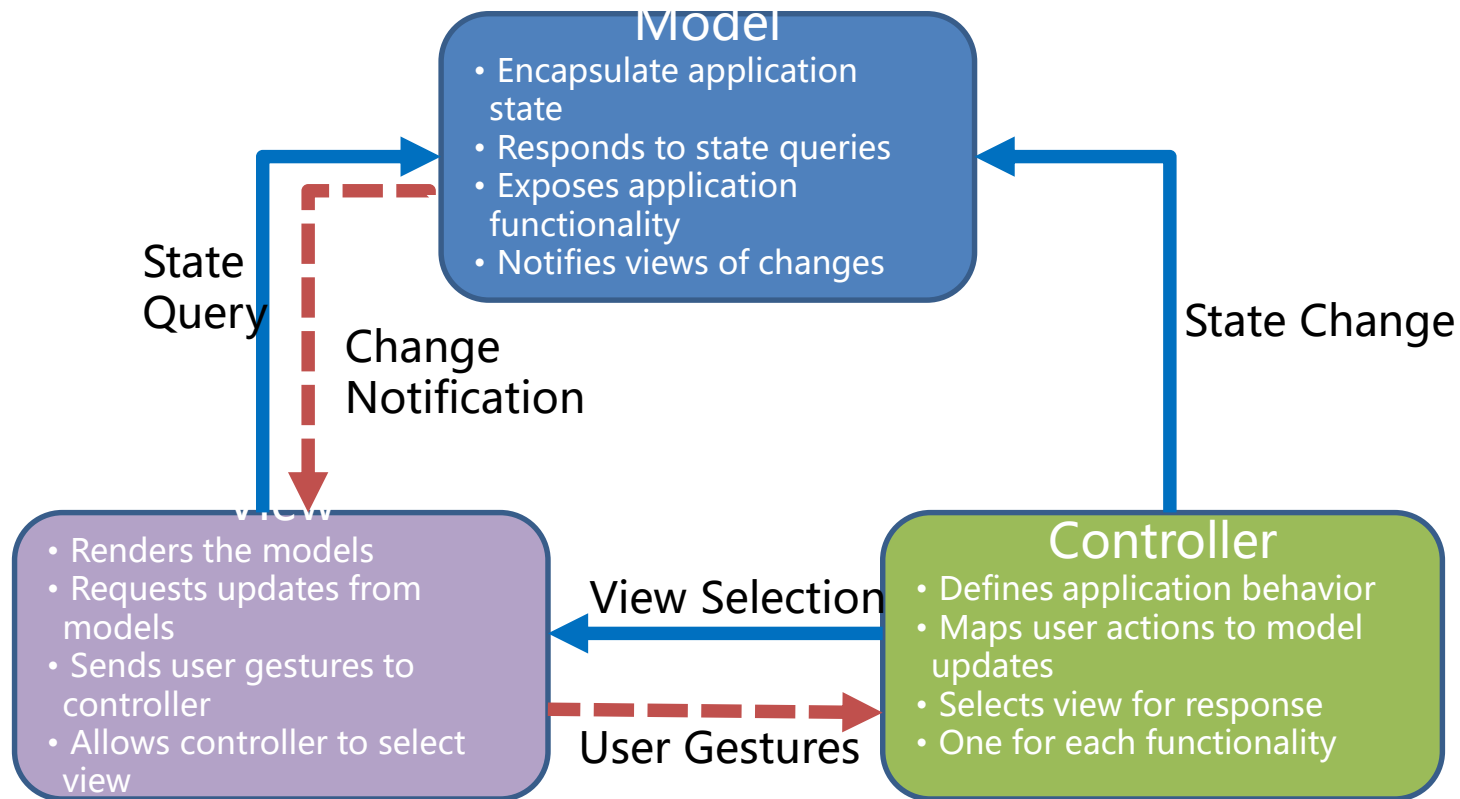
- Litmus test: Swapping out user interface
 - Can the model be used, without modification, by a completely different UI?
 - e.g. Swing vs console text interface
- Model can be easily tested with JUnit
- Model actions should be quick
 - GUI is frozen while model executes
 - Alternative: multithreading

Summary

- MVC pattern is composed of the Observer, Strategy, and Composite patterns.
 - Model: makes use of the **Observer** pattern so that it can keep observers updated, yet stay decoupled from them.
 - Controller: is the **Strategy** for the view. The view can use different implementations of the controller to get different behavior.
 - View: uses **Composite** Pattern to implement the user interface, which usually consists of nested components like panels, frames, and buttons.
- These patterns work together to decouple the three players in the MVC model, which keeps designs clear and flexible.

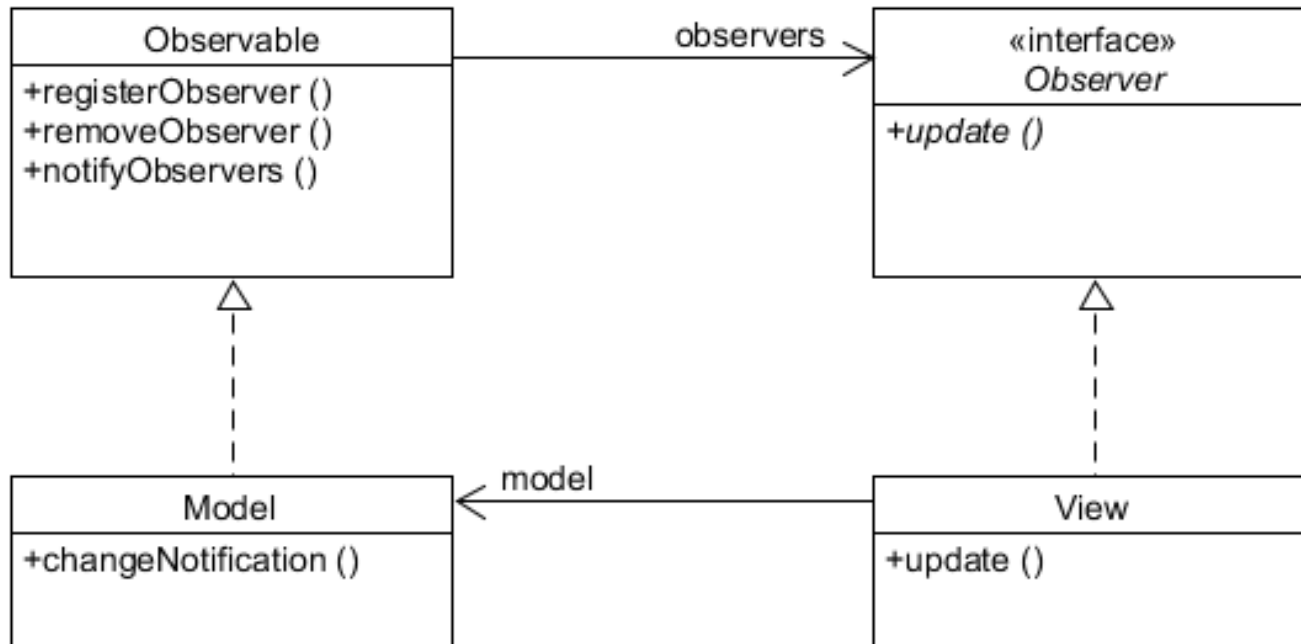
MVC Pattern Quiz : Problem

MVC 패턴의 클래스 다이어그램을 보면 Model이 View에 대한 약한 결합 관계를 보이고 있는데 이러한 제한이 왜 필요한지, 또한 구체적으로 어떻게 구현되는지 설명하시오.



MVC Pattern Quiz : Answer

```
class Model extends Observable {  
    // ...  
    void changeNotification () {  
        setChanged();  
        notifyObservers();  
    }  
    // ...  
}
```



필요성: 잦은 UI의 변화가 Model의 변화로 자동 파급되지 않도록함.
UI가 교체되더라도 Model은 그대로 재사용 가능

PATTERN SUMMARY

2020. 04. 16

중앙대학교 소프트웨어학부

이찬근 교수

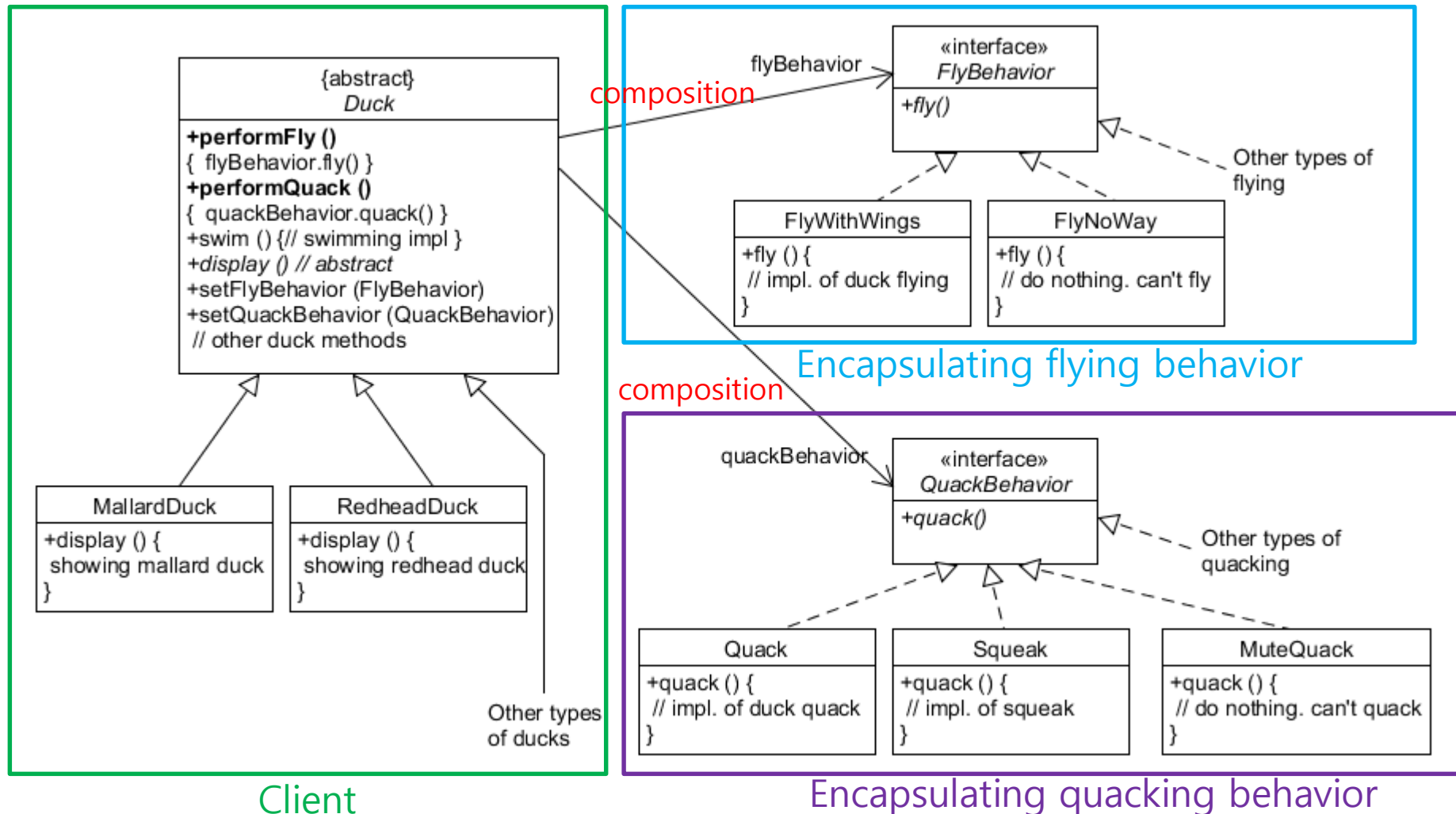


Contents

- 1. Patterns: Design for change**
- 2. Category of GOF patterns**
- 3. Pattern pitfalls**
- 4. Journeys ahead**

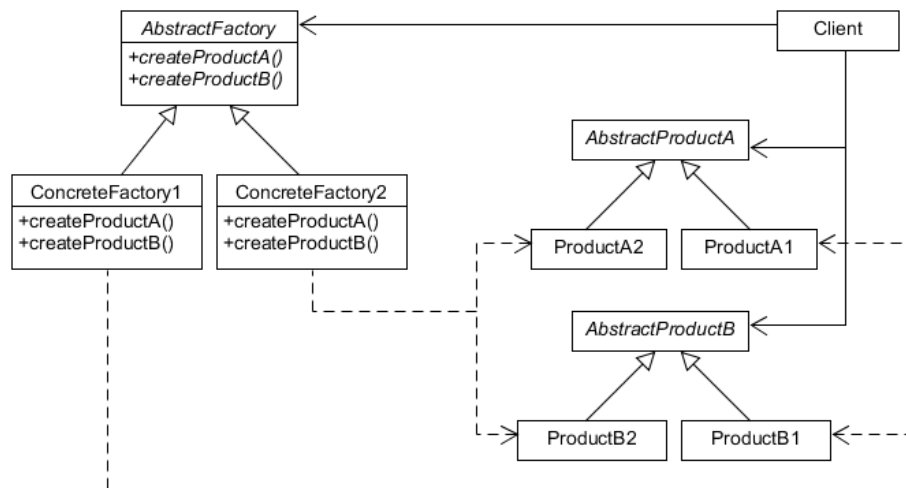
Designing for Change

- Design patterns help to ensure that a system can change in specific ways → easier to change

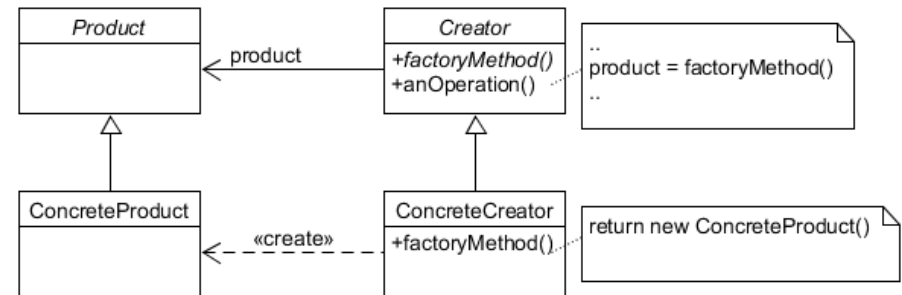


Creating an Object by Specifying a Class Explicitly

- complicates future changes
- Design patterns : [Abstract Factory](#), [Factory Method](#), [Builder](#)



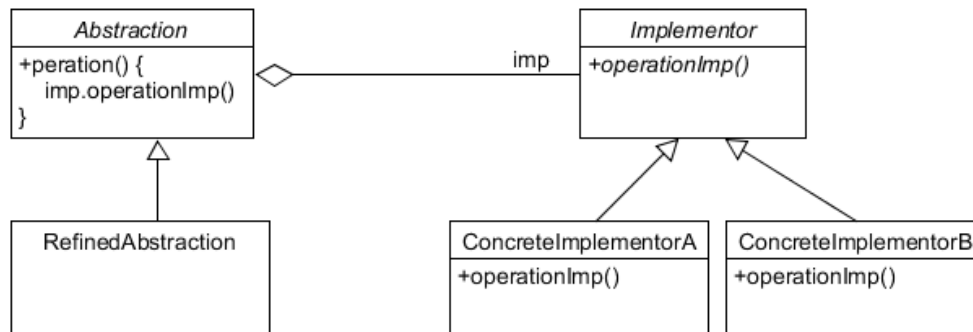
Abstract Factory pattern



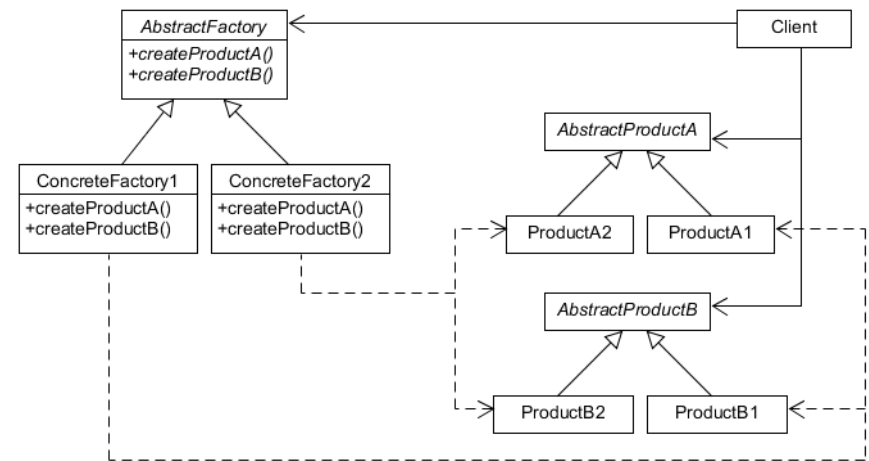
Factory Method pattern

Dependence on Hardware and Software Platform

- harder to port to other platforms
- harder to keep it up to date on its native platform
- Design patterns : **Bridge**, **Abstract Factory**



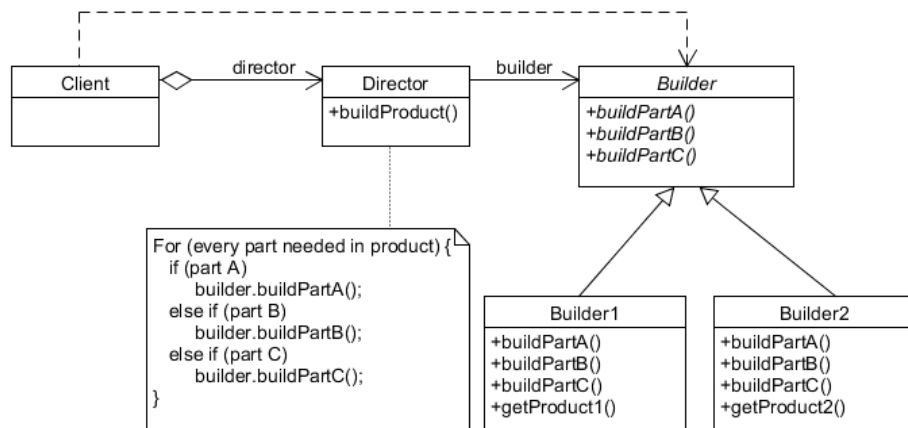
Bridge pattern



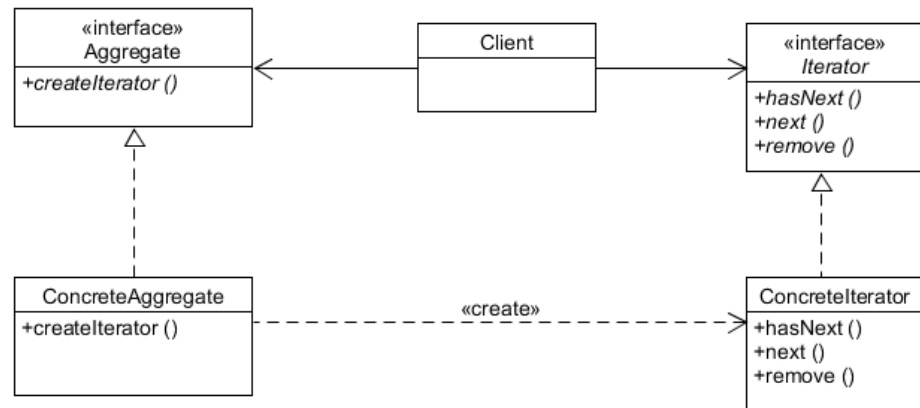
Abstract Factory pattern

Algorithmic Dependencies

- algorithm changes → object changes
 - algorithm that are likely to change should be isolated
- Design patterns : **Builder**, **Iterator**, **Strategy**, **Template Method**

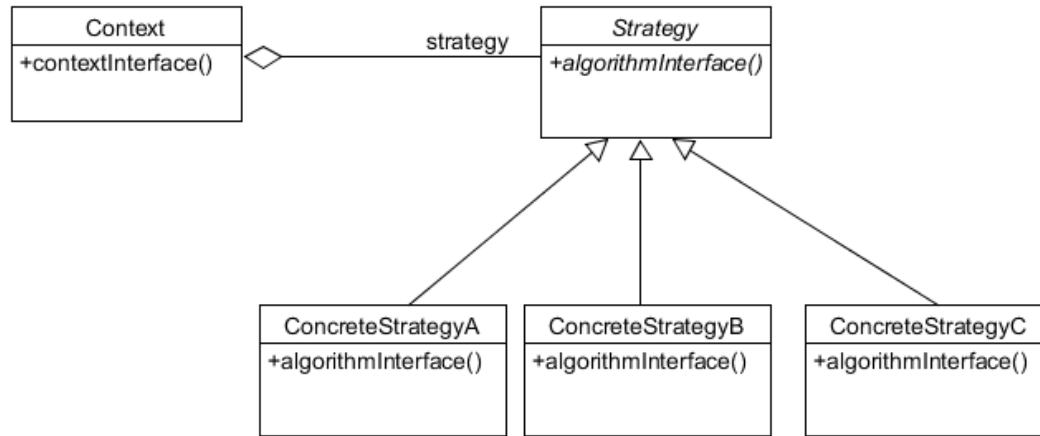


Builder pattern

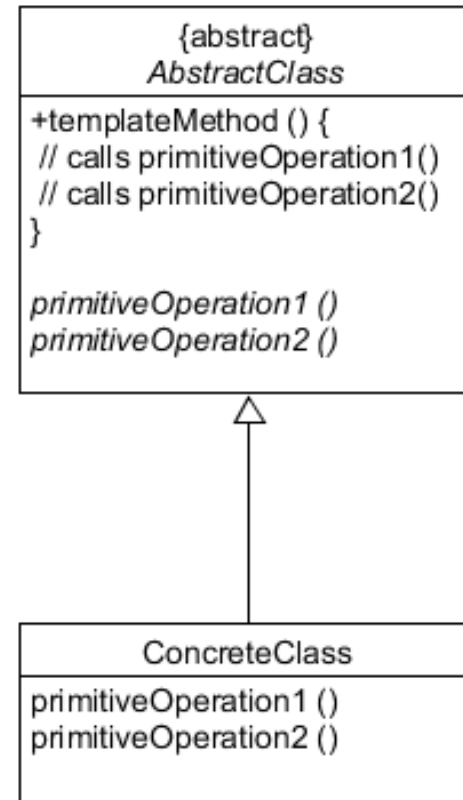


Iterator pattern

Algorithmic Dependencies



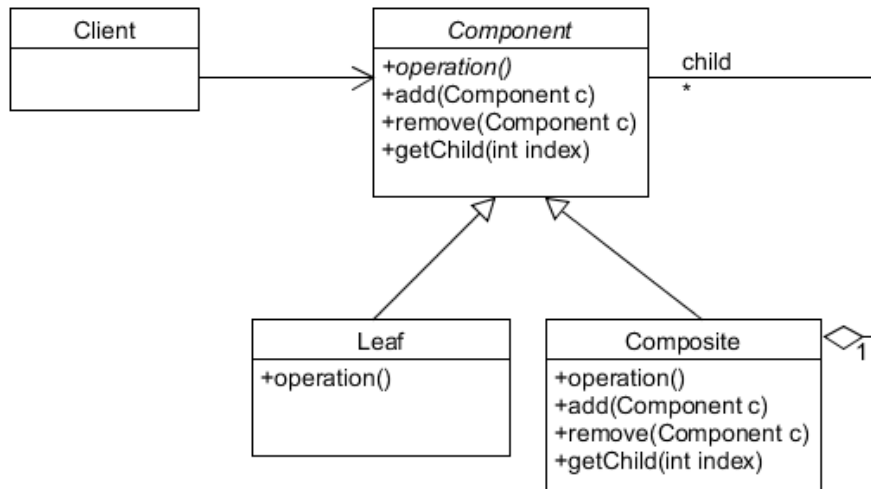
Strategy pattern



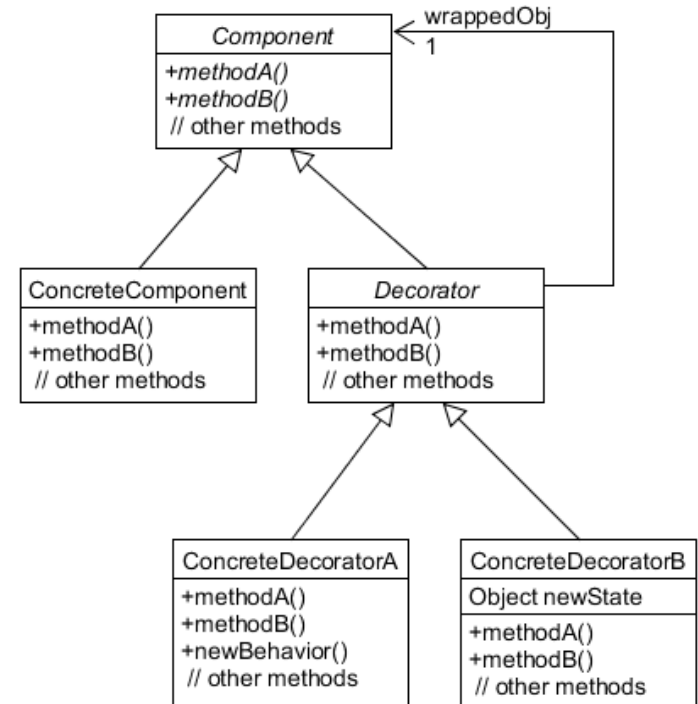
Template Methods pattern

Extending Functionality by Subclassing

- subclassing can lead to explosion of subclasses
 - object composition and delegation provide flexible alternatives
- Design patterns : [Bridge](#), [Composite](#), [Decorator](#), [Strategy](#)



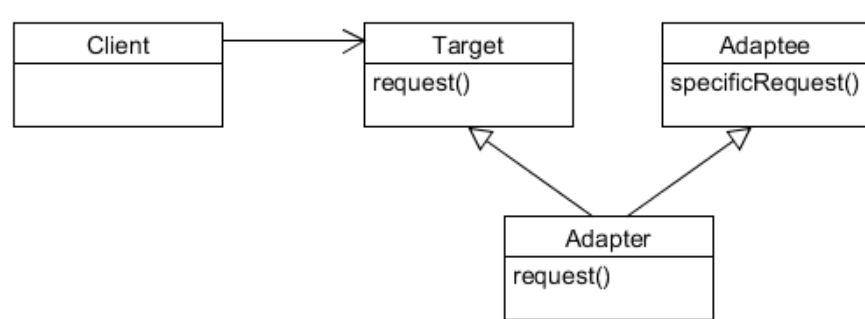
Composite pattern



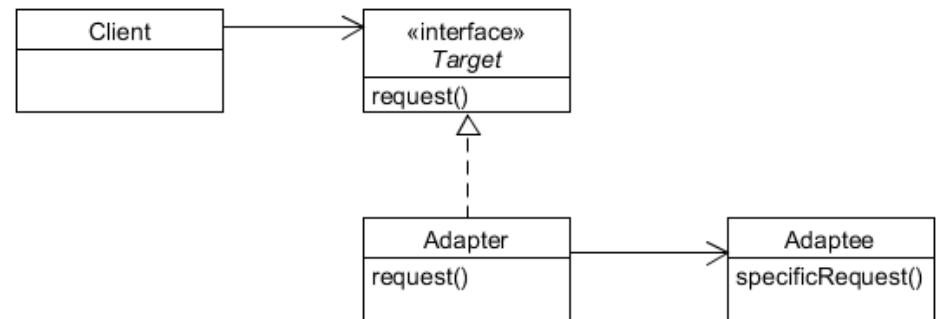
Decorator pattern

Inability to Alter Classes Conveniently

- When modifying a class that cannot be modified conveniently
 - no source code
 - any change would require modifying lots of existing subclasses
- Design patterns : [Adapter](#), [Decorator](#)



Class Adapter



Object Adapter

Category of GoF Patterns

		Purpose		
		Defer object creation to another class or object	Describe ways to assemble objects	Address problems of assigning responsibilities
		Creational	Structural	Behavioral
Scope	Class	Factory Method	Adapter	Interpreter Template
	Object	Abstract Factory Builder Prototype Singleton	Adapter Bridge Composite Decorator Façade Flyweight Proxy	CoR Command Iterator Mediator Memento Observer State Strategy Visitor

Need Practice

- Design patterns can't guarantee a good overall architecture
 - They're just micro-architectures
- Creativity still required
 - not all design decisions are covered by patterns
 - the solutions of some patterns look similar
 - composition & delegation → State, Strategy, Bridge, ...
 - but the problem/intent they address is different
- Learning the patterns takes time
 - You have to experience the problem to appreciate the solution

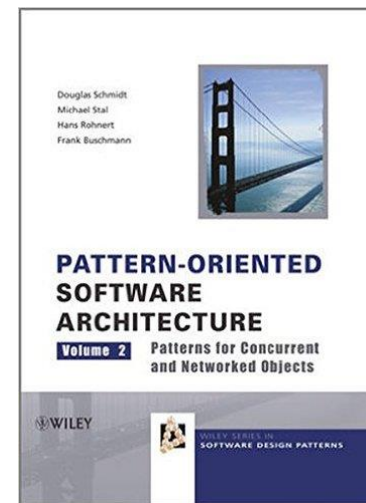
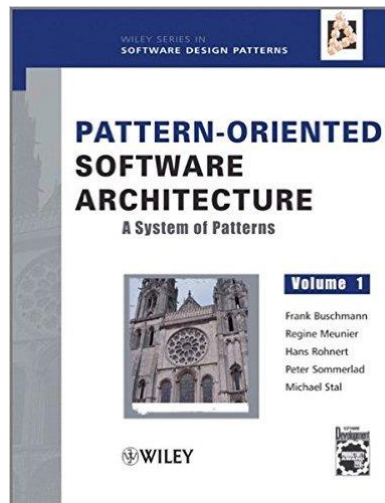
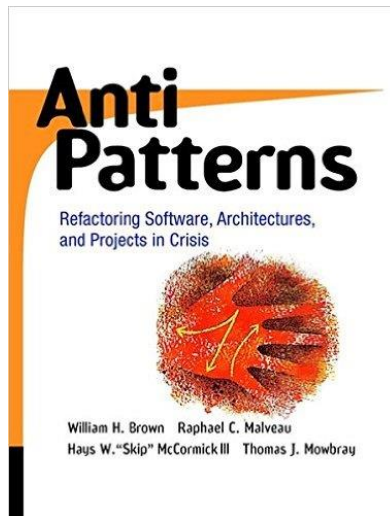
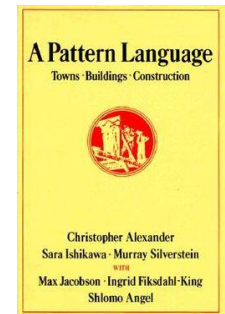
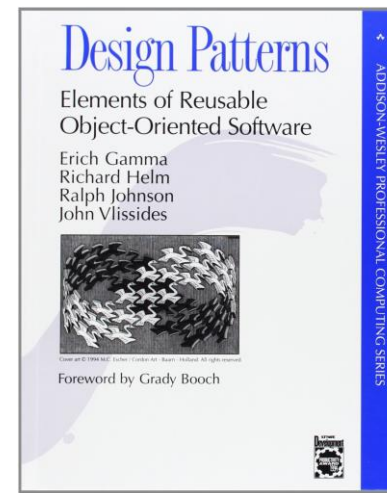
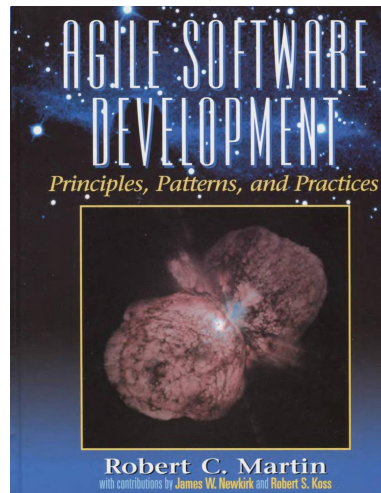
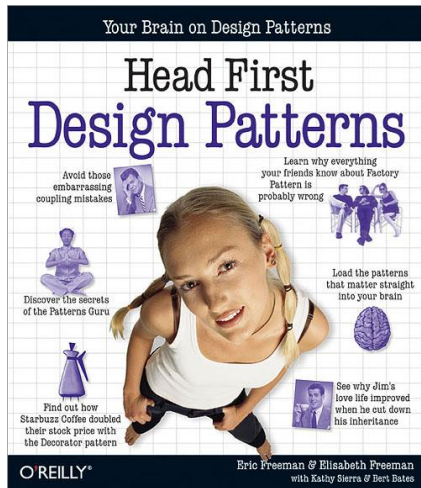
Pattern Pitfalls

- Overenthusiasm
 - patterns have costs (indirection, complexity)
 - design should be as flexible as needed, not as flexible as possible
 - “Complex systems that work evolved from simple systems that worked.”
- Booch
 - “Start stupid and evolve.” - Beck
- Overly dense application
 - E.g., a class that participates in all 23 patterns!

Minds on Patterns

- Beginner
 - uses patterns everywhere
- Intermediate
 - sometimes tries to squeeze patterns to the problems
 - sometimes also sees where patterns are needed and where they aren't
- Zen
 - is able to see patterns where they fit naturally.
 - looks for **simple solutions** that best solve the problem
 - thinks in terms of object **principles** and their **trade-offs**
 - When a **need for a pattern naturally arises**, the Zen mind applies it knowing well that it may require **adaptation**
 - Sees relationships to **similar patterns** and understands the subtleties of differences in the **intent of the related patterns**
 - does not let all that pattern knowledge overly influence **design decisions**

Journey Ahead



19. PATTERN SUMMARY

2020. 04. 16

중앙대학교 소프트웨어학부

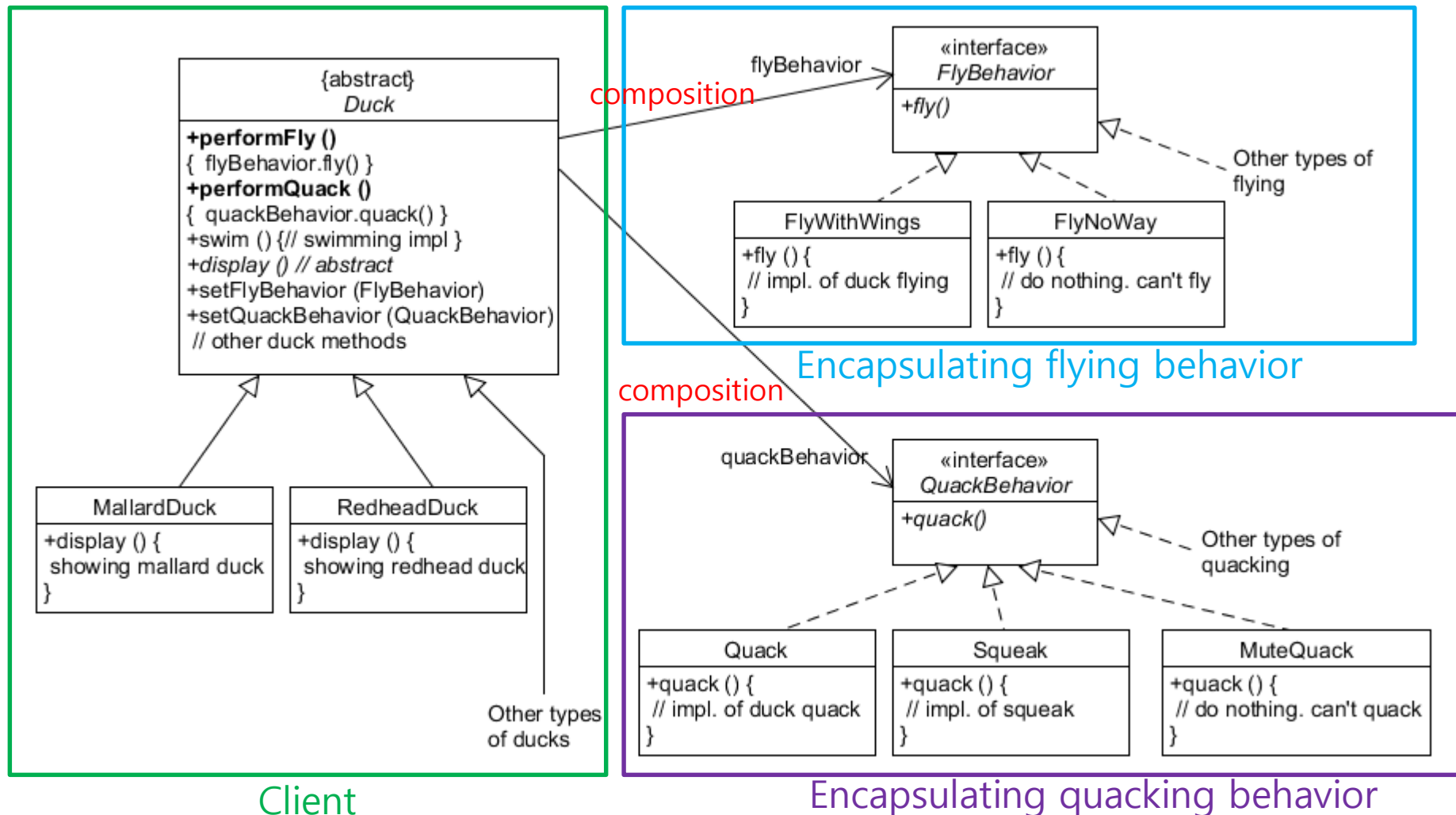
이찬근 교수

Contents

- Patterns: Design for change
- Category of GOF patterns
- Pattern pitfalls
- Journeys ahead

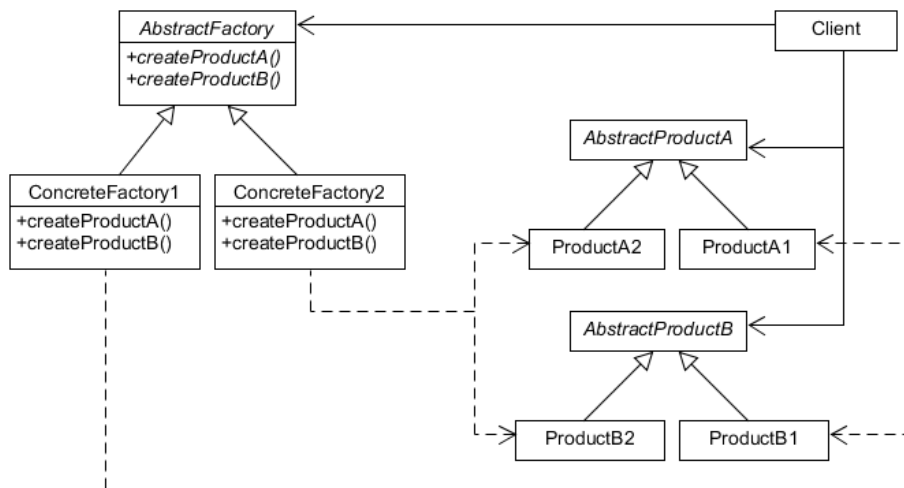
Designing for Change

- Design patterns help to ensure that a system can change in specific ways → easier to change

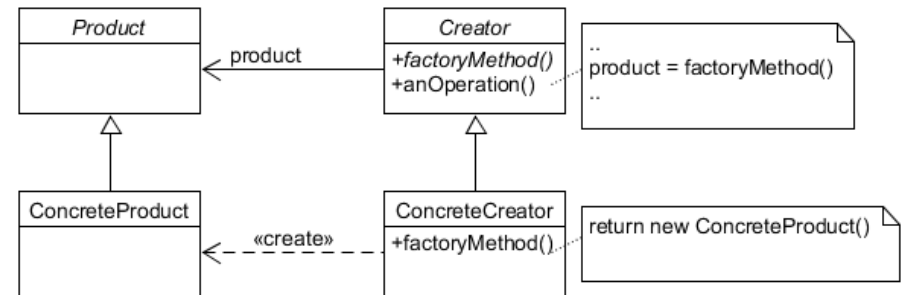


Creating an Object by Specifying a Class Explicitly

- complicates future changes
- Design patterns : [Abstract Factory](#), [Factory Method](#), [Builder](#)



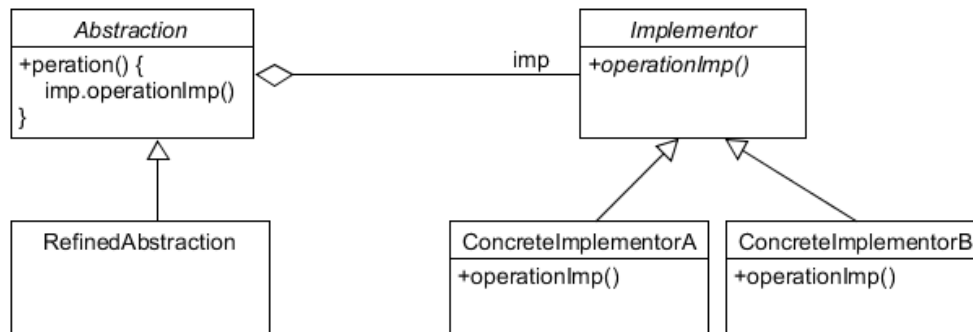
Abstract Factory pattern



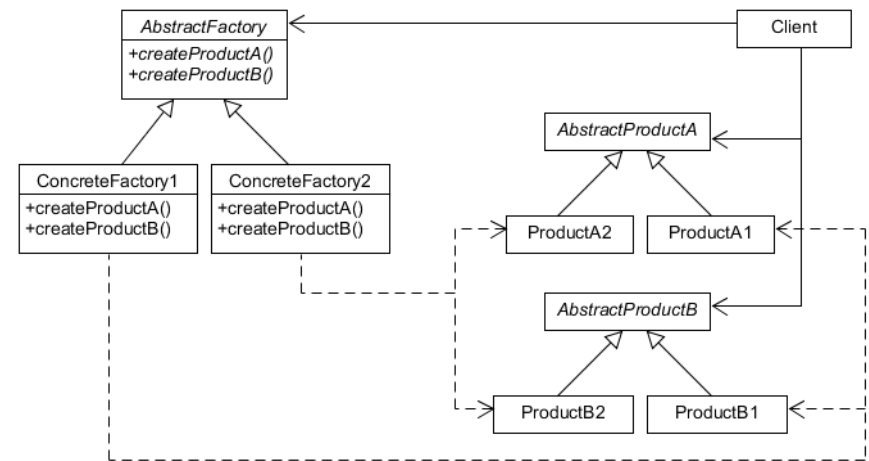
Factory Method pattern

Dependence on Hardware and Software Platform

- harder to port to other platforms
- harder to keep it up to date on its native platform
- Design patterns : **Bridge**, **Abstract Factory**



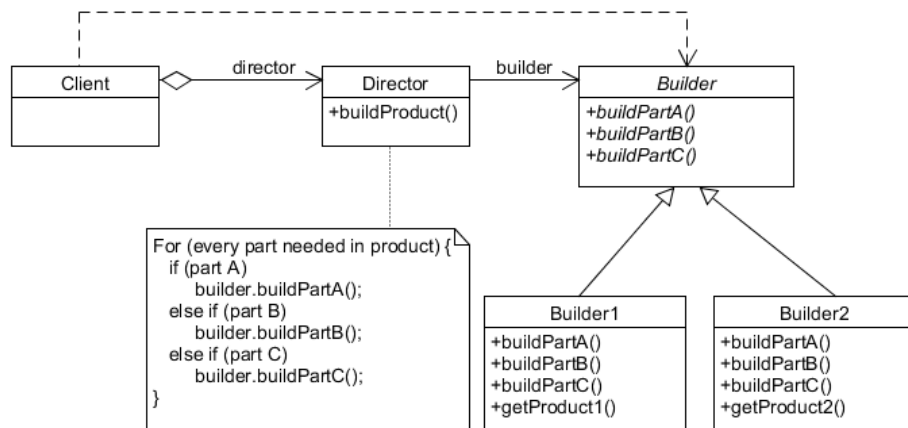
Bridge pattern



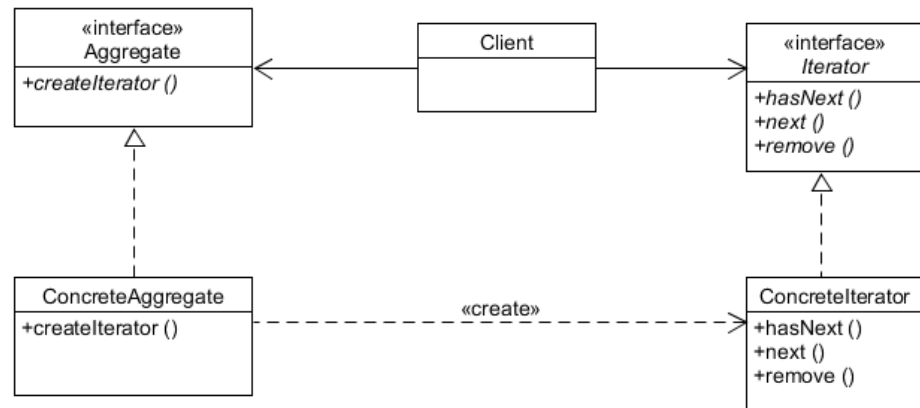
Abstract Factory pattern

Algorithmic Dependencies

- algorithm changes → object changes
 - algorithm that are likely to change should be isolated
- Design patterns : **Builder**, **Iterator**, **Strategy**, **Template Method**

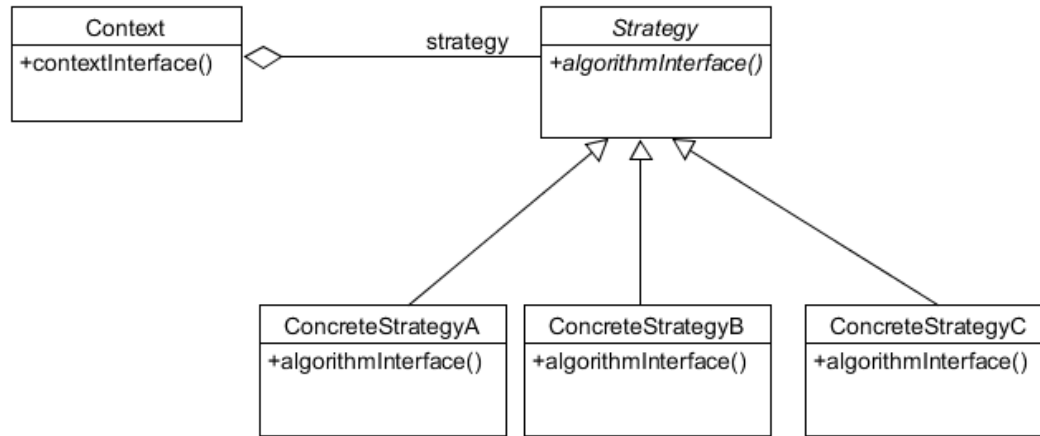


Builder pattern

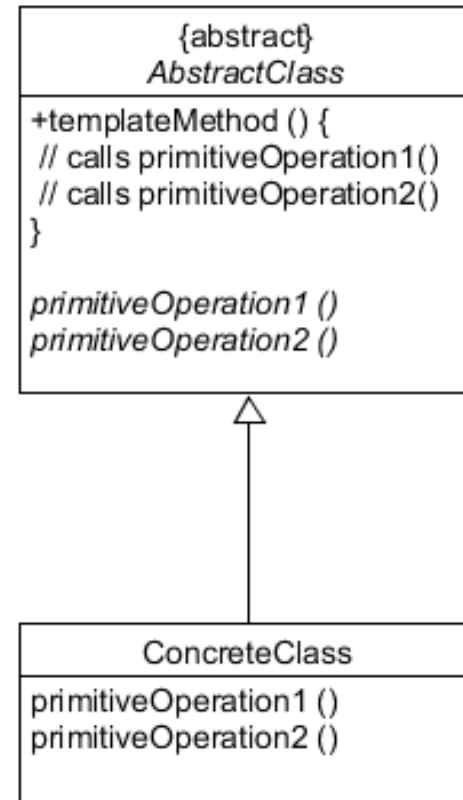


Iterator pattern

Algorithmic Dependencies



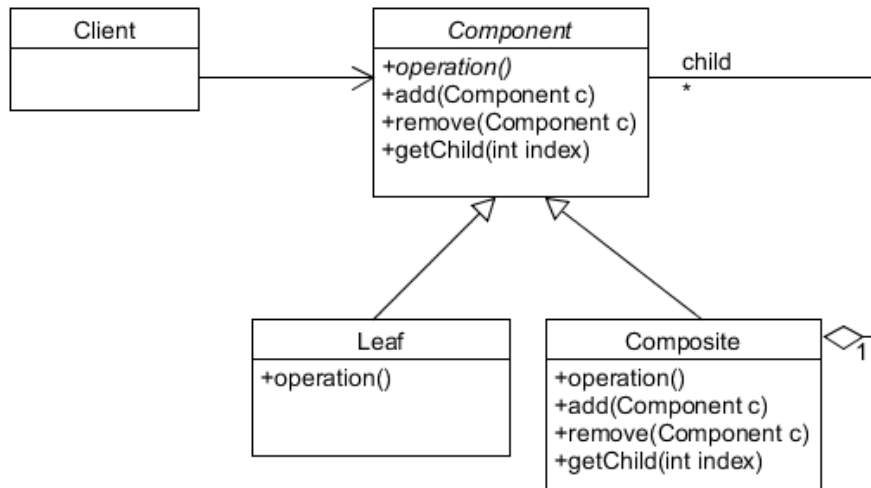
Strategy pattern



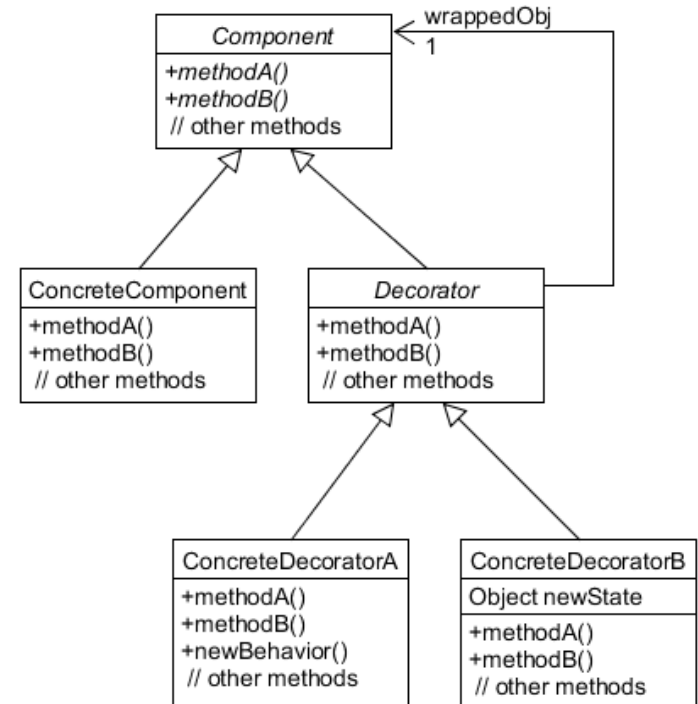
Template Methods pattern

Extending Functionality by Subclassing

- subclassing can lead to explosion of subclasses
 - object composition and delegation provide flexible alternatives
- Design patterns : [Bridge](#), [Composite](#), [Decorator](#), [Strategy](#)



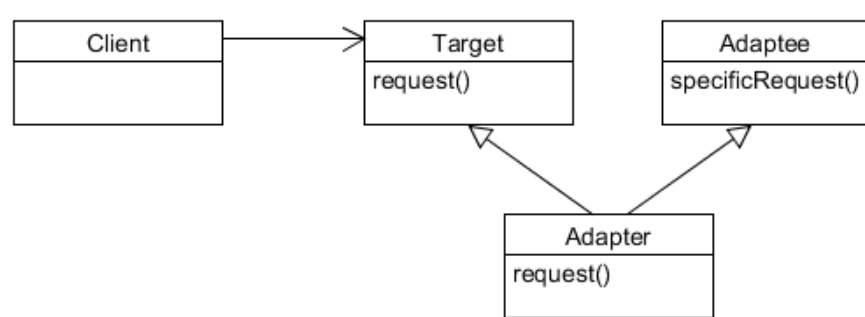
Composite pattern



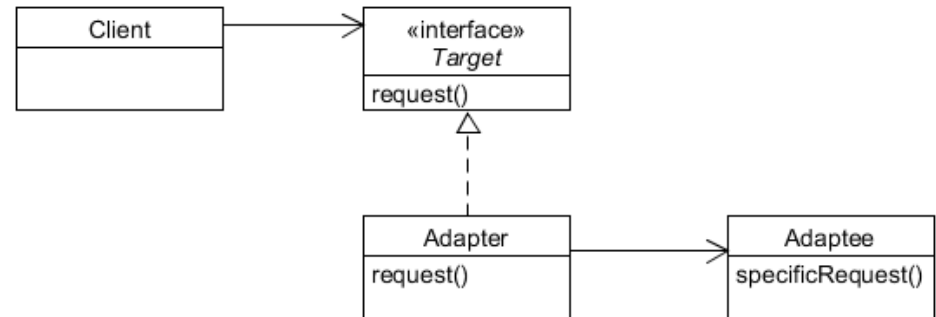
Decorator pattern

Inability to Alter Classes Conveniently

- When modifying a class that cannot be modified conveniently
 - no source code
 - any change would require modifying lots of existing subclasses
- Design patterns : [Adapter](#), [Decorator](#)



Class Adapter



Object Adapter

Category of GoF Patterns

		Purpose		
		Defer object creation to another class or object	Describe ways to assemble objects	Address problems of assigning responsibilities
		Creational	Structural	Behavioral
Scope	Class	Factory Method	Adapter	Interpreter Template
	Object	Abstract Factory Builder Prototype Singleton	Adapter Bridge Composite Decorator Façade Flyweight Proxy	CoR Command Iterator Mediator Memento Observer State Strategy Visitor

Need Practice

- Design patterns can't guarantee a good overall architecture
 - They're just micro-architectures
- Creativity still required
 - not all design decisions are covered by patterns
 - the solutions of some patterns look similar
 - composition & delegation → State, Strategy, Bridge, ...
 - but the problem/intent they address is different
- Learning the patterns takes time
 - You have to experience the problem to appreciate the solution

Pattern Pitfalls

- Overenthusiasm
 - patterns have costs (indirection, complexity)
 - design should be as flexible as needed, not as flexible as possible
 - “Complex systems that work evolved from simple systems that worked.”
- Booch
 - “Start stupid and evolve.” - Beck
- Overly dense application
 - E.g., a class that participates in all 23 patterns!

Minds on Patterns

- Beginner
 - uses patterns everywhere
- Intermediate
 - sometimes tries to squeeze patterns to the problems
 - sometimes also sees where patterns are needed and where they aren't
- Zen
 - is able to see patterns where they fit naturally.
 - looks for **simple solutions** that best solve the problem
 - thinks in terms of object **principles** and their **trade-offs**
 - When a **need for a pattern naturally arises**, the Zen mind applies it knowing well that it may require **adaptation**
 - Sees relationships to **similar patterns** and understands the subtleties of differences in the **intent of the related patterns**
 - does not let all that pattern knowledge overly influence **design decisions**

Journey Ahead

